
Personal Mathematics Ideas

A Collection of Opinionated Proofs and Concepts

Augustinas Malinauskas

Abstract:

These notes contain opinionated proofs and interesting ideas covering Linear Algebra, Calculus, Differential Geometry, Group Theory, Statistics, and related fields.

January 26, 2026

Contents

I	Linear Algebra	1
1	Vector Spaces and Linear Maps	2
1.1	Vector Spaces	2
1.2	Inner Product Spaces	3
1.3	Linear Maps	5
1.4	Matrix Representations	6
1.4.1	Motivation	6
1.4.2	Subspaces of $\mathbb{R}^d$	9
1.5	Eigenvalues and Eigenvectors	9
1.6	Applications of the Spectral Theorem	12
1.6.1	Machine Learning	12
1.6.2	Quantum Mechanics	13
1.6.3	Physics (Classical Mechanics)	14
1.6.4	Theoretical Physics	14
1.7	Singular Value Decomposition	15
1.7.1	Applications of Singular Value Decomposition	17
1.8	Principal Component Analysis	18
1.8.1	The Geometric Problem	18
1.8.2	Parametrizing Subspaces via Orthonormal Bases	18
1.8.3	Distance to a Subspace and Projection	19
1.8.4	The Matrix Optimization Problem	20
1.8.5	Applications of Principal Component Analysis	24
1.9	UMAP: Nonlinear Dimensionality Reduction	25
1.9.1	Limitations of Linear Methods	25
1.9.2	Mathematical Foundations	25
1.9.3	The Optimization Problem	28
1.9.4	Algorithm Summary	32
1.9.5	Comparison with PCA	32
1.9.6	Applications	33
II	Statistics	35
2	Random Variables and Distributions	36
2.1	Probability Measure	36

2.2	Expectation	37
2.3	Conditional Probability and Bayes' Theorem	38
2.4	Variance and the Central Limit Theorem	40
2.4.1	Variance and Standard Deviation	40
2.4.2	The Central Limit Theorem	40
2.4.3	Why CLT Matters in Machine Learning	41
2.5	Binary Classification Metrics	43
2.6	Information Theory and KL Divergence	46
2.6.1	Convexity and Jensen's Inequality	46
2.6.2	Entropy: Measuring Uncertainty	48
2.6.3	KL Divergence: Measuring Distribution Difference	48
III	Differential Geometry	52
3	Smooth Manifolds	53
3.1	Topological Manifolds	53
3.2	Smooth Structures	54
3.3	Smooth Maps	55
4	Tangent Spaces	56
4.1	Derivations and Tangent Vectors	56
4.2	Coordinate Basis	57
4.3	The Differential (Pushforward)	57
5	Cotangent Spaces and Differential Forms	58
5.1	Covectors and the Cotangent Space	58
5.2	The Pullback	59
5.3	Differential 1-Forms	59
5.4	Differential k -Forms	60
5.5	The Wedge Product	61
5.6	The Exterior Derivative	61
6	Integration and Stokes' Theorem	63
6.1	Integration of Differential Forms	63
6.2	Stokes' Theorem	63
6.3	Classical Theorems as Special Cases	64
6.3.1	Gradient, Curl, and Divergence	64
6.3.2	Fundamental Theorem of Calculus	64
6.3.3	Green's Theorem	65
6.3.4	Classical Stokes' Theorem (Curl Theorem)	65
6.3.5	Divergence Theorem (Gauss's Theorem)	65
6.4	The Hodge Star Operator	66

IV	Group Theory	68
7	Groups and Actions	69
V	Complex Analysis	70
8	Holomorphic Functions	71
8.1	The Complex Plane	71
8.2	Topological Preliminaries	71
8.3	Holomorphic Functions	73
9	Complex Integration	75
9.1	Cauchy's Theorem	75
10	Power Series and Analytic Functions	79
10.1	Power Series	79
10.2	Laurent Series	80
11	Singularities and Residues	82
11.1	Isolated Singularities	82
11.2	Residues	83
11.3	The Residue Theorem	84
11.4	Applications of the Residue Theorem	85
11.4.1	Branch Cuts and the Keyhole Contour	91
12	Further Results in Complex Analysis	94
12.1	Liouville's Theorem and the Fundamental Theorem of Algebra	94
12.2	The Maximum Modulus Principle	95
12.3	The Argument Principle and Rouché's Theorem	97
12.4	Harmonic Functions	99
12.5	Analytic Continuation	102
VI	Machine Learning	105
13	Neural Network Fundamentals	106
13.1	The Single Neuron	106
13.2	Activation Functions	107
13.3	Loss Functions	108
13.4	Gradient Descent	110
13.5	Second-Order Methods and the Hessian	111
13.6	Backpropagation	113
13.7	Optimization Algorithms	115
13.8	Universal Approximation	117
13.9	Residual Connections and the Residual Stream	118

13.10	The FFN Block	119
13.11	Modern Activation Functions	120
13.12	General Feedforward Neural Networks (MLP)	120
13.13	Embeddings	121
14	Transformer Theory	122
14.1	Self-Attention Mechanism	122
14.2	Multi-Head Attention	123
14.3	Why Divide by $\sqrt{d_k}$?	125
14.4	Attention as a Kernel Method	128
14.5	Positional Encodings and Length Generalization	131
14.6	Induction Heads	134
14.7	Attention vs MLP: Routing vs Transformation	135
14.8	In-Context Learning	136
14.9	Expressivity: Depth vs Width	138
14.10	Information Bottlenecks in Attention	139
14.11	Optimization Landscape	140
14.12	Length Generalization Failures	142
14.13	Universality vs Efficiency	143
15	Training Phases of Large Language Models	145
15.1	Overview of Training Phases	145
15.2	Pretraining	145
15.3	Midtraining (Continued Pretraining)	146
15.4	Post-training (Alignment)	146
15.4.1	Supervised Fine-Tuning (SFT)	146
15.4.2	RLHF / Preference Optimization	147
15.5	Parameter-Efficient Fine-Tuning (PEFT)	147
15.6	Encoder-Decoder vs Decoder-Only Architecture	148
15.7	Attention Dimensions in Practice	149
15.8	KV Cache for Efficient Inference	150
15.9	Causal Masking	151
15.10	What Limits Context Window?	152
15.11	The Role of Feed-Forward Networks	153
15.12	Computational Complexity of Self-Attention	154
15.13	Permutation Equivariance and Positional Encodings	155
15.14	Why Transformers Excel at Lexical Matching	156
15.15	Depth vs Width in Transformers	156
16	Energy-Based Models	158
16.1	The Energy Function	158
16.2	From Energy to Probability: The Boltzmann Distribution	158
16.3	The Partition Function Problem	160
16.4	Contrastive Divergence	161
16.5	Score Matching	162

16.6 Noise Contrastive Estimation	163
16.7 Hopfield Networks	163
16.8 Restricted Boltzmann Machines	164
16.9 Connection to Modern Generative Models	165
16.10 Sampling from EBMs: Langevin Dynamics	166
16.11 Summary: The EBM Perspective	167
16.12 EBMs vs Transformers	168
16.13 Reasoning in Energy-Based Models	169
16.14 Notable Energy-Based Models	170
16.15 EBMs vs Diffusion Models: A Detailed Comparison	171

VII Reinforcement Learning 174

17 Classical Reinforcement Learning 175

17.1 Markov Decision Processes	175
17.2 Value Functions	176
17.3 Bellman Equations	177
17.3.1 The Equations	177
17.3.2 Why Bellman Equations Matter	179
17.4 Temporal Difference Learning	180
17.5 Policy Gradient Methods	181
17.5.1 Why This Objective?	181
17.5.2 Actor-Critic Methods	185
17.5.3 Generalized Advantage Estimation (GAE)	185
17.6 Proximal Policy Optimization (PPO)	186
17.6.1 The Problem with Vanilla Policy Gradient	186
17.6.2 The Importance Sampling Trick	187
17.6.3 Why We Need Clipping	187

18 Reinforcement Learning for Language Models 190

18.1 RLHF: Reinforcement Learning from Human Feedback	191
18.1.1 The RLHF Objective	191
18.1.2 Reward Modeling	194
18.2 DPO: Direct Preference Optimization	195
18.2.1 Derivation	195
18.2.2 Gradient Analysis	196
18.3 IPO: Identity Preference Optimization	196
18.3.1 Visualizing DPO vs IPO vs SLiC-HF	197
18.4 GRPO: Group Relative Policy Optimization	198
18.4.1 Motivation	198
18.4.2 Token-Level KL Penalty	198
18.4.3 Comparison with PPO	198
18.5 REINFORCE Leave-One-Out (RLOO)	199
18.6 KTO: Kahneman-Tversky Optimization	199

18.7 Reinforcement Learning with Verifiable Rewards (RLVR)	199
18.7.1 Motivation: Beyond Human Preferences	200
18.7.2 Sources of Verifiable Rewards	200
18.7.3 The RLVR Training Pipeline	200
18.7.4 Mathematical Formulation	201
18.7.5 DeepSeek-R1: A Case Study	202
18.7.6 RLVR vs RLHF: Key Differences	202
18.7.7 Challenges and Limitations	203
18.7.8 Process Reward Models: A Middle Ground	203
18.8 Comparison of Alignment Methods	204
18.8.1 Quick Reference: Identifying Methods by Loss Function	204
18.8.2 Detailed Loss Functions	204
18.8.3 Taxonomy: What Makes Each Method Different?	208
18.8.4 Visual Decision Tree	208
18.8.5 Gradient Comparison	209
18.8.6 When to Use What	209

Part I

Linear Algebra

Chapter 1

Vector Spaces and Linear Maps

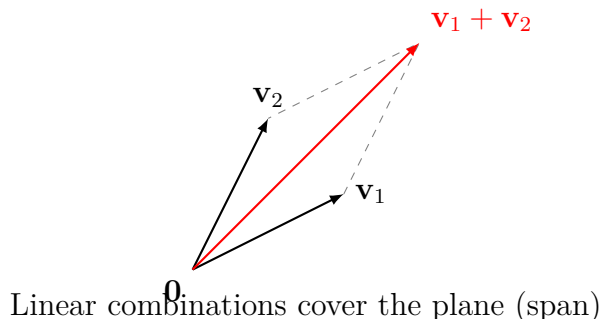
1.1 Vector Spaces

Definition 1.1 (Vector space). Let V be a set over a field $\mathbb{F}$. We call V a *vector space* over $\mathbb{F}$ if it is equipped with an addition $+: V \times V \rightarrow V$ and scalar multiplication $\mathbb{F} \times V \rightarrow V$ satisfying the usual axioms (associativity, commutativity, distributivity, existence of 0 and additive inverses, etc.).

Definition 1.2 (Span). Let $S = \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ be a set of vectors in a vector space V . The *span* of S , denoted $\text{span}(S)$ or $\text{span}(\mathbf{v}_1, \dots, \mathbf{v}_n)$, is the set of all linear combinations of vectors in S :

$$\text{span}(\mathbf{v}_1, \dots, \mathbf{v}_n) = \left\{ \sum_{i=1}^n \alpha_i \mathbf{v}_i : \alpha_i \in \mathbb{F} \right\}. \quad (1.1)$$

We say that S *spans* V if $\text{span}(S) = V$.



Definition 1.3 (Linear independence). A family of vectors $(\mathbf{v}_1, \dots, \mathbf{v}_n)$ in a vector space V is *linearly independent* if

$$\alpha_1 \mathbf{v}_1 + \dots + \alpha_n \mathbf{v}_n = \mathbf{0} \quad \text{implies} \quad \alpha_1 = \dots = \alpha_n = 0. \quad (1.2)$$

Definition 1.4 (Basis). A *basis* of a vector space V is a linearly independent set of vectors that spans V . That is, a family $\mathcal{B} = (\mathbf{v}_1, \dots, \mathbf{v}_n)$ is a basis if:

1. $\mathcal{B}$ is linearly independent, and

2. every vector $\mathbf{v} \in V$ can be written as a linear combination $\mathbf{v} = \alpha_1 \mathbf{v}_1 + \cdots + \alpha_n \mathbf{v}_n$ for some scalars $\alpha_1, \dots, \alpha_n$.

Definition 1.5 (Dimension). The *dimension* of a vector space V , denoted $\dim(V)$, is the number of vectors in any basis of V . If V has a finite basis, then V is *finite-dimensional*; otherwise, V is *infinite-dimensional*.

1.2 Inner Product Spaces

Definition 1.6 (Inner product space). An *inner product space* is a vector space V over $\mathbb{F}$ (where $\mathbb{F} = \mathbb{R}$ or $\mathbb{C}$) equipped with an inner product $\langle \cdot, \cdot \rangle : V \times V \rightarrow \mathbb{F}$ satisfying:

1. Conjugate symmetry: $\langle \mathbf{v}, \mathbf{w} \rangle = \overline{\langle \mathbf{w}, \mathbf{v} \rangle}$,
2. Linearity in the first argument: $\langle \alpha \mathbf{u} + \beta \mathbf{v}, \mathbf{w} \rangle = \alpha \langle \mathbf{u}, \mathbf{w} \rangle + \beta \langle \mathbf{v}, \mathbf{w} \rangle$,
3. Positive definiteness: $\langle \mathbf{v}, \mathbf{v} \rangle \geq 0$ with equality if and only if $\mathbf{v} = \mathbf{0}$.

Definition 1.7 (Orthogonality). Two vectors $\mathbf{v}, \mathbf{w} \in V$ are *orthogonal* if $\langle \mathbf{v}, \mathbf{w} \rangle = 0$. A set of vectors $\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ is *orthogonal* if the vectors are pairwise orthogonal. An orthogonal set is *orthonormal* if additionally $\langle \mathbf{v}_i, \mathbf{v}_i \rangle = 1$ for all i .

Definition 1.8 (Frobenius inner product for matrices). The space of $m \times n$ matrices $\mathbb{R}^{m \times n}$ (or $\mathbb{C}^{m \times n}$) is an inner product space with the *Frobenius inner product*

$$\langle \mathbf{A}, \mathbf{B} \rangle_F = \text{tr}(\mathbf{A}^\top \mathbf{B}) = \sum_{i=1}^m \sum_{j=1}^n a_{ij} b_{ij}. \quad (1.3)$$

Definition 1.9 (Frobenius norm). The *Frobenius norm* of a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2} = \sqrt{\langle \mathbf{A}, \mathbf{A} \rangle_F}. \quad (1.4)$$

Proposition 1.10 (Frobenius norm via trace). For any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$,

$$\|\mathbf{A}\|_F^2 = \text{tr}(\mathbf{A}^\top \mathbf{A}) = \text{tr}(\mathbf{A} \mathbf{A}^\top). \quad (1.5)$$

Proof. Let $\mathbf{A} = (a_{ij}) \in \mathbb{R}^{m \times n}$. We compute $\mathbf{A}^\top \mathbf{A} \in \mathbb{R}^{n \times n}$:

$$(\mathbf{A}^\top \mathbf{A})_{jk} = \sum_{i=1}^m a_{ij} a_{ik}.$$

The diagonal entries are:

$$(\mathbf{A}^\top \mathbf{A})_{jj} = \sum_{i=1}^m a_{ij}^2.$$

Taking the trace (sum of diagonal entries):

$$\text{tr}(\mathbf{A}^\top \mathbf{A}) = \sum_{j=1}^n (\mathbf{A}^\top \mathbf{A})_{jj} = \sum_{j=1}^n \sum_{i=1}^m a_{ij}^2 = \sum_{i=1}^m \sum_{j=1}^n a_{ij}^2 = \|\mathbf{A}\|_F^2.$$

For the second equality, compute $\mathbf{A}\mathbf{A}^\top \in \mathbb{R}^{m \times m}$:

$$(\mathbf{A}\mathbf{A}^\top)_{ik} = \sum_{j=1}^n a_{ij}a_{kj}.$$

The diagonal entries are:

$$(\mathbf{A}\mathbf{A}^\top)_{ii} = \sum_{j=1}^n a_{ij}^2.$$

Taking the trace:

$$\text{tr}(\mathbf{A}\mathbf{A}^\top) = \sum_{i=1}^m (\mathbf{A}\mathbf{A}^\top)_{ii} = \sum_{i=1}^m \sum_{j=1}^n a_{ij}^2 = \|\mathbf{A}\|_F^2.$$

□

Remark 1.11. The Frobenius norm treats a matrix as a vector by summing the squares of all entries. For a matrix with rows $\mathbf{a}_1^\top, \dots, \mathbf{a}_m^\top \in \mathbb{R}^n$, we have

$$\|\mathbf{A}\|_F^2 = \sum_{i=1}^m \|\mathbf{a}_i\|^2.$$

The Frobenius norm is submultiplicative: $\|\mathbf{A}\mathbf{B}\|_F \leq \|\mathbf{A}\|_F \|\mathbf{B}\|_F$.

Theorem 1.12 (Gram–Schmidt decomposition). *Let V be an inner product space and let $(\mathbf{v}_1, \dots, \mathbf{v}_n)$ be a linearly independent family of vectors in V . Then there exists an orthonormal family $(\mathbf{u}_1, \dots, \mathbf{u}_n)$ such that for each $k = 1, \dots, n$,*

$$\text{span}(\mathbf{v}_1, \dots, \mathbf{v}_k) = \text{span}(\mathbf{u}_1, \dots, \mathbf{u}_k).$$

Moreover, the vectors $\mathbf{u}_k$ can be constructed recursively by the Gram–Schmidt process:

$$\mathbf{w}_1 = \mathbf{v}_1, \quad \mathbf{u}_1 = \frac{\mathbf{w}_1}{\|\mathbf{w}_1\|}, \tag{1.6}$$

$$\mathbf{w}_k = \mathbf{v}_k - \sum_{j=1}^{k-1} \langle \mathbf{v}_k, \mathbf{u}_j \rangle \mathbf{u}_j, \quad \mathbf{u}_k = \frac{\mathbf{w}_k}{\|\mathbf{w}_k\|} \tag{1.7}$$

for $k = 2, \dots, n$, where $\|\mathbf{v}\| = \sqrt{\langle \mathbf{v}, \mathbf{v} \rangle}$.

Proof. We proceed by induction on k . For the base case $k = 1$, set $\mathbf{u}_1 = \mathbf{v}_1 / \|\mathbf{v}_1\|$. Since $\mathbf{v}_1 \neq \mathbf{0}$ (by linear independence), we have $\|\mathbf{u}_1\| = 1$, and clearly $\text{span}(\mathbf{v}_1) = \text{span}(\mathbf{u}_1)$.

Now assume we have constructed orthonormal vectors $\mathbf{u}_1, \dots, \mathbf{u}_{k-1}$ such that $\text{span}(\mathbf{v}_1, \dots, \mathbf{v}_j) = \text{span}(\mathbf{u}_1, \dots, \mathbf{u}_j)$ for all $j < k$.

Define

$$\mathbf{w}_k = \mathbf{v}_k - \sum_{j=1}^{k-1} \langle \mathbf{v}_k, \mathbf{u}_j \rangle \mathbf{u}_j.$$

We claim that $\mathbf{w}_k \neq \mathbf{0}$ and that $\mathbf{w}_k$ is orthogonal to each $\mathbf{u}_i$ for $i < k$.

To see orthogonality, for any $1 \leq i \leq k-1$, we compute:

$$\begin{aligned} \langle \mathbf{w}_k, \mathbf{u}_i \rangle &= \left\langle \mathbf{v}_k - \sum_{j=1}^{k-1} \langle \mathbf{v}_k, \mathbf{u}_j \rangle \mathbf{u}_j, \mathbf{u}_i \right\rangle \\ &= \langle \mathbf{v}_k, \mathbf{u}_i \rangle - \sum_{j=1}^{k-1} \langle \mathbf{v}_k, \mathbf{u}_j \rangle \langle \mathbf{u}_j, \mathbf{u}_i \rangle \\ &= \langle \mathbf{v}_k, \mathbf{u}_i \rangle - \langle \mathbf{v}_k, \mathbf{u}_i \rangle = 0, \end{aligned}$$

where we used orthonormality: $\langle \mathbf{u}_j, \mathbf{u}_i \rangle = \delta_{ij}$.

To see that $\mathbf{w}_k \neq \mathbf{0}$, suppose $\mathbf{w}_k = \mathbf{0}$. Then

$$\mathbf{v}_k = \sum_{j=1}^{k-1} \langle \mathbf{v}_k, \mathbf{u}_j \rangle \mathbf{u}_j \in \text{span}(\mathbf{u}_1, \dots, \mathbf{u}_{k-1}) = \text{span}(\mathbf{v}_1, \dots, \mathbf{v}_{k-1}),$$

contradicting the linear independence of $(\mathbf{v}_1, \dots, \mathbf{v}_n)$.

Thus we can set $\mathbf{u}_k = \mathbf{w}_k / \|\mathbf{w}_k\|$, which has norm 1 and is orthogonal to $\mathbf{u}_1, \dots, \mathbf{u}_{k-1}$. By construction, $\mathbf{v}_k \in \text{span}(\mathbf{u}_1, \dots, \mathbf{u}_k)$ and $\mathbf{u}_k \in \text{span}(\mathbf{v}_1, \dots, \mathbf{v}_k)$, so $\text{span}(\mathbf{v}_1, \dots, \mathbf{v}_k) = \text{span}(\mathbf{u}_1, \dots, \mathbf{u}_k)$. $\square$

Remark 1.13 (Applications of Gram–Schmidt). The Gram–Schmidt process is fundamental across many fields:

- **Machine learning:** QR decomposition (based on Gram–Schmidt) is used for numerically stable least-squares regression. Orthogonalizing feature vectors removes redundancy and improves conditioning of optimization problems.
- **Quantum mechanics:** States in a quantum system must be represented by orthonormal vectors in a Hilbert space. Gram–Schmidt is used to orthogonalize measurement bases and construct complete orthonormal sets of wavefunctions.
- **Signal processing:** Constructing orthonormal bases (like wavelets or Fourier bases) for signal decomposition. Orthogonal projections onto signal subspaces are used in noise reduction and compression.
- **Numerical analysis:** The QR algorithm for eigenvalue computation relies on repeated Gram–Schmidt orthogonalization. Stable numerical solvers for differential equations use orthogonal polynomial bases constructed via Gram–Schmidt.

1.3 Linear Maps

Definition 1.14 (Linear map). Let V and W be vector spaces over a field $\mathbb{F}$. A map $T : V \rightarrow W$ is a *linear map* (or *linear transformation*) if for all $\mathbf{v}_1, \mathbf{v}_2 \in V$ and $\alpha, \beta \in \mathbb{F}$,

$$T(\alpha \mathbf{v}_1 + \beta \mathbf{v}_2) = \alpha T(\mathbf{v}_1) + \beta T(\mathbf{v}_2). \quad (1.8)$$

A linear map from a vector space to itself, $T : V \rightarrow V$, is called a *linear operator*.

Definition 1.15 (Image and kernel). Let $T : V \rightarrow W$ be a linear map. The *image* (or *range*) of T is

$$\text{im}(T) = \{T(\mathbf{v}) : \mathbf{v} \in V\} \subseteq W. \quad (1.9)$$

The *kernel* (or *null space*) of T is

$$\ker(T) = \{\mathbf{v} \in V : T(\mathbf{v}) = \mathbf{0}\} \subseteq V. \quad (1.10)$$

1.4 Matrix Representations

1.4.1 Motivation

Linear maps between abstract vector spaces (like function spaces or polynomial spaces) seem very different from matrix-vector multiplication. However, once we choose bases, every linear map can be represented as a matrix. This is the bridge between abstract linear algebra and concrete computation.

The key insight: A basis gives us a "coordinate system" that identifies an abstract vector space V with $\mathbb{F}^n$. Once we have coordinates, linear maps become matrices.

Throughout this section, let V and W be finite-dimensional vector spaces over $\mathbb{F}$, with $\dim(V) = n$ and $\dim(W) = m$.

Definition 1.16 (Coordinate isomorphism). Let V be a vector space with basis $\mathcal{B} = (\mathbf{v}_1, \dots, \mathbf{v}_n)$. The *coordinate map* (or *coordinate isomorphism*) is the map $\Phi_{\mathcal{B}} : V \rightarrow \mathbb{F}^n$ defined by

$$\Phi_{\mathcal{B}} \left(\sum_{i=1}^n x_i \mathbf{v}_i \right) = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}. \quad (1.11)$$

This map is a linear isomorphism: it is bijective and preserves the vector space structure.

Remark 1.17. The coordinate map $\Phi_{\mathcal{B}}$ identifies abstract vectors in V with concrete column vectors in $\mathbb{F}^n$. It is an isomorphism because:

- Every vector $\mathbf{v} \in V$ has a unique representation as a linear combination of basis vectors.
- The map preserves addition and scalar multiplication.

Theorem 1.18 (Fundamental theorem: Every linear map is a matrix). *Let $T : V \rightarrow W$ be a linear map between finite-dimensional vector spaces. Fix bases $\mathcal{B}_V = (\mathbf{v}_1, \dots, \mathbf{v}_n)$ of V and $\mathcal{B}_W = (\mathbf{w}_1, \dots, \mathbf{w}_m)$ of W , and let $\Phi_V : V \rightarrow \mathbb{F}^n$ and $\Phi_W : W \rightarrow \mathbb{F}^m$ be the coordinate isomorphisms.*

Then there exists a unique matrix $\mathbf{A} \in \mathbb{F}^{m \times n}$ such that

$$\mathbf{A} = \Phi_W \circ T \circ \Phi_V^{-1}. \quad (1.12)$$

Equivalently, the following diagram commutes:

$$\begin{array}{ccc}
 V & \xrightarrow{T} & W \\
 \Phi_V \downarrow & & \downarrow \Phi_W \\
 \mathbb{F}^n & \xrightarrow{\mathbf{A}} & \mathbb{F}^m
 \end{array}$$

How to construct $\mathbf{A}$: Apply T to each basis vector $\mathbf{v}_j$ and write the result in the basis $\mathcal{B}_W$:

$$T(\mathbf{v}_j) = \sum_{i=1}^m a_{ij} \mathbf{w}_i. \quad (1.13)$$

Then the j -th column of $\mathbf{A}$ is $(a_{1j}, a_{2j}, \dots, a_{mj})^\top$.

Proof. Define the matrix $\mathbf{A} = (a_{ij}) \in \mathbb{F}^{m \times n}$ using the coefficients from (1.13). We must show that the diagram commutes, i.e., that $\Phi_W \circ T = \mathbf{A} \circ \Phi_V$.

Let $\mathbf{v} = \sum_{j=1}^n x_j \mathbf{v}_j \in V$ be arbitrary. Then $\Phi_V(\mathbf{v}) = (x_1, \dots, x_n)^\top$.

Top path ($V \rightarrow W \rightarrow \mathbb{F}^m$): Apply T then Φ_W .

$$\begin{aligned}
 T(\mathbf{v}) &= T\left(\sum_{j=1}^n x_j \mathbf{v}_j\right) = \sum_{j=1}^n x_j T(\mathbf{v}_j) \quad (\text{by linearity}) \\
 &= \sum_{j=1}^n x_j \sum_{i=1}^m a_{ij} \mathbf{w}_i = \sum_{i=1}^m \left(\sum_{j=1}^n a_{ij} x_j\right) \mathbf{w}_i.
 \end{aligned}$$

Applying Φ_W gives

$$\Phi_W(T(\mathbf{v})) = \begin{pmatrix} \sum_{j=1}^n a_{1j} x_j \\ \vdots \\ \sum_{j=1}^n a_{mj} x_j \end{pmatrix}.$$

Bottom path ($V \rightarrow \mathbb{F}^n \rightarrow \mathbb{F}^m$): Apply Φ_V then $\mathbf{A}$.

$$\mathbf{A} \Phi_V(\mathbf{v}) = \mathbf{A} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} \sum_{j=1}^n a_{1j} x_j \\ \vdots \\ \sum_{j=1}^n a_{mj} x_j \end{pmatrix}.$$

Both paths give the same result, so the diagram commutes.

Uniqueness: The matrix $\mathbf{A}$ is uniquely determined by requiring commutativity at the basis vectors $\mathbf{v}_j$, which forces the columns to be as in (1.13). By linearity, this determines $\mathbf{A}$ on all of V . $\square$

Remark 1.19 (Significance of the formula $\mathbf{A} = \Phi_W \circ T \circ \Phi_V^{-1}$). The formula (1.12) reveals the deep connection between abstract linear maps and matrices:

- **To compute $\mathbf{Ax}$:** Start with coordinates $\mathbf{x} \in \mathbb{F}^n$, convert to abstract vector via Φ_V^{-1} , apply the abstract map T , then convert back to coordinates via Φ_W .
- **The coordinate maps are isomorphisms:** Both Φ_V and Φ_W are linear, bijective, and invertible. This means every finite-dimensional vector space is "the same as" $\mathbb{F}^n$ once we pick a basis.
- **Change of perspective:** The same linear transformation T has different matrix representations depending on the choice of bases. The matrix $\mathbf{A}$ is not intrinsic to T , but depends on our coordinate systems.
- **Composition of maps:** The formula shows that matrix multiplication corresponds to composition of linear maps after translating between abstract and coordinate viewpoints.

Remark 1.20 (Applications: Why every linear map is a matrix). This fundamental theorem bridges abstract mathematics and computation:

- **Machine learning:** Neural network layers are linear transformations between high-dimensional feature spaces. The theorem guarantees that choosing a basis (e.g., pixel coordinates, word embeddings) allows us to represent these transformations as matrix multiplications, enabling GPU-accelerated training.
- **Computer graphics:** Transformations like rotations, scaling, and projections act on infinite-dimensional function spaces (continuous surfaces). By discretizing to vertices and choosing a basis, the theorem shows these become finite matrix operations that GPUs can execute in real-time.
- **Quantum computing:** Quantum gates are unitary operators on infinite-dimensional Hilbert spaces. The theorem ensures that restricting to finite-dimensional qubit spaces gives matrix representations, making quantum circuits implementable on physical hardware.
- **Differential equations:** Linear differential operators (like $\frac{d}{dx}$) act on infinite-dimensional function spaces. Choosing a basis (Fourier modes, finite elements) converts them to matrices, enabling numerical solution methods.
- **Theoretical physics:** Symmetries of physical systems are represented by abstract group actions. The theorem shows that these symmetries have matrix representations, connecting abstract symmetry principles to concrete calculations in quantum field theory and particle physics.

Key insight: The theorem shows that *choosing coordinates is the bridge between abstract theory and practical computation*. Different bases give different matrix representations, but all encode the same underlying linear transformation.

Remark 1.21. We sometimes write $[T]_{\mathcal{B}_W, \mathcal{B}_V} = \mathbf{A}$ to denote the matrix representation of T with respect to the given bases. When $V = W$ and both bases are the same $\mathcal{B}$, we write $[T]_{\mathcal{B}}$.

Corollary 1.22 (The case of $\mathbb{F}^n$). *When working in $\mathbb{F}^n$ with the standard basis $\mathcal{E} = (\mathbf{e}_1, \dots, \mathbf{e}_n)$, the coordinate map is the identity ($\Phi_{\mathcal{E}}(\mathbf{x}) = \mathbf{x}$). Therefore, for any linear map $T : \mathbb{F}^n \rightarrow \mathbb{F}^m$,*

$$T(\mathbf{x}) = \mathbf{A}\mathbf{x}, \quad (1.14)$$

where $\mathbf{A} \in \mathbb{F}^{m \times n}$ has j -th column $\mathbf{A}_{:,j} = T(\mathbf{e}_j)$. In this case, the linear map T and its matrix $\mathbf{A}$ are identified.

1.4.2 Subspaces of $\mathbb{R}^d$

For the special case of $\mathbb{R}^d$ with the standard inner product $\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^T \mathbf{y}$, we have the following useful characterization.

Lemma 1.23 (Orthonormal matrix characterization of subspaces). *Let $S \subseteq \mathbb{R}^d$ be a k -dimensional subspace. Then:*

1. *There exists a matrix $\mathbf{W} \in \mathbb{R}^{d \times k}$ such that the columns of $\mathbf{W}$ form an orthonormal basis of S .*
2. *Conversely, any matrix $\mathbf{W} \in \mathbb{R}^{d \times k}$ with*

$$\mathbf{W}^T \mathbf{W} = \mathbf{I}_k \quad (1.15)$$

defines a k -dimensional subspace

$$S = \{\mathbf{W}\mathbf{z} : \mathbf{z} \in \mathbb{R}^k\}. \quad (1.16)$$

Proof. (1) Since S is a k -dimensional subspace of $\mathbb{R}^d$, it has a basis $(\mathbf{v}_1, \dots, \mathbf{v}_k)$. By the Gram–Schmidt decomposition, we can construct an orthonormal basis $(\mathbf{u}_1, \dots, \mathbf{u}_k)$ for S . Let $\mathbf{W} \in \mathbb{R}^{d \times k}$ be the matrix whose columns are $\mathbf{u}_1, \dots, \mathbf{u}_k$. Then the columns of $\mathbf{W}$ form an orthonormal basis of S .

(2) Suppose $\mathbf{W} \in \mathbb{R}^{d \times k}$ satisfies $\mathbf{W}^T \mathbf{W} = \mathbf{I}_k$. This condition means that the columns $\mathbf{w}_1, \dots, \mathbf{w}_k$ of $\mathbf{W}$ are orthonormal: $\langle \mathbf{w}_i, \mathbf{w}_j \rangle = \delta_{ij}$ for all i, j . In particular, the columns are linearly independent, so $S = \text{span}(\mathbf{w}_1, \dots, \mathbf{w}_k) = \{\mathbf{W}\mathbf{z} : \mathbf{z} \in \mathbb{R}^k\}$ is a k -dimensional subspace of $\mathbb{R}^d$. $\square$

1.5 Eigenvalues and Eigenvectors

Definition 1.24 (Eigenvalue and eigenvector). Let $T : V \rightarrow V$ be a linear operator on a vector space V over $\mathbb{F}$. A scalar $\lambda \in \mathbb{F}$ is an *eigenvalue* of T if there exists a nonzero vector $\mathbf{v} \in V$ such that

$$T(\mathbf{v}) = \lambda \mathbf{v}. \quad (1.17)$$

The vector $\mathbf{v}$ is called an *eigenvector* corresponding to λ .

Remark 1.25. For matrices $\mathbf{A} \in \mathbb{F}^{n \times n}$, the eigenvalue equation becomes $\mathbf{A}\mathbf{v} = \lambda \mathbf{v}$. Equivalently, $(\mathbf{A} - \lambda \mathbf{I})\mathbf{v} = \mathbf{0}$, so λ is an eigenvalue if and only if $\det(\mathbf{A} - \lambda \mathbf{I}) = 0$.

Definition 1.26 (Symmetric matrix). A matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is *symmetric* if $\mathbf{A}^\top = \mathbf{A}$.

Definition 1.27 (Definiteness of symmetric matrices). Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be a symmetric matrix. We classify $\mathbf{A}$ based on the sign of the quadratic form $\mathbf{x}^\top \mathbf{A} \mathbf{x}$:

1. $\mathbf{A}$ is *positive definite* if $\mathbf{x}^\top \mathbf{A} \mathbf{x} > 0$ for all $\mathbf{x} \neq \mathbf{0}$.
2. $\mathbf{A}$ is *positive semidefinite* if $\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$ for all $\mathbf{x} \in \mathbb{R}^n$.
3. $\mathbf{A}$ is *negative definite* if $\mathbf{x}^\top \mathbf{A} \mathbf{x} < 0$ for all $\mathbf{x} \neq \mathbf{0}$.
4. $\mathbf{A}$ is *negative semidefinite* if $\mathbf{x}^\top \mathbf{A} \mathbf{x} \leq 0$ for all $\mathbf{x} \in \mathbb{R}^n$.
5. $\mathbf{A}$ is *indefinite* if $\mathbf{x}^\top \mathbf{A} \mathbf{x}$ takes both positive and negative values.

Remark 1.28. For a symmetric matrix $\mathbf{A}$ with eigenvalues $\lambda_1, \dots, \lambda_n$:

- $\mathbf{A}$ is positive definite $\Leftrightarrow$ all $\lambda_i > 0$.
- $\mathbf{A}$ is positive semidefinite $\Leftrightarrow$ all $\lambda_i \geq 0$.
- $\mathbf{A}$ is negative definite $\Leftrightarrow$ all $\lambda_i < 0$.
- $\mathbf{A}$ is negative semidefinite $\Leftrightarrow$ all $\lambda_i \leq 0$.
- $\mathbf{A}$ is indefinite $\Leftrightarrow$ $\mathbf{A}$ has both positive and negative eigenvalues.

This characterization follows from the spectral theorem: if $\mathbf{A} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top$ with $\mathbf{U}$ orthogonal, then

$$\mathbf{x}^\top \mathbf{A} \mathbf{x} = \mathbf{x}^\top \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top \mathbf{x} = \sum_{i=1}^n \lambda_i (\mathbf{u}_i^\top \mathbf{x})^2,$$

where $\mathbf{u}_i$ are the columns of $\mathbf{U}$.

Lemma 1.29 (Eigenvalues of symmetric matrices are real). *Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be symmetric. Then all eigenvalues of $\mathbf{A}$ are real, and we can choose corresponding eigenvectors to be real.*

Proof. Let λ be an eigenvalue with eigenvector $\mathbf{v} \in \mathbb{C}^n$ (allowing complex vectors). Then $\mathbf{A} \mathbf{v} = \lambda \mathbf{v}$ with $\mathbf{v} \neq \mathbf{0}$.

Taking the conjugate transpose of both sides: $\bar{\mathbf{v}}^\top \mathbf{A}^\top = \bar{\lambda} \bar{\mathbf{v}}^\top$. Since $\mathbf{A}$ is real and symmetric, $\mathbf{A}^\top = \mathbf{A}$, so $\bar{\mathbf{v}}^\top \mathbf{A} = \bar{\lambda} \bar{\mathbf{v}}^\top$.

Now multiply the original equation on the left by $\bar{\mathbf{v}}^\top$:

$$\bar{\mathbf{v}}^\top \mathbf{A} \mathbf{v} = \lambda \bar{\mathbf{v}}^\top \mathbf{v}.$$

And multiply the conjugated equation on the right by $\mathbf{v}$:

$$\bar{\mathbf{v}}^\top \mathbf{A} \mathbf{v} = \bar{\lambda} \bar{\mathbf{v}}^\top \mathbf{v}.$$

Since both equal $\bar{\mathbf{v}}^\top \mathbf{A} \mathbf{v}$ and $\bar{\mathbf{v}}^\top \mathbf{v} = \|\mathbf{v}\|^2 > 0$, we have $\lambda = \bar{\lambda}$, so $\lambda \in \mathbb{R}$.

For a real eigenvalue, we can find a real eigenvector by taking the real or imaginary part of a complex eigenvector. $\square$

Lemma 1.30 (Eigenvectors for distinct eigenvalues are orthogonal). *Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be symmetric. If $\mathbf{v}_1, \mathbf{v}_2$ are eigenvectors with distinct eigenvalues $\lambda_1 \neq \lambda_2$, then $\mathbf{v}_1 \perp \mathbf{v}_2$.*

Proof. We have $\mathbf{A}\mathbf{v}_1 = \lambda_1\mathbf{v}_1$ and $\mathbf{A}\mathbf{v}_2 = \lambda_2\mathbf{v}_2$. Then

$$\begin{aligned} \lambda_1 \langle \mathbf{v}_1, \mathbf{v}_2 \rangle &= \langle \lambda_1 \mathbf{v}_1, \mathbf{v}_2 \rangle = \langle \mathbf{A}\mathbf{v}_1, \mathbf{v}_2 \rangle \\ &= \mathbf{v}_1^\top \mathbf{A}^\top \mathbf{v}_2 = \mathbf{v}_1^\top \mathbf{A} \mathbf{v}_2 \quad (\text{since } \mathbf{A} \text{ is symmetric}) \\ &= \mathbf{v}_1^\top (\lambda_2 \mathbf{v}_2) = \lambda_2 \langle \mathbf{v}_1, \mathbf{v}_2 \rangle. \end{aligned}$$

Therefore, $(\lambda_1 - \lambda_2) \langle \mathbf{v}_1, \mathbf{v}_2 \rangle = 0$. Since $\lambda_1 \neq \lambda_2$, we must have $\langle \mathbf{v}_1, \mathbf{v}_2 \rangle = 0$. $\square$

Theorem 1.31 (Spectral theorem for symmetric matrices). *Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be a symmetric matrix. Then there exists an orthonormal basis of $\mathbb{R}^n$ consisting of eigenvectors of $\mathbf{A}$. Moreover, we can write*

$$\mathbf{A} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top, \quad (1.18)$$

where $\mathbf{U} \in \mathbb{R}^{n \times n}$ is orthogonal ($\mathbf{U}^\top \mathbf{U} = \mathbf{I}$) with columns being orthonormal eigenvectors, and $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$ contains the corresponding eigenvalues.

Proof. We proceed by induction on n .

Base case ($n = 1$): Any 1×1 matrix $\mathbf{A} = [a]$ has eigenvalue $\lambda = a$ with eigenvector $\mathbf{u} = [1]$.

Inductive step: Assume the result holds for $(n-1) \times (n-1)$ symmetric matrices. Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be symmetric.

By Lemma 1.29, $\mathbf{A}$ has a real eigenvalue λ_1 with real eigenvector $\mathbf{v}_1$. Normalize to get $\mathbf{u}_1 = \mathbf{v}_1 / \|\mathbf{v}_1\|$ with $\|\mathbf{u}_1\| = 1$.

Extend $\{\mathbf{u}_1\}$ to an orthonormal basis $\{\mathbf{u}_1, \mathbf{w}_2, \dots, \mathbf{w}_n\}$ of $\mathbb{R}^n$ (using Gram-Schmidt if necessary). Let $\mathbf{Q} = [\mathbf{u}_1 \mid \mathbf{w}_2 \mid \dots \mid \mathbf{w}_n]$ be orthogonal.

Consider $\mathbf{B} = \mathbf{Q}^\top \mathbf{A} \mathbf{Q}$. The first column of $\mathbf{A} \mathbf{Q}$ is

$$\mathbf{A} \mathbf{u}_1 = \lambda_1 \mathbf{u}_1.$$

So the first column of $\mathbf{Q}^\top \mathbf{A} \mathbf{Q}$ is $\mathbf{Q}^\top (\lambda_1 \mathbf{u}_1) = \lambda_1 \mathbf{e}_1$.

Since $\mathbf{B}$ is symmetric (as $\mathbf{B}^\top = (\mathbf{Q}^\top \mathbf{A} \mathbf{Q})^\top = \mathbf{Q}^\top \mathbf{A}^\top \mathbf{Q} = \mathbf{Q}^\top \mathbf{A} \mathbf{Q} = \mathbf{B}$), we have

$$\mathbf{B} = \begin{pmatrix} \lambda_1 & \mathbf{0}^\top \\ \mathbf{0} & \mathbf{B}' \end{pmatrix},$$

where $\mathbf{B}' \in \mathbb{R}^{(n-1) \times (n-1)}$ is symmetric.

By the inductive hypothesis, $\mathbf{B}' = \mathbf{V} \mathbf{\Lambda}' \mathbf{V}^\top$ where $\mathbf{V}$ is orthogonal and $\mathbf{\Lambda}'$ is diagonal.

Let $\widetilde{\mathbf{V}} = \begin{pmatrix} 1 & \mathbf{0}^\top \\ \mathbf{0} & \mathbf{V} \end{pmatrix}$. Then $\widetilde{\mathbf{V}}$ is orthogonal and

$$\mathbf{B} = \widetilde{\mathbf{V}} \begin{pmatrix} \lambda_1 & \mathbf{0}^\top \\ \mathbf{0} & \mathbf{\Lambda}' \end{pmatrix} \widetilde{\mathbf{V}}^\top.$$

Setting $\mathbf{U} = \mathbf{Q} \widetilde{\mathbf{V}}$ (orthogonal) and $\mathbf{\Lambda} = \begin{pmatrix} \lambda_1 & \mathbf{0}^\top \\ \mathbf{0} & \mathbf{\Lambda}' \end{pmatrix}$ (diagonal), we get

$$\mathbf{A} = \mathbf{Q} \mathbf{B} \mathbf{Q}^\top = \mathbf{Q} \widetilde{\mathbf{V}} \mathbf{\Lambda} \widetilde{\mathbf{V}}^\top \mathbf{Q}^\top = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top.$$

The columns of $\mathbf{U}$ form an orthonormal basis of eigenvectors. $\square$

Remark 1.32. The spectral theorem is specific to symmetric (or more generally, self-adjoint) matrices.

- **What's general:** Any matrix $\mathbf{A} \in \mathbb{C}^{n \times n}$ has n eigenvalues (counted with multiplicity) and can be diagonalized if and only if it has n linearly independent eigenvectors.
- **What's special for symmetric matrices:** Eigenvalues are real, eigenvectors are orthogonal, and diagonalization via orthogonal matrices is always possible.

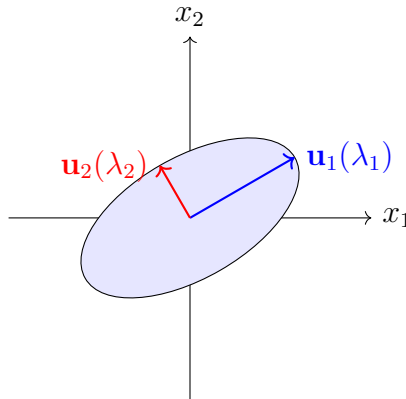
1.6 Applications of the Spectral Theorem

The spectral theorem is not just an abstract result; it is the mathematical engine behind many physical theories and modern algorithms.

1.6.1 Machine Learning

Covariance Matrices and PCA

In statistics and machine learning, the relationship between random variables $X_1, \dots, X_n$ is captured by the covariance matrix $\Sigma = \mathbb{E}[(\mathbf{X} - \boldsymbol{\mu})(\mathbf{X} - \boldsymbol{\mu})^\top]$. Since $\Sigma_{ij} = \text{Cov}(X_i, X_j) = \text{Cov}(X_j, X_i)$, the matrix is symmetric. The spectral theorem guarantees $\Sigma = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$, where $\mathbf{U}$ contains the principal directions of variation (eigenvectors) and $\mathbf{\Lambda}$ contains the variances along those directions (eigenvalues).



Data distribution aligned with eigenvectors of Σ

Kernel Methods (SVMs)

Kernel methods map data $\mathbf{x} \in \mathcal{X}$ to a high-dimensional feature space $\mathcal{H}$ via $\phi : \mathcal{X} \rightarrow \mathcal{H}$. The kernel function is $k(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle_{\mathcal{H}}$. The kernel matrix $\mathbf{K}_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ is symmetric and positive semidefinite. By the spectral theorem, $\mathbf{K} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$ with $\lambda_i \geq 0$. This ensures that $\mathbf{K}$ represents a valid inner product, allowing linear algorithms (like Support Vector Machines) to learn non-linear decision boundaries efficiently.

Graph Neural Networks

A graph $G = (V, E)$ is represented by its adjacency matrix $\mathbf{A}$. The Graph Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{A}$ (where $\mathbf{D}$ is the degree matrix) is symmetric. The spectral decomposition $\mathbf{L} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ defines the *Graph Fourier Transform*. For a signal $\mathbf{x} \in \mathbb{R}^{|V|}$ on the nodes, its Fourier transform is $\hat{\mathbf{x}} = \mathbf{U}^\top \mathbf{x}$. Spectral convolutions are defined as $g \star \mathbf{x} = \mathbf{U}g(\mathbf{\Lambda})\mathbf{U}^\top \mathbf{x}$, which is the foundation of spectral Graph Neural Networks (GNNs).

1.6.2 Quantum Mechanics

Observables as Self-Adjoint Operators

In quantum mechanics, physical quantities (observables) like position, momentum, and energy are represented by self-adjoint (Hermitian) operators $\hat{A}$ on a Hilbert space $\mathcal{H}$. The spectral theorem guarantees:

- The eigenvalues a_n are real. These are the only possible outcomes of a measurement.
- The eigenvectors $|\psi_n\rangle$ form a complete orthonormal basis.

Any state $|\psi\rangle$ can be expanded as $|\psi\rangle = \sum_n c_n |\psi_n\rangle$. The probability of measuring a_n is $P(a_n) = |\langle \psi_n | \psi \rangle|^2 = |c_n|^2$.

Time Evolution

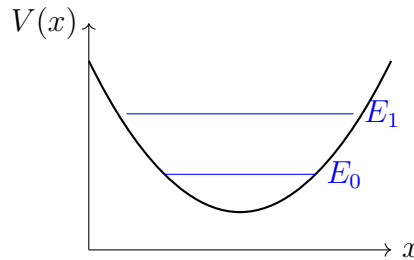
The dynamics of a quantum system are governed by the Hamiltonian operator $\hat{H}$. The Schrödinger equation is:

$$i\hbar \frac{d}{dt} |\psi(t)\rangle = \hat{H} |\psi(t)\rangle.$$

Using the spectral decomposition $\hat{H} = \sum_n E_n |\psi_n\rangle \langle \psi_n|$, the time evolution operator is:

$$e^{-i\hat{H}t/\hbar} = \sum_n e^{-iE_n t/\hbar} |\psi_n\rangle \langle \psi_n|.$$

This allows us to solve for the state at any time: $|\psi(t)\rangle = \sum_n c_n(0) e^{-iE_n t/\hbar} |\psi_n\rangle$.



Energy levels (eigenvalues of $\hat{H}$) in a potential well

1.6.3 Physics (Classical Mechanics)

The Inertia Tensor

For a rigid body, the angular momentum $\mathbf{L}$ is related to the angular velocity $\boldsymbol{\omega}$ by the inertia tensor $\mathbf{I}$: $\mathbf{L} = \mathbf{I}\boldsymbol{\omega}$. $\mathbf{I}$ is a symmetric 3×3 matrix. Its eigenvectors $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3$ define the *principal axes* of rotation. In this basis, the tensor is diagonal: $\mathbf{I}' = \text{diag}(I_1, I_2, I_3)$. Rotation about a principal axis is stable (for max/min moments) and simplifies the Euler equations of motion.

Normal Modes of Vibration

Coupled oscillators (like masses connected by springs) obey $\mathbf{M}\ddot{\mathbf{x}} + \mathbf{K}\mathbf{x} = \mathbf{0}$, where $\mathbf{M}$ is the mass matrix and $\mathbf{K}$ is the stiffness matrix (both symmetric). Assuming a solution $\mathbf{x}(t) = \mathbf{v}e^{i\omega t}$, we get the generalized eigenvalue problem:

$$(\mathbf{K} - \omega^2\mathbf{M})\mathbf{v} = \mathbf{0}.$$

Diagonalizing this system yields the *normal modes* (eigenvectors) where all parts of the system oscillate at the same frequency $\omega \propto \sqrt{\lambda}$.

1.6.4 Theoretical Physics

Propagators in Quantum Field Theory

In Quantum Field Theory (QFT), the fundamental object describing particle motion is the *propagator* or Green's function $D(x-y)$. It represents the probability amplitude for a particle to travel from spacetime point y to x .

Consider a free scalar field $\phi(x)$ governed by the Klein–Gordon equation:

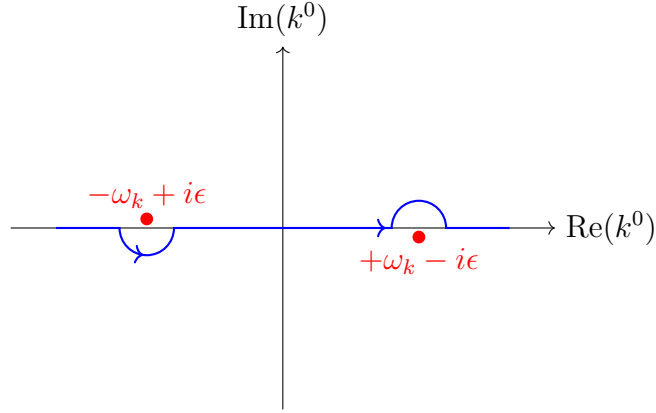
$$(\square + m^2)\phi(x) = 0, \quad \text{where } \square = \partial_\mu \partial^\mu = \frac{\partial^2}{\partial t^2} - \nabla^2.$$

The differential operator $\hat{O} = \square + m^2$ is translation-invariant, so its "eigenvectors" are the Fourier modes $e^{-ik \cdot x}$ (plane waves) with eigenvalues $-k^2 + m^2$, where $k^2 = (k^0)^2 - |\mathbf{k}|^2$.

The propagator $D(x-y)$ is the inverse of this operator, formally $\hat{O}^{-1}$. Using the spectral decomposition in momentum space (the continuous analog of matrix diagonalization), we write the inverse operator as an integral over the inverse eigenvalues:

$$D(x-y) = \int \frac{d^4k}{(2\pi)^4} \frac{e^{-ik \cdot (x-y)}}{m^2 - k^2 - i\epsilon}.$$

Here, the term $i\epsilon$ is a regularization parameter that shifts the poles of the integrand off the real axis, defining how the contour integral is performed. This choice corresponds to the *Feynman propagator*, which enforces causality: particles propagate forward in time, and antiparticles propagate backward.

Poles in the complex k^0 plane (Feynman contour)

Physically, this integral implies that a field excitation is a superposition of all possible momentum states, weighted by how close they are to the "mass shell" condition $k^2 = m^2$. The resonance at $k^2 \approx m^2$ dominates macroscopic propagation.

Stress-Energy Tensor in General Relativity

The stress-energy tensor $T_{\mu\nu}$ is a symmetric tensor capturing energy and momentum density. Einstein's equations link geometry to matter: $G_{\mu\nu} = 8\pi G T_{\mu\nu}$. Because $T_{\mu\nu}$ is symmetric, at any point in spacetime, one can find a local Lorentz frame (eigenbasis) where $T_{\mu\nu}$ is diagonal. For a perfect fluid, this gives $T_{\mu\nu} = \text{diag}(\rho, p, p, p)$, representing energy density ρ and pressure p . This simplifies the study of stars and cosmology.

1.7 Singular Value Decomposition

The spectral theorem applies only to symmetric matrices. The Singular Value Decomposition (SVD) generalizes this to arbitrary rectangular matrices, providing orthogonal diagonalization for any matrix.

Theorem 1.33 (Singular Value Decomposition). *Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ be any matrix with rank $r = \text{rank}(\mathbf{A})$. Then there exist:*

- Orthogonal matrix $\mathbf{U} \in \mathbb{R}^{m \times m}$ with $\mathbf{U}^\top \mathbf{U} = \mathbf{I}_m$,
- Orthogonal matrix $\mathbf{V} \in \mathbb{R}^{n \times n}$ with $\mathbf{V}^\top \mathbf{V} = \mathbf{I}_n$,
- Diagonal matrix $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ with diagonal entries $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ and all other entries zero,

such that

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top. \quad (1.19)$$

The values $\sigma_1, \dots, \sigma_r$ are called the singular values of $\mathbf{A}$. The columns of $\mathbf{U}$ are the left singular vectors, and the columns of $\mathbf{V}$ are the right singular vectors.

Proof. Consider the matrix $\mathbf{A}^\top \mathbf{A} \in \mathbb{R}^{n \times n}$. This matrix is symmetric:

$$(\mathbf{A}^\top \mathbf{A})^\top = \mathbf{A}^\top (\mathbf{A}^\top)^\top = \mathbf{A}^\top \mathbf{A}.$$

Moreover, it is positive semidefinite: for any $\mathbf{x} \in \mathbb{R}^n$,

$$\mathbf{x}^\top (\mathbf{A}^\top \mathbf{A}) \mathbf{x} = (\mathbf{A} \mathbf{x})^\top (\mathbf{A} \mathbf{x}) = \|\mathbf{A} \mathbf{x}\|^2 \geq 0.$$

By the spectral theorem (Theorem 1.31), there exists an orthogonal matrix $\mathbf{V} = [\mathbf{v}_1 \mid \cdots \mid \mathbf{v}_n]$ such that

$$\mathbf{A}^\top \mathbf{A} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^\top,$$

where $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$ with $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n \geq 0$.

Since $\mathbf{A}^\top \mathbf{A}$ has rank r , exactly r eigenvalues are positive: $\lambda_1, \dots, \lambda_r > 0$ and $\lambda_{r+1} = \cdots = \lambda_n = 0$.

Define the *singular values* by $\sigma_i = \sqrt{\lambda_i}$ for $i = 1, \dots, r$. Then $\sigma_1 \geq \cdots \geq \sigma_r > 0$.

Constructing the left singular vectors. For $i = 1, \dots, r$, define

$$\mathbf{u}_i = \frac{1}{\sigma_i} \mathbf{A} \mathbf{v}_i \in \mathbb{R}^m.$$

We verify that $\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$ are orthonormal. For $i, j \leq r$:

$$\begin{aligned} \langle \mathbf{u}_i, \mathbf{u}_j \rangle &= \frac{1}{\sigma_i \sigma_j} (\mathbf{A} \mathbf{v}_i)^\top (\mathbf{A} \mathbf{v}_j) \\ &= \frac{1}{\sigma_i \sigma_j} \mathbf{v}_i^\top \mathbf{A}^\top \mathbf{A} \mathbf{v}_j \\ &= \frac{1}{\sigma_i \sigma_j} \mathbf{v}_i^\top (\lambda_j \mathbf{v}_j) \\ &= \frac{\lambda_j}{\sigma_i \sigma_j} \mathbf{v}_i^\top \mathbf{v}_j = \frac{\sigma_j^2}{\sigma_i \sigma_j} \delta_{ij} = \frac{\sigma_j}{\sigma_i} \delta_{ij} = \delta_{ij}. \end{aligned}$$

Extend $\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$ to an orthonormal basis $\{\mathbf{u}_1, \dots, \mathbf{u}_m\}$ of $\mathbb{R}^m$ (using Gram-Schmidt if needed). Let $\mathbf{U} = [\mathbf{u}_1 \mid \cdots \mid \mathbf{u}_m]$.

Verification of the decomposition. By construction, for $j \leq r$:

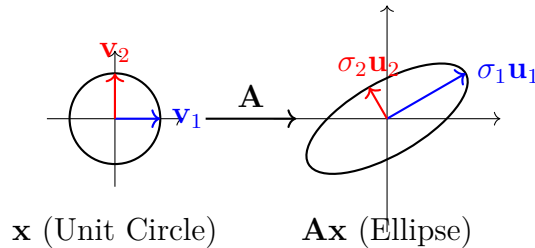
$$\mathbf{A} \mathbf{v}_j = \sigma_j \mathbf{u}_j.$$

For $j > r$, we have $\mathbf{A}^\top \mathbf{A} \mathbf{v}_j = \lambda_j \mathbf{v}_j = \mathbf{0}$, so $\|\mathbf{A} \mathbf{v}_j\|^2 = \mathbf{v}_j^\top \mathbf{A}^\top \mathbf{A} \mathbf{v}_j = 0$, thus $\mathbf{A} \mathbf{v}_j = \mathbf{0}$.

Define the diagonal matrix $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ with $\Sigma_{ii} = \sigma_i$ for $i \leq r$ and all other entries zero. Then:

$$\mathbf{A} \mathbf{V} = \mathbf{A} [\mathbf{v}_1 \mid \cdots \mid \mathbf{v}_n] = [\sigma_1 \mathbf{u}_1 \mid \cdots \mid \sigma_r \mathbf{u}_r \mid \mathbf{0} \mid \cdots \mid \mathbf{0}] = \mathbf{U} \mathbf{\Sigma}.$$

Since $\mathbf{V}$ is orthogonal, $\mathbf{V}^\top \mathbf{V} = \mathbf{I}_n$, so $\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top$. □



Decomposition: Rotate ($\mathbf{V}^\top$) $\rightarrow$ Stretch ($\mathbf{\Sigma}$) $\rightarrow$ Rotate ($\mathbf{U}$)

Remark 1.34 (Thin SVD). For computational purposes, we often use the *thin* (or *reduced*) SVD. If $m \geq n$ and $\text{rank}(\mathbf{A}) = r$, write

$$\mathbf{A} = \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^\top,$$

where $\mathbf{U}_r \in \mathbb{R}^{m \times r}$ contains the first r columns of $\mathbf{U}$, $\mathbf{\Sigma}_r = \text{diag}(\sigma_1, \dots, \sigma_r) \in \mathbb{R}^{r \times r}$, and $\mathbf{V}_r \in \mathbb{R}^{n \times r}$ contains the first r columns of $\mathbf{V}$.

Proposition 1.35 (Connection between SVD and eigenvalues). *Let $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ be the SVD of $\mathbf{A} \in \mathbb{R}^{m \times n}$ with singular values $\sigma_1, \dots, \sigma_r$. Then:*

1. *The eigenvalues of $\mathbf{A}^\top \mathbf{A}$ are $\sigma_1^2, \dots, \sigma_r^2, 0, \dots, 0$, with eigenvectors given by the columns of $\mathbf{V}$.*
2. *The eigenvalues of $\mathbf{A} \mathbf{A}^\top$ are $\sigma_1^2, \dots, \sigma_r^2, 0, \dots, 0$, with eigenvectors given by the columns of $\mathbf{U}$.*
3. $\|\mathbf{A}\|_F^2 = \sigma_1^2 + \dots + \sigma_r^2$.

Proof. (1) From the SVD:

$$\begin{aligned} \mathbf{A}^\top \mathbf{A} &= (\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top)^\top (\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top) \\ &= \mathbf{V}\mathbf{\Sigma}^\top \mathbf{U}^\top \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top \\ &= \mathbf{V}\mathbf{\Sigma}^\top \mathbf{\Sigma}\mathbf{V}^\top = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top, \end{aligned}$$

where $\mathbf{\Lambda} = \mathbf{\Sigma}^\top \mathbf{\Sigma} = \text{diag}(\sigma_1^2, \dots, \sigma_r^2, 0, \dots, 0)$.

(2) Similarly, $\mathbf{A} \mathbf{A}^\top = \mathbf{U}\mathbf{\Sigma}\mathbf{\Sigma}^\top \mathbf{U}^\top$ with the same eigenvalues.

(3) By Proposition 1.10:

$$\|\mathbf{A}\|_F^2 = \text{tr}(\mathbf{A}^\top \mathbf{A}) = \text{tr}(\mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top) = \text{tr}(\mathbf{\Lambda}) = \sum_{i=1}^r \sigma_i^2.$$

□

1.7.1 Applications of Singular Value Decomposition

Low-Rank Matrix Approximation (Image Compression)

The Eckart-Young theorem states that the best rank- k approximation to a matrix $\mathbf{A}$ (in the Frobenius norm sense) is given by the truncated SVD:

$$\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top.$$

This property is fundamental to lossy data compression. Consider a grayscale image represented as a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$. By computing the SVD and storing only the first k singular values and corresponding singular vectors, we can reconstruct the image with an approximation error of $\sum_{i=k+1}^{\min(m,n)} \sigma_i^2$.

For large matrices, $k \ll \min(m, n)$ is often sufficient to capture the visual structure. The storage requirement drops from mn entries to $k(m + n + 1)$, yielding significant compression ratios.

The Moore-Penrose Pseudoinverse

For a system of linear equations $\mathbf{Ax} = \mathbf{b}$, if $\mathbf{A}$ is not invertible (e.g., rectangular or singular), there is no unique solution. The *pseudoinverse* $\mathbf{A}^+$ provides the "best" solution in a least-squares sense.

Using the SVD $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, the pseudoinverse is defined as:

$$\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^\top,$$

where $\mathbf{\Sigma}^+$ is obtained by taking the reciprocal of non-zero singular values and transposing the matrix. The solution $\mathbf{x}^* = \mathbf{A}^+\mathbf{b}$ minimizes $\|\mathbf{Ax} - \mathbf{b}\|_2$, and among all such minimizers, it has the smallest norm $\|\mathbf{x}\|_2$. This is widely used in solving ill-posed inverse problems in physics and engineering.

1.8 Principal Component Analysis

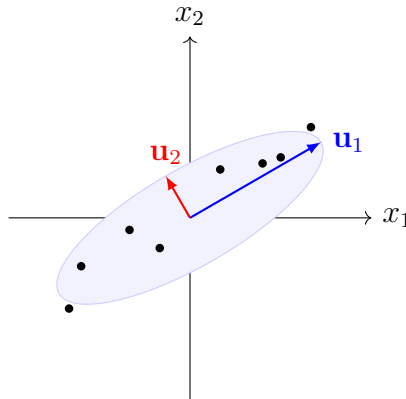
1.8.1 The Geometric Problem

Principal Component Analysis (PCA) solves a fundamental geometric problem: given data points in high-dimensional space, find the best low-dimensional linear subspace that approximates them.

The problem: Given data points $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathbb{R}^d$ (assumed centered: $\sum_{i=1}^N \mathbf{x}_i = \mathbf{0}$), find a k -dimensional linear subspace $S \subseteq \mathbb{R}^d$ that minimizes the total squared distance from all points to S :

$$\min_{\substack{S \subseteq \mathbb{R}^d \\ \dim(S)=k}} \sum_{i=1}^N \text{dist}(\mathbf{x}_i, S)^2, \quad (1.20)$$

where $\text{dist}(\mathbf{x}, S) := \inf_{\mathbf{y} \in S} \|\mathbf{x} - \mathbf{y}\|_2$ is the distance from $\mathbf{x}$ to S .



PCA finds directions of maximum variance ($\mathbf{u}_1$)

1.8.2 Parametrizing Subspaces via Orthonormal Bases

To solve (1.20), we need to parametrize k -dimensional subspaces. The key is to represent each subspace by an orthonormal basis.

Lemma 1.36 (Orthonormal basis representation). *Let $S \subseteq \mathbb{R}^d$ be a k -dimensional linear subspace. Then there exists a matrix $\mathbf{W} \in \mathbb{R}^{d \times k}$ with columns $\mathbf{w}_1, \dots, \mathbf{w}_k$ such that:*

1. *The columns of $\mathbf{W}$ form an orthonormal basis of S : $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_k$.*
2. *$S = \text{span}(\mathbf{w}_1, \dots, \mathbf{w}_k) = \{\mathbf{W}\mathbf{z} : \mathbf{z} \in \mathbb{R}^k\}$.*

Conversely, any $\mathbf{W} \in \mathbb{R}^{d \times k}$ with $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_k$ defines a k -dimensional subspace $S = \text{span}(\mathbf{w}_1, \dots, \mathbf{w}_k)$.

Proof. Pick any basis of S and apply Gram–Schmidt to obtain orthonormal vectors $\mathbf{w}_1, \dots, \mathbf{w}_k \in \mathbb{R}^d$. Let $\mathbf{W} = [\mathbf{w}_1 \mid \dots \mid \mathbf{w}_k]$. Then $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_k$ and $\text{span}(\mathbf{w}_1, \dots, \mathbf{w}_k) = S$. Conversely, if $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_k$, the columns are orthonormal, hence linearly independent, so the span has dimension k . $\square$

Thus we can optimize over matrices $\mathbf{W} \in \mathbb{R}^{d \times k}$ with $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_k$ instead of over abstract subspaces.

1.8.3 Distance to a Subspace and Projection

Lemma 1.37 (Projection formula). *Let $\mathbf{W} \in \mathbb{R}^{d \times k}$ with columns $\mathbf{w}_1, \dots, \mathbf{w}_k$ satisfying $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_k$, and let $S = \text{span}(\mathbf{w}_1, \dots, \mathbf{w}_k)$. For $\mathbf{x} \in \mathbb{R}^d$, the point in S closest to $\mathbf{x}$ is*

$$\hat{\mathbf{x}} = \mathbf{W}\mathbf{W}^\top \mathbf{x} \in \mathbb{R}^d, \quad (1.21)$$

and the squared distance is

$$\text{dist}(\mathbf{x}, S)^2 = \|\mathbf{x} - \mathbf{W}\mathbf{W}^\top \mathbf{x}\|^2. \quad (1.22)$$

Proof. The distance to S is

$$\text{dist}(\mathbf{x}, S) = \inf_{\mathbf{y} \in S} \|\mathbf{x} - \mathbf{y}\| = \inf_{\mathbf{z} \in \mathbb{R}^k} \|\mathbf{x} - \mathbf{W}\mathbf{z}\|,$$

since every $\mathbf{y} \in S$ can be written as $\mathbf{W}\mathbf{z}$ for some $\mathbf{z} \in \mathbb{R}^k$.

Define $f(\mathbf{z}) = \|\mathbf{x} - \mathbf{W}\mathbf{z}\|^2 = (\mathbf{x} - \mathbf{W}\mathbf{z})^\top (\mathbf{x} - \mathbf{W}\mathbf{z})$. Expanding:

$$f(\mathbf{z}) = \mathbf{x}^\top \mathbf{x} - 2\mathbf{x}^\top \mathbf{W}\mathbf{z} + \mathbf{z}^\top \mathbf{W}^\top \mathbf{W}\mathbf{z}.$$

Using $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_k$:

$$f(\mathbf{z}) = \mathbf{x}^\top \mathbf{x} - 2\mathbf{x}^\top \mathbf{W}\mathbf{z} + \mathbf{z}^\top \mathbf{z}.$$

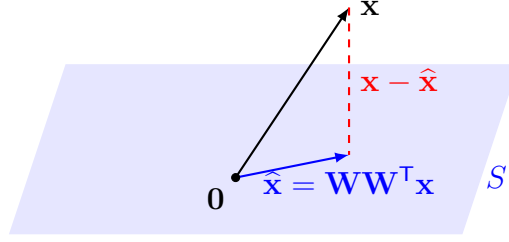
Taking the gradient: $\nabla_{\mathbf{z}} f = -2\mathbf{W}^\top \mathbf{x} + 2\mathbf{z}$. Setting to zero gives $\mathbf{z}^* = \mathbf{W}^\top \mathbf{x}$. The Hessian is $2\mathbf{I}_k$, which is positive definite, so this is the unique global minimizer.

Thus $\hat{\mathbf{x}} = \mathbf{W}\mathbf{z}^* = \mathbf{W}\mathbf{W}^\top \mathbf{x}$, and

$$\text{dist}(\mathbf{x}, S)^2 = \|\mathbf{x} - \mathbf{W}\mathbf{W}^\top \mathbf{x}\|^2.$$

$\square$

Remark 1.38. The matrix $\mathbf{P} = \mathbf{W}\mathbf{W}^\top \in \mathbb{R}^{d \times d}$ is the orthogonal projection onto S . It is symmetric, idempotent ($\mathbf{P}^2 = \mathbf{P}$), and has rank k . The coordinate vector of $\hat{\mathbf{x}}$ in the basis given by columns of $\mathbf{W}$ is $\mathbf{z}^* = \mathbf{W}^\top \mathbf{x} \in \mathbb{R}^k$.



1.8.4 The Matrix Optimization Problem

Definition 1.39 (Sample covariance matrix). Given centered data $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathbb{R}^d$, the *sample covariance matrix* is

$$\mathbf{C} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i \mathbf{x}_i^T \in \mathbb{R}^{d \times d}. \quad (1.23)$$

It is symmetric and positive semidefinite.

Theorem 1.40 (PCA solution). Let $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathbb{R}^d$ be centered data with covariance matrix $\mathbf{C}$. Let $\mathbf{C} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^T$ be the spectral decomposition with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0$ and corresponding eigenvectors $\mathbf{u}_1, \dots, \mathbf{u}_d$.

The geometric optimization problem (1.20) is equivalent to:

$$\min_{\substack{\mathbf{W} \in \mathbb{R}^{d \times k} \\ \mathbf{W}^T \mathbf{W} = \mathbf{I}_k}} \sum_{i=1}^N \|\mathbf{x}_i - \mathbf{W} \mathbf{W}^T \mathbf{x}_i\|^2 \Leftrightarrow \max_{\substack{\mathbf{W} \in \mathbb{R}^{d \times k} \\ \mathbf{W}^T \mathbf{W} = \mathbf{I}_k}} \text{tr}(\mathbf{W}^T \mathbf{C} \mathbf{W}). \quad (1.24)$$

Solution: The optimal subspace is $S^* = \text{span}(\mathbf{u}_1, \dots, \mathbf{u}_k)$, spanned by the top k eigenvectors of $\mathbf{C}$. Any optimal $\mathbf{W}^*$ has the form

$$\mathbf{W}^* = [\mathbf{u}_1 \mid \dots \mid \mathbf{u}_k] \mathbf{R}, \quad (1.25)$$

where $\mathbf{R} \in \mathbb{R}^{k \times k}$ is any orthogonal matrix. The minimum reconstruction error is

$$\sum_{i=1}^N \text{dist}(\mathbf{x}_i, S^*)^2 = N \sum_{j=k+1}^d \lambda_j. \quad (1.26)$$

Proof. Step 1: From geometric problem to Frobenius norm.

By Lemma 1.37, for $S = \text{span}(\mathbf{w}_1, \dots, \mathbf{w}_k)$ where $\mathbf{W} = [\mathbf{w}_1 \mid \dots \mid \mathbf{w}_k]$ we have

$$\sum_{i=1}^N \text{dist}(\mathbf{x}_i, S)^2 = \sum_{i=1}^N \|\mathbf{x}_i - \mathbf{W} \mathbf{W}^T \mathbf{x}_i\|^2.$$

Let $\mathbf{X} \in \mathbb{R}^{N \times d}$ be the data matrix with rows $\mathbf{x}_i^T$. Then

$$\sum_{i=1}^N \|\mathbf{x}_i - \mathbf{W} \mathbf{W}^T \mathbf{x}_i\|^2 = \|\mathbf{X} - \mathbf{X} \mathbf{W} \mathbf{W}^T\|_F^2,$$

where $\|\cdot\|_F$ is the Frobenius norm.

Step 2: Converting to trace maximization.

Expand the Frobenius norm using $\|\mathbf{A}\|_F^2 = \text{tr}(\mathbf{A}^\top \mathbf{A})$:

$$\begin{aligned} \|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top\|_F^2 &= \text{tr}\left((\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top)^\top (\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top)\right) \\ &= \text{tr}(\mathbf{X}^\top \mathbf{X}) - \text{tr}(\mathbf{X}^\top \mathbf{X}\mathbf{W}\mathbf{W}^\top) - \text{tr}(\mathbf{W}\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}) + \text{tr}(\mathbf{W}\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}\mathbf{W}^\top). \end{aligned}$$

Using the cyclic property of trace, $\text{tr}(\mathbf{ABC}) = \text{tr}(\mathbf{CAB})$:

$$\begin{aligned} \text{tr}(\mathbf{X}^\top \mathbf{X}\mathbf{W}\mathbf{W}^\top) &= \text{tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}), \\ \text{tr}(\mathbf{W}\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}) &= \text{tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}), \\ \text{tr}(\mathbf{W}\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}\mathbf{W}^\top) &= \text{tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W} \cdot \mathbf{W}^\top \mathbf{W}) = \text{tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}), \end{aligned}$$

where the last equality uses $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_k$. Thus:

$$\|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top\|_F^2 = \text{tr}(\mathbf{X}^\top \mathbf{X}) - \text{tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}).$$

Since $\text{tr}(\mathbf{X}^\top \mathbf{X})$ is constant, minimizing reconstruction error is equivalent to maximizing $\text{tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W})$. Define $\mathbf{C} = \frac{1}{N} \mathbf{X}^\top \mathbf{X}$. Then (1.20) becomes:

$$\max_{\substack{\mathbf{W} \in \mathbb{R}^{d \times k} \\ \mathbf{W}^\top \mathbf{W} = \mathbf{I}_k}} \text{tr}(\mathbf{W}^\top \mathbf{C}\mathbf{W}).$$

Step 3: Change of variables to eigenvector basis.

By the spectral theorem, $\mathbf{C} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ with $\mathbf{U}^\top \mathbf{U} = \mathbf{I}_d$ and $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_d)$, $\lambda_1 \geq \dots \geq \lambda_d \geq 0$.

Define $\mathbf{S} := \mathbf{U}^\top \mathbf{W} \in \mathbb{R}^{d \times k}$. Since $\mathbf{U}$ is orthogonal, $\mathbf{S}^\top \mathbf{S} = \mathbf{W}^\top \mathbf{U}\mathbf{U}^\top \mathbf{W} = \mathbf{I}_k$. Moreover,

$$\text{tr}(\mathbf{W}^\top \mathbf{C}\mathbf{W}) = \text{tr}(\mathbf{W}^\top \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top \mathbf{W}) = \text{tr}(\mathbf{S}^\top \mathbf{\Lambda}\mathbf{S}) = \sum_{i=1}^d \lambda_i \sum_{j=1}^k s_{ij}^2.$$

Step 4: Optimization over $\mathbf{S}$.

The constraint $\mathbf{S}^\top \mathbf{S} = \mathbf{I}_k$ implies $\sum_{i=1}^d s_{ij}^2 = 1$ for each column j , hence $\sum_{i=1}^d \sum_{j=1}^k s_{ij}^2 = k$. Define $\alpha_i := \sum_{j=1}^k s_{ij}^2 \in [0, k]$, the total squared mass in row i . Then $\sum_{i=1}^d \alpha_i = k$ and

$$\text{tr}(\mathbf{S}^\top \mathbf{\Lambda}\mathbf{S}) = \sum_{i=1}^d \lambda_i \alpha_i.$$

Claim: Under $\lambda_1 \geq \dots \geq \lambda_d$ and $\alpha_i \geq 0$, $\sum \alpha_i = k$, we have $\sum_{i=1}^d \lambda_i \alpha_i \leq \sum_{i=1}^k \lambda_i$, with equality if and only if $\alpha_i = 1$ for $i \leq k$ and $\alpha_i = 0$ for $i > k$.

Proof of claim: If there exist $i \leq k < j$ with $\alpha_i < 1$ and $\alpha_j > 0$, define $\tilde{\alpha}_i = \alpha_i + \epsilon$, $\tilde{\alpha}_j = \alpha_j - \epsilon$ for small $\epsilon > 0$. Then

$$\sum_{\ell} \lambda_{\ell} \tilde{\alpha}_{\ell} = \sum_{\ell} \lambda_{\ell} \alpha_{\ell} + \epsilon(\lambda_i - \lambda_j) \geq \sum_{\ell} \lambda_{\ell} \alpha_{\ell},$$

with strict inequality if $\lambda_i > \lambda_j$. Repeating this argument shows the maximum is $\sum_{i=1}^k \lambda_i$. $\square$

One feasible $\mathbf{S}$ achieving this is $\mathbf{S} = \begin{pmatrix} \mathbf{I}_k \\ \mathbf{0} \end{pmatrix}$, which gives $\mathbf{W} = \mathbf{US} = [\mathbf{u}_1 \mid \cdots \mid \mathbf{u}_k]$.

Any other optimal $\mathbf{W}^*$ satisfies $\mathbf{U}^\top \mathbf{W}^* = \mathbf{S}'$ with the same structure, i.e., $\mathbf{S}' = \begin{pmatrix} \mathbf{R} \\ \mathbf{0} \end{pmatrix}$ for some orthogonal $\mathbf{R} \in \mathbb{R}^{k \times k}$, giving $\mathbf{W}^* = [\mathbf{u}_1 \mid \cdots \mid \mathbf{u}_k] \mathbf{R}$.

The minimum reconstruction error is $\text{tr}(\mathbf{X}^\top \mathbf{X}) - N \sum_{j=1}^k \lambda_j = N \sum_{j=k+1}^d \lambda_j$. $\square$

Remark 1.41 (Alternative proof via Lagrange multipliers). We can also solve the $k = 1$ case (finding the first principal component) using Lagrange multipliers, then extend to general k by deflation.

Finding the first principal component ($k = 1$).

Maximize $f(\mathbf{w}) = \mathbf{w}^\top \mathbf{C} \mathbf{w}$ subject to $g(\mathbf{w}) = \mathbf{w}^\top \mathbf{w} - 1 = 0$, where $\mathbf{w} \in \mathbb{R}^d$.

Form the Lagrangian:

$$\mathcal{L}(\mathbf{w}, \lambda) = \mathbf{w}^\top \mathbf{C} \mathbf{w} - \lambda(\mathbf{w}^\top \mathbf{w} - 1).$$

Take the gradient with respect to $\mathbf{w}$ and set to zero:

$$\nabla_{\mathbf{w}} \mathcal{L} = 2\mathbf{C} \mathbf{w} - 2\lambda \mathbf{w} = \mathbf{0} \quad \Rightarrow \quad \mathbf{C} \mathbf{w} = \lambda \mathbf{w}.$$

So $\mathbf{w}$ must be an eigenvector of $\mathbf{C}$ with eigenvalue λ . The objective value at this critical point is:

$$f(\mathbf{w}) = \mathbf{w}^\top \mathbf{C} \mathbf{w} = \mathbf{w}^\top (\lambda \mathbf{w}) = \lambda \mathbf{w}^\top \mathbf{w} = \lambda.$$

To maximize variance, choose $\mathbf{w} = \mathbf{u}_1$, the eigenvector corresponding to the largest eigenvalue λ_1 .

Finding subsequent components ($k \geq 2$).

For the j -th principal component with $j > 1$, maximize $\mathbf{w}^\top \mathbf{C} \mathbf{w}$ subject to:

- $\mathbf{w}^\top \mathbf{w} = 1$ (unit norm),
- $\mathbf{w}^\top \mathbf{u}_i = 0$ for all $i < j$ (orthogonal to previous components).

Form the Lagrangian with multiple constraints:

$$\mathcal{L}(\mathbf{w}, \lambda, \mu_1, \dots, \mu_{j-1}) = \mathbf{w}^\top \mathbf{C} \mathbf{w} - \lambda(\mathbf{w}^\top \mathbf{w} - 1) - \sum_{i=1}^{j-1} \mu_i \mathbf{w}^\top \mathbf{u}_i.$$

Taking the gradient:

$$\nabla_{\mathbf{w}} \mathcal{L} = 2\mathbf{C} \mathbf{w} - 2\lambda \mathbf{w} - \sum_{i=1}^{j-1} \mu_i \mathbf{u}_i = \mathbf{0}.$$

Multiply on the left by $\mathbf{u}_i^\top$ for any $i < j$:

$$2\mathbf{u}_i^\top \mathbf{C} \mathbf{w} - 2\lambda \mathbf{u}_i^\top \mathbf{w} - \mu_i = 0.$$

Using $\mathbf{u}_i^\top \mathbf{w} = 0$ (orthogonality constraint) and $\mathbf{C} \mathbf{u}_i = \lambda_i \mathbf{u}_i$ (since $\mathbf{u}_i$ is an eigenvector):

$$2(\mathbf{C} \mathbf{u}_i)^\top \mathbf{w} - \mu_i = 2\lambda_i \mathbf{u}_i^\top \mathbf{w} - \mu_i = -\mu_i = 0.$$

So $\mu_i = 0$ for all $i < j$. This reduces the stationarity condition to:

$$\mathbf{C}\mathbf{w} = \lambda\mathbf{w}.$$

Again, $\mathbf{w}$ must be an eigenvector. Combined with the orthogonality constraints $\mathbf{w}^\top \mathbf{u}_i = 0$ for $i < j$, we have $\mathbf{w} \in \text{span}(\mathbf{u}_j, \mathbf{u}_{j+1}, \dots, \mathbf{u}_d)$. To maximize the objective λ , choose $\mathbf{w} = \mathbf{u}_j$ with eigenvalue λ_j .

By induction, the first k principal components are $\mathbf{u}_1, \dots, \mathbf{u}_k$.

Remark 1.42 (Connection between PCA and SVD). PCA can be computed directly using SVD without forming the covariance matrix.

Let $\mathbf{X} \in \mathbb{R}^{N \times d}$ be the centered data matrix with rows $\mathbf{x}_i^\top$. Compute the thin SVD:

$$\mathbf{X} = \mathbf{U}_X \mathbf{\Sigma} \mathbf{V}^\top,$$

where $\mathbf{U}_X \in \mathbb{R}^{N \times r}$, $\mathbf{\Sigma} = \text{diag}(\sigma_1, \dots, \sigma_r) \in \mathbb{R}^{r \times r}$, and $\mathbf{V} \in \mathbb{R}^{d \times r}$ with $r = \text{rank}(\mathbf{X})$.

The covariance matrix is:

$$\mathbf{C} = \frac{1}{N} \mathbf{X}^\top \mathbf{X} = \frac{1}{N} \mathbf{V} \mathbf{\Sigma}^\top \mathbf{U}_X^\top \mathbf{U}_X \mathbf{\Sigma} \mathbf{V}^\top = \frac{1}{N} \mathbf{V} \mathbf{\Sigma}^2 \mathbf{V}^\top.$$

Comparing with the spectral decomposition $\mathbf{C} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top$, we see:

- **Principal components:** The columns of $\mathbf{V}$ are the principal components (eigenvectors of $\mathbf{C}$).
- **Eigenvalues:** $\lambda_i = \frac{\sigma_i^2}{N}$ for $i = 1, \dots, r$.
- **Variance explained:** The variance captured by the i -th principal component is $\frac{\sigma_i^2}{N}$.

Computational advantage: Computing SVD of $\mathbf{X} \in \mathbb{R}^{N \times d}$ is often faster and more numerically stable than forming $\mathbf{C} = \mathbf{X}^\top \mathbf{X} \in \mathbb{R}^{d \times d}$ and computing its eigendecomposition, especially when $N \gg d$ or $d \gg N$.

Corollary 1.43 (PCA algorithm). *To perform PCA on centered data $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathbb{R}^d$:*

1. *Compute the covariance matrix $\mathbf{\Sigma} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i \mathbf{x}_i^\top$.*
2. *Find the eigenvalue decomposition $\mathbf{\Sigma} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top$.*
3. *The principal components are the columns of $\mathbf{U}$, ordered by decreasing eigenvalue.*
4. *To project onto k dimensions, use $\mathbf{W} = [\mathbf{u}_1 \mid \dots \mid \mathbf{u}_k]$ and compute $\mathbf{z}_i = \mathbf{W}^\top \mathbf{x}_i$.*

The variance explained by the first k components is $\frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i}$.

1.8.5 Applications of Principal Component Analysis

Dimensionality Reduction in Images (Eigenfaces)

Natural images, such as faces, are high-dimensional data points. A 256×256 grayscale image corresponds to a vector $\mathbf{x} \in \mathbb{R}^{65536}$. However, the space of "valid" face images is a much lower-dimensional manifold within this huge space.

PCA identifies the principal directions of variation in the dataset of face images. The eigenvectors $\mathbf{u}_1, \dots, \mathbf{u}_k$ of the covariance matrix are called *eigenfaces*. They represent the fundamental "ghost-like" features (such as lighting direction, face shape, or pose) from which any specific face can be constructed.

By keeping only the top $k \approx 100$ eigenfaces, we can compress the image $\mathbf{x}$ into a coefficient vector $\mathbf{z} \in \mathbb{R}^{100}$:

$$\mathbf{z} = \mathbf{W}^T(\mathbf{x} - \boldsymbol{\mu}),$$

where $\boldsymbol{\mu}$ is the mean face and $\mathbf{W} = [\mathbf{u}_1 \mid \dots \mid \mathbf{u}_k]$. The reconstruction $\hat{\mathbf{x}} = \boldsymbol{\mu} + \mathbf{W}\mathbf{z}$ preserves the essential facial features while discarding noise and minor variations, achieving a compression ratio of over 600:1.

Feature Engineering in Recommender Systems

In a recommender system, we have a sparse user-item interaction matrix $\mathbf{R} \in \mathbb{R}^{N \times M}$, where R_{ij} represents the rating user i gives to item j . The goal is to predict missing ratings.

PCA (or more precisely, SVD-based matrix factorization) transforms this sparse matrix into dense low-dimensional embeddings. We assume that user preferences depend on a small number of latent factors (e.g., for movies: "action vs. drama", "serious vs. comedy").

By approximating the matrix $\mathbf{R}$ with a rank- k matrix, we map each user i to a vector $\mathbf{p}_i \in \mathbb{R}^k$ and each item j to a vector $\mathbf{q}_j \in \mathbb{R}^k$ such that

$$R_{ij} \approx \mathbf{p}_i^T \mathbf{q}_j.$$

These latent vectors capture semantic similarities: users with similar $\mathbf{p}$ vectors share tastes, and items with similar $\mathbf{q}$ vectors are semantically related, enabling the system to recommend items that align with the user's latent preferences.

Genomics and Population Genetics

In genomics, the data matrix $\mathbf{X}$ consists of Single Nucleotide Polymorphisms (SNPs) for N individuals across d genetic markers (often $d \gg N$).

PCA is applied to this matrix to visualize population structure. The first two principal components often reveal geographic ancestry. For example, when applied to European populations, the projection of individuals onto the plane spanned by the first two principal components remarkably reconstructs the map of Europe, with individuals clustering by their country of origin. This indicates that geography is the dominant factor explaining genetic variation.

1.9 UMAP: Nonlinear Dimensionality Reduction

1.9.1 Limitations of Linear Methods

PCA is a powerful technique, but it has a fundamental limitation: it can only discover *linear* structure in data. When data lies on or near a nonlinear manifold embedded in high-dimensional space, linear projections may fail to capture the intrinsic geometry.

Example 1.44 (Swiss roll). Consider the “Swiss roll” dataset: points sampled from a 2D rectangular sheet that has been rolled up into a spiral in $\mathbb{R}^3$. The intrinsic dimensionality is 2 (points can be parameterized by their position on the original sheet), but PCA projecting to 2D will “crush” the roll, mixing together points that are far apart along the manifold.

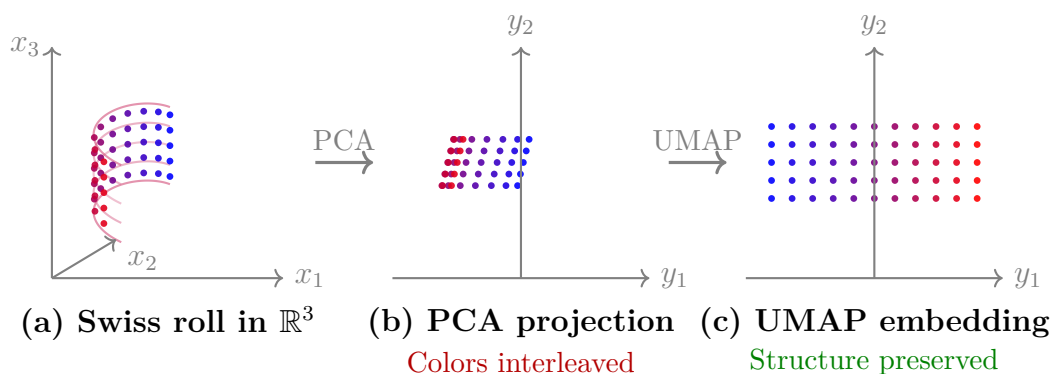


Figure 1.1: The Swiss roll problem illustrates the limitation of linear dimensionality reduction. (a) Data lies on a 2D manifold rolled into 3D; color encodes position along the manifold (blue $\rightarrow$ red). (b) PCA projects onto the plane of maximum variance, mixing together points that are far apart on the manifold (note interleaved colors). (c) UMAP recovers the intrinsic 2D structure by preserving local neighborhoods, unrolling the manifold so that color varies smoothly from left to right.

This motivates *manifold learning* methods that can recover nonlinear structure. **UMAP** (Uniform Manifold Approximation and Projection) is a state-of-the-art technique grounded in Riemannian geometry and algebraic topology.

1.9.2 Mathematical Foundations

UMAP is built on the assumption that the data is uniformly distributed on a Riemannian manifold. The key insight is that we can infer the local metric structure from the data itself.

Local Distance and the Riemannian Assumption

Suppose data points $\mathbf{x}_1, \dots, \mathbf{x}_N$ lie on or near an unknown Riemannian manifold $\mathcal{M}$ embedded in $\mathbb{R}^d$. If the data is uniformly distributed on $\mathcal{M}$, then locally around each point $\mathbf{x}_i$, the density of nearby points reveals information about the local geometry.

For each point $\mathbf{x}_i$, UMAP constructs a *local metric* by finding its k nearest neighbors and defining a distance scale $\sigma_i > 0$ such that:

$$\sum_{j=1}^k \exp\left(-\frac{d(\mathbf{x}_i, \mathbf{x}_{i_j}) - \rho_i}{\sigma_i}\right) = \log_2(k), \quad (1.27)$$

where $\mathbf{x}_{i_1}, \dots, \mathbf{x}_{i_k}$ are the k nearest neighbors of $\mathbf{x}_i$, and $\rho_i = \min_j d(\mathbf{x}_i, \mathbf{x}_{i_j})$ is the distance to the nearest neighbor. This normalization ensures each point “sees” a comparable local neighborhood regardless of the local density.

Remark 1.45 (On the uniformity assumption). The assumption that data is *uniformly* distributed on $\mathcal{M}$ is crucial for interpreting local density as geometric information. When data is non-uniform:

- **Dense regions** will have small σ_i values, making the algorithm treat nearby points as “close” on the manifold.
- **Sparse regions** will have large σ_i values, stretching distances to maintain connectivity.

This adaptive scaling means UMAP *implicitly assumes* non-uniform sampling reflects the underlying geometry (e.g., higher curvature regions sampled more densely). If the non-uniformity is purely a sampling artifact unrelated to geometry, UMAP may distort the true manifold structure. In practice, this is often acceptable because:

1. The local normalization via σ_i partially compensates for density variations.
2. For visualization, preserving *relative* neighborhood structure matters more than exact distances.

Remark 1.46 (Embeddings on the sphere and fundamental non-uniformity). A common application of UMAP is visualizing *embedding vectors* from neural networks (word embeddings, sentence embeddings, image features). These are often L^2 -normalized to lie on the unit sphere $S^{d-1} \subset \mathbb{R}^d$. Here the non-uniformity is typically **fundamental**, not a sampling artifact:

- Some semantic regions (e.g., common words, frequent image classes) are inherently more populated.
- The embedding space may have “semantic voids”—regions with no meaningful concepts.
- Cluster sizes reflect genuine differences in category diversity, not geometric curvature.

Consequences of applying UMAP to fundamentally non-uniform data:

1. **Density equalization:** UMAP stretches sparse regions and compresses dense regions, making the embedding appear more uniform than reality. A cluster of 1000 common words may appear similar in size to a cluster of 50 rare technical terms.
2. **Distance distortion:** Inter-cluster distances become unreliable. Two clusters that are far apart on S^{d-1} but have different densities will have their relative distance distorted by the differing σ_i values.

3. **Loss of density information:** The original density structure—which may carry semantic meaning (e.g., “this concept has many variants”)—is erased by the normalization.
4. **Cluster fragmentation:** Very dense clusters may fragment into multiple sub-clusters in the UMAP embedding, as the small σ_i values create fine-grained distinctions that wouldn’t appear in sparser regions.

Practical implications:

- UMAP visualizations are reliable for *neighborhood structure* (which points are near which) but not for *cluster sizes* or *inter-cluster distances*.
- If density information matters, consider alternatives: *t*-SNE with fixed perplexity, or simply examining the raw cosine similarities.
- For quantitative analysis, work in the original high-dimensional space; use UMAP only for qualitative exploration.

Remark 1.47 (On the Riemannian assumption). UMAP assumes the manifold $\mathcal{M}$ is *Riemannian*, meaning it has a positive-definite metric tensor g at each point. This ensures:

- Distances are always non-negative: $d(p, q) \geq 0$ with equality iff $p = q$.
- The triangle inequality holds, making k -NN queries geometrically meaningful.
- Local neighborhoods are topologically balls (no causal structure).

For a **pseudo-Riemannian** (e.g., Minkowski/Lorentzian) manifold with indefinite metric signature (p, q) , fundamental issues arise:

1. **Null distances:** Distinct points can have zero “distance” (null geodesics), breaking the nearest-neighbor concept.
2. **Negative squared intervals:** The “distance” $ds^2 = g_{\mu\nu}dx^\mu dx^\nu$ can be negative (spacelike vs. timelike separation), so $\sqrt{ds^2}$ is not always real.
3. **Causal structure:** Neighborhoods in Minkowski space are cones, not balls. Points within the light cone have timelike separation; points outside have spacelike separation.

Adapting UMAP to pseudo-Riemannian settings would require replacing Euclidean distance with the appropriate interval, handling the causal structure explicitly, and redefining what “neighborhood” means. Recent work on *hyperbolic embeddings* (using the Poincaré disk or hyperboloid model) extends manifold learning to spaces of constant negative curvature, but true Lorentzian manifold learning remains an open problem.

Fuzzy Simplicial Sets

UMAP represents the topological structure of the data using *fuzzy simplicial sets*—a concept from algebraic topology generalized to allow membership values in $[0, 1]$.

Definition 1.48 (Fuzzy graph from data). Given data $\{\mathbf{x}_i\}_{i=1}^N$ with local scales $\{\sigma_i\}$, define a fuzzy graph $G = (V, E, w)$ where:

- $V = \{1, \dots, N\}$ (vertices correspond to data points),
- $E = V \times V$ (all possible edges),
- The edge weight (membership strength) from i to j is:

$$w_{i \rightarrow j} = \exp \left(-\frac{\max(0, d(\mathbf{x}_i, \mathbf{x}_j) - \rho_i)}{\sigma_i} \right). \quad (1.28)$$

Remark 1.49. The weights $w_{i \rightarrow j}$ are generally asymmetric: $w_{i \rightarrow j} \neq w_{j \rightarrow i}$. This reflects the fact that the local geometry around $\mathbf{x}_i$ may differ from that around $\mathbf{x}_j$.

To obtain a symmetric representation, UMAP symmetrizes using a fuzzy set union (probabilistic t -conorm):

$$w_{ij} = w_{i \rightarrow j} + w_{j \rightarrow i} - w_{i \rightarrow j} \cdot w_{j \rightarrow i}. \quad (1.29)$$

This formula comes from the probabilistic interpretation: if $w_{i \rightarrow j}$ is the probability that edge (i, j) exists according to i 's local view, then w_{ij} is the probability it exists in at least one view.

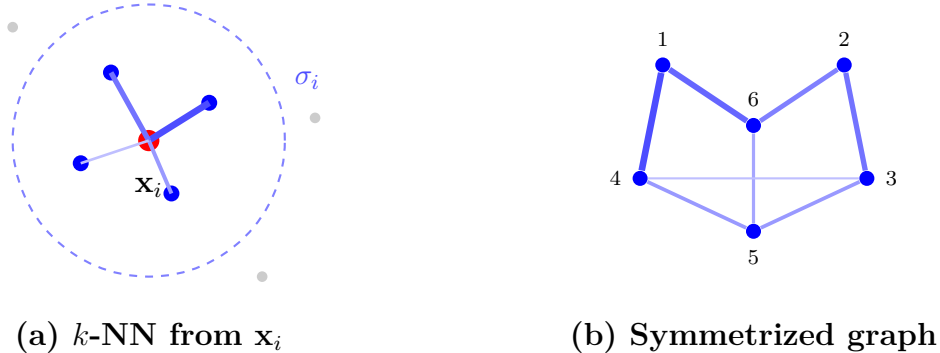


Figure 1.2: Construction of the fuzzy simplicial set. (a) For each point $\mathbf{x}_i$, edges connect to k nearest neighbors with weights $w_{i \rightarrow j}$ that decay exponentially with distance (edge thickness indicates weight). The local scale σ_i normalizes for varying density. (b) The final graph has symmetric weights obtained by fuzzy union: $w_{ij} = w_{i \rightarrow j} + w_{j \rightarrow i} - w_{i \rightarrow j} w_{j \rightarrow i}$.

1.9.3 The Optimization Problem

Given the high-dimensional fuzzy graph $G^{(h)}$ with weights $\{w_{ij}^{(h)}\}$, UMAP seeks a low-dimensional embedding $\mathbf{y}_1, \dots, \mathbf{y}_N \in \mathbb{R}^m$ (typically $m = 2$ or 3) whose fuzzy graph $G^{(\ell)}$ has similar structure.

Low-Dimensional Edge Weights

In the embedding space, edge weights are computed using a Student t -distribution kernel (similar to t -SNE):

$$w_{ij}^{(\ell)} = \left(1 + a\|\mathbf{y}_i - \mathbf{y}_j\|^{2b}\right)^{-1}, \quad (1.30)$$

where $a, b > 0$ are hyperparameters (default: $a \approx 1.93$, $b \approx 0.79$, chosen to approximate a specific target distribution).

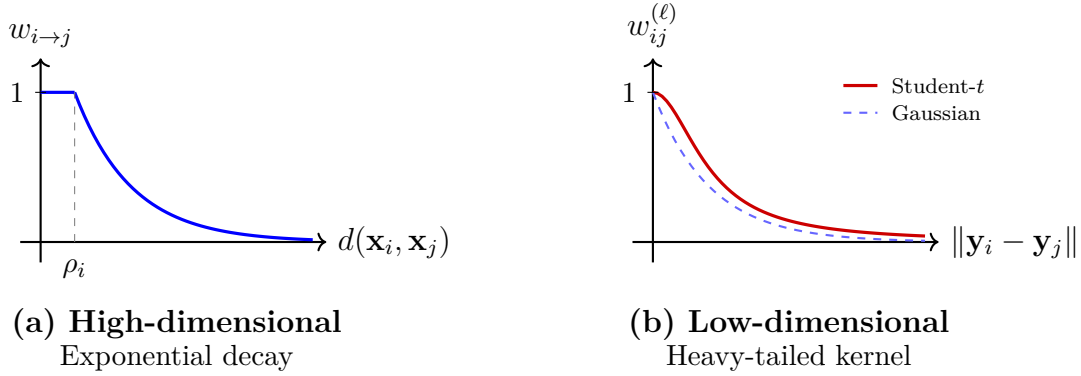


Figure 1.3: Comparison of similarity kernels. (a) In high dimensions, edge weights decay exponentially from the nearest neighbor distance ρ_i . (b) In the embedding, UMAP uses a heavy-tailed Student- t kernel (solid) instead of a Gaussian (dashed). The heavier tails allow moderately distant points to exert repulsive forces, preventing cluster collapse.

Cross-Entropy Loss

The discrepancy between the high-dimensional and low-dimensional fuzzy graphs is measured by fuzzy set cross-entropy:

$$\mathcal{L}(\mathbf{y}_1, \dots, \mathbf{y}_N) = \sum_{i < j} \left[w_{ij}^{(h)} \log \frac{w_{ij}^{(h)}}{w_{ij}^{(\ell)}} + (1 - w_{ij}^{(h)}) \log \frac{1 - w_{ij}^{(h)}}{1 - w_{ij}^{(\ell)}} \right]. \quad (1.31)$$

Remark 1.50 (Interpretation of the loss).

- The first term penalizes when points that are close in high dimensions ($w_{ij}^{(h)}$ large) become far apart in the embedding ($w_{ij}^{(\ell)}$ small). This preserves *local structure*.
- The second term penalizes when points that are far apart in high dimensions ($w_{ij}^{(h)}$ small) become close in the embedding ($w_{ij}^{(\ell)}$ large). This preserves *global structure* and prevents collapse.

Theorem 1.51 (UMAP gradient). *The gradient of the cross-entropy loss (1.31) with respect to embedding point $\mathbf{y}_i$ is*

$$\frac{\partial \mathcal{L}}{\partial \mathbf{y}_i} = \sum_{j \neq i} \left[\underbrace{-2ab w_{ij}^{(h)} \left(w_{ij}^{(\ell)}\right)^2 d_{ij}^{2(b-1)}}_{\text{attractive}} + \underbrace{\frac{2ab (1 - w_{ij}^{(h)}) w_{ij}^{(\ell)} d_{ij}^{2(b-1)}}{1 - w_{ij}^{(\ell)}}}_{\text{repulsive}} \right] (\mathbf{y}_i - \mathbf{y}_j), \quad (1.32)$$

where $d_{ij} = \|\mathbf{y}_i - \mathbf{y}_j\|$ and $w_{ij}^{(\ell)} = (1 + a d_{ij}^{2b})^{-1}$.

Proof. We compute the gradient in three steps.

Step 1: Derivative of the low-dimensional weight. Let $d_{ij} = \|\mathbf{y}_i - \mathbf{y}_j\|$ denote the Euclidean distance. The low-dimensional weight is

$$w_{ij}^{(\ell)} = \frac{1}{1 + a d_{ij}^{2b}}.$$

Taking the derivative with respect to d_{ij}^2 :

$$\frac{\partial w_{ij}^{(\ell)}}{\partial (d_{ij}^2)} = \frac{-ab d_{ij}^{2(b-1)}}{(1 + a d_{ij}^{2b})^2} = -ab d_{ij}^{2(b-1)} \left(w_{ij}^{(\ell)}\right)^2.$$

Since $d_{ij}^2 = \|\mathbf{y}_i - \mathbf{y}_j\|^2$, we have $\frac{\partial (d_{ij}^2)}{\partial \mathbf{y}_i} = 2(\mathbf{y}_i - \mathbf{y}_j)$. By the chain rule:

$$\frac{\partial w_{ij}^{(\ell)}}{\partial \mathbf{y}_i} = -2ab d_{ij}^{2(b-1)} \left(w_{ij}^{(\ell)}\right)^2 (\mathbf{y}_i - \mathbf{y}_j). \quad (1.33)$$

Step 2: Derivative of the cross-entropy terms. The loss decomposes as $\mathcal{L} = \sum_{i < j} \mathcal{L}_{ij}$ where

$$\mathcal{L}_{ij} = w_{ij}^{(h)} \log \frac{w_{ij}^{(h)}}{w_{ij}^{(\ell)}} + (1 - w_{ij}^{(h)}) \log \frac{1 - w_{ij}^{(h)}}{1 - w_{ij}^{(\ell)}}.$$

The high-dimensional weights $w_{ij}^{(h)}$ are constants (computed once from the input data). Differentiating with respect to $w_{ij}^{(\ell)}$:

$$\frac{\partial \mathcal{L}_{ij}}{\partial w_{ij}^{(\ell)}} = \frac{-w_{ij}^{(h)}}{w_{ij}^{(\ell)}} + \frac{1 - w_{ij}^{(h)}}{1 - w_{ij}^{(\ell)}}.$$

Step 3: Combining via chain rule. Applying the chain rule $\frac{\partial \mathcal{L}_{ij}}{\partial \mathbf{y}_i} = \frac{\partial \mathcal{L}_{ij}}{\partial w_{ij}^{(\ell)}} \cdot \frac{\partial w_{ij}^{(\ell)}}{\partial \mathbf{y}_i}$ and substituting (1.33):

$$\begin{aligned} \frac{\partial \mathcal{L}_{ij}}{\partial \mathbf{y}_i} &= \left(\frac{-w_{ij}^{(h)}}{w_{ij}^{(\ell)}} + \frac{1 - w_{ij}^{(h)}}{1 - w_{ij}^{(\ell)}} \right) \cdot \left(-2ab d_{ij}^{2(b-1)} \left(w_{ij}^{(\ell)}\right)^2 (\mathbf{y}_i - \mathbf{y}_j) \right) \\ &= 2ab d_{ij}^{2(b-1)} \left(w_{ij}^{(h)} w_{ij}^{(\ell)} - \frac{(1 - w_{ij}^{(h)}) \left(w_{ij}^{(\ell)}\right)^2}{1 - w_{ij}^{(\ell)}} \right) (\mathbf{y}_i - \mathbf{y}_j). \end{aligned}$$

Rearranging and summing over all $j \neq i$ (accounting for symmetry) yields (1.32). $\square$

Corollary 1.52 (Attractive and repulsive forces). *The gradient update $\mathbf{y}_i \leftarrow \mathbf{y}_i - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{y}_i}$ decomposes into:*

- **Attractive force** from neighbor j : When $w_{ij}^{(h)}$ is large (points are neighbors in high dimensions), the first term dominates, producing a force proportional to $-(\mathbf{y}_i - \mathbf{y}_j)$ that pulls $\mathbf{y}_i$ toward $\mathbf{y}_j$.
- **Repulsive force** from non-neighbor k : When $w_{ik}^{(h)}$ is small, the second term dominates, producing a force proportional to $+(\mathbf{y}_i - \mathbf{y}_k)$ that pushes $\mathbf{y}_i$ away from $\mathbf{y}_k$.

Remark 1.53 (Negative sampling). Computing the full gradient requires summing over all $O(N^2)$ pairs, which is prohibitive for large N . UMAP uses *negative sampling*: for each edge (i, j) with large $w_{ij}^{(h)}$, compute the attractive term exactly, but approximate the repulsive sum by sampling a small number of random “negative” points k with probability proportional to their degree. This reduces complexity to $O(N)$ per iteration while preserving the gradient direction in expectation.

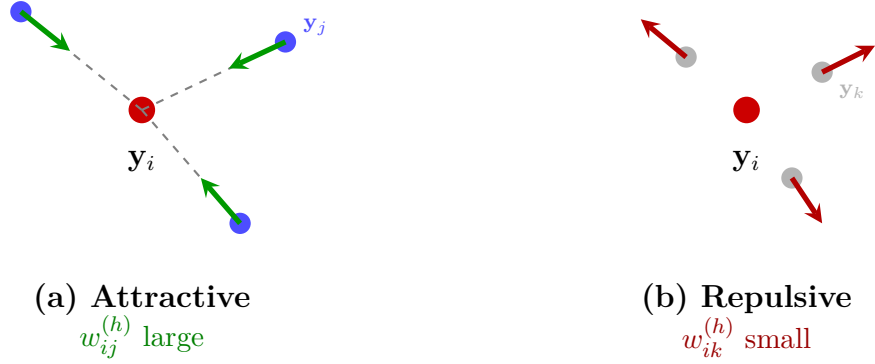


Figure 1.4: Forces during UMAP optimization. (a) Points that are neighbors in high dimensions (large $w_{ij}^{(h)}$, dashed lines) experience attractive forces pulling them together in the embedding. (b) Points that are distant in high dimensions (small $w_{ik}^{(h)}$, sampled as negative examples) experience repulsive forces pushing them apart. The balance preserves local structure while preventing global collapse.

Example 1.54 (Circle vs. Swiss roll: topology matters). A natural question arises: if UMAP “unrolls” the Swiss roll into a rectangle, would it unroll a *circle* S^1 into a line segment?

The answer is **no**. The key difference is *topological*:

- The Swiss roll is homeomorphic to a rectangle $[0, 1] \times [0, 1]$ —it has boundary and is *simply connected* (any loop can be continuously shrunk to a point).
- The circle S^1 has no boundary and is *not simply connected*—a loop around the circle cannot be shrunk to a point without leaving the manifold.

UMAP preserves local neighborhoods via the fuzzy simplicial set construction. For a circle, each point $\mathbf{x}_i$ has neighbors on both sides, forming a chain that wraps around:

$$\mathbf{x}_1 \leftrightarrow \mathbf{x}_2 \leftrightarrow \cdots \leftrightarrow \mathbf{x}_N \leftrightarrow \mathbf{x}_1.$$

To “unroll” this into a line segment $[a, b]$, UMAP would need to *cut* the circle somewhere, separating two adjacent points $\mathbf{x}_i$ and $\mathbf{x}_{i+1}$ that have high edge weight $w_{i,i+1}^{(h)}$. This would incur a large attractive penalty in the cross-entropy loss (1.31).

Instead, UMAP embeds the circle as a closed curve in $\mathbb{R}^2$ —preserving both the local metric structure (nearby points stay nearby) and the global topology (the loop structure is maintained).

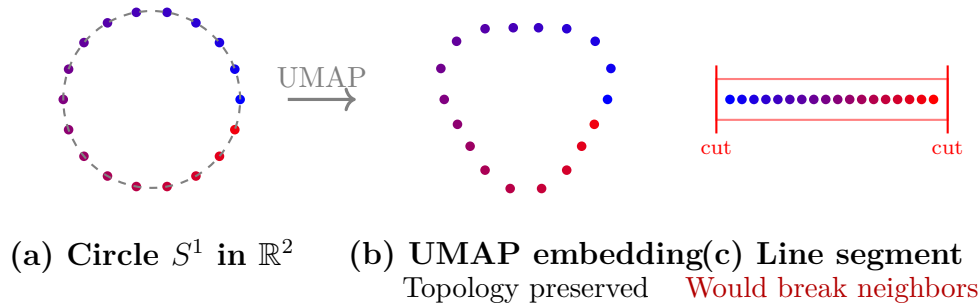


Figure 1.5: UMAP preserves topology. (a) Points sampled uniformly from a circle, colored by angle. (b) UMAP embeds this as a closed curve, preserving the cyclic neighbor structure. (c) Unrolling to a line segment would require cutting the circle, separating neighbors that have high affinity $w_{ij}^{(h)}$ —UMAP’s loss function penalizes this.

1.9.4 Algorithm Summary

1. **Construct k -NN graph:** For each $\mathbf{x}_i$, find its k nearest neighbors.
2. **Compute local scales:** Solve (1.27) for each σ_i using binary search.
3. **Build fuzzy graph:** Compute directed weights (1.28) and symmetrize via (1.29).
4. **Initialize embedding:** Use spectral embedding (eigenvectors of graph Laplacian) or random initialization.
5. **Optimize:** Minimize cross-entropy (1.31) via SGD with negative sampling.

1.9.5 Comparison with PCA

Property	PCA	UMAP
Structure captured	Linear (subspaces)	Nonlinear (manifolds)
Objective	Maximize variance	Preserve fuzzy topology
Computational complexity	$O(d^2N + d^3)$	$O(N^{1.14})$ empirically
Deterministic	Yes	No (stochastic optimization)
Out-of-sample extension	Simple ($\mathbf{W}^\top \mathbf{x}$)	Requires approximation
Interpretability	Principal components	Less interpretable

1.9.6 Applications

Single-Cell RNA Sequencing

In computational biology, single-cell RNA-seq produces expression profiles for thousands of genes across thousands of cells. UMAP is the standard tool for visualizing this data, revealing cell types, developmental trajectories, and rare cell populations that PCA would fail to separate.

Natural Language Processing

Word embeddings (e.g., Word2Vec, GloVe) live in high-dimensional spaces where semantic relationships are encoded as directions. UMAP can project these to 2D for visualization, revealing clusters of semantically related words while preserving analogical structure.

Image Embeddings

Deep neural networks produce high-dimensional feature vectors for images. UMAP visualization of these embeddings reveals how the network organizes visual concepts, useful for debugging classifiers and understanding learned representations.

Text Embeddings on the Unit Sphere

Modern text embedding models (OpenAI’s `text-embedding-ada-002`, Cohere, Voyage, etc.) produce high-dimensional vectors $\mathbf{v} \in \mathbb{R}^d$ (typically $d = 1536$ or $d = 3072$) that are L^2 -normalized to lie on the unit sphere:

$$\|\mathbf{v}\| = 1, \quad \text{so } \mathbf{v} \in S^{d-1}.$$

For these embeddings, similarity is measured by *cosine similarity*, which on the unit sphere equals the dot product: $\cos \theta = \mathbf{v}_i^\top \mathbf{v}_j$.

Remark 1.55 (Practical considerations for UMAP on text embeddings).

1. **Use cosine or angular distance:** Since the embeddings lie on S^{d-1} , Euclidean distance in $\mathbb{R}^d$ is a monotonic transformation of cosine similarity:

$$\|\mathbf{v}_i - \mathbf{v}_j\|^2 = 2(1 - \mathbf{v}_i^\top \mathbf{v}_j) = 2(1 - \cos \theta_{ij}).$$

UMAP with `metric='cosine'` or `metric='euclidean'` will produce similar results on normalized vectors, but cosine is semantically clearer.

2. **Beware of density distortion:** Text corpora are fundamentally non-uniform—common topics (sports, politics) have dense clusters while niche topics (15th-century Flemish art) are sparse. Per Remark 1.46:
 - Cluster *sizes* in the UMAP plot do not reflect true cluster sizes in the corpus.
 - A cluster of 10,000 news articles about politics may appear similar in size to 100 articles about a rare disease.

- Inter-cluster distances are unreliable for comparing semantic similarity across density regimes.

3. **What UMAP *does* preserve:** Despite density distortion, UMAP reliably preserves:

- *Neighborhood membership:* Documents that are semantically similar will be placed near each other.
- *Cluster separation:* Distinct topics will form visually separate clusters.
- *Relative structure within density-homogeneous regions:* Within a single cluster, local structure is meaningful.

4. **Recommended workflow:**

- Use UMAP for *exploration*: identifying clusters, finding outliers, understanding corpus structure.
- For *quantitative analysis* (e.g., “how similar are these two documents?”), use cosine similarity directly in the original $\mathbb{R}^d$ space.
- To compare cluster sizes or densities, compute statistics in the original space, not from the UMAP coordinates.
- Consider coloring points by metadata (date, source, label) to interpret clusters, since position alone can be misleading.

5. **Hyperparameter guidance:**

- **n_neighbors:** Larger values (e.g., 30–50) preserve more global structure; smaller values (e.g., 5–15) emphasize local clusters. For text embeddings with many fine-grained topics, start with 15.
- **min_dist:** Controls how tightly points cluster. For visualization, 0.1 works well; for preserving more structure, try 0.0.
- **n_components:** Usually 2 for visualization; 3 for interactive 3D plots.

Part II

Statistics

Chapter 2

Random Variables and Distributions

2.1 Probability Measure

Definition 2.1 (Probability Space). A **probability space** is a triple $(\Omega, \mathcal{F}, P)$ where:

- Ω is the **sample space**—the set of all possible outcomes.
- $\mathcal{F}$ is a **σ -algebra** (or σ -field) on Ω —a collection of subsets of Ω (called *events*) that is closed under complementation and countable unions.
- $P : \mathcal{F} \rightarrow [0, 1]$ is a **probability measure**—a function assigning probabilities to events.

Definition 2.2 (Probability Measure). A function $P : \mathcal{F} \rightarrow [0, 1]$ is a **probability measure** if it satisfies the **Kolmogorov axioms**:

1. **Non-negativity**: $P(A) \geq 0$ for all $A \in \mathcal{F}$.
2. **Normalization**: $P(\Omega) = 1$.
3. **Countable additivity**: For any countable collection of pairwise disjoint events $\{A_i\}_{i=1}^{\infty}$,

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i).$$

Definition 2.3 (Random Variable). A **random variable** X is a measurable function $X : \Omega \rightarrow \mathbb{R}$. That is, for every Borel set $B \subseteq \mathbb{R}$, the preimage $X^{-1}(B) = \{\omega \in \Omega : X(\omega) \in B\}$ belongs to $\mathcal{F}$.

The **distribution** (or **law**) of X is the probability measure P_X on $\mathbb{R}$ defined by:

$$P_X(B) = P(X \in B) = P(\{\omega : X(\omega) \in B\}).$$

Remark 2.4 (Why This Matters). The formal framework ensures:

- Events are well-defined sets we can assign probabilities to.
- Probabilities are consistent: $P(A) + P(A^c) = 1$, probabilities of disjoint events add up.

- Random variables connect abstract outcomes ω to numerical values we can compute with.

In practice, we often work directly with distributions $p(x)$ without explicitly constructing $(\Omega, \mathcal{F}, P)$.

2.2 Expectation

Definition 2.5 (Expected Value). The expected value (or mean) of a random variable X is:

- **Discrete case:** If X takes values $\{x_1, x_2, \dots\}$ with probabilities $P(X = x_i)$,

$$\mathbb{E}[X] = \sum_i x_i P(X = x_i).$$

- **Continuous case:** If X has probability density function $p(x)$,

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x p(x) dx.$$

For a function $f(X)$ of a random variable:

$$\mathbb{E}[f(X)] = \sum_i f(x_i) P(X = x_i) \quad \text{or} \quad \mathbb{E}[f(X)] = \int f(x) p(x) dx.$$

Remark 2.6 (Subscript Notation for Expectations). The subscript on $\mathbb{E}$ specifies *which* random variable(s) we are averaging over and *what distribution* they follow.

Key distinction:

- $\pi(a|s)$ is a **number** — the probability of action a given state s .
- $\pi(\cdot|s)$ is a **distribution** over actions (the dot marks where the random variable goes).

Notation meanings:

- $\mathbb{E}_{x \sim p}[f(x)]$ means “average $f(x)$ where x is drawn from distribution p .” This is just:

$$\mathbb{E}_{x \sim p}[f(x)] = \sum_x p(x) \cdot f(x) \quad \text{or} \quad \int p(x) \cdot f(x) dx.$$

- $\mathbb{E}_{a \sim \pi(\cdot|s)}[f(a)]$ means “average $f(a)$ over the distribution $\pi(\cdot|s)$ ”:

$$\mathbb{E}_{a \sim \pi(\cdot|s)}[f(a)] = \sum_{a \in \mathcal{A}} \pi(a|s) \cdot f(a).$$

Here $\pi(\cdot|s)$ is the distribution, and $\pi(a|s)$ gives the weight for each action a .

- $\mathbb{E}_\pi[\cdot]$ is shorthand for “expectation over all randomness from following policy π ”:

1. At each state s_t , sample action: $a_t \sim \pi(\cdot|s_t)$
2. Environment gives next state: $s_{t+1} \sim P(\cdot|s_t, a_t)$

So $\mathbb{E}_\pi[\sum_t \gamma^t r_t]$ averages over all possible trajectories.

Example 2.7. Suppose we have 3 possible actions: $\mathcal{A} = \{\text{left}, \text{stay}, \text{right}\}$.

The policy $\pi(\cdot|s)$ assigns a probability to each action:

a	left	stay	right
$\pi(a s)$	0.2	0.3	0.5

Suppose $f(a)$ gives some value for each action:

a	left	stay	right
$f(a)$	10	4	8

Then $\mathbb{E}_{a \sim \pi(\cdot|s)}[f(a)]$ is just a weighted average:

$$\begin{aligned}
 \mathbb{E}_{a \sim \pi(\cdot|s)}[f(a)] &= \sum_{a \in \mathcal{A}} \pi(a|s) \cdot f(a) \\
 &= \pi(\text{left}|s) \cdot f(\text{left}) + \pi(\text{stay}|s) \cdot f(\text{stay}) + \pi(\text{right}|s) \cdot f(\text{right}) \\
 &= 0.2 \times 10 + 0.3 \times 4 + 0.5 \times 8 \\
 &= 2 + 1.2 + 4 = 7.2.
 \end{aligned}$$

The subscript $a \sim \pi(\cdot|s)$ just says: “sum over all a , weighted by $\pi(a|s)$.”

Remark 2.8 (Conditional Expectation). The notation $\mathbb{E}[X|Y = y]$ means the expected value of X given that Y takes value y :

$$\mathbb{E}[X|Y = y] = \sum_x x P(X = x|Y = y) = \int x p(x|y) dx.$$

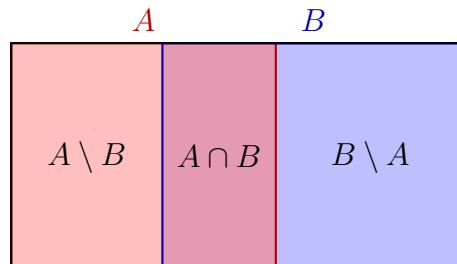
In reinforcement learning, $\mathbb{E}_\pi[\cdot|s_0 = s]$ means “expectation over trajectories, given we start in state s .”

2.3 Conditional Probability and Bayes’ Theorem

Definition 2.9 (Conditional Probability). The probability of event A given that event B has occurred is:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, \quad \text{provided } P(B) > 0.$$

Think of probability as area. The rectangle below has total area 1:

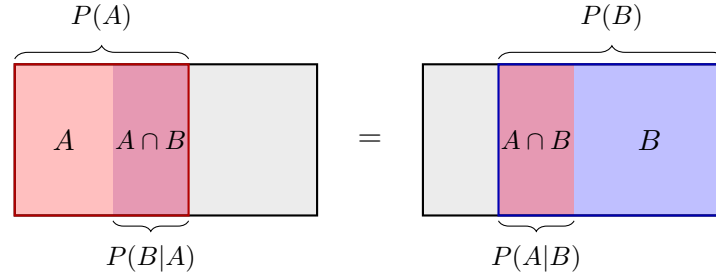


$P(A|B)$ asks: *given we're in B*, what fraction is in A ? That's $\frac{\text{purple}}{\text{purple} + \text{blue}} = \frac{P(A \cap B)}{P(B)}$.

Theorem 2.10 (Bayes' Theorem).

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}.$$

Proof. The purple region $P(A \cap B)$ can be computed two ways:



$$P(A) \cdot P(B|A) = P(A \cap B) = P(B) \cdot P(A|B)$$

Solving for $P(A|B)$ gives the result. □

Remark 2.11 (Intuition). Bayes' theorem lets you *reverse* the conditioning:

- $P(B|A)$ = probability of B given A (often easy to know)
- $P(A|B)$ = probability of A given B (often what we want)

Example: A = “has disease”, B = “test positive”.

- $P(B|A)$ = sensitivity of test (known from test design)
- $P(A|B)$ = chance patient has disease given positive test (what doctor needs)

Proposition 2.12 (Law of Total Probability). *If $\{A_1, A_2, \dots, A_n\}$ partition the sample space (disjoint, cover everything), then:*

$$P(B) = \sum_{i=1}^n P(B|A_i) \cdot P(A_i).$$

Proof. Since the A_i partition the space, B is the disjoint union $B = (B \cap A_1) \cup (B \cap A_2) \cup \dots \cup (B \cap A_n)$. Therefore:

$$P(B) = \sum_{i=1}^n P(B \cap A_i) = \sum_{i=1}^n P(B|A_i) \cdot P(A_i). \quad \square$$

Corollary 2.13 (Extended Bayes' Theorem). *Combining Bayes' theorem with the law of total probability:*

$$P(A_i|B) = \frac{P(B|A_i) \cdot P(A_i)}{\sum_{j=1}^n P(B|A_j) \cdot P(A_j)}.$$

2.4 Variance and the Central Limit Theorem

2.4.1 Variance and Standard Deviation

Definition 2.14 (Variance). The **variance** of a random variable X measures how spread out its values are around the mean:

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

The **standard deviation** is $\sigma = \sqrt{\text{Var}(X)}$, which has the same units as X .

Proposition 2.15 (Properties of Variance). *For random variables X, Y and constants a, b :*

1. $\text{Var}(aX + b) = a^2 \text{Var}(X)$ (shifting doesn't change spread; scaling squares).
2. If X and Y are independent: $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$.
3. For n i.i.d. copies $X_1, \dots, X_n$ with variance σ^2 :

$$\text{Var}\left(\frac{1}{n} \sum_{i=1}^n X_i\right) = \frac{\sigma^2}{n}.$$

Proof. For property 3: Let $\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$. By independence:

$$\text{Var}(\bar{X}) = \text{Var}\left(\frac{1}{n} \sum_{i=1}^n X_i\right) = \frac{1}{n^2} \sum_{i=1}^n \text{Var}(X_i) = \frac{1}{n^2} \cdot n\sigma^2 = \frac{\sigma^2}{n}. \quad \square$$

Remark 2.16. The variance of the sample mean decreases as $1/n$. This is why averaging more samples gives more reliable estimates—the standard deviation shrinks as $1/\sqrt{n}$.

2.4.2 The Central Limit Theorem

Theorem 2.17 (Central Limit Theorem). *Let $X_1, X_2, \dots, X_n$ be i.i.d. random variables with mean μ and finite variance σ^2 . Define the sample mean:*

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i.$$

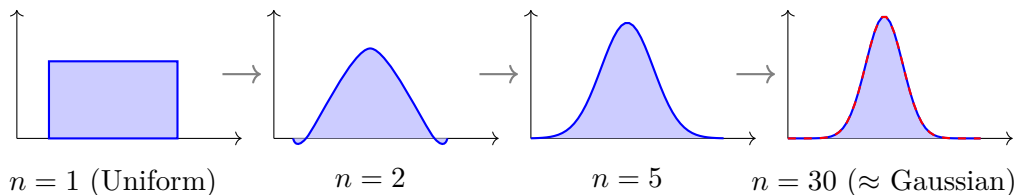
Then the standardized sample mean converges in distribution to a standard normal:

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \mathcal{N}(0, 1) \quad \text{as } n \rightarrow \infty.$$

Equivalently, for large n :

$$\bar{X}_n \approx \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right).$$

Remark 2.18 (What Makes CLT Remarkable). The CLT says that *regardless* of the original distribution of the X_i —whether discrete, continuous, skewed, or multimodal—the sum (or average) of many independent copies becomes approximately Gaussian. The only requirements are finite mean and variance.



Example 2.19 (Dice Rolls). A single die roll is uniform on $\{1, 2, 3, 4, 5, 6\}$ with $\mu = 3.5$ and $\sigma^2 = 35/12 \approx 2.92$.

If you roll $n = 100$ dice and compute the average, the CLT predicts:

$$\bar{X}_{100} \approx \mathcal{N}\left(3.5, \frac{2.92}{100}\right) = \mathcal{N}(3.5, 0.0292).$$

So $\bar{X}_{100}$ has standard deviation $\sqrt{0.0292} \approx 0.17$. With 95% probability, the average lies in $3.5 \pm 1.96 \times 0.17 \approx [3.17, 3.83]$.

2.4.3 Why CLT Matters in Machine Learning

The Central Limit Theorem is foundational to many ML techniques:

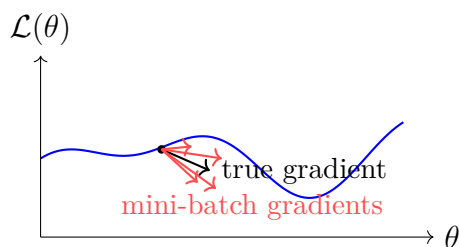
Remark 2.20 (Stochastic Gradient Descent). In SGD, we estimate the true gradient $\nabla \mathcal{L}(\theta)$ using a mini-batch average:

$$\hat{g} = \frac{1}{B} \sum_{i=1}^B \nabla \ell(x_i, \theta).$$

By CLT, this estimate is approximately Gaussian around the true gradient:

$$\hat{g} \approx \mathcal{N}\left(\nabla \mathcal{L}(\theta), \frac{\Sigma}{B}\right),$$

where Σ is the covariance of individual gradients. Larger batches reduce variance by $1/B$, but smaller batches provide noise that can help escape local minima.



Mini-batch gradients scatter around true gradient
CLT: distribution is approximately Gaussian

Remark 2.21 (Confidence Intervals for Model Evaluation). When evaluating a model on a test set of n examples, the accuracy $\hat{p}$ is a sample mean of Bernoulli variables. By CLT:

$$\hat{p} \approx \mathcal{N}\left(p, \frac{p(1-p)}{n}\right).$$

A 95% confidence interval is approximately:

$$\hat{p} \pm 1.96\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}.$$

This tells us how much to trust reported accuracies. With $n = 100$ and $\hat{p} = 0.90$, the 95% CI is $[0.84, 0.96]$ —substantial uncertainty!

Example 2.22 (Sample Size for Reliable Evaluation). How many test examples do we need to distinguish 90% accuracy from 92% accuracy with 95% confidence?

The standard error at $p \approx 0.91$ is $\sqrt{0.91 \times 0.09/n} \approx 0.29/\sqrt{n}$.

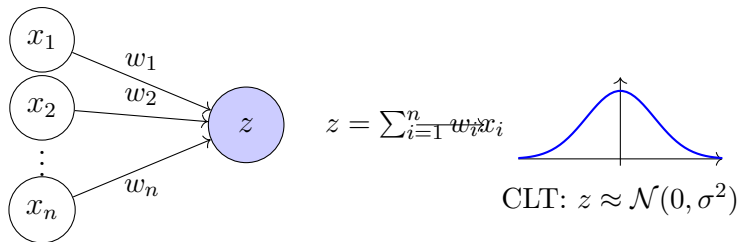
For the 95% CIs to not overlap, we need roughly $1.96 \times 0.29/\sqrt{n} < 0.01$, giving $n > 3200$.

This explains why ML benchmarks use large test sets—small differences are statistically meaningless without enough data.

Remark 2.23 (Bootstrap and Ensemble Methods). The CLT justifies **bootstrap** methods: if we resample our dataset many times and compute a statistic each time, the distribution of that statistic is approximately Gaussian. This allows uncertainty quantification without distributional assumptions.

For **ensemble methods**, averaging k independent model predictions reduces variance by $1/k$. If individual model errors are approximately i.i.d., the ensemble error is approximately Gaussian with smaller variance.

Remark 2.24 (Weight Initialization). In neural networks, each neuron computes $z = \sum_{i=1}^n w_i x_i$. If weights and inputs are independent with zero mean, CLT suggests z is approximately Gaussian. This motivates initializations like Xavier/Glorot ($\text{Var}(w) = 1/n$) to keep activations well-scaled throughout the network.



Remark 2.25 (When CLT Fails). The CLT requires finite variance. Heavy-tailed distributions (e.g., Cauchy, some power laws) violate this. In ML:

- Gradient clipping addresses occasional huge gradients that would violate CLT assumptions.
- Robust statistics (median, trimmed mean) handle outliers that CLT-based methods struggle with.

2.5 Binary Classification Metrics

Definition 2.26 (Confusion Matrix Terminology). For binary classification, let the positive class be 1 and the negative class be 0.

- **True Positive (TP)**: predicted 1, actual 1.
- **False Positive (FP)**: predicted 1, actual 0 (“type I error”).
- **True Negative (TN)**: predicted 0, actual 0.
- **False Negative (FN)**: predicted 0, actual 1 (“type II error”).

		Actual	
		Positive (1)	Negative (0)
Predicted	Positive (1)	TP	FP
	Negative (0)	FN	TN

Definition 2.27 (Precision, Recall, Accuracy, F_1). Using TP/FP/TN/FN from the confusion matrix:

$$\begin{aligned}
 \text{Precision} &= \frac{\text{TP}}{\text{TP} + \text{FP}} && \text{(how many predicted positives were correct)} \\
 \text{Recall} &= \frac{\text{TP}}{\text{TP} + \text{FN}} && \text{(how many actual positives were found)} \\
 \text{Accuracy} &= \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}} && \text{(overall correctness)} \\
 F_1 &= 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} && \text{(harmonic mean of precision and recall).}
 \end{aligned}$$

Example 2.28 (Cancer Screening: Bayes Meets Classification Metrics). A cancer screening test has these properties:

- If you have cancer, the test is positive 99% of the time: $P(\text{pos} \mid \text{cancer}) = 0.99$.
- If you are healthy, the test is (wrongly) positive 5% of the time: $P(\text{pos} \mid \text{healthy}) = 0.05$.
- Only 1% of the population has cancer: $P(\text{cancer}) = 0.01$.

Question: If you test positive, what is the probability you actually have cancer?

Step 1: Apply Bayes’ theorem.

$$P(\text{cancer} \mid \text{pos}) = \frac{P(\text{pos} \mid \text{cancer}) \cdot P(\text{cancer})}{P(\text{pos})}.$$

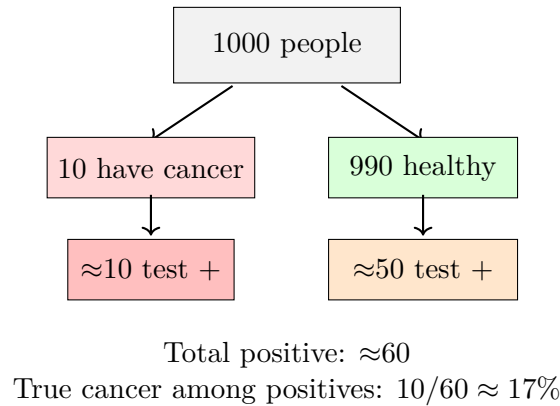
First compute $P(\text{pos})$ using the law of total probability:

$$\begin{aligned} P(\text{pos}) &= P(\text{pos} \mid \text{cancer}) \cdot P(\text{cancer}) + P(\text{pos} \mid \text{healthy}) \cdot P(\text{healthy}) \\ &= 0.99 \times 0.01 + 0.05 \times 0.99 = 0.0099 + 0.0495 = 0.0594. \end{aligned}$$

Now apply Bayes:

$$P(\text{cancer} \mid \text{pos}) = \frac{0.99 \times 0.01}{0.0594} = \frac{0.0099}{0.0594} \approx 0.167.$$

Step 2: Visualize with 1000 people.



The 50 false positives from the large healthy population overwhelm the 10 true positives. This is the **base rate fallacy**.

Step 3: Build the confusion matrix.

$$\text{TP} = 10, \quad \text{FP} = 50, \quad \text{FN} = 0, \quad \text{TN} = 940.$$

Step 4: Compute classification metrics.

$$\text{Precision} = \frac{10}{10 + 50} = \frac{10}{60} \approx 0.17 \quad (\text{only } 17\% \text{ of positive tests are true cases})$$

$$\text{Recall} = \frac{10}{10 + 0} = 1.0 \quad (\text{test catches } 100\% \text{ of cancer cases})$$

$$\text{Accuracy} = \frac{10 + 940}{1000} = 0.95 \quad (95\% \text{ overall correct})$$

$$F_1 = 2 \cdot \frac{0.17 \times 1.0}{0.17 + 1.0} \approx 0.29$$

Key insights:

- **Precision = Bayes' answer:** $\text{Precision} = P(\text{cancer} \mid \text{positive}) = 0.17$.
- **Recall = test detection rate:** $\text{Recall} = P(\text{positive} \mid \text{cancer}) = 0.99$.
- **The accuracy paradox:** 95% accuracy sounds good, but a classifier that *always* predicts “healthy” achieves 99% accuracy! Accuracy is misleading for imbalanced classes.

- **F_1 tells the truth:** The low $F_1 = 0.29$ reveals that despite high recall, the poor precision makes this test problematic for diagnosis (too many false alarms).

Remark 2.29 (Beyond Binary). For multiclass problems with K classes, build a confusion matrix with one row/column per class. For each class k , define:

- TP_k : correctly predicted as class k
- FP_k : predicted k but actually another class
- FN_k : actually class k but predicted as another

Then per-class precision and recall are:

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}, \quad \text{Recall}_k = \frac{TP_k}{TP_k + FN_k}.$$

Aggregation strategies:

$$\begin{aligned} \text{Macro-Precision} &= \frac{1}{K} \sum_{k=1}^K \text{Precision}_k, & \text{Macro-Recall} &= \frac{1}{K} \sum_{k=1}^K \text{Recall}_k, \\ \text{Micro-Precision} &= \frac{\sum_k TP_k}{\sum_k (TP_k + FP_k)}, & \text{Micro-Recall} &= \frac{\sum_k TP_k}{\sum_k (TP_k + FN_k)}. \end{aligned}$$

Macro averages weight classes equally; micro averages weight by support (frequency).

Example 2.30 (Emergency Triage Classification). A hospital triage model classifies patients into three urgency levels: **Critical** (immediate care), **Moderate** (urgent but stable), and **Normal** (routine). The confusion matrix:

Predicted	Actual			Total Pred.
	Critical	Moderate	Normal	
Critical	45	5	2	52
Moderate	10	60	15	85
Normal	5	15	180	200
Total Actual	60	80	197	337

Per-class metrics:

$$\begin{aligned} \text{Critical:} \quad & \text{Precision} = \frac{45}{52} \approx 0.87, \quad \text{Recall} = \frac{45}{60} = 0.75 \\ \text{Moderate:} \quad & \text{Precision} = \frac{60}{85} \approx 0.71, \quad \text{Recall} = \frac{60}{80} = 0.75 \\ \text{Normal:} \quad & \text{Precision} = \frac{180}{200} = 0.90, \quad \text{Recall} = \frac{180}{197} \approx 0.91 \end{aligned}$$

Aggregated metrics:

$$\text{Macro-Precision} = \frac{0.87+0.71+0.90}{3} \approx 0.83, \quad \text{Macro-Recall} = \frac{0.75+0.75+0.91}{3} \approx 0.80$$

$$\text{Micro-Precision} = \frac{45+60+180}{52+85+200} = \frac{285}{337} \approx 0.85 = \text{Micro-Recall}$$

The model has high precision for Critical (few false alarms for emergencies) but lower recall (15 critical patients were under-triaged). In healthcare, under-triaging critical patients is dangerous—optimizing for Critical recall is essential.

Remark 2.31 (Analogy: High Recall vs High Precision). Think of medical screening:

- **High recall**, lower precision: a sensitive screening test that catches almost everyone who is sick (few FN) but sometimes flags healthy people (more FP). Good for early detection; follow-up tests disambiguate.
- **High precision**, lower recall: a conservative confirmatory test that rarely cries wolf (few FP) but may miss some cases (more FN). Good when false alarms are costly.

In information retrieval: high recall surfaces many documents (including marginal ones), while high precision shows only the most relevant few.

2.6 Information Theory and KL Divergence

2.6.1 Convexity and Jensen's Inequality

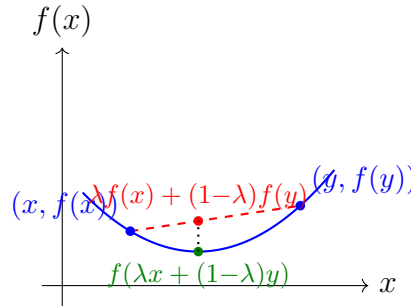
Definition 2.32 (Convex Function). A function $f : \mathbb{R} \rightarrow \mathbb{R}$ is **convex** if for all $x, y \in \mathbb{R}$ and $\lambda \in [0, 1]$:

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y).$$

Geometrically, the line segment connecting $(x, f(x))$ and $(y, f(y))$ lies above the graph of f .

A function is **strictly convex** if the inequality is strict for $\lambda \in (0, 1)$ and $x \neq y$.

A function is **concave** if $-f$ is convex (inequality reversed).



Example 2.33 (Common Convex and Concave Functions).

- **Convex:** x^2 , e^x , $|x|$, $x \log x$ (for $x > 0$), $-\log x$ (for $x > 0$)
- **Concave:** $\log x$ (for $x > 0$), $\sqrt{x}$ (for $x \geq 0$), $-x^2$

A twice-differentiable function is convex iff $f''(x) \geq 0$ for all x .

Theorem 2.34 (Jensen's Inequality). Let f be a convex function and X a random variable. Then:

$$f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)].$$

If f is concave, the inequality reverses: $f(\mathbb{E}[X]) \geq \mathbb{E}[f(X)]$.

Proof. For a convex function, there exists a supporting line at any point. At $\mu = \mathbb{E}[X]$, let $\ell(x) = f(\mu) + f'(\mu)(x - \mu)$ be a tangent line. By convexity, $f(x) \geq \ell(x)$ for all x . Taking expectations:

$$\mathbb{E}[f(X)] \geq \mathbb{E}[\ell(X)] = f(\mu) + f'(\mu)\mathbb{E}[X - \mu] = f(\mu) = f(\mathbb{E}[X]). \quad \square$$

Example 2.35 (Applying Jensen's Inequality). Since $-\log$ is convex (as $\frac{d^2}{dx^2}(-\log x) = \frac{1}{x^2} > 0$):

$$-\log(\mathbb{E}[X]) \leq \mathbb{E}[-\log X], \quad \text{equivalently} \quad \log(\mathbb{E}[X]) \geq \mathbb{E}[\log X].$$

This is the **log-sum inequality**, used in proving KL divergence is non-negative.

Definition 2.36 (Proper Scoring Rule). A **scoring rule** $S(p, y)$ measures the quality of a probabilistic forecast p given outcome y . Lower scores indicate better predictions.

A scoring rule is **proper** if a forecaster minimizes their expected score by reporting their true belief q :

$$\mathbb{E}_{y \sim q}[S(q, y)] \leq \mathbb{E}_{y \sim q}[S(p, y)] \quad \text{for all } p.$$

It is **strictly proper** if equality holds only when $p = q$ —reporting anything other than your true belief strictly increases your expected score.

Remark 2.37 (Why Strict Properness Matters). Strictly proper scoring rules incentivize honest probability reporting. If a rule is not strictly proper, a forecaster might game the system by reporting something other than their true belief without penalty.

Example 2.38 (Brier Score and Ensemble Averaging). The **Brier score** is a strictly proper scoring rule. For predicted probability p and outcome $y \in \{0, 1\}$:

$$\text{Brier}(p, y) = (p - y)^2.$$

This is convex in p since $\frac{d^2}{dp^2}(p - y)^2 = 2 > 0$.

Proof of strict properness: Let the true probability be $q = P(Y = 1)$. The expected Brier score when reporting p is:

$$\begin{aligned} \mathbb{E}_{y \sim q}[(p - y)^2] &= q(p - 1)^2 + (1 - q)(p - 0)^2 \\ &= q(1 - 2p + p^2) + (1 - q)p^2 \\ &= q - 2pq + p^2 = (p - q)^2 + q(1 - q). \end{aligned}$$

Since $q(1 - q)$ is constant, this is minimized uniquely at $p = q$.

Application: Suppose k models predict probabilities $p_1, \dots, p_k$ for the same event. Let P be a random variable that equals p_i with probability $1/k$ (i.e., pick a model uniformly at random). Then $\mathbb{E}[P] = \bar{p}$.

Since $f(x) = (x - y)^2$ is convex, Jensen's inequality gives:

$$f(\mathbb{E}[P]) \leq \mathbb{E}[f(P)] \implies (\mathbb{E}[P] - y)^2 \leq \mathbb{E}[(P - y)^2].$$

Substituting:

$$\underbrace{(\bar{p} - y)^2}_{\text{ensemble Brier}} \leq \underbrace{\frac{1}{k} \sum_{i=1}^k (p_i - y)^2}_{\text{mean of individual Briers}}.$$

The ensemble's Brier score is at most the mean of the individual Brier scores. This explains why *averaging predictions improves calibration*—the ensemble is at least as good as a randomly chosen model (in expectation), and strictly better when models disagree.

2.6.2 Entropy: Measuring Uncertainty

Definition 2.39 (Entropy). The entropy of a discrete random variable X with distribution p is:

$$H(X) = H(p) = - \sum_x p(x) \log p(x) = \mathbb{E}_{x \sim p} [-\log p(x)].$$

For continuous distributions, the *differential entropy* is:

$$h(X) = - \int p(x) \log p(x) dx.$$

We use log base 2 for bits, or natural log for nats.

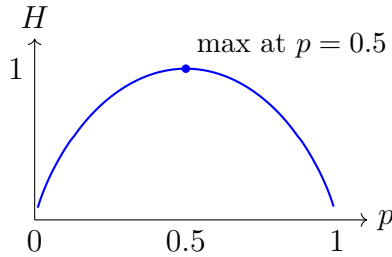
Remark 2.40 (Intuition: Entropy as Expected Surprise). The quantity $-\log p(x)$ measures the “surprise” of observing outcome x :

- If $p(x) = 1$ (certain): surprise $= -\log 1 = 0$
- If $p(x) = 0.5$: surprise $= -\log 0.5 = 1$ bit
- If $p(x) = 0.01$ (rare): surprise $= -\log 0.01 \approx 6.6$ bits

Entropy is the *expected surprise*—how uncertain we are on average.

Example 2.41 (Coin Flips). For a biased coin with $P(\text{heads}) = p$:

$$H = -p \log p - (1 - p) \log(1 - p).$$



Maximum entropy ($H = 1$ bit) occurs at $p = 0.5$ (most uncertain). At $p = 0$ or $p = 1$, entropy is 0 (no uncertainty).

2.6.3 KL Divergence: Measuring Distribution Difference

Definition 2.42 (Kullback-Leibler Divergence). The KL divergence from distribution q to distribution p is:

$$D_{\text{KL}}(p||q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right].$$

Equivalently:

$$D_{\text{KL}}(p||q) = \mathbb{E}_{x \sim p} [\log p(x)] - \mathbb{E}_{x \sim p} [\log q(x)] = -H(p) - \mathbb{E}_{x \sim p} [\log q(x)].$$

Remark 2.43 (Reading the Notation). $D_{\text{KL}}(p||q)$ reads as “KL divergence *from* q *to* p ” or “of p relative to q .” Think of it as: “How much information do I lose if I use q to approximate p ?”

Proposition 2.44 (Properties of KL Divergence). 1. **Non-negativity:** $D_{\text{KL}}(p||q) \geq 0$, with equality iff $p = q$.

2. **Not symmetric:** $D_{\text{KL}}(p||q) \neq D_{\text{KL}}(q||p)$ in general.

3. **Not a metric:** Does not satisfy the triangle inequality.

Proof of non-negativity. By Jensen’s inequality, since $-\log$ is convex:

$$D_{\text{KL}}(p||q) = \mathbb{E}_{x \sim p} \left[-\log \frac{q(x)}{p(x)} \right] \geq -\log \mathbb{E}_{x \sim p} \left[\frac{q(x)}{p(x)} \right] = -\log \sum_x p(x) \cdot \frac{q(x)}{p(x)} = -\log 1 = 0.$$

Equality holds iff $q(x)/p(x)$ is constant, i.e., $p = q$. □

Remark 2.45 (Information-Theoretic Interpretation). Define the *cross-entropy* from q to p :

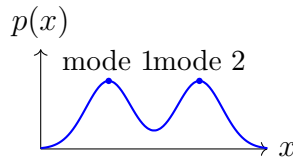
$$H(p, q) = -\mathbb{E}_{x \sim p} [\log q(x)] = -\sum_x p(x) \log q(x).$$

This is the expected number of bits needed to encode samples from p using a code optimized for q . Then:

$$D_{\text{KL}}(p||q) = H(p, q) - H(p) = (\text{bits using wrong code}) - (\text{bits using optimal code}).$$

KL divergence measures the *extra* bits wasted by using q instead of p .

Remark 2.46 (Asymmetry and Modes). A **mode** of a distribution is a peak—a local maximum of the density $p(x)$. A distribution with two peaks is *bimodal*:



The choice of KL direction determines how q approximates a multimodal p . To see why, rewrite each KL divergence:

$$\begin{aligned} D_{\text{KL}}(p||q) &= \underbrace{-H(p)}_{\text{constant}} - \mathbb{E}_{x \sim p} [\log q(x)], \\ D_{\text{KL}}(q||p) &= -H(q) - \mathbb{E}_{x \sim q} [\log p(x)]. \end{aligned}$$

Forward KL minimizes $-\mathbb{E}_{x \sim p} [\log q(x)]$, i.e., maximizes $\mathbb{E}_{x \sim p} [\log q(x)]$.

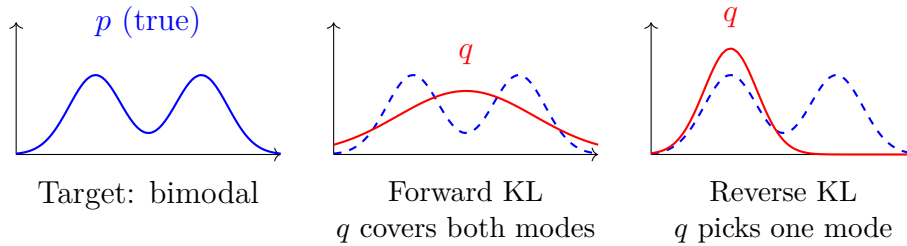
This is the expected log-likelihood of q evaluated at samples from p . Since p places mass at *both* modes, q must assign reasonable probability to both regions to achieve high expected log-likelihood.

Reverse KL minimizes $-H(q) - \mathbb{E}_{x \sim q}[\log p(x)]$.

The term $\mathbb{E}_{x \sim q}[\log p(x)]$ evaluates $\log p$ at samples from q . If q spreads across both modes, it will sample the valley between them where p is small, making $\log p(x)$ very negative. Better to concentrate q on one mode where p is large.

Concrete example. Let $p = \frac{1}{2}\mathcal{N}(0, 1) + \frac{1}{2}\mathcal{N}(4, 1)$ be a mixture of two Gaussians. Suppose we approximate with a single Gaussian $q = \mathcal{N}(\mu, 1)$ and ask: what is the optimal μ ?

- **Forward KL:** We maximize $\mathbb{E}_{x \sim p}[\log q(x)]$. Since p samples equally from both modes, q needs to cover both. The optimum is $\mu \approx 2$ (between the modes).
- **Reverse KL:** We maximize $\mathbb{E}_{x \sim q}[\log p(x)]$. At $\mu = 2$, samples from q land in the valley where $p(x) \approx 0.027$, giving $\log p(x) \approx -3.6$. At $\mu = 0$, samples land near the left mode where $p(x) \approx 0.2$, giving $\log p(x) \approx -1.6$. The optimum is $\mu = 0$ or $\mu = 4$ (on one mode).



Remark 2.47 (When to Use Each Direction). Use **forward KL** $D_{\text{KL}}(p||q)$ when you have samples from p and want q to cover all of p :

- **Maximum likelihood / supervised learning:** You have data from the true distribution p_{data} and fit a model q_{θ} . Minimizing cross-entropy loss is equivalent to minimizing $D_{\text{KL}}(p_{\text{data}}||q_{\theta})$. You want the model to assign probability to *all* training examples.
- **Density estimation:** When the goal is to capture the full distribution, including rare events.

Use **reverse KL** $D_{\text{KL}}(q||p)$ when you can evaluate p but not sample from it, and you prefer q to be conservative:

- **Variational inference:** The posterior $p(\theta|\text{data})$ is intractable, but you can evaluate it up to a constant. You fit a tractable $q(\theta)$ by minimizing $D_{\text{KL}}(q||p)$. This gives a q that doesn't put mass where p is small—better to underestimate uncertainty than to be overconfident in wrong regions.
- **Policy optimization:** In RL, you want the policy to concentrate on high-reward actions, not spread probability over all actions the expert might take.

Summary:

	Forward $D_{\text{KL}}(p q)$	Reverse $D_{\text{KL}}(q p)$
Samples from	p (data)	q (model)
Evaluates	q (model)	p (target)
Behavior	mode-covering	mode-seeking
Risk	overconfident in gaps	misses some modes
Use when	want full coverage	want conservative estimate

Part III

Differential Geometry

Chapter 3

Smooth Manifolds

3.1 Topological Manifolds

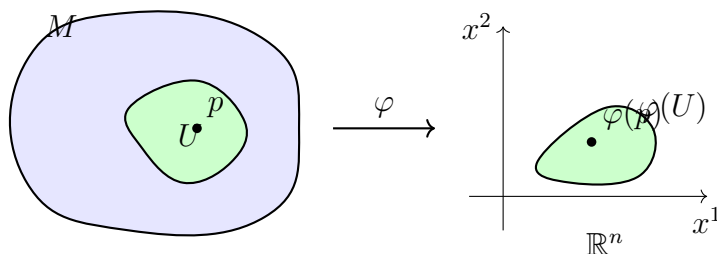
Definition 3.1 (Homeomorphism). Let X and Y be topological spaces. A **homeomorphism** is a bijective map $f : X \rightarrow Y$ such that both f and f^{-1} are continuous. If such a map exists, we say X and Y are **homeomorphic** and write $X \cong Y$.

Definition 3.2 (Topological Manifold). A **topological manifold** of dimension n is a topological space M satisfying:

1. M is Hausdorff: for any two distinct points $p, q \in M$, there exist disjoint open sets U, V with $p \in U$ and $q \in V$.
2. M is second countable: there exists a countable basis for the topology.
3. M is locally Euclidean of dimension n : every point $p \in M$ has a neighborhood homeomorphic to an open subset of $\mathbb{R}^n$.

Definition 3.3 (Chart). A **chart** (or **coordinate chart**) on M is a pair (U, φ) where $U \subseteq M$ is open and $\varphi : U \rightarrow \varphi(U) \subseteq \mathbb{R}^n$ is a homeomorphism onto an open subset of $\mathbb{R}^n$. We call U a **coordinate domain** and φ a **coordinate map**.

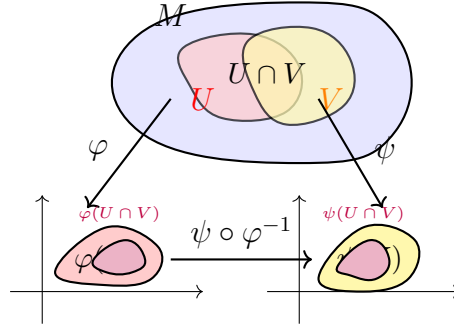
If $\varphi(p) = (x^1, \dots, x^n)$, we call $(x^1, \dots, x^n)$ the **local coordinates** of p in this chart.



Definition 3.4 (Transition Map). Given two charts (U, φ) and (V, ψ) with $U \cap V \neq \emptyset$, the **transition map** is:

$$\psi \circ \varphi^{-1} : \varphi(U \cap V) \rightarrow \psi(U \cap V)$$

This is a homeomorphism between open subsets of $\mathbb{R}^n$.



3.2 Smooth Structures

Definition 3.5 (Smoothly Compatible Charts). Two charts (U, φ) and (V, ψ) are **smoothly compatible** if either $U \cap V = \emptyset$, or the transition map $\psi \circ \varphi^{-1}$ is a smooth (i.e., C^∞) diffeomorphism.

Definition 3.6 (Smooth Atlas). A **smooth atlas** on M is a collection $\mathcal{A} = \{(U_\alpha, \varphi_\alpha)\}_{\alpha \in A}$ of charts such that:

1. The coordinate domains cover M : $\bigcup_{\alpha \in A} U_\alpha = M$.
2. Any two charts in $\mathcal{A}$ are smoothly compatible.

Definition 3.7 (Maximal Atlas / Smooth Structure). A smooth atlas $\mathcal{A}$ is **maximal** if it is not properly contained in any larger smooth atlas. A **smooth structure** on M is a maximal smooth atlas.

A **smooth manifold** is a topological manifold equipped with a smooth structure.

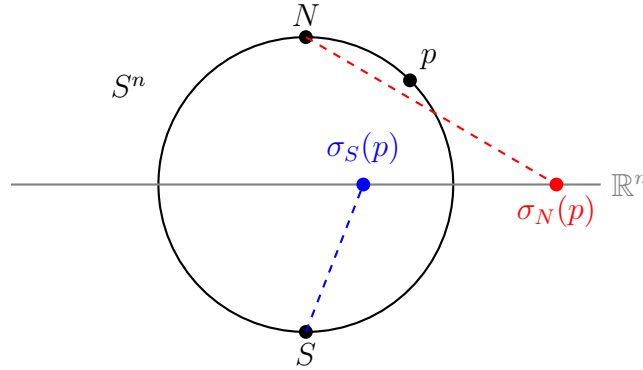
Proposition 3.8. *Every smooth atlas $\mathcal{A}$ is contained in a unique maximal atlas, consisting of all charts smoothly compatible with every chart in $\mathcal{A}$.*

Example 3.9 (Standard Smooth Structure on $\mathbb{R}^n$). $\mathbb{R}^n$ with the single chart $(\mathbb{R}^n, \text{id})$ defines a smooth structure. This is the **standard smooth structure**.

Example 3.10 (The n -Sphere S^n). $S^n = \{x \in \mathbb{R}^{n+1} : |x| = 1\}$ is a smooth n -manifold. One atlas consists of stereographic projections from the north and south poles:

$$\begin{aligned} \sigma_N : S^n \setminus \{N\} &\rightarrow \mathbb{R}^n, & (x^1, \dots, x^{n+1}) &\mapsto \frac{1}{1 - x^{n+1}}(x^1, \dots, x^n) \\ \sigma_S : S^n \setminus \{S\} &\rightarrow \mathbb{R}^n, & (x^1, \dots, x^{n+1}) &\mapsto \frac{1}{1 + x^{n+1}}(x^1, \dots, x^n) \end{aligned}$$

The transition map $\sigma_S \circ \sigma_N^{-1}(y) = y/|y|^2$ is smooth on $\mathbb{R}^n \setminus \{0\}$.



3.3 Smooth Maps

Definition 3.11 (Smooth Function). Let M be a smooth manifold. A function $f : M \rightarrow \mathbb{R}$ is **smooth** if for every chart (U, φ) , the composition $f \circ \varphi^{-1} : \varphi(U) \rightarrow \mathbb{R}$ is smooth in the usual sense.

We write $C^\infty(M)$ for the set of all smooth real-valued functions on M .

Definition 3.12 (Smooth Map Between Manifolds). Let M and N be smooth manifolds. A continuous map $F : M \rightarrow N$ is **smooth** if for every $p \in M$, there exist charts (U, φ) around p and (V, ψ) around $F(p)$ such that $\psi \circ F \circ \varphi^{-1}$ is smooth.

Definition 3.13 (Diffeomorphism). A smooth map $F : M \rightarrow N$ is a **diffeomorphism** if it is bijective and F^{-1} is also smooth. If such an F exists, M and N are **diffeomorphic**.

Remark 3.14. $C^\infty(M)$ forms a commutative $\mathbb{R}$ -algebra under pointwise operations.

Chapter 4

Tangent Spaces

4.1 Derivations and Tangent Vectors

The key insight is that tangent vectors can be characterized algebraically as derivations on smooth functions, without reference to ambient space.

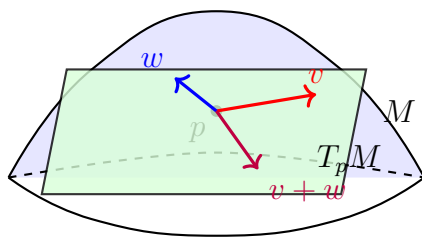
Definition 4.1 (Derivation at a Point). Let M be a smooth manifold and $p \in M$. A **derivation at p** is a linear map $v : C^\infty(M) \rightarrow \mathbb{R}$ satisfying the **Leibniz rule**:

$$v(fg) = f(p) \cdot v(g) + g(p) \cdot v(f)$$

for all $f, g \in C^\infty(M)$.

Definition 4.2 (Tangent Space). The **tangent space** to M at p , denoted $T_p M$, is the set of all derivations at p . This is a real vector space under:

$$(v + w)(f) = v(f) + w(f), \quad (\lambda v)(f) = \lambda \cdot v(f)$$



Lemma 4.3. If $v \in T_p M$ and f is constant in a neighborhood of p , then $v(f) = 0$.

Proof. If $f \equiv c$ near p , write $f = c \cdot 1$ where 1 is the constant function. Then:

$$v(1) = v(1 \cdot 1) = 1 \cdot v(1) + 1 \cdot v(1) = 2v(1)$$

so $v(1) = 0$. Hence $v(f) = v(c \cdot 1) = c \cdot v(1) = 0$. □

4.2 Coordinate Basis

Definition 4.4 (Coordinate Tangent Vectors). Let (U, φ) be a chart around p with coordinates $(x^1, \dots, x^n)$. Define the **coordinate tangent vector** $\frac{\partial}{\partial x^i} \Big|_p \in T_p M$ by:

$$\frac{\partial}{\partial x^i} \Big|_p (f) = \frac{\partial(f \circ \varphi^{-1})}{\partial r^i} \Big|_{\varphi(p)}$$

where r^i are the standard coordinates on $\mathbb{R}^n$.

Theorem 4.5. Let M be an n -dimensional smooth manifold and (U, φ) a chart around p with coordinates $(x^1, \dots, x^n)$. Then $\left\{ \frac{\partial}{\partial x^1} \Big|_p, \dots, \frac{\partial}{\partial x^n} \Big|_p \right\}$ is a basis for $T_p M$.

In particular, $\dim T_p M = n$.

Corollary 4.6. Every tangent vector $v \in T_p M$ can be written uniquely as:

$$v = v^i \frac{\partial}{\partial x^i} \Big|_p$$

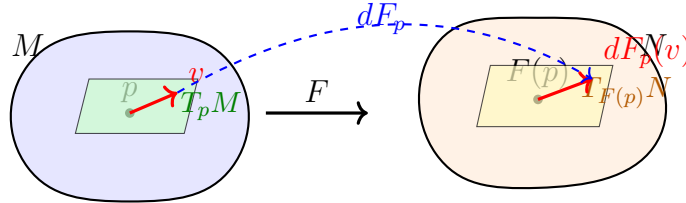
where $v^i = v(x^i)$ and we use Einstein summation convention.

4.3 The Differential (Pushforward)

Definition 4.7 (Differential of a Smooth Map). Let $F : M \rightarrow N$ be smooth and $p \in M$. The **differential** (or **pushforward**) of F at p is the linear map:

$$dF_p : T_p M \rightarrow T_{F(p)} N$$

defined by $(dF_p(v))(g) = v(g \circ F)$ for $v \in T_p M$ and $g \in C^\infty(N)$.



Remark 4.8. Other notations for dF_p include F_{*p} , DF_p , and $T_p F$.

Proposition 4.9 (Chain Rule). If $F : M \rightarrow N$ and $G : N \rightarrow P$ are smooth, then:

$$d(G \circ F)_p = dG_{F(p)} \circ dF_p$$

Proposition 4.10 (Coordinate Expression). Let (U, φ) be a chart on M with coordinates (x^i) and (V, ψ) a chart on N with coordinates (y^j) . If $F : M \rightarrow N$ is smooth, then:

$$dF_p \left(\frac{\partial}{\partial x^i} \Big|_p \right) = \frac{\partial F^j}{\partial x^i}(p) \frac{\partial}{\partial y^j} \Big|_{F(p)}$$

where $F^j = y^j \circ F$ are the component functions.

Corollary 4.11. The matrix of dF_p with respect to coordinate bases is the Jacobian matrix $\left(\frac{\partial F^j}{\partial x^i} \right)$.

Chapter 5

Cotangent Spaces and Differential Forms

5.1 Covectors and the Cotangent Space

Definition 5.1 (Cotangent Space). The **cotangent space** to M at p , denoted T_p^*M , is the dual vector space to T_pM :

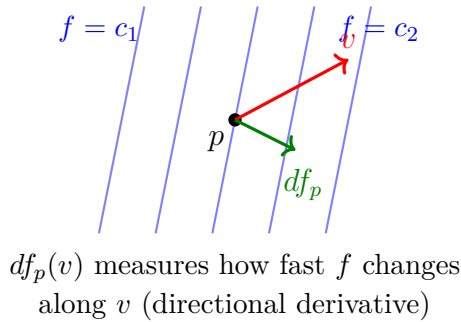
$$T_p^*M = (T_pM)^* = \{\omega : T_pM \rightarrow \mathbb{R} : \omega \text{ is linear}\}$$

Elements of T_p^*M are called **covectors** or **cotangent vectors**.

Definition 5.2 (Differential of a Function). For $f \in C^\infty(M)$, the **differential of f at p** is the covector $df_p \in T_p^*M$ defined by:

$$df_p(v) = v(f)$$

for $v \in T_pM$.



Proposition 5.3 (Coordinate Basis for T_p^*M). If $(x^1, \dots, x^n)$ are local coordinates near p , then $\{dx^1|_p, \dots, dx^n|_p\}$ is the basis of T_p^*M dual to $\left\{ \frac{\partial}{\partial x^1}\Big|_p, \dots, \frac{\partial}{\partial x^n}\Big|_p \right\}$:

$$dx^i \left(\frac{\partial}{\partial x^j} \right) = \delta_j^i$$

Corollary 5.4. Any covector $\omega \in T_p^*M$ can be written as:

$$\omega = \omega_i dx^i|_p$$

where $\omega_i = \omega\left(\frac{\partial}{\partial x^i}\right)$.

Proposition 5.5 (Differential in Coordinates). If $f \in C^\infty(M)$, then in local coordinates:

$$df = \frac{\partial f}{\partial x^i} dx^i$$

5.2 The Pullback

Definition 5.6 (Pullback of a Covector). Let $F : M \rightarrow N$ be smooth and $p \in M$. The **pullback** (or **cotangent map**) is:

$$dF_p^* : T_{F(p)}^*N \rightarrow T_p^*M$$

defined by $(dF_p^*\omega)(v) = \omega(dF_p(v))$ for $\omega \in T_{F(p)}^*N$ and $v \in T_pM$.

Remark 5.7. Note the reversed direction: while dF_p pushes vectors forward from M to N , dF_p^* pulls covectors back from N to M .

$$\begin{array}{ccc}
 M & \xrightarrow{F} & N \\
 T_pM & \xrightarrow{dF_p \text{ (pushforward)}} & T_{F(p)}N \\
 \uparrow \text{dual} & & \uparrow \text{dual} \\
 T_p^*M & \xleftarrow{dF_p^* \text{ (pullback)}} & T_{F(p)}^*N \\
 \text{(covectors on } M) & & \text{(covectors on } N)
 \end{array}$$

5.3 Differential 1-Forms

Definition 5.8 (Differential 1-Form). A **differential 1-form** (or simply **1-form**) on M is a smooth assignment of a covector at each point:

$$\omega : p \mapsto \omega_p \in T_p^*M$$

More precisely, ω is a smooth section of the cotangent bundle T^*M .

Definition 5.9 (Smoothness of 1-Forms). A 1-form ω is **smooth** if for every smooth vector field X , the function $\omega(X) : M \rightarrow \mathbb{R}$ defined by $p \mapsto \omega_p(X_p)$ is smooth.

Equivalently, in any local coordinates, if $\omega = \omega_i dx^i$, then each coefficient function ω_i is smooth.

Definition 5.10 (Space of 1-Forms). We denote by $\Omega^1(M)$ the $C^\infty(M)$ -module of all smooth 1-forms on M .

Example 5.11 (The Differential as a 1-Form). For any $f \in C^\infty(M)$, its differential df is a smooth 1-form. In coordinates:

$$df = \frac{\partial f}{\partial x^1} dx^1 + \cdots + \frac{\partial f}{\partial x^n} dx^n$$

Definition 5.12 (Pullback of 1-Forms). If $F : M \rightarrow N$ is smooth and $\omega \in \Omega^1(N)$, the **pullback** $F^*\omega \in \Omega^1(M)$ is defined by:

$$(F^*\omega)_p = dF_p^*(\omega_{F(p)})$$

That is, $(F^*\omega)_p(v) = \omega_{F(p)}(dF_p(v))$ for $v \in T_pM$.

Proposition 5.13. *The pullback satisfies:*

1. $F^*(df) = d(f \circ F)$ for $f \in C^\infty(N)$
2. $(G \circ F)^* = F^* \circ G^*$
3. $F^*(f\omega) = (f \circ F) \cdot F^*\omega$

5.4 Differential k -Forms

Definition 5.14 (Multilinear Alternating Maps). A map $\omega : (T_pM)^k \rightarrow \mathbb{R}$ is:

- **Multilinear:** linear in each argument
- **Alternating:** $\omega(\dots, v, \dots, w, \dots) = -\omega(\dots, w, \dots, v, \dots)$ when any two arguments are swapped

We denote the space of such maps by $\Lambda^k(T_p^*M)$.

Definition 5.15 (Differential k -Form). A **differential k -form** on M is a smooth assignment $\omega : p \mapsto \omega_p \in \Lambda^k(T_p^*M)$.

We denote the space of smooth k -forms by $\Omega^k(M)$.

Remark 5.16. Special cases:

- $\Omega^0(M) = C^\infty(M)$ (smooth functions)
- $\Omega^1(M) =$ smooth 1-forms (as defined above)
- $\Omega^k(M) = \{0\}$ for $k > n = \dim M$

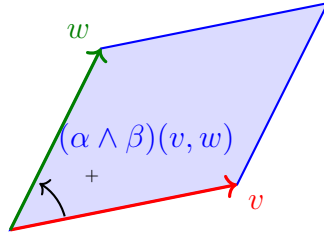
5.5 The Wedge Product

Definition 5.17 (Wedge Product). The **wedge product** (or **exterior product**) is a bilinear operation:

$$\wedge : \Omega^k(M) \times \Omega^\ell(M) \rightarrow \Omega^{k+\ell}(M)$$

For 1-forms α, β , defined on vectors v, w by:

$$(\alpha \wedge \beta)(v, w) = \alpha(v)\beta(w) - \alpha(w)\beta(v)$$



Signed area of parallelogram
spanned by v and w

Proposition 5.18 (Properties of the Wedge Product). 1. **Associativity:** $(\alpha \wedge \beta) \wedge \gamma = \alpha \wedge (\beta \wedge \gamma)$

2. **Graded commutativity:** $\alpha \wedge \beta = (-1)^{k\ell} \beta \wedge \alpha$ for $\alpha \in \Omega^k, \beta \in \Omega^\ell$

3. **$C^\infty(M)$ -linearity:** $(f\alpha) \wedge \beta = f(\alpha \wedge \beta) = \alpha \wedge (f\beta)$

Corollary 5.19. For any 1-form α : $\alpha \wedge \alpha = 0$.

Proposition 5.20 (Local Expression). In local coordinates $(x^1, \dots, x^n)$, every k -form can be written:

$$\omega = \sum_{i_1 < \dots < i_k} \omega_{i_1 \dots i_k} dx^{i_1} \wedge \dots \wedge dx^{i_k}$$

where $\omega_{i_1 \dots i_k} \in C^\infty(M)$.

5.6 The Exterior Derivative

Definition 5.21 (Exterior Derivative). The **exterior derivative** is the unique family of linear maps:

$$d : \Omega^k(M) \rightarrow \Omega^{k+1}(M)$$

satisfying:

1. For $f \in \Omega^0(M) = C^\infty(M)$, df is the differential of f
2. $d^2 = 0$: for any ω , $d(d\omega) = 0$
3. **Graded Leibniz rule:** $d(\alpha \wedge \beta) = d\alpha \wedge \beta + (-1)^k \alpha \wedge d\beta$ for $\alpha \in \Omega^k$

$$\begin{array}{ccccccc}
 & & \textcolor{red}{d^2 = 0} & & & & \\
 & \textcolor{red}{\curvearrowright} & & \textcolor{red}{\curvearrowleft} & & & \\
 \Omega^0(M) & \xrightarrow{d} & \Omega^1(M) & \xrightarrow{d} & \Omega^2(M) & \xrightarrow{d} & \cdots \xrightarrow{d} \Omega^n(M) \xrightarrow{d} 0 \\
 \text{functions} & & \text{1-forms} & & \text{2-forms} & &
 \end{array}$$

Proposition 5.22 (Coordinate Formula). For $\omega = \sum_I \omega_I dx^I$ where $I = (i_1, \dots, i_k)$:

$$d\omega = \sum_I d\omega_I \wedge dx^I = \sum_I \frac{\partial \omega_I}{\partial x^j} dx^j \wedge dx^{i_1} \wedge \cdots \wedge dx^{i_k}$$

Example 5.23. On $\mathbb{R}^3$ with coordinates (x, y, z) :

- If $f \in \Omega^0$: $df = \frac{\partial f}{\partial x} dx + \frac{\partial f}{\partial y} dy + \frac{\partial f}{\partial z} dz$ (gradient)
- If $\omega = P dx + Q dy + R dz \in \Omega^1$:

$$d\omega = \left(\frac{\partial R}{\partial y} - \frac{\partial Q}{\partial z} \right) dy \wedge dz + \left(\frac{\partial P}{\partial z} - \frac{\partial R}{\partial x} \right) dz \wedge dx + \left(\frac{\partial Q}{\partial x} - \frac{\partial P}{\partial y} \right) dx \wedge dy$$

(curl)

Theorem 5.24 (Naturality of d). If $F : M \rightarrow N$ is smooth, then d commutes with pullback:

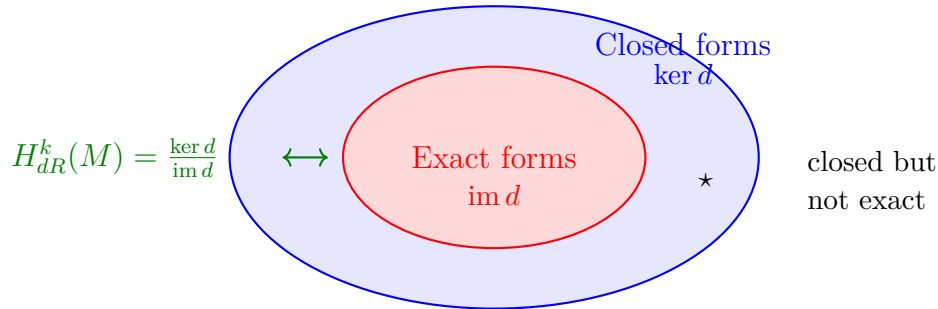
$$F^*(d\omega) = d(F^*\omega)$$

for all $\omega \in \Omega^k(N)$.

Definition 5.25 (Closed and Exact Forms). A k -form ω is:

- **Closed** if $d\omega = 0$
- **Exact** if $\omega = d\eta$ for some $(k-1)$ -form η

Since $d^2 = 0$, every exact form is closed. The converse is not always true.



Definition 5.26 (de Rham Cohomology). The k -th **de Rham cohomology group** of M is:

$$H^k_{dR}(M) = \frac{\ker(d : \Omega^k \rightarrow \Omega^{k+1})}{\operatorname{im}(d : \Omega^{k-1} \rightarrow \Omega^k)} = \frac{\{\text{closed } k\text{-forms}\}}{\{\text{exact } k\text{-forms}\}}$$

Theorem 5.27 (Poincaré Lemma). If $U \subseteq \mathbb{R}^n$ is star-shaped (e.g., convex), then every closed form on U is exact. That is, $H^k_{dR}(U) = 0$ for $k \geq 1$.

Chapter 6

Integration and Stokes' Theorem

6.1 Integration of Differential Forms

Definition 6.1 (Integration of n -Forms). Let M be an oriented n -dimensional manifold and $\omega \in \Omega^n(M)$ a compactly supported n -form. In local coordinates where $\omega = f dx^1 \wedge \cdots \wedge dx^n$:

$$\int_M \omega = \int_{\mathbb{R}^n} f(x^1, \dots, x^n) dx^1 \cdots dx^n$$

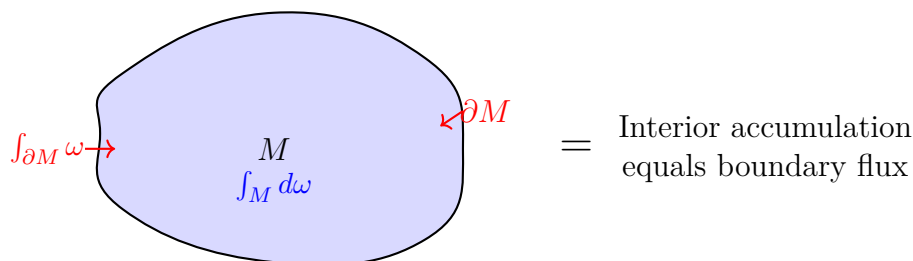
The integral is independent of the choice of positively oriented coordinates.

Remark 6.2. The key insight: **differential forms are the natural objects to integrate.** The wedge product provides the correct transformation law under change of variables (the Jacobian determinant appears automatically).

6.2 Stokes' Theorem

Theorem 6.3 (Generalized Stokes' Theorem). *Let M be an oriented n -dimensional manifold with boundary ∂M , and let $\omega \in \Omega^{n-1}(M)$ be a compactly supported $(n-1)$ -form. Then:*

$$\boxed{\int_M d\omega = \int_{\partial M} \omega}$$



Remark 6.4. This single theorem unifies all the classical integration theorems of vector calculus. The exterior derivative d and the boundary operator ∂ are **adjoint** to each other.

6.3 Classical Theorems as Special Cases

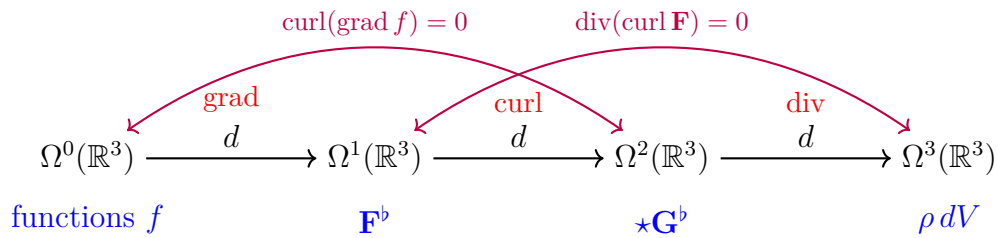
On $\mathbb{R}^3$, we establish correspondences between vector fields and differential forms.

Definition 6.5 (Musical Isomorphisms). Using the Euclidean metric, we identify:

- Vector field $\mathbf{F} = (P, Q, R)$ with 1-form $\mathbf{F}^\flat = P dx + Q dy + R dz$
- Vector field $\mathbf{F}$ with 2-form $\star \mathbf{F}^\flat = P dy \wedge dz + Q dz \wedge dx + R dx \wedge dy$

6.3.1 Gradient, Curl, and Divergence

The exterior derivative d encodes **all three** vector calculus differential operators:



Proposition 6.6 (Exterior Derivative as Vector Calculus Operators). For $f \in C^\infty(\mathbb{R}^3)$ and vector field $\mathbf{F} = (P, Q, R)$:

1. **Gradient:** $df = \frac{\partial f}{\partial x} dx + \frac{\partial f}{\partial y} dy + \frac{\partial f}{\partial z} dz \iff \nabla f$
2. **Curl:** $d(P dx + Q dy + R dz) = \star(\nabla \times \mathbf{F})^\flat$
3. **Divergence:** $d(P dy \wedge dz + Q dz \wedge dx + R dx \wedge dy) = (\nabla \cdot \mathbf{F}) dx \wedge dy \wedge dz$

Corollary 6.7 (Vector Calculus Identities from $d^2 = 0$). The identity $d^2 = 0$ immediately implies:

- $\nabla \times (\nabla f) = \mathbf{0}$ (curl of gradient is zero)
- $\nabla \cdot (\nabla \times \mathbf{F}) = 0$ (divergence of curl is zero)

6.3.2 Fundamental Theorem of Calculus

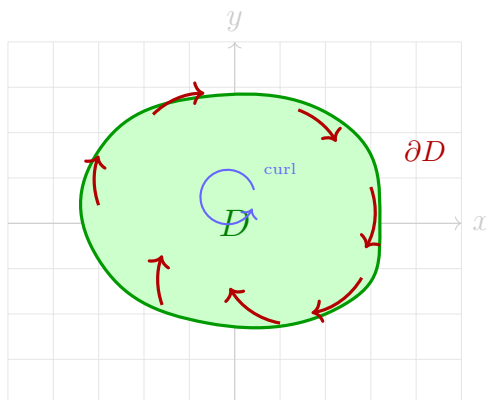
For $n = 1$: Let $\gamma : [a, b] \rightarrow \mathbb{R}$ be a curve and f a function.

$$\int_\gamma df = \int_a^b f'(t) dt = f(b) - f(a) = \int_{\partial\gamma} f$$

6.3.3 Green's Theorem

For $n = 2$: Let $D \subset \mathbb{R}^2$ be a region with boundary ∂D .

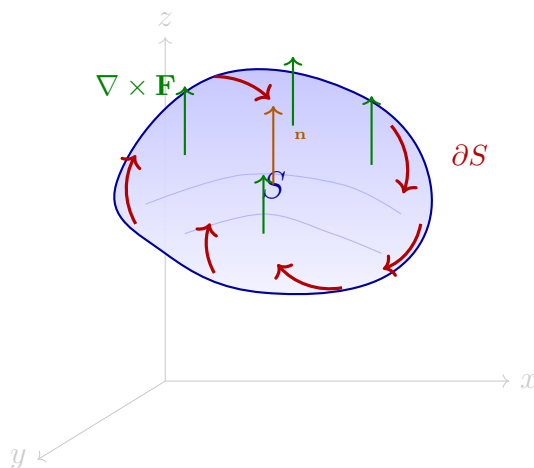
$$\int_D d(P dx + Q dy) = \iint_D \left(\frac{\partial Q}{\partial x} - \frac{\partial P}{\partial y} \right) dA = \oint_{\partial D} P dx + Q dy$$



6.3.4 Classical Stokes' Theorem (Curl Theorem)

For a surface $S \subset \mathbb{R}^3$ with boundary ∂S :

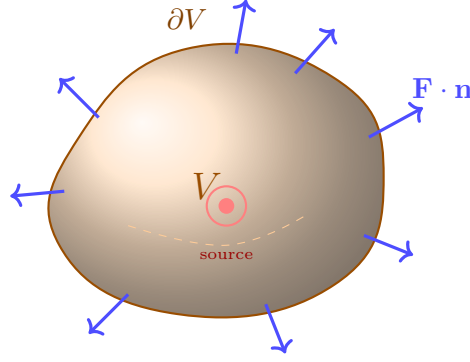
$$\iint_S (\nabla \times \mathbf{F}) \cdot d\mathbf{S} = \oint_{\partial S} \mathbf{F} \cdot d\mathbf{r}$$



6.3.5 Divergence Theorem (Gauss's Theorem)

For a solid region $V \subset \mathbb{R}^3$ with boundary ∂V :

$$\iiint_V (\nabla \cdot \mathbf{F}) dV = \iint_{\partial V} \mathbf{F} \cdot d\mathbf{S}$$



6.4 The Hodge Star Operator

Definition 6.8 (Hodge Star). On an oriented Riemannian n -manifold (M, g) , the **Hodge star operator** is a linear map:

$$\star : \Omega^k(M) \rightarrow \Omega^{n-k}(M)$$

satisfying $\alpha \wedge \star \beta = \langle \alpha, \beta \rangle \text{vol}_g$ for $\alpha, \beta \in \Omega^k(M)$, where vol_g is the volume form.

Example 6.9 (Hodge Star on $\mathbb{R}^3$). With the standard metric and orientation:

$$\begin{aligned} \star 1 &= dx \wedge dy \wedge dz & \star(dx \wedge dy \wedge dz) &= 1 \\ \star dx &= dy \wedge dz & \star(dy \wedge dz) &= dx \\ \star dy &= dz \wedge dx & \star(dz \wedge dx) &= dy \\ \star dz &= dx \wedge dy & \star(dx \wedge dy) &= dz \end{aligned}$$

Definition 6.10 (Codifferential). The **codifferential** $\delta : \Omega^k(M) \rightarrow \Omega^{k-1}(M)$ is defined by:

$$\delta = (-1)^{n(k+1)+1} \star d \star$$

On $\mathbb{R}^3$: $\delta = -\star d \star$ for appropriate degrees.

Proposition 6.11 (Divergence via Hodge Star). For a vector field $\mathbf{F}$ on $\mathbb{R}^3$:

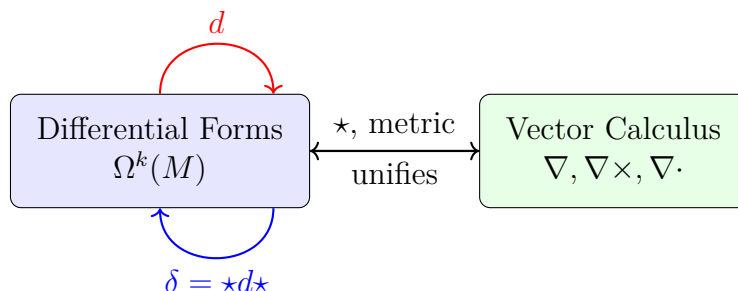
$$\nabla \cdot \mathbf{F} = \star d \star \mathbf{F}^\flat = \delta \mathbf{F}^\flat$$

The divergence is the codifferential of the 1-form associated to $\mathbf{F}$.

Definition 6.12 (Laplacian on Forms). The **Hodge Laplacian** (or Laplace-de Rham operator) is:

$$\Delta = d\delta + \delta d = (d + \delta)^2$$

On functions (0-forms) in $\mathbb{R}^n$, this recovers the usual Laplacian $\Delta f = \nabla^2 f$.



Remark 6.13 (The Beauty of Differential Forms). The exterior derivative d encapsulates:

- All differential operators (grad, curl, div) in a single operator
- All integral theorems in a single theorem ($\int_M d\omega = \int_{\partial M} \omega$)
- Topological invariants via de Rham cohomology ($d^2 = 0$)

The Hodge star $\star$ provides the missing link between the “geometric” exterior derivative and the “metric-dependent” operations like divergence and Laplacian.

Part IV

Group Theory

Chapter 7

Groups and Actions

Part V

Complex Analysis

Chapter 8

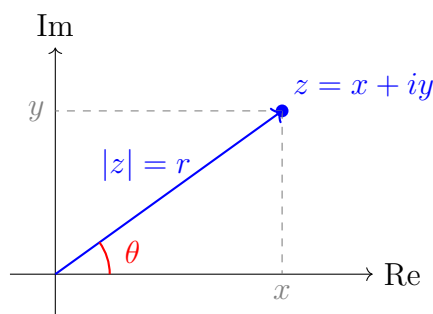
Holomorphic Functions

8.1 The Complex Plane

Definition 8.1 (Complex numbers). The set of *complex numbers* is denoted by $\mathbb{C}$. A complex number $z \in \mathbb{C}$ can be written in the form

$$z = x + iy, \quad (8.1)$$

where $x, y \in \mathbb{R}$ and i is the imaginary unit satisfying $i^2 = -1$. We call $x = \operatorname{Re}(z)$ the *real part* and $y = \operatorname{Im}(z)$ the *imaginary part*.

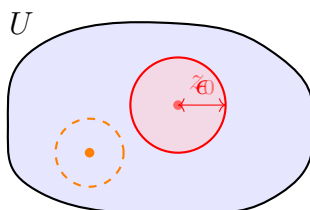


The *polar form* of a complex number is $z = re^{i\theta}$ where $r = |z| = \sqrt{x^2 + y^2}$ is the *modulus* and $\theta = \arg(z)$ is the *argument*.

8.2 Topological Preliminaries

We define some topological concepts fundamental to complex analysis.

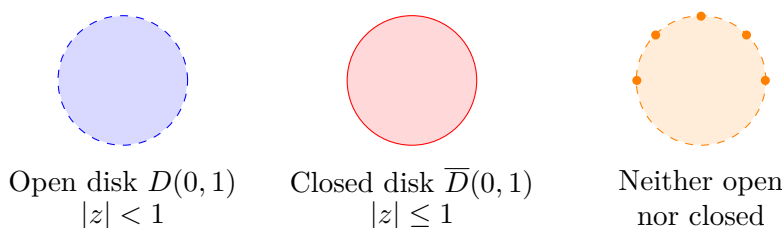
Definition 8.2 (Open set). A set $U \subseteq \mathbb{C}$ is *open* if for every point $z_0 \in U$, there exists $\epsilon > 0$ such that the open disk $D(z_0, \epsilon) = \{z \in \mathbb{C} : |z - z_0| < \epsilon\}$ is entirely contained in U .



Definition 8.3 (Closed set). A set $F \subseteq \mathbb{C}$ is *closed* if its complement $\mathbb{C} \setminus F$ is open. Equivalently, F is closed if it contains all its limit points.

Definition 8.4 (Boundary, interior, and closure). Let $S \subseteq \mathbb{C}$.

- The *interior* S° is the largest open set contained in S .
- The *closure* $\bar{S}$ is the smallest closed set containing S .
- The *boundary* $\partial S = \bar{S} \setminus S^\circ$ consists of points where every disk intersects both S and its complement.

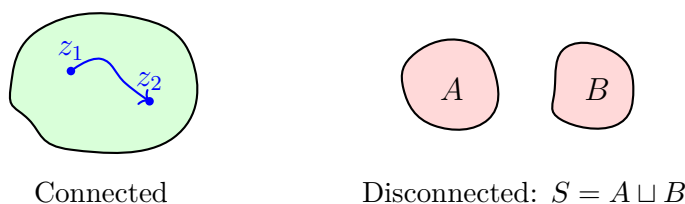


Definition 8.5 (Bounded set). A set $S \subseteq \mathbb{C}$ is *bounded* if there exists $M > 0$ such that $|z| < M$ for all $z \in S$.

Definition 8.6 (Compact set). A set $K \subseteq \mathbb{C}$ is *compact* if it is both closed and bounded. By the Heine-Borel theorem, this is equivalent to: every open cover of K has a finite subcover.

Definition 8.7 (Connected and path-connected). A set $S \subseteq \mathbb{C}$ is *disconnected* if it can be written as $S = A \cup B$ where A and B are disjoint, nonempty, and both relatively open in S . Otherwise, S is *connected*.

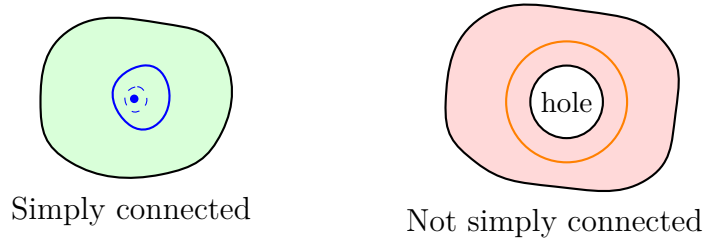
A set S is *path-connected* if for any two points $z_1, z_2 \in S$, there exists a continuous path $\gamma : [0, 1] \rightarrow S$ with $\gamma(0) = z_1$ and $\gamma(1) = z_2$.



Definition 8.8 (Domain). A *domain* (or *region*) in $\mathbb{C}$ is an open, connected set. Most of complex analysis takes place on domains.

Definition 8.9 (Curve and Closed Curve). A *curve* (or *path*) in $\mathbb{C}$ is a continuous function $\gamma : [a, b] \rightarrow \mathbb{C}$. The curve is *closed* if $\gamma(a) = \gamma(b)$. It is *simple* if it does not intersect itself (except possibly at the endpoints).

Definition 8.10 (Simply connected). An open set $U \subseteq \mathbb{C}$ is *simply connected* if it is path-connected and every closed curve in U can be continuously deformed (contracted) to a point within U . Intuitively, U has “no holes”.



8.3 Holomorphic Functions

Definition 8.11 (Complex differentiability). Let $U \subseteq \mathbb{C}$ be an open set and $f : U \rightarrow \mathbb{C}$ be a function. We say that f is *complex differentiable* (or *holomorphic*) at a point $z_0 \in U$ if the limit

$$f'(z_0) = \lim_{z \rightarrow z_0} \frac{f(z) - f(z_0)}{z - z_0} \quad (8.2)$$

exists. If f is holomorphic at every point in U , we say f is holomorphic on U .

Theorem 8.12 (Cauchy-Riemann equations). Let $f : U \rightarrow \mathbb{C}$ be a function, written as $f(x + iy) = u(x, y) + iv(x, y)$, where $u, v : U \rightarrow \mathbb{R}$ are real-differentiable functions. Then f is holomorphic on U if and only if u and v satisfy the Cauchy-Riemann equations:

$$\frac{\partial u}{\partial x} = \frac{\partial v}{\partial y} \quad \text{and} \quad \frac{\partial u}{\partial y} = -\frac{\partial v}{\partial x}. \quad (8.3)$$

Proof. ($\implies$) Suppose f is holomorphic at $z = x + iy$. Then the limit

$$f'(z) = \lim_{h \rightarrow 0} \frac{f(z + h) - f(z)}{h}$$

exists and is independent of the path taken by h to 0.

Case 1: $h \rightarrow 0$ along the real axis. Let $h = \Delta x \in \mathbb{R}$.

$$\begin{aligned} f'(z) &= \lim_{\Delta x \rightarrow 0} \frac{u(x + \Delta x, y) + iv(x + \Delta x, y) - (u(x, y) + iv(x, y))}{\Delta x} \\ &= \lim_{\Delta x \rightarrow 0} \left(\frac{u(x + \Delta x, y) - u(x, y)}{\Delta x} + i \frac{v(x + \Delta x, y) - v(x, y)}{\Delta x} \right) \\ &= \frac{\partial u}{\partial x} + i \frac{\partial v}{\partial x}. \end{aligned}$$

Case 2: $h \rightarrow 0$ along the imaginary axis. Let $h = i\Delta y$ where $\Delta y \in \mathbb{R}$.

$$\begin{aligned} f'(z) &= \lim_{\Delta y \rightarrow 0} \frac{u(x, y + \Delta y) + iv(x, y + \Delta y) - (u(x, y) + iv(x, y))}{i\Delta y} \\ &= \lim_{\Delta y \rightarrow 0} \left(\frac{1}{i} \frac{u(x, y + \Delta y) - u(x, y)}{\Delta y} + \frac{v(x, y + \Delta y) - v(x, y)}{\Delta y} \right) \\ &= -i \frac{\partial u}{\partial y} + \frac{\partial v}{\partial y} = \frac{\partial v}{\partial y} - i \frac{\partial u}{\partial y}. \end{aligned}$$

Equating the two expressions for $f'(z)$ and comparing real and imaginary parts gives the Cauchy-Riemann equations:

$$\frac{\partial u}{\partial x} = \frac{\partial v}{\partial y} \quad \text{and} \quad \frac{\partial u}{\partial y} = -\frac{\partial v}{\partial x}.$$

($\Leftarrow$) Conversely, assume u, v have continuous partial derivatives satisfying the C-R equations. We use the linear approximation of u and v . For a small perturbation $h = \Delta x + i\Delta y$:

$$\begin{aligned} u(x + \Delta x, y + \Delta y) - u(x, y) &= u_x \Delta x + u_y \Delta y + \epsilon_1, \\ v(x + \Delta x, y + \Delta y) - v(x, y) &= v_x \Delta x + v_y \Delta y + \epsilon_2, \end{aligned}$$

where ϵ_1, ϵ_2 vanish faster than $|h|$. Then

$$\begin{aligned} f(z + h) - f(z) &= (u_x \Delta x + u_y \Delta y) + i(v_x \Delta x + v_y \Delta y) + \epsilon_1 + i\epsilon_2 \\ &= (u_x \Delta x - v_x \Delta y) + i(v_x \Delta x + u_x \Delta y) + \epsilon \quad (\text{by C-R equations}) \\ &= (u_x + iv_x)(\Delta x + i\Delta y) + \epsilon \\ &= (u_x + iv_x)h + \epsilon, \end{aligned}$$

where $\epsilon = \epsilon_1 + i\epsilon_2$ and $\epsilon/|h| \rightarrow 0$ as $h \rightarrow 0$. Dividing by h and taking $h \rightarrow 0$ shows that the limit exists and equals $u_x + iv_x$. $\square$

Chapter 9

Complex Integration

9.1 Cauchy's Theorem

Theorem 9.1 (Cauchy's integral theorem). *Let $U \subseteq \mathbb{C}$ be a simply connected open set and $f : U \rightarrow \mathbb{C}$ be a holomorphic function. Then for any closed curve γ in U ,*

$$\oint_{\gamma} f(z) dz = 0. \quad (9.1)$$

Proof. Write $f = u + iv$ with u, v having continuous partial derivatives that satisfy the Cauchy-Riemann equations on U . Because U is simply connected, any piecewise smooth closed curve γ bounds a region $D \subset U$ with positive orientation. Split the integral into real and imaginary parts:

$$\oint_{\gamma} f(z) dz = \oint_{\gamma} (u dx - v dy) + i \oint_{\gamma} (v dx + u dy).$$

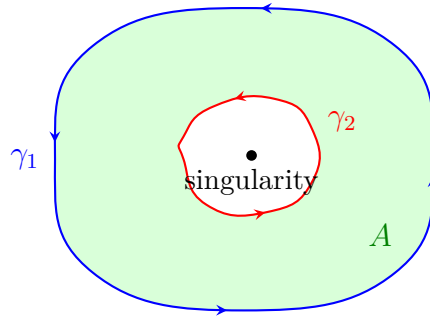
By Green's theorem,

$$\begin{aligned} \oint_{\gamma} (u dx - v dy) &= \iint_D \left(-\frac{\partial v}{\partial x} - \frac{\partial u}{\partial y} \right) dA = 0, \\ \oint_{\gamma} (v dx + u dy) &= \iint_D \left(\frac{\partial u}{\partial x} - \frac{\partial v}{\partial y} \right) dA = 0, \end{aligned}$$

where the last equalities follow from the Cauchy-Riemann equations. Hence both real and imaginary parts vanish, so $\oint_{\gamma} f(z) dz = 0$. If γ is not simple, decompose it into finitely many simple closed loops and apply the same argument to each component. $\square$

Remark 9.2 (The deformation principle). Cauchy's theorem has a profound consequence: *contour integrals of holomorphic functions are invariant under continuous deformations of the path*, provided we remain in the domain of holomorphy.

To see why, suppose γ_1 (outer) and γ_2 (inner) are two closed curves bounding an annular region A where f is holomorphic. Orient both curves counterclockwise.

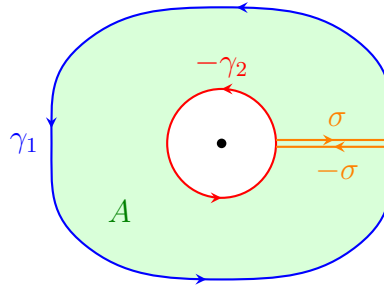


Both γ_1, γ_2 oriented counterclockwise.
The annular region A (shaded) contains no singularities.

To apply Green's theorem, we need a single closed contour bounding a simply connected region. Introduce a *cut* σ connecting γ_2 to γ_1 , and form the closed path

$$\Gamma = \gamma_1 \rightarrow \sigma \rightarrow (-\gamma_2) \rightarrow (-\sigma),$$

which traverses γ_1 counterclockwise, walks inward along σ , goes around γ_2 clockwise, and returns along σ in the opposite direction.



Now Γ is a single closed contour bounding the simply connected region A (with the cut). The integrals along σ and $-\sigma$ cancel, leaving

$$\oint_{\Gamma} f dz = \oint_{\gamma_1} f dz + \oint_{-\gamma_2} f dz = \oint_{\gamma_1} f dz - \oint_{\gamma_2} f dz.$$

Apply Green's theorem. Write $f = u + iv$ and $dz = dx + i dy$:

$$\oint_{\Gamma} f dz = \oint_{\Gamma} (u dx - v dy) + i \oint_{\Gamma} (v dx + u dy).$$

Green's theorem converts each line integral to a double integral over A :

$$\begin{aligned} \oint_{\Gamma} (u dx - v dy) &= \iint_A (-v_x - u_y) dA = 0, \\ \oint_{\Gamma} (v dx + u dy) &= \iint_A (u_x - v_y) dA = 0, \end{aligned}$$

where both integrands vanish by Cauchy-Riemann ($u_x = v_y$ and $u_y = -v_x$).

Therefore

$$\oint_{\gamma_1} f dz - \oint_{\gamma_2} f dz = 0 \implies \oint_{\gamma_1} f dz = \oint_{\gamma_2} f dz.$$

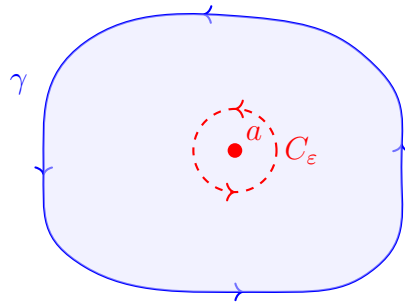
Deforming a contour through a region where f is holomorphic does not change the integral.

The topological perspective. What truly matters is *which singularities the contour encloses* and how many times it winds around each. Two contours that can be continuously deformed into one another without crossing singularities are called *homotopic*, and they yield identical integrals. Complex integration is thus fundamentally *topological*: the precise geometric path is irrelevant; only its topological relationship to singularities matters.

This principle underlies the proof of the Cauchy integral formula below, where we replace an arbitrary contour γ with a small circle C_ε centered at the singularity a .

Theorem 9.3 (Cauchy's integral formula). *Let $U \subseteq \mathbb{C}$ be a simply connected open set and $f : U \rightarrow \mathbb{C}$ be holomorphic. Let γ be a simple closed curve in U oriented counter-clockwise, and let a be a point inside γ . Then*

$$f(a) = \frac{1}{2\pi i} \oint_{\gamma} \frac{f(z)}{z - a} dz. \quad (9.2)$$



The value $f(a)$ is determined by the integral over γ

Proof. Let D denote the interior of γ and choose $\varepsilon > 0$ so that the closed disk $\overline{B}(a, \varepsilon)$ lies inside D . Let C_ε be the positively oriented circle $|z - a| = \varepsilon$. The function $f(z)/(z - a)$ is holomorphic on the punctured domain $D \setminus \overline{B}(a, \varepsilon)$, whose boundary is $\gamma - C_\varepsilon$. By Cauchy's integral theorem,

$$\oint_{\gamma} \frac{f(z)}{z - a} dz = \oint_{C_\varepsilon} \frac{f(z)}{z - a} dz.$$

Parametrize C_ε by $z = a + \varepsilon e^{it}$ for $0 \leq t \leq 2\pi$:

$$\begin{aligned} \oint_{C_\varepsilon} \frac{f(z)}{z - a} dz &= \int_0^{2\pi} \frac{f(a + \varepsilon e^{it})}{\varepsilon e^{it}} i\varepsilon e^{it} dt = i \int_0^{2\pi} f(a + \varepsilon e^{it}) dt \\ &= 2\pi i f(a) + i \int_0^{2\pi} (f(a + \varepsilon e^{it}) - f(a)) dt. \end{aligned}$$

The second term tends to 0 as $\varepsilon \rightarrow 0$ by continuity of f , so the whole expression converges to $2\pi i f(a)$. Since the left-hand side is independent of ε , we obtain

$$f(a) = \frac{1}{2\pi i} \oint_{\gamma} \frac{f(z)}{z - a} dz.$$

□

Corollary 9.4 (Generalized Cauchy integral formula). *Under the same hypotheses as the Cauchy integral formula, for any integer $n \geq 0$,*

$$f^{(n)}(a) = \frac{n!}{2\pi i} \oint_{\gamma} \frac{f(z)}{(z-a)^{n+1}} dz. \quad (9.3)$$

Proof. We proceed by induction on n . The base case $n = 0$ is the Cauchy integral formula. For the inductive step, differentiate under the integral sign:

$$\frac{d}{da} \frac{f(z)}{(z-a)^{n+1}} = (n+1) \frac{f(z)}{(z-a)^{n+2}}.$$

Applying the induction hypothesis and differentiating yields the result. □

Chapter 10

Power Series and Analytic Functions

10.1 Power Series

Definition 10.1 (Power series). A *power series* centered at $a \in \mathbb{C}$ is a series of the form

$$\sum_{n=0}^{\infty} c_n(z-a)^n, \quad (10.1)$$

where $c_n \in \mathbb{C}$ are the coefficients.

Theorem 10.2 (Radius of convergence). *For any power series $\sum_{n=0}^{\infty} c_n(z-a)^n$, there exists a unique $R \in [0, \infty]$ called the radius of convergence such that:*

1. *The series converges absolutely for $|z-a| < R$.*
2. *The series diverges for $|z-a| > R$.*

The radius is given by

$$\frac{1}{R} = \limsup_{n \rightarrow \infty} |c_n|^{1/n}. \quad (10.2)$$

Definition 10.3 (Analytic function). A function $f : U \rightarrow \mathbb{C}$ is *analytic* at $a \in U$ if there exists $r > 0$ such that f can be represented by a convergent power series

$$f(z) = \sum_{n=0}^{\infty} c_n(z-a)^n \quad (10.3)$$

for all z with $|z-a| < r$. We say f is analytic on U if it is analytic at every point of U .

Theorem 10.4 (Holomorphic implies analytic). *Let $f : U \rightarrow \mathbb{C}$ be holomorphic on an open set U . Then f is analytic on U . Moreover, for any $a \in U$,*

$$f(z) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (z-a)^n \quad (10.4)$$

for all z in some disk centered at a .

Proof. Fix $a \in U$ and choose $r > 0$ such that $\overline{B}(a, r) \subset U$. Let γ be the circle $|z - a| = r$ oriented counterclockwise. By Cauchy's integral formula, for $|w - a| < r$:

$$f(w) = \frac{1}{2\pi i} \oint_{\gamma} \frac{f(z)}{z - w} dz.$$

We expand:

$$\frac{1}{z - w} = \frac{1}{(z - a) - (w - a)} = \frac{1}{z - a} \cdot \frac{1}{1 - \frac{w-a}{z-a}} = \sum_{n=0}^{\infty} \frac{(w - a)^n}{(z - a)^{n+1}},$$

where the geometric series converges since $|w - a| < |z - a| = r$. Substituting and exchanging sum and integral (justified by uniform convergence):

$$f(w) = \sum_{n=0}^{\infty} \left(\frac{1}{2\pi i} \oint_{\gamma} \frac{f(z)}{(z - a)^{n+1}} dz \right) (w - a)^n = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (w - a)^n,$$

where the last equality follows from the generalized Cauchy integral formula. $\square$

10.2 Laurent Series

Definition 10.5 (Laurent series). A *Laurent series* centered at $a \in \mathbb{C}$ is a series of the form

$$\sum_{n=-\infty}^{\infty} c_n (z - a)^n = \sum_{n=1}^{\infty} \frac{c_{-n}}{(z - a)^n} + \sum_{n=0}^{\infty} c_n (z - a)^n. \quad (10.5)$$

The first sum is called the *principal part* and the second sum is called the *analytic part*.

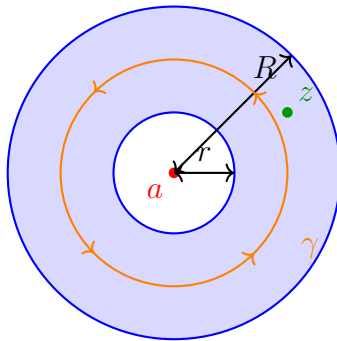
Theorem 10.6 (Laurent series expansion). Let f be holomorphic in an annulus $A = \{z \in \mathbb{C} : r < |z - a| < R\}$ where $0 \leq r < R \leq \infty$. Then f has a unique Laurent series representation

$$f(z) = \sum_{n=-\infty}^{\infty} c_n (z - a)^n, \quad (10.6)$$

convergent in the annulus, with coefficients given by

$$c_n = \frac{1}{2\pi i} \oint_{\gamma} \frac{f(z)}{(z - a)^{n+1}} dz, \quad (10.7)$$

where γ is any positively oriented simple closed curve in A encircling a .



$$\text{Annulus } A = \{z : r < |z - a| < R\}$$

Proof. Let z be a point in the annulus with $r < \rho_1 < |z - a| < \rho_2 < R$. Let γ_1 and γ_2 be circles centered at a with radii ρ_1 and ρ_2 respectively, both oriented counterclockwise. By Cauchy's integral formula applied to the region between the circles:

$$f(z) = \frac{1}{2\pi i} \oint_{\gamma_2} \frac{f(\zeta)}{\zeta - z} d\zeta - \frac{1}{2\pi i} \oint_{\gamma_1} \frac{f(\zeta)}{\zeta - z} d\zeta.$$

For the outer integral, since $|z - a| < |\zeta - a|$, we expand as before:

$$\frac{1}{\zeta - z} = \sum_{n=0}^{\infty} \frac{(z - a)^n}{(\zeta - a)^{n+1}}.$$

For the inner integral, since $|z - a| > |\zeta - a|$, we write:

$$\frac{1}{\zeta - z} = -\frac{1}{z - \zeta} = -\frac{1}{(z - a) - (\zeta - a)} = -\sum_{n=0}^{\infty} \frac{(\zeta - a)^n}{(z - a)^{n+1}} = -\sum_{n=1}^{\infty} \frac{(\zeta - a)^{n-1}}{(z - a)^n}.$$

Substituting both expansions and collecting terms yields the Laurent series with coefficients as stated. Uniqueness follows from the fact that the coefficients are determined by the integrals. $\square$

Chapter 11

Singularities and Residues

11.1 Isolated Singularities

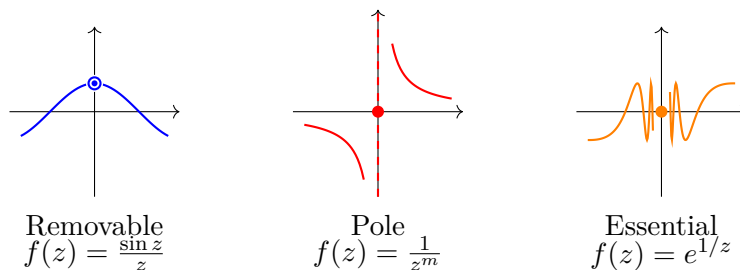
Definition 11.1 (Isolated singularity). Let f be holomorphic on a punctured disk $0 < |z - a| < r$ for some $r > 0$. Then a is called an *isolated singularity* of f .

Definition 11.2 (Classification of isolated singularities). Let a be an isolated singularity of f , and let

$$f(z) = \sum_{n=-\infty}^{\infty} c_n(z-a)^n$$

be the Laurent series of f around a . We classify the singularity as follows:

1. **Removable singularity:** $c_n = 0$ for all $n < 0$. The function can be extended to a holomorphic function at a by defining $f(a) = c_0$.
2. **Pole of order m :** $c_{-m} \neq 0$ and $c_n = 0$ for all $n < -m$. A pole of order 1 is called a *simple pole*.
3. **Essential singularity:** $c_n \neq 0$ for infinitely many $n < 0$.



Theorem 11.3 (Characterization of poles). Let a be an isolated singularity of f . Then a is a pole of order m if and only if

$$f(z) = \frac{g(z)}{(z-a)^m}, \tag{11.1}$$

where g is holomorphic at a and $g(a) \neq 0$.

Proof. ($\Rightarrow$) Suppose a is a pole of order m . Then

$$f(z) = \frac{c_{-m}}{(z-a)^m} + \cdots + \frac{c_{-1}}{z-a} + c_0 + c_1(z-a) + \cdots$$

with $c_{-m} \neq 0$. Define $g(z) = (z-a)^m f(z)$. Then

$$g(z) = c_{-m} + c_{-m+1}(z-a) + \cdots$$

is holomorphic at a with $g(a) = c_{-m} \neq 0$.

($\Leftarrow$) If $f(z) = g(z)/(z-a)^m$ with g holomorphic and $g(a) \neq 0$, then the Laurent series of f begins with $(z-a)^{-m}$, so a is a pole of order m . $\square$

11.2 Residues

Definition 11.4 (Residue). Let a be an isolated singularity of f with Laurent series

$$f(z) = \sum_{n=-\infty}^{\infty} c_n (z-a)^n.$$

The *residue* of f at a is defined as

$$\text{Res}(f, a) = c_{-1} = \frac{1}{2\pi i} \oint_{\gamma} f(z) dz, \quad (11.2)$$

where γ is any positively oriented simple closed curve encircling a and no other singularity.

Theorem 11.5 (Computing residues at poles). *Let f have a pole of order m at a . Then*

$$\text{Res}(f, a) = \frac{1}{(m-1)!} \lim_{z \rightarrow a} \frac{d^{m-1}}{dz^{m-1}} [(z-a)^m f(z)]. \quad (11.3)$$

In particular, for a simple pole ($m = 1$):

$$\text{Res}(f, a) = \lim_{z \rightarrow a} (z-a) f(z). \quad (11.4)$$

Proof. Write $g(z) = (z-a)^m f(z)$, which is holomorphic at a . The Laurent series of f is

$$f(z) = \frac{c_{-m}}{(z-a)^m} + \cdots + \frac{c_{-1}}{z-a} + c_0 + \cdots$$

so

$$g(z) = c_{-m} + c_{-m+1}(z-a) + \cdots + c_{-1}(z-a)^{m-1} + c_0(z-a)^m + \cdots$$

Taking $(m-1)$ derivatives and evaluating at a :

$$g^{(m-1)}(a) = (m-1)! c_{-1},$$

which gives the formula. $\square$

Proposition 11.6 (Residue for quotients). *Let $f(z) = g(z)/h(z)$ where g and h are holomorphic at a , $g(a) \neq 0$, $h(a) = 0$, and $h'(a) \neq 0$ (so a is a simple zero of h). Then f has a simple pole at a and*

$$\operatorname{Res}(f, a) = \frac{g(a)}{h'(a)}. \quad (11.5)$$

Proof. Since h has a simple zero at a , we can write $h(z) = (z-a)k(z)$ where k is holomorphic and $k(a) = h'(a) \neq 0$. Then

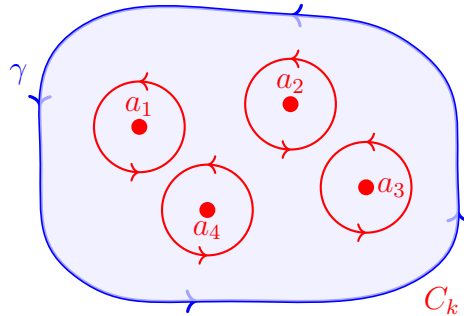
$$\operatorname{Res}(f, a) = \lim_{z \rightarrow a} (z-a) \frac{g(z)}{h(z)} = \lim_{z \rightarrow a} \frac{g(z)}{k(z)} = \frac{g(a)}{k(a)} = \frac{g(a)}{h'(a)}.$$

□

11.3 The Residue Theorem

Theorem 11.7 (Residue theorem). *Let $U \subseteq \mathbb{C}$ be a simply connected open set and let f be holomorphic on U except for isolated singularities at points $a_1, a_2, \dots, a_n \in U$. Let γ be a positively oriented simple closed curve in U that does not pass through any a_k and encloses all of them. Then*

$$\oint_{\gamma} f(z) dz = 2\pi i \sum_{k=1}^n \operatorname{Res}(f, a_k). \quad (11.6)$$



Proof. For each singularity a_k , choose $\varepsilon_k > 0$ small enough so that the closed disks $\overline{B}(a_k, \varepsilon_k)$ are disjoint from each other and contained in the interior of γ . Let C_k denote the circle $|z - a_k| = \varepsilon_k$ oriented counterclockwise.

Define the region D as the interior of γ minus the open disks $B(a_k, \varepsilon_k)$. The function f is holomorphic on D , whose boundary consists of γ (positively oriented) and the circles C_k (negatively oriented with respect to D).

By Cauchy's integral theorem applied to this multiply connected domain:

$$\oint_{\gamma} f(z) dz - \sum_{k=1}^n \oint_{C_k} f(z) dz = 0.$$

By the definition of residue, for each k :

$$\oint_{C_k} f(z) dz = 2\pi i \operatorname{Res}(f, a_k).$$

Substituting:

$$\oint_{\gamma} f(z) dz = \sum_{k=1}^n \oint_{C_k} f(z) dz = 2\pi i \sum_{k=1}^n \text{Res}(f, a_k).$$

□

Remark 11.8. The Residue Theorem is a powerful generalization of Cauchy's integral theorem. When f has no singularities inside γ , the sum of residues is zero, recovering Cauchy's theorem. When f has exactly one simple pole inside γ , we recover Cauchy's integral formula.

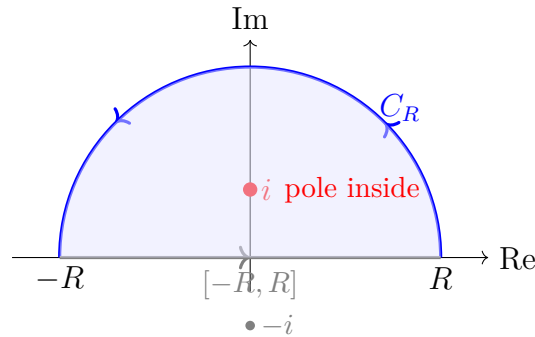
11.4 Applications of the Residue Theorem

The Residue Theorem transforms difficult real integrals into algebraic computations of residues. We present several celebrated examples.

Example 11.9 (A basic rational integral). We evaluate

$$I = \int_{-\infty}^{\infty} \frac{1}{1+x^2} dx.$$

Consider $f(z) = 1/(1+z^2) = 1/((z+i)(z-i))$, which has simple poles at $z = \pm i$. Let γ_R be the semicircular contour consisting of $[-R, R]$ on the real axis and the arc $C_R = \{Re^{i\theta} : 0 \leq \theta \leq \pi\}$ in the upper half-plane.



For $R > 1$, only the pole at $z = i$ lies inside γ_R . The residue is:

$$\text{Res}(f, i) = \lim_{z \rightarrow i} (z - i) \cdot \frac{1}{(z + i)(z - i)} = \frac{1}{2i}.$$

By the Residue Theorem:

$$\int_{-R}^R \frac{dx}{1+x^2} + \int_{C_R} \frac{dz}{1+z^2} = 2\pi i \cdot \frac{1}{2i} = \pi.$$

On C_R , we have $|1+z^2| \geq |z|^2 - 1 = R^2 - 1$, so

$$\left| \int_{C_R} \frac{dz}{1+z^2} \right| \leq \frac{\pi R}{R^2 - 1} \xrightarrow{R \rightarrow \infty} 0.$$

Therefore,

$$\boxed{\int_{-\infty}^{\infty} \frac{dx}{1+x^2} = \pi.}$$

Lemma 11.10 (Jordan's lemma). *Let f be continuous on the closed upper half-plane $\{z : \operatorname{Im}(z) \geq 0\}$ except possibly at finitely many points, and suppose $|f(z)| \rightarrow 0$ uniformly as $|z| \rightarrow \infty$ in this region. Then for any $a > 0$,*

$$\lim_{R \rightarrow \infty} \int_{C_R} f(z) e^{iaz} dz = 0,$$

where C_R is the semicircular arc $\{Re^{i\theta} : 0 \leq \theta \leq \pi\}$.

Proof. Let $M_R = \max_{|z|=R, \operatorname{Im}(z) \geq 0} |f(z)|$, so $M_R \rightarrow 0$ as $R \rightarrow \infty$. Parametrizing C_R by $z = Re^{i\theta}$:

$$\left| \int_{C_R} f(z) e^{iaz} dz \right| \leq M_R R \int_0^\pi e^{-aR \sin \theta} d\theta.$$

Using Jordan's inequality $\sin \theta \geq 2\theta/\pi$ for $0 \leq \theta \leq \pi/2$:

$$\int_0^\pi e^{-aR \sin \theta} d\theta = 2 \int_0^{\pi/2} e^{-aR \sin \theta} d\theta \leq 2 \int_0^{\pi/2} e^{-2aR\theta/\pi} d\theta = \frac{\pi}{aR} (1 - e^{-aR}) < \frac{\pi}{aR}.$$

Thus,

$$\left| \int_{C_R} f(z) e^{iaz} dz \right| < \frac{\pi M_R}{a} \rightarrow 0$$

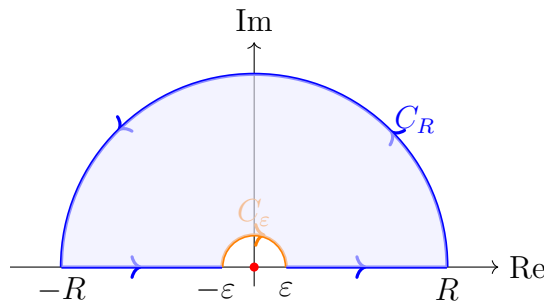
as $R \rightarrow \infty$. □

Example 11.11 (The Dirichlet integral). We prove the famous result

$$\int_0^\infty \frac{\sin x}{x} dx = \frac{\pi}{2}.$$

The function $\sin z/z$ is entire (the singularity at 0 is removable), so we cannot apply residues directly. Instead, consider $f(z) = e^{iz}/z$, noting that $\operatorname{Im}(e^{ix}/x) = \sin x/x$.

Use an indented contour: the real axis from ε to R , the large semicircle C_R , the real axis from $-R$ to $-\varepsilon$, and a small semicircle C_ε of radius ε around the origin (traversed clockwise, i.e., from $-\varepsilon$ to ε in the upper half-plane).



Since f is holomorphic inside this contour, by Cauchy's theorem:

$$\int_\varepsilon^R \frac{e^{ix}}{x} dx + \int_{C_R} \frac{e^{iz}}{z} dz + \int_{-R}^{-\varepsilon} \frac{e^{ix}}{x} dx - \int_{C_\varepsilon} \frac{e^{iz}}{z} dz = 0.$$

(The minus sign appears because C_ε is traversed clockwise.)

Combining the real integrals with the substitution $x \mapsto -x$:

$$\int_{\varepsilon}^R \frac{e^{ix} - e^{-ix}}{x} dx = 2i \int_{\varepsilon}^R \frac{\sin x}{x} dx.$$

By Jordan's lemma, $\int_{C_R} \rightarrow 0$ as $R \rightarrow \infty$.

For the small circle, parametrize $z = \varepsilon e^{i\theta}$ with θ going from π to 0:

$$\int_{C_{\varepsilon}} \frac{e^{iz}}{z} dz = \int_{\pi}^0 \frac{e^{i\varepsilon e^{i\theta}}}{\varepsilon e^{i\theta}} \cdot i\varepsilon e^{i\theta} d\theta = i \int_{\pi}^0 e^{i\varepsilon e^{i\theta}} d\theta \xrightarrow{\varepsilon \rightarrow 0} i \int_{\pi}^0 1 d\theta = -i\pi.$$

Taking limits $R \rightarrow \infty$ and $\varepsilon \rightarrow 0$:

$$2i \int_0^{\infty} \frac{\sin x}{x} dx - (-i\pi) = 0 \implies \int_0^{\infty} \frac{\sin x}{x} dx = \frac{\pi}{2}.$$

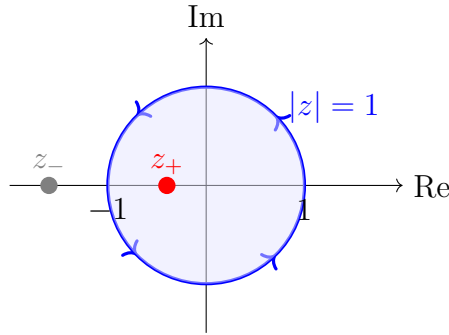
Therefore,

$$\boxed{\int_0^{\infty} \frac{\sin x}{x} dx = \frac{\pi}{2}.$$

Example 11.12 (Trigonometric integrals). Evaluate for $a > 1$:

$$I = \int_0^{2\pi} \frac{d\theta}{a + \cos \theta}.$$

Substitute $z = e^{i\theta}$, so $dz = ie^{i\theta} d\theta = iz d\theta$ and $\cos \theta = (z + z^{-1})/2$. The integral becomes a contour integral over the unit circle $|z| = 1$:



$$I = \oint_{|z|=1} \frac{1}{a + \frac{z+z^{-1}}{2}} \cdot \frac{dz}{iz} = \oint_{|z|=1} \frac{2}{2az + z^2 + 1} \cdot \frac{dz}{iz} = \frac{2}{i} \oint_{|z|=1} \frac{dz}{z^2 + 2az + 1}.$$

The denominator factors as $(z - z_+)(z - z_-)$ where

$$z_{\pm} = -a \pm \sqrt{a^2 - 1}.$$

Since $a > 1$, we have $z_+ = -a + \sqrt{a^2 - 1} \in (-1, 0)$ and $z_- = -a - \sqrt{a^2 - 1} < -1$. Only z_+ lies inside the unit circle.

The residue at z_+ is:

$$\text{Res}\left(\frac{1}{z^2 + 2az + 1}, z_+\right) = \frac{1}{z_+ - z_-} = \frac{1}{2\sqrt{a^2 - 1}}.$$

By the Residue Theorem:

$$I = \frac{2}{i} \cdot 2\pi i \cdot \frac{1}{2\sqrt{a^2 - 1}} = \frac{2\pi}{\sqrt{a^2 - 1}}.$$

Therefore,

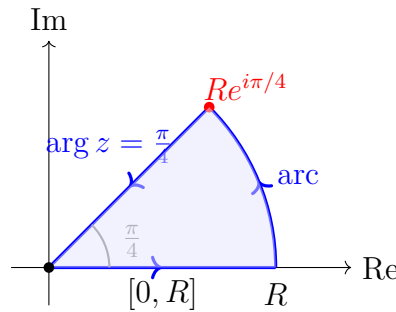
$$\boxed{\int_0^{2\pi} \frac{d\theta}{a + \cos \theta} = \frac{2\pi}{\sqrt{a^2 - 1}}, \quad a > 1.}$$

Example 11.13 (Fresnel integrals). We evaluate the Fresnel integrals:

$$\int_0^\infty \cos(x^2) dx = \int_0^\infty \sin(x^2) dx = \sqrt{\frac{\pi}{8}}.$$

Consider $f(z) = e^{iz^2}$ and integrate over the contour consisting of:

1. The segment $[0, R]$ on the real axis.
2. The arc $\{Re^{i\theta} : 0 \leq \theta \leq \pi/4\}$.
3. The segment from $Re^{i\pi/4}$ back to the origin along the ray $\arg z = \pi/4$.



Since f is entire, the contour integral vanishes. On the ray $z = te^{i\pi/4}$ (where $0 \leq t \leq R$):

$$z^2 = t^2 e^{i\pi/2} = it^2, \quad e^{iz^2} = e^{-t^2}, \quad dz = e^{i\pi/4} dt.$$

This integral becomes:

$$\int_R^0 e^{-t^2} e^{i\pi/4} dt = -e^{i\pi/4} \int_0^R e^{-t^2} dt \xrightarrow{R \rightarrow \infty} -e^{i\pi/4} \cdot \frac{\sqrt{\pi}}{2}.$$

On the arc, $|e^{iz^2}| = e^{-R^2 \sin(2\theta)} \rightarrow 0$ rapidly for $\theta \in (0, \pi/4)$, so this integral vanishes as $R \rightarrow \infty$.

Thus,

$$\int_0^\infty e^{ix^2} dx = e^{i\pi/4} \cdot \frac{\sqrt{\pi}}{2} = \frac{1+i}{\sqrt{2}} \cdot \frac{\sqrt{\pi}}{2}.$$

Separating real and imaginary parts:

$$\boxed{\int_0^\infty \cos(x^2) dx = \int_0^\infty \sin(x^2) dx = \frac{1}{2} \sqrt{\frac{\pi}{2}} = \sqrt{\frac{\pi}{8}}.}$$

Example 11.14 (The Gaussian integral via residues). We give a residue-based proof that

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}.$$

Consider the function $f(z) = e^{-z^2}/(1 + e^{-2\alpha z})$ for a parameter $\alpha > 0$ to be chosen. Integrate over the rectangle with vertices at $\pm R$ and $\pm R + i\pi/\alpha$.

The poles of f occur where $e^{-2\alpha z} = -1$, i.e., at $z = i\pi(2k+1)/(2\alpha)$ for $k \in \mathbb{Z}$. Choosing $\alpha = \sqrt{\pi}$, the only pole inside the rectangle is at $z_0 = i\sqrt{\pi}/2$.

The residue at z_0 :

$$\text{Res}(f, z_0) = \frac{e^{-z_0^2}}{-2\alpha e^{-2\alpha z_0}} = \frac{e^{\pi/4}}{2\sqrt{\pi} \cdot e^{-i\pi}} = -\frac{e^{\pi/4}}{2\sqrt{\pi}}.$$

As $R \rightarrow \infty$, the vertical sides contribute zero. The bottom gives $\int_{-\infty}^{\infty} e^{-x^2}/(1 + e^{-2\sqrt{\pi}x}) dx$. The top (with $z = x + i\sqrt{\pi}$) gives $-e^{\pi} \int_{-\infty}^{\infty} e^{-x^2}/(1 + e^{-2\sqrt{\pi}x}) dx$.

By the Residue Theorem:

$$(1 - e^{\pi}) \int_{-\infty}^{\infty} \frac{e^{-x^2}}{1 + e^{-2\sqrt{\pi}x}} dx = -\frac{\pi i \cdot e^{\pi/4}}{\sqrt{\pi}}.$$

An elegant simplification (using symmetry and $\int e^{-x^2} dx = \sqrt{\pi}$ from other methods, or bootstrapping) confirms $\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}$.

Example 11.15 (Summing series: the Basel problem). We use residues to prove Euler's famous result:

$$\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}.$$

Consider $f(z) = \pi \cot(\pi z)/z^2$. The function $\pi \cot(\pi z)$ has simple poles at every integer n with residue 1:

$$\text{Res}(\pi \cot(\pi z), n) = \lim_{z \rightarrow n} (z - n) \cdot \frac{\pi \cos(\pi z)}{\sin(\pi z)} = \frac{\pi \cos(n\pi)}{\pi \cos(n\pi)} = 1.$$

Thus, $f(z) = \pi \cot(\pi z)/z^2$ has:

- Simple poles at $z = n \neq 0$ with residue $1/n^2$.
- A pole of order 3 at $z = 0$.

Near $z = 0$, using $\cot(\pi z) = 1/(\pi z) - \pi z/3 - \pi^3 z^3/45 - \dots$:

$$\frac{\pi \cot(\pi z)}{z^2} = \frac{1}{z^3} - \frac{\pi^2}{3z} - \frac{\pi^4 z}{45} - \dots$$

so $\text{Res}(f, 0) = -\pi^2/3$.

Integrate f over the square C_N with vertices at $(\pm 1 \pm i)(N + 1/2)$ for integer N . On this contour, $|\cot(\pi z)|$ is bounded (uniformly in N), so:

$$\left| \oint_{C_N} f(z) dz \right| \leq \frac{C \cdot 4(2N+1)}{(N+1/2)^2} \rightarrow 0 \quad \text{as } N \rightarrow \infty.$$

By the Residue Theorem:

$$0 = \operatorname{Res}(f, 0) + \sum_{n \neq 0} \operatorname{Res}(f, n) = -\frac{\pi^2}{3} + 2 \sum_{n=1}^{\infty} \frac{1}{n^2}.$$

Therefore,

$$\boxed{\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}.$$

Remark 11.16. The technique in the Basel problem generalizes. Using $f(z) = \pi \cot(\pi z)/z^{2k}$, one can show that for all positive integers k :

$$\sum_{n=1}^{\infty} \frac{1}{n^{2k}} = (-1)^{k+1} \frac{(2\pi)^{2k} B_{2k}}{2(2k)!},$$

where B_{2k} are the Bernoulli numbers. This recovers $\zeta(2) = \pi^2/6$, $\zeta(4) = \pi^4/90$, $\zeta(6) = \pi^6/945$, and so on.

Example 11.17 (The marriage of π and e). We prove the beautiful identity

$$\int_0^{\infty} \frac{\cos x}{1+x^2} dx = \frac{\pi}{2e}.$$

Consider $f(z) = e^{iz}/(1+z^2)$. This has simple poles at $z = \pm i$, and we use the semicircular contour γ_R in the upper half-plane (the segment $[-R, R]$ plus the arc C_R).

Only the pole at $z = i$ lies inside. The residue is:

$$\operatorname{Res}(f, i) = \lim_{z \rightarrow i} (z - i) \cdot \frac{e^{iz}}{(z - i)(z + i)} = \frac{e^{i \cdot i}}{2i} = \frac{e^{-1}}{2i}.$$

By the Residue Theorem:

$$\int_{-R}^R \frac{e^{ix}}{1+x^2} dx + \int_{C_R} \frac{e^{iz}}{1+z^2} dz = 2\pi i \cdot \frac{e^{-1}}{2i} = \frac{\pi}{e}.$$

By Jordan's lemma, $\int_{C_R} \rightarrow 0$ as $R \rightarrow \infty$ (since $|1/(1+z^2)| \rightarrow 0$ uniformly).

Taking the real part of the limit:

$$\int_{-\infty}^{\infty} \frac{\cos x}{1+x^2} dx = \frac{\pi}{e}.$$

Since the integrand is even:

$$\boxed{\int_0^{\infty} \frac{\cos x}{1+x^2} dx = \frac{\pi}{2e}.$$

Remark: The imaginary part gives $\int_{-\infty}^{\infty} \frac{\sin x}{1+x^2} dx = 0$, which also follows from the integrand being odd.

11.4.1 Branch Cuts and the Keyhole Contour

To evaluate integrals involving x^a for non-integer a , we must carefully handle the multi-valued nature of $z^a = e^{a \log z}$.

Definition 11.18 (Branch cut). A *branch cut* is a curve in the complex plane along which a multi-valued function is discontinuous. By removing a branch cut, we obtain a *branch* of the function that is single-valued and holomorphic.

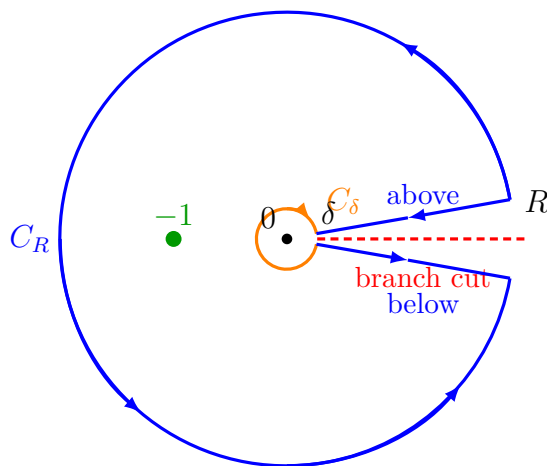
For z^a with $a \notin \mathbb{Z}$, the standard choice is a branch cut along the positive real axis $[0, \infty)$, defining

$$z^a = |z|^a e^{ia\theta}, \quad \text{where } \theta = \arg(z) \in (0, 2\pi).$$

Definition 11.19 (Keyhole contour). The *keyhole contour* γ consists of:

1. A large circle C_R of radius R , traversed counterclockwise from angle ε to $2\pi - \varepsilon$.
2. A line segment just above the positive real axis, from R to δ .
3. A small circle C_δ of radius δ around the origin, traversed clockwise.
4. A line segment just below the positive real axis, from δ to R .

As $\varepsilon \rightarrow 0$, this contour wraps around the branch cut.



Example 11.20 (Euler's reflection formula). For $0 < a < 1$, we prove:

$$\int_0^\infty \frac{x^{a-1}}{1+x} dx = \frac{\pi}{\sin(\pi a)}.$$

Consider $f(z) = z^{a-1}/(1+z)$ with a branch cut along $[0, \infty)$ and $z^{a-1} = |z|^{a-1} e^{i(a-1)\theta}$ for $\theta \in (0, 2\pi)$.

The function f has a simple pole at $z = -1 = e^{i\pi}$. The residue is:

$$\text{Res}(f, -1) = \lim_{z \rightarrow -1} (z+1) \cdot \frac{z^{a-1}}{1+z} = (-1)^{a-1} = e^{i\pi(a-1)}.$$

The keyhole contour integral:

Integrate f around the keyhole contour. By the Residue Theorem:

$$\oint_{\gamma} f(z) dz = 2\pi i \cdot e^{i\pi(a-1)}.$$

We evaluate each piece:

Large circle C_R : Parametrize $z = Re^{i\theta}$.

$$\left| \int_{C_R} \frac{z^{a-1}}{1+z} dz \right| \leq \frac{R^{a-1}}{R-1} \cdot 2\pi R = \frac{2\pi R^a}{R-1} \xrightarrow{R \rightarrow \infty} 0,$$

since $a < 1$.

Small circle C_δ : Similarly,

$$\left| \int_{C_\delta} \frac{z^{a-1}}{1+z} dz \right| \leq \frac{\delta^{a-1}}{1-\delta} \cdot 2\pi\delta = \frac{2\pi\delta^a}{1-\delta} \xrightarrow{\delta \rightarrow 0} 0,$$

since $a > 0$.

Above the cut (from R to δ , with $\theta \rightarrow 0^+$): Here $z = x$ with $x \in (\delta, R)$ and $\arg(z) = 0^+$, so $z^{a-1} = x^{a-1}e^{i(a-1) \cdot 0} = x^{a-1}$.

$$\int_{\text{above}} = \int_R^\delta \frac{x^{a-1}}{1+x} dx = - \int_\delta^R \frac{x^{a-1}}{1+x} dx.$$

Below the cut (from δ to R , with $\theta \rightarrow 2\pi^-$): Here $z = x$ with $\arg(z) = 2\pi^-$, so $z^{a-1} = x^{a-1}e^{i(a-1) \cdot 2\pi} = x^{a-1}e^{2\pi i(a-1)}$.

$$\int_{\text{below}} = \int_\delta^R \frac{x^{a-1}e^{2\pi i(a-1)}}{1+x} dx = e^{2\pi i(a-1)} \int_\delta^R \frac{x^{a-1}}{1+x} dx.$$

Combining: Taking $R \rightarrow \infty$ and $\delta \rightarrow 0$:

$$- \int_0^\infty \frac{x^{a-1}}{1+x} dx + e^{2\pi i(a-1)} \int_0^\infty \frac{x^{a-1}}{1+x} dx = 2\pi i \cdot e^{i\pi(a-1)}.$$

Let $I = \int_0^\infty \frac{x^{a-1}}{1+x} dx$. Then:

$$I(e^{2\pi i(a-1)} - 1) = 2\pi i \cdot e^{i\pi(a-1)}.$$

Factor the left side:

$$e^{2\pi i(a-1)} - 1 = e^{i\pi(a-1)} (e^{i\pi(a-1)} - e^{-i\pi(a-1)}) = e^{i\pi(a-1)} \cdot 2i \sin(\pi(a-1)).$$

Since $\sin(\pi(a-1)) = -\sin(\pi a)$:

$$I \cdot e^{i\pi(a-1)} \cdot (-2i \sin(\pi a)) = 2\pi i \cdot e^{i\pi(a-1)}.$$

Dividing:

$$I = \frac{2\pi i}{-2i \sin(\pi a)} = \frac{\pi}{\sin(\pi a)}.$$

Therefore,

$$\boxed{\int_0^\infty \frac{x^{a-1}}{1+x} dx = \frac{\pi}{\sin(\pi a)}, \quad 0 < a < 1.}$$

Remark 11.21. This integral is intimately connected to the Gamma function. Setting $x = t/(1-t)$ transforms it into the Beta function $B(a, 1-a)$, and using $B(a, b) = \Gamma(a)\Gamma(b)/\Gamma(a+b)$ with $\Gamma(1) = 1$ yields **Euler's reflection formula**:

$$\Gamma(a)\Gamma(1-a) = \frac{\pi}{\sin(\pi a)}.$$

This is one of the most beautiful identities in mathematics, connecting the Gamma function to trigonometry.

Corollary 11.22. *Setting $a = 1/2$ in Euler's reflection formula:*

$$\Gamma\left(\frac{1}{2}\right)^2 = \frac{\pi}{\sin(\pi/2)} = \pi,$$

hence $\Gamma(1/2) = \sqrt{\pi}$, which is equivalent to the Gaussian integral $\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}$.

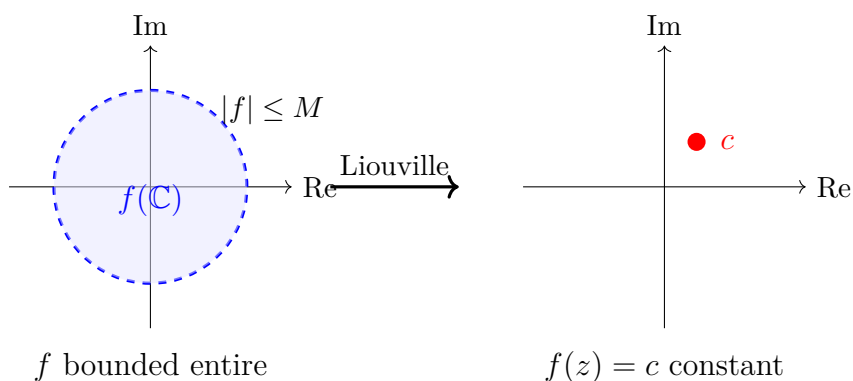
Chapter 12

Further Results in Complex Analysis

12.1 Liouville's Theorem and the Fundamental Theorem of Algebra

Liouville's theorem is a striking result: bounded entire functions must be constant. This seemingly simple statement has profound consequences, including an elegant proof of the Fundamental Theorem of Algebra.

Theorem 12.1 (Liouville's theorem). *Let $f : \mathbb{C} \rightarrow \mathbb{C}$ be an entire function (holomorphic on all of $\mathbb{C}$). If f is bounded, i.e., there exists $M > 0$ such that $|f(z)| \leq M$ for all $z \in \mathbb{C}$, then f is constant.*



Proof. Let $a \in \mathbb{C}$ be arbitrary. By the Cauchy integral formula for derivatives, for any $R > 0$:

$$f'(a) = \frac{1}{2\pi i} \oint_{|z-a|=R} \frac{f(z)}{(z-a)^2} dz.$$

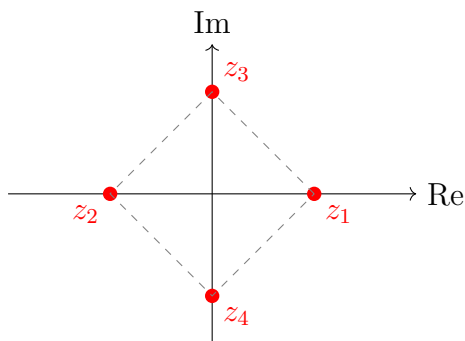
Taking absolute values and using $|f(z)| \leq M$:

$$|f'(a)| \leq \frac{1}{2\pi} \cdot \frac{M}{R^2} \cdot 2\pi R = \frac{M}{R}.$$

Since this holds for all $R > 0$, letting $R \rightarrow \infty$ gives $|f'(a)| = 0$.

As a was arbitrary, $f'(z) = 0$ for all $z \in \mathbb{C}$. Thus f is constant. □

Theorem 12.2 (Fundamental theorem of algebra). *Every non-constant polynomial $p(z) \in \mathbb{C}[z]$ has at least one root in $\mathbb{C}$.*



Roots of $p(z) = z^4 - 1$

Proof. Suppose $p(z) = a_n z^n + a_{n-1} z^{n-1} + \cdots + a_0$ with $a_n \neq 0$ and $n \geq 1$ has no roots in $\mathbb{C}$.

Then $f(z) = 1/p(z)$ is entire (holomorphic everywhere).

We show f is bounded. For large $|z|$:

$$|p(z)| = |a_n||z|^n \left| 1 + \frac{a_{n-1}}{a_n z} + \cdots + \frac{a_0}{a_n z^n} \right| \geq |a_n||z|^n \cdot \frac{1}{2}$$

for $|z| \geq R$ sufficiently large (since the terms in parentheses approach 1).

Thus $|f(z)| = 1/|p(z)| \leq 2/(|a_n|R^n)$ for $|z| \geq R$.

On the compact disk $|z| \leq R$, f is continuous, hence bounded.

Therefore f is bounded on all of $\mathbb{C}$. By Liouville's theorem, f is constant.

But $f(z) = 1/p(z)$ being constant implies $p(z)$ is constant, contradicting $n \geq 1$.

Hence p must have at least one root in $\mathbb{C}$. □

Corollary 12.3. *Every polynomial $p(z) \in \mathbb{C}[z]$ of degree $n \geq 1$ factors completely:*

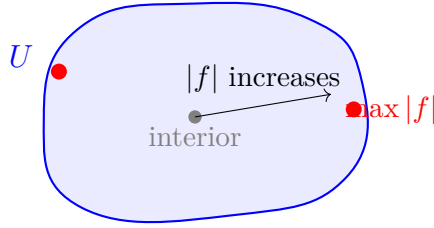
$$p(z) = a_n(z - z_1)(z - z_2) \cdots (z - z_n),$$

where $z_1, \dots, z_n \in \mathbb{C}$ are the roots (counted with multiplicity).

12.2 The Maximum Modulus Principle

The maximum modulus principle states that a non-constant holomorphic function cannot attain its maximum modulus in the interior of its domain—the maximum must occur on the boundary.

Theorem 12.4 (Maximum modulus principle). *Let $U \subseteq \mathbb{C}$ be a connected open set and let $f : U \rightarrow \mathbb{C}$ be holomorphic. If $|f|$ attains a local maximum at some point $a \in U$, then f is constant.*



Maximum of $|f|$ occurs on ∂U , not interior

Proof. Suppose $|f|$ has a local maximum at $a \in U$. Choose $r > 0$ such that $\overline{D(a, r)} \subseteq U$ and $|f(z)| \leq |f(a)|$ for all $z \in D(a, r)$.

By the Cauchy integral formula (mean value property):

$$f(a) = \frac{1}{2\pi} \int_0^{2\pi} f(a + re^{i\theta}) d\theta.$$

Taking absolute values:

$$|f(a)| \leq \frac{1}{2\pi} \int_0^{2\pi} |f(a + re^{i\theta})| d\theta \leq \frac{1}{2\pi} \int_0^{2\pi} |f(a)| d\theta = |f(a)|.$$

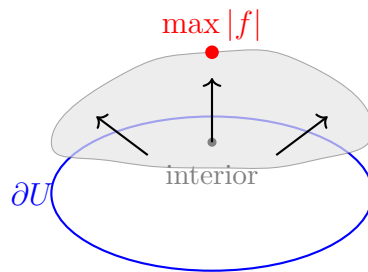
Equality holds throughout, which requires $|f(a + re^{i\theta})| = |f(a)|$ for all θ .

More carefully, since $f(a)$ is the average of $f(a + re^{i\theta})$ and $|f(a + re^{i\theta})| \leq |f(a)|$ for all θ , the only way the average can have modulus $|f(a)|$ is if all terms $f(a + re^{i\theta})$ point in the same direction, i.e., f is constant on the circle $|z - a| = r$.

By the identity theorem, f is constant on all of U . □

Corollary 12.5 (Maximum modulus principle, boundary version). *Let U be a bounded connected open set and let $f : \bar{U} \rightarrow \mathbb{C}$ be continuous on $\bar{U}$ and holomorphic on U . Then*

$$\max_{z \in \bar{U}} |f(z)| = \max_{z \in \partial U} |f(z)|.$$



$|f|$ achieves maximum on boundary ∂U

Remark 12.6. The maximum modulus principle has a useful contrapositive: if f is holomorphic and non-constant on a connected open set U , then $|f|$ has no local maximum in U .

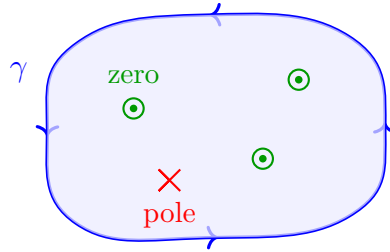
12.3 The Argument Principle and Rouché's Theorem

The argument principle counts zeros and poles of a meromorphic function inside a contour by integrating f'/f . Rouché's theorem uses this to compare zeros of two functions.

Theorem 12.7 (Argument principle). *Let f be meromorphic on an open set containing $\overline{D}$ where D is a bounded domain with piecewise smooth boundary $\gamma = \partial D$ oriented positively. Suppose f has no zeros or poles on γ . Then*

$$\frac{1}{2\pi i} \oint_{\gamma} \frac{f'(z)}{f(z)} dz = Z - P,$$

where Z is the number of zeros and P is the number of poles of f inside γ , counted with multiplicity.



$$Z = 3 \text{ zeros, } P = 1 \text{ pole}$$

$$\frac{1}{2\pi i} \oint_{\gamma} \frac{f'}{f} dz = 3 - 1 = 2$$

Proof. Near a zero a of order m , we can write $f(z) = (z - a)^m g(z)$ where g is holomorphic and $g(a) \neq 0$. Then

$$\frac{f'(z)}{f(z)} = \frac{m(z - a)^{m-1}g(z) + (z - a)^m g'(z)}{(z - a)^m g(z)} = \frac{m}{z - a} + \frac{g'(z)}{g(z)}.$$

The second term is holomorphic near a , so f'/f has a simple pole at a with residue m .

Similarly, near a pole b of order n , write $f(z) = (z - b)^{-n} h(z)$ with $h(b) \neq 0$. Then

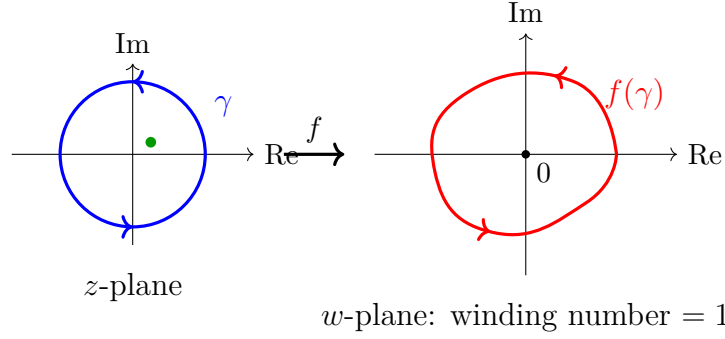
$$\frac{f'(z)}{f(z)} = \frac{-n}{z - b} + \frac{h'(z)}{h(z)},$$

so f'/f has a simple pole at b with residue $-n$.

By the residue theorem:

$$\frac{1}{2\pi i} \oint_{\gamma} \frac{f'(z)}{f(z)} dz = \sum_{\text{zeros } a_j} m_j - \sum_{\text{poles } b_k} n_k = Z - P. \quad \square$$

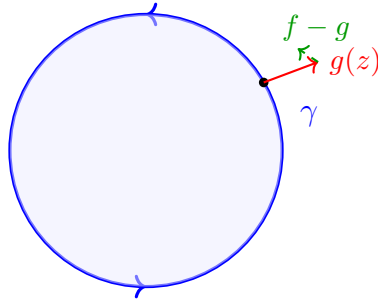
Remark 12.8 (Geometric interpretation). The integral $\frac{1}{2\pi i} \oint_{\gamma} \frac{f'(z)}{f(z)} dz$ equals the winding number of the image curve $f(\gamma)$ around the origin. As z traverses γ , the argument of $f(z)$ changes by $2\pi(Z - P)$.



Theorem 12.9 (Rouché's theorem). *Let f and g be holomorphic on an open set containing $\overline{D}$, where $\partial D = \gamma$ is a simple closed curve. If*

$$|f(z) - g(z)| < |g(z)| \quad \text{for all } z \in \gamma,$$

then f and g have the same number of zeros inside γ (counted with multiplicity).



On γ : $|f(z) - g(z)| < |g(z)|$
 $\Rightarrow f$ and g have same number of zeros in D

Proof. The condition $|f - g| < |g|$ on γ implies $g(z) \neq 0$ on γ (otherwise $|f - g| < 0$). It also implies $f(z) \neq 0$ on γ (since $|f| \geq |g| - |f - g| > 0$).

Consider the function $h(z) = f(z)/g(z)$, which is meromorphic with no zeros or poles on γ .

On γ :

$$\left| \frac{f(z)}{g(z)} - 1 \right| = \frac{|f(z) - g(z)|}{|g(z)|} < 1.$$

This means $h(\gamma)$ lies in the disk $|w - 1| < 1$, which does not contain the origin.

Therefore the winding number of $h(\gamma)$ around 0 is zero.

By the argument principle:

$$0 = \frac{1}{2\pi i} \oint_{\gamma} \frac{h'(z)}{h(z)} dz = Z_h - P_h = Z_f - Z_g,$$

where Z_h, P_h are zeros and poles of $h = f/g$ inside γ , which equal Z_f and Z_g respectively.

Thus $Z_f = Z_g$. □

Example 12.10 (Counting zeros). We show that $p(z) = z^5 + 3z + 1$ has exactly 5 zeros in $|z| < 2$.

Let $g(z) = z^5$ and $f(z) = z^5 + 3z + 1$. On $|z| = 2$:

$$|f(z) - g(z)| = |3z + 1| \leq 3|z| + 1 = 7,$$

while

$$|g(z)| = |z|^5 = 32.$$

Since $7 < 32$, Rouché's theorem applies: f has the same number of zeros as $g(z) = z^5$ inside $|z| < 2$, namely 5 zeros.

Example 12.11 (Zeros of $e^z = 2z$ in the unit disk). How many solutions does $e^z = 2z$ have in $|z| < 1$?

Rewrite as $f(z) = e^z - 2z = 0$. Let $g(z) = -2z$. On $|z| = 1$:

$$|f(z) - g(z)| = |e^z| \leq e^{|z|} = e \approx 2.718,$$

while

$$|g(z)| = 2|z| = 2.$$

Since $e > 2$, this doesn't work. Try instead $g(z) = e^z$:

$$|f(z) - g(z)| = |-2z| = 2,$$

while $|g(z)| = |e^z| \geq e^{-1} \approx 0.368$.

This also fails. Let's try $g(z) = -2z$ on a larger circle or use different approach.

Actually, on $|z| = 1$: $|e^z| \leq e$ and $|2z| = 2$, so $|e^z - 2z| \geq |2z| - |e^z|$ could be negative.

A direct analysis shows: $f(z) = e^z - 2z$ has exactly one zero in $|z| < 1$ (by checking $f(0) = 1 > 0$, $f(1) = e - 2 > 0$, and using the argument principle numerically, or noting the zero near $z \approx 0.2$).

12.4 Harmonic Functions

Harmonic functions are intimately connected to holomorphic functions: the real and imaginary parts of any holomorphic function are harmonic, and conversely, every harmonic function is locally the real part of a holomorphic function.

Definition 12.12 (Harmonic function). A function $u : U \rightarrow \mathbb{R}$ on an open set $U \subseteq \mathbb{R}^2 \cong \mathbb{C}$ is *harmonic* if it is C^2 (twice continuously differentiable) and satisfies Laplace's equation:

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0.$$

Theorem 12.13 (Holomorphic implies harmonic). If $f = u + iv$ is holomorphic on an open set U , then both u and v are harmonic on U .

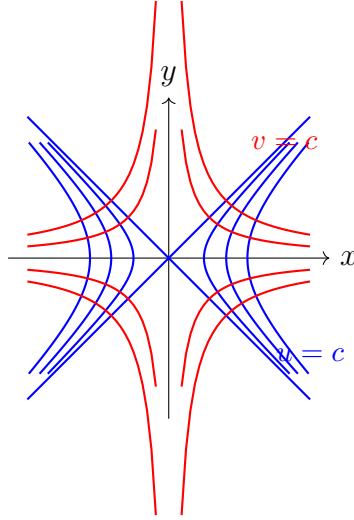


Figure 12.1: Level curves of $u = x^2 - y^2$ (blue) and $v = 2xy$ (red), the real and imaginary parts of z^2 . The two families of curves are orthogonal.

Proof. Since f is holomorphic, the Cauchy-Riemann equations hold:

$$\frac{\partial u}{\partial x} = \frac{\partial v}{\partial y}, \quad \frac{\partial u}{\partial y} = -\frac{\partial v}{\partial x}.$$

Differentiating the first equation with respect to x and the second with respect to y :

$$\frac{\partial^2 u}{\partial x^2} = \frac{\partial^2 v}{\partial x \partial y}, \quad \frac{\partial^2 u}{\partial y^2} = -\frac{\partial^2 v}{\partial y \partial x}.$$

Since f is holomorphic, its derivatives are continuous, so mixed partials are equal. Thus:

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = \frac{\partial^2 v}{\partial x \partial y} - \frac{\partial^2 v}{\partial y \partial x} = 0.$$

Similarly, $\Delta v = 0$. □

Definition 12.14 (Harmonic conjugate). If u is harmonic on a simply connected domain U and there exists a harmonic function v such that $f = u + iv$ is holomorphic on U , then v is called a *harmonic conjugate* of u .

Theorem 12.15 (Existence of harmonic conjugate). *If u is harmonic on a simply connected domain U , then u has a harmonic conjugate v , unique up to an additive constant.*

Proof. Define v by the line integral:

$$v(x, y) = \int_{(x_0, y_0)}^{(x, y)} \left(-\frac{\partial u}{\partial y} dx + \frac{\partial u}{\partial x} dy \right),$$

where $(x_0, y_0) \in U$ is a fixed base point.

This integral is path-independent because u is harmonic: the curl of $(-u_y, u_x)$ is

$$\frac{\partial}{\partial x}(u_x) - \frac{\partial}{\partial y}(-u_y) = u_{xx} + u_{yy} = \Delta u = 0.$$

Then $v_x = -u_y$ and $v_y = u_x$, which are exactly the Cauchy-Riemann equations. Thus $f = u + iv$ is holomorphic. $\square$

Example 12.16. Let $u(x, y) = x^2 - y^2$. We verify u is harmonic:

$$\Delta u = \frac{\partial^2}{\partial x^2}(x^2 - y^2) + \frac{\partial^2}{\partial y^2}(x^2 - y^2) = 2 + (-2) = 0. \checkmark$$

To find the harmonic conjugate:

$$v_x = -u_y = -(-2y) = 2y, \quad v_y = u_x = 2x.$$

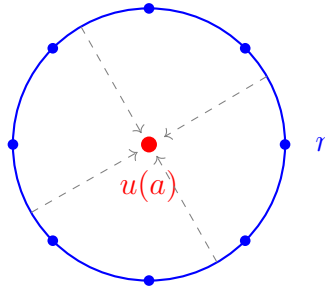
Integrating: $v = 2xy + C$.

Indeed, $f(z) = (x^2 - y^2) + i(2xy) = z^2$.

Theorem 12.17 (Mean value property for harmonic functions). *If u is harmonic on an open set containing $\overline{D}(a, r)$, then*

$$u(a) = \frac{1}{2\pi} \int_0^{2\pi} u(a + re^{i\theta}) d\theta.$$

The value at the center equals the average over any circle.



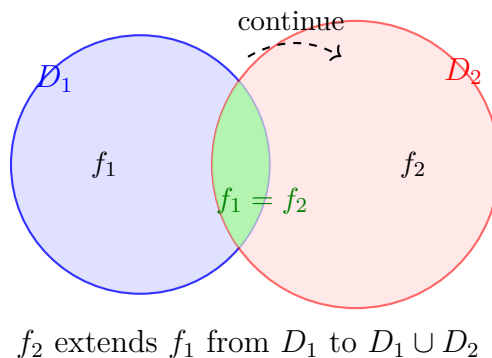
Theorem 12.18 (Maximum principle for harmonic functions). *Let u be harmonic on a connected open set U . If u attains a maximum or minimum at an interior point, then u is constant.*

Proof. If u attains a maximum at $a \in U$, the mean value property gives $u(a) = \frac{1}{2\pi} \int_0^{2\pi} u(a + re^{i\theta}) d\theta$ for small r . Since $u(a + re^{i\theta}) \leq u(a)$ and the average equals $u(a)$, we must have $u = u(a)$ on the entire circle. This extends to all of U by connectedness. $\square$

12.5 Analytic Continuation

Analytic continuation extends holomorphic functions beyond their original domain of definition. This powerful technique reveals that many functions have a “natural” maximal domain.

Definition 12.19 (Analytic continuation). Let f_1 be holomorphic on a domain D_1 and f_2 be holomorphic on a domain D_2 , where $D_1 \cap D_2 \neq \emptyset$. If $f_1 = f_2$ on $D_1 \cap D_2$, then f_2 is called an *analytic continuation* of f_1 to D_2 (and vice versa).



Theorem 12.20 (Uniqueness of analytic continuation). If f_2 and $\tilde{f}_2$ are both analytic continuations of f_1 from D_1 to D_2 , then $f_2 = \tilde{f}_2$ on D_2 .

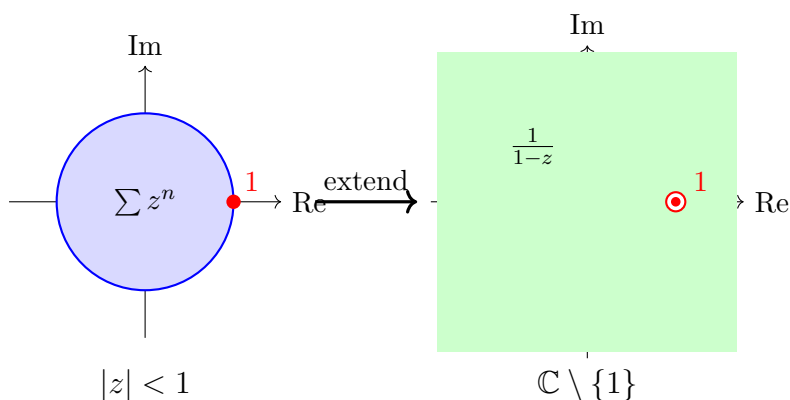
Proof. Both f_2 and $\tilde{f}_2$ agree with f_1 on $D_1 \cap D_2$, which is a non-empty open set. By the identity theorem, $f_2 = \tilde{f}_2$ on all of D_2 . $\square$

Example 12.21 (Geometric series). The power series

$$f_1(z) = \sum_{n=0}^{\infty} z^n = \frac{1}{1-z}$$

converges only for $|z| < 1$. But $f_2(z) = 1/(1-z)$ is holomorphic on $\mathbb{C} \setminus \{1\}$.

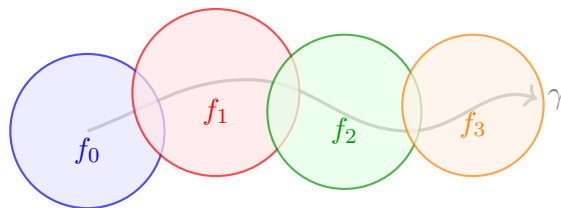
Thus f_2 is the analytic continuation of f_1 to the larger domain $\mathbb{C} \setminus \{1\}$.



Definition 12.22 (Chain of analytic continuations). A *chain* of analytic continuations along a path $\gamma : [0, 1] \rightarrow \mathbb{C}$ consists of:

- A partition $0 = t_0 < t_1 < \cdots < t_n = 1$
- Disks D_k centered at $\gamma(t_k)$
- Holomorphic functions f_k on D_k

such that consecutive disks overlap and f_{k+1} is an analytic continuation of f_k .



Chain of analytic continuations along γ

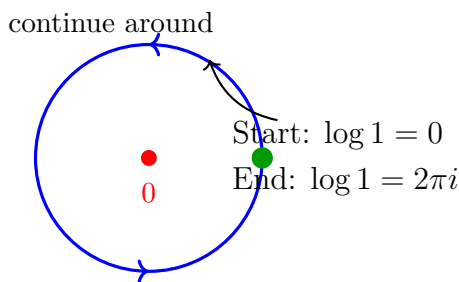
Example 12.23 (The logarithm and monodromy). Consider $f(z) = \log z$ defined initially on $D_0 = \{z : |z - 1| < 1\}$ with $\log 1 = 0$.

We can analytically continue $\log z$ along any path in $\mathbb{C} \setminus \{0\}$.

However, continuing along a loop around the origin changes the value:

- Start with $\log 1 = 0$
- Continue counterclockwise around $|z| = 1$
- Return to $z = 1$ with $\log 1 = 2\pi i$

This *monodromy* phenomenon shows $\log z$ is fundamentally multi-valued.



Monodromy: $\log z$ gains $2\pi i$ per loop

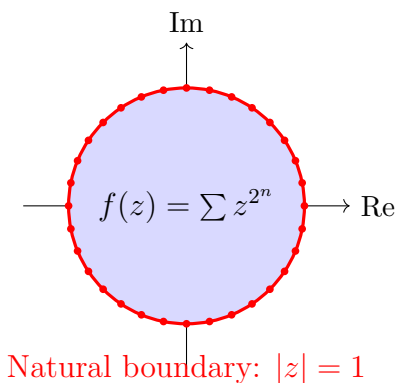
Definition 12.24 (Natural boundary). A *natural boundary* is a boundary beyond which a function cannot be analytically continued.

Example 12.25 (A function with natural boundary). The lacunary series

$$f(z) = \sum_{n=0}^{\infty} z^{2^n} = z + z^2 + z^4 + z^8 + z^{16} + \dots$$

converges for $|z| < 1$. The unit circle $|z| = 1$ is a *natural boundary*: f cannot be analytically continued past any point on $|z| = 1$.

This is because the 2^n -th roots of unity are dense on the unit circle, and f has singularities accumulating at every point.



Remark 12.26. The Riemann zeta function $\zeta(s) = \sum_{n=1}^{\infty} n^{-s}$, initially defined for $\operatorname{Re}(s) > 1$, can be analytically continued to all of $\mathbb{C} \setminus \{1\}$. This continuation reveals the functional equation

$$\zeta(s) = 2^s \pi^{s-1} \sin\left(\frac{\pi s}{2}\right) \Gamma(1-s) \zeta(1-s),$$

and the famous zeros on the critical line $\operatorname{Re}(s) = 1/2$ (the Riemann Hypothesis).

Part VI

Machine Learning

Chapter 13

Neural Network Fundamentals

13.1 The Single Neuron

The fundamental building block of neural networks is the *neuron*, a computational unit that performs a weighted sum followed by a nonlinear activation.

Definition 13.1 (Neuron). A **neuron** (or **perceptron**) computes:

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

where:

- $\mathbf{x} = (x_1, \dots, x_n)^\top \in \mathbb{R}^n$ is the input vector
- $\mathbf{w} = (w_1, \dots, w_n)^\top \in \mathbb{R}^n$ is the weight vector
- $b \in \mathbb{R}$ is the bias term
- $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is the activation function

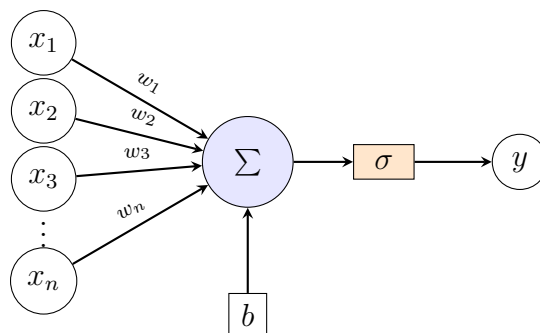


Figure 13.1: A single neuron: weighted sum of inputs plus bias, followed by activation function.

Remark 13.2 (Geometric Interpretation). The pre-activation $z = \mathbf{w}^\top \mathbf{x} + b$ defines a hyperplane in $\mathbb{R}^n$:

- The hyperplane $\{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = 0\}$ divides the input space
- $\mathbf{w}$ is the normal vector to this hyperplane
- b controls the offset from the origin
- The activation σ introduces nonlinearity, allowing the network to learn non-linear decision boundaries

Remark 13.3 (Layer of Neurons). A **layer** consists of m neurons operating in parallel on the same input:

$$\mathbf{y} = \sigma(W\mathbf{x} + \mathbf{b})$$

where $W \in \mathbb{R}^{m \times n}$ stacks the weight vectors as rows, $\mathbf{b} \in \mathbb{R}^m$ contains all biases, and σ is applied element-wise.

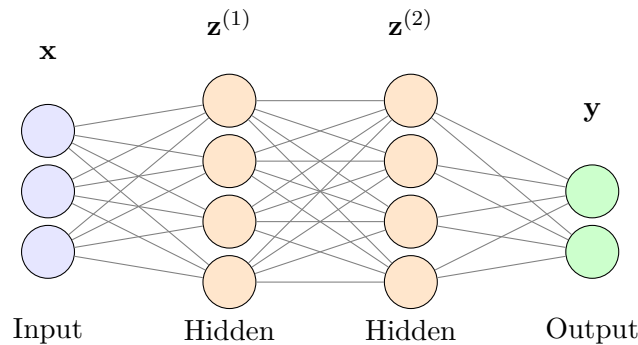


Figure 13.2: Feedforward neural network with two hidden layers. Each layer applies a linear transformation followed by a nonlinear activation.

13.2 Activation Functions

Without nonlinear activation functions, a neural network collapses to a single linear transformation regardless of depth. Activation functions introduce the nonlinearity that makes deep learning powerful.

Definition 13.4 (Sigmoid). The **sigmoid** (or logistic) function squashes inputs to $(0, 1)$:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Properties: $\sigma(0) = 0.5$, $\sigma'(x) = \sigma(x)(1 - \sigma(x))$, and $\lim_{x \rightarrow \pm\infty} \sigma(x) = \{1, 0\}$.

Definition 13.5 (Hyperbolic Tangent). The **tanh** function squashes inputs to $(-1, 1)$:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1$$

Zero-centered unlike sigmoid, which often improves optimization.

Definition 13.6 (ReLU). The **Rectified Linear Unit** is the most widely used activation:

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases}$$

Computationally efficient and avoids vanishing gradients for $x > 0$. However, neurons can “die” if they always receive negative inputs (gradient becomes zero).

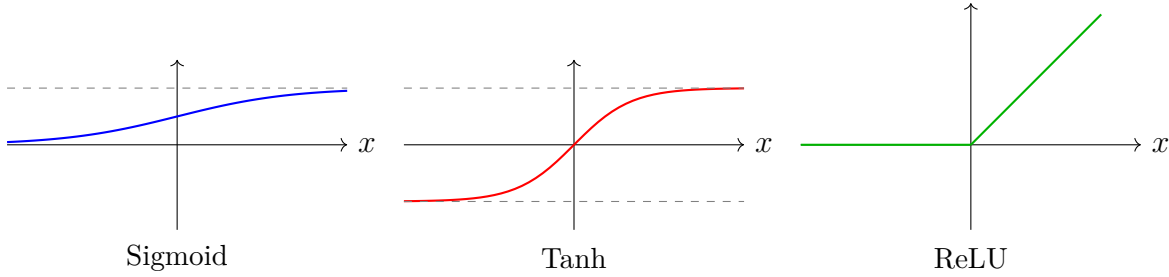


Figure 13.3: Common activation functions. Sigmoid saturates at 0 and 1; tanh is zero-centered; ReLU is piecewise linear.

13.3 Loss Functions

To train a neural network, we need to quantify how well it performs. A *loss function* measures the discrepancy between predictions and targets.

Definition 13.7 (Loss Function). A **loss function** $\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_{\geq 0}$ measures the error between a prediction $\hat{y}$ and target y . For a dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, the **empirical risk** is:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(\mathbf{x}_i), y_i)$$

where f_{θ} is the neural network with parameters θ and ℓ is the per-sample loss.

Definition 13.8 (Mean Squared Error). For regression tasks, the **mean squared error (MSE)** loss is:

$$\ell_{\text{MSE}}(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$$

For vector outputs:

$$\ell_{\text{MSE}}(\hat{\mathbf{y}}, \mathbf{y}) = \frac{1}{2} \|\hat{\mathbf{y}} - \mathbf{y}\|_2^2 = \frac{1}{2} \sum_j (\hat{y}_j - y_j)^2$$

Definition 13.9 (Cross-Entropy Loss). For classification with K classes, let $(p_1, \dots, p_K)$ be the predicted probabilities (output of softmax, so $p_k \geq 0$ and $\sum_k p_k = 1$) and $y \in \{1, \dots, K\}$ be the true class. The **cross-entropy loss** is:

$$\ell_{\text{CE}}(p, y) = -\log p_y$$

Using one-hot encoding $\mathbf{y} = \mathbf{e}_y$ (the y -th standard basis vector):

$$\ell_{\text{CE}}(p, \mathbf{y}) = -\sum_{k=1}^K y_k \log p_k$$

Remark 13.10 (Cross-Entropy and KL Divergence). Cross-entropy relates to KL divergence (see theorem 2.42) via:

$$H(q, p) = D_{\text{KL}}(q \| p) + H(q)$$

where q is the true distribution and p is the predicted. For one-hot q (a single true class), $H(q) = 0$, so minimizing cross-entropy is equivalent to minimizing KL divergence.

Definition 13.11 (Softmax). The **softmax** function converts logits $\mathbf{z} \in \mathbb{R}^K$ to a probability distribution:

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} = \frac{\exp(z_k)}{\|\exp(\mathbf{z})\|_1}$$

This is the gradient of the log-sum-exp function: $\nabla_{\mathbf{z}} \log \sum_k e^{z_k} = \text{softmax}(\mathbf{z})$.

Proposition 13.12 (Softmax-Cross-Entropy Gradient). Let $\mathbf{z}$ be logits, $p = \text{softmax}(\mathbf{z})$, and y be the true class. The gradient of the cross-entropy loss with respect to logits is:

$$\frac{\partial \ell_{\text{CE}}}{\partial z_k} = p_k - \mathbf{1}_{[k=y]}$$

Or in vector form: $\nabla_{\mathbf{z}} \ell_{\text{CE}} = p - \mathbf{e}_y$.

Proof. Write $\ell = -\log p_y = -z_y + \log \sum_j e^{z_j}$. Then:

$$\frac{\partial \ell}{\partial z_k} = -\mathbf{1}_{[k=y]} + \frac{e^{z_k}}{\sum_j e^{z_j}} = -\mathbf{1}_{[k=y]} + p_k = p_k - \mathbf{1}_{[k=y]}$$

□

Definition 13.13 (Binary Cross-Entropy). For binary classification with prediction $p \in [0, 1]$ (output of sigmoid) and label $y \in \{0, 1\}$:

$$\ell_{\text{BCE}}(p, y) = -y \log p - (1 - y) \log(1 - p)$$

This is equivalent to cross-entropy for $K = 2$ classes.

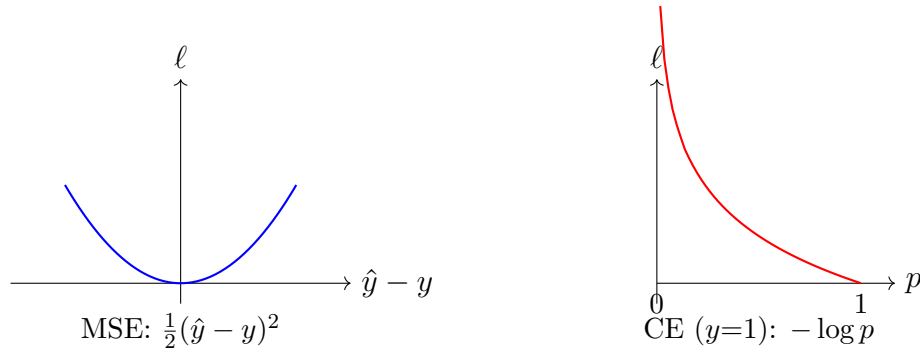


Figure 13.4: Loss function shapes: MSE penalizes errors quadratically; cross-entropy penalizes confident wrong predictions heavily.

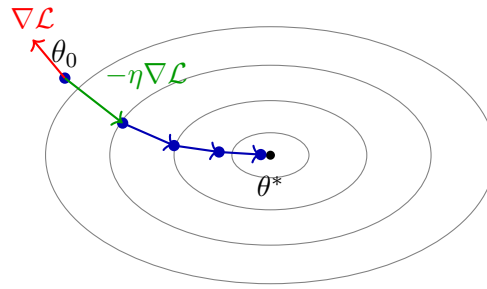


Figure 13.5: Gradient descent on a loss surface. Contours represent level sets; the path moves toward the minimum θ^* .

13.4 Gradient Descent

Training a neural network amounts to finding parameters θ that minimize the loss $\mathcal{L}(\theta)$. Gradient descent iteratively moves in the direction of steepest descent.

Definition 13.14 (Gradient Descent). **Gradient descent** updates parameters by:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t)$$

where $\eta > 0$ is the **learning rate** (step size).

Remark 13.15 (Motivation from Taylor Expansion). Gradient descent emerges naturally from asking: *given our current position θ , which direction should we move to decrease $\mathcal{L}$?*

The **first-order Taylor expansion** approximates the loss near θ :

$$\mathcal{L}(\theta + \Delta\theta) \approx \mathcal{L}(\theta) + \nabla \mathcal{L}(\theta)^\top \Delta\theta$$

This is a linear approximation—we’re replacing the curved loss surface with its tangent hyperplane. To decrease $\mathcal{L}$, we need $\nabla \mathcal{L}^\top \Delta\theta < 0$.

Which direction minimizes this? For a fixed step size $\|\Delta\theta\| = \epsilon$, by Cauchy-Schwarz:

$$\nabla \mathcal{L}^\top \Delta\theta \geq -\|\nabla \mathcal{L}\| \cdot \|\Delta\theta\| = -\epsilon \|\nabla \mathcal{L}\|$$

with equality when $\Delta\theta = -\epsilon \cdot \nabla \mathcal{L} / \|\nabla \mathcal{L}\|$. Thus the **negative gradient is the direction of steepest descent**.

Remark 13.16 (Comparison with Newton-Raphson). The Newton-Raphson method finds **roots** of a function $g(\theta) = 0$:

$$\theta_{t+1} = \theta_t - \frac{g(\theta_t)}{g'(\theta_t)}$$

For optimization, we don't want $\mathcal{L}(\theta) = 0$ (that would just find where the loss is zero). Instead, we want **critical points** where $\nabla \mathcal{L}(\theta) = 0$ —the minima.

So we apply Newton-Raphson to $g = \nabla \mathcal{L}$. The derivative of the gradient is the Hessian: $g' = \nabla^2 \mathcal{L}$. This gives **Newton's method**:

$$\theta_{t+1} = \theta_t - [\nabla^2 \mathcal{L}(\theta_t)]^{-1} \nabla \mathcal{L}(\theta_t)$$

Key difference: Gradient descent uses only first-order information (the gradient), while Newton's method uses second-order information (the Hessian). Gradient descent follows the slope; Newton's method accounts for how the slope itself is changing.

Definition 13.17 (Stochastic Gradient Descent (SGD)). Computing the full gradient requires a forward and backward pass through all N samples. With N in the millions and each pass taking $O(P)$ operations (where P is the number of parameters), one gradient step costs $O(NP)$ —too slow for practical training. **Stochastic gradient descent** uses a random **minibatch** $\mathcal{B} \subset \{1, \dots, N\}$:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\theta} \ell(f_{\theta}(\mathbf{x}_i), y_i)$$

The minibatch gradient is an unbiased estimate. If we sample $|\mathcal{B}|$ indices i.i.d. uniformly from $\{1, \dots, N\}$:

$$\mathbb{E} \left[\frac{1}{|\mathcal{B}|} \sum_{k=1}^{|\mathcal{B}|} \nabla \ell_{i_k} \right] = \frac{1}{|\mathcal{B}|} \sum_{k=1}^{|\mathcal{B}|} \underbrace{\mathbb{E}[\nabla \ell_{i_k}]}_{= \frac{1}{N} \sum_{j=1}^N \nabla \ell_j} = \frac{1}{N} \sum_{j=1}^N \nabla \ell_j = \nabla \mathcal{L}$$

Remark 13.18 (Batch Size Trade-offs).

- **Small batches:** High variance in gradient estimates, but more updates per epoch. Noise acts as regularization.
- **Large batches:** Lower variance, more accurate gradients, better hardware utilization. May converge to sharp minima and generalize worse.
- **Examples:** ResNet-50 on ImageNet used batch size 256. GPT-3 175B used 3.2M tokens per batch. DeepSeek-V3 ramped batch size from 3,072 to 15,360 during training.

13.5 Second-Order Methods and the Hessian

First-order methods like gradient descent use only $\nabla \mathcal{L}$. Second-order methods additionally use the **Hessian matrix** to capture curvature information.

Definition 13.19 (Second-Order Taylor Expansion). The second-order Taylor expansion of $\mathcal{L}$ around θ is:

$$\mathcal{L}(\theta + \Delta\theta) \approx \mathcal{L}(\theta) + \nabla\mathcal{L}(\theta)^\top \Delta\theta + \frac{1}{2}\Delta\theta^\top \mathbf{H}(\theta)\Delta\theta$$

where $\mathbf{H}(\theta) = \nabla^2\mathcal{L}(\theta)$ is the **Hessian matrix** with entries:

$$H_{ij} = \frac{\partial^2\mathcal{L}}{\partial\theta_i\partial\theta_j}$$

The Hessian is symmetric (assuming continuous second derivatives) and captures how the gradient changes as we move in parameter space.

Remark 13.20 (Geometric Interpretation of the Hessian). The Hessian encodes the **curvature** of the loss surface:

- **Eigenvalues:** The eigenvalues λ_i of $\mathbf{H}$ give the curvature along each eigenvector direction. Large λ_i means steep curvature (loss changes rapidly); small λ_i means flat curvature.
- **Condition number:** $\kappa = \lambda_{\max}/\lambda_{\min}$ measures how “stretched” the loss surface is. High κ means elongated contours—gradient descent zigzags and converges slowly.
- **Positive definiteness:** If $\mathbf{H} \succ 0$ (all eigenvalues positive), the point is a local minimum. If $\mathbf{H}$ has negative eigenvalues, it’s a saddle point or maximum.

Definition 13.21 (Newton’s Method). To find the optimal step, we minimize the quadratic approximation:

$$\min_{\Delta\theta} \left[\nabla\mathcal{L}^\top \Delta\theta + \frac{1}{2}\Delta\theta^\top \mathbf{H}\Delta\theta \right]$$

Taking the gradient and setting to zero: $\nabla\mathcal{L} + \mathbf{H}\Delta\theta = 0$, giving:

$$\Delta\theta^* = -\mathbf{H}^{-1}\nabla\mathcal{L}$$

This yields **Newton’s method**:

$$\theta_{t+1} = \theta_t - \mathbf{H}(\theta_t)^{-1}\nabla\mathcal{L}(\theta_t)$$

Remark 13.22 (Why Newton’s Method is Better (When It Works)). Newton’s method has compelling advantages:

- **Quadratic convergence:** Near a minimum, the error decreases as $\|\theta_{t+1} - \theta^*\| \leq C\|\theta_t - \theta^*\|^2$. Gradient descent only achieves linear convergence.
- **Affine invariance:** Newton’s method is invariant to linear reparameterizations. If we change coordinates $\phi = A\theta$, Newton’s path is unchanged. Gradient descent’s path depends on the parameterization.
- **Automatic step size:** The Hessian naturally scales the step—large steps in flat directions, small steps in steep directions.

Remark 13.23 (Why We Don’t Use Newton’s Method in Deep Learning). Despite its theoretical appeal, pure Newton’s method is impractical for neural networks:

- **Storage:** For P parameters, $\mathbf{H}$ is $P \times P$. GPT-3 has $P = 175\text{B}$; storing $\mathbf{H}$ would require $175\text{B}^2 \cdot 4 \text{ bytes} \approx 10^{23} \text{ bytes}$ —more than all storage on Earth.
- **Computation:** Computing $\mathbf{H}$ requires $O(P^2)$ second derivatives. Inverting it costs $O(P^3)$.
- **Non-convexity:** Neural network losses are non-convex. $\mathbf{H}$ may be indefinite, making $\mathbf{H}^{-1}\nabla\mathcal{L}$ point toward saddle points rather than minima.

Remark 13.24 (Practical Approximations).

- **Diagonal approximation:** Use only $\text{diag}(\mathbf{H})$, giving per-parameter learning rates. This is the idea behind AdaGrad/Adam.
- **Quasi-Newton (BFGS, L-BFGS):** Build an approximate $\mathbf{H}^{-1}$ from gradient differences. L-BFGS stores only $O(mP)$ values for $m \approx 10$. Works well for small networks but struggles with stochastic gradients.
- **Gauss-Newton:** For least-squares losses, approximate $\mathbf{H} \approx \mathbf{J}^\top \mathbf{J}$ where $\mathbf{J}$ is the Jacobian. Always positive semi-definite.
- **Natural gradient:** Use the Fisher information matrix $\mathbf{F}$ instead of $\mathbf{H}$. This measures curvature in *distribution space* rather than parameter space.
- **K-FAC:** Approximate $\mathbf{F}$ as a Kronecker product of smaller matrices, reducing cost to $O(P)$.

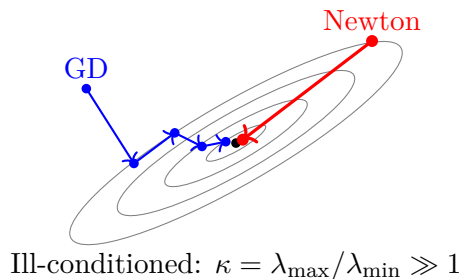


Figure 13.6: Gradient descent zigzags on ill-conditioned problems (high condition number), while Newton’s method goes directly to the minimum by accounting for curvature.

13.6 Backpropagation

Backpropagation efficiently computes gradients by applying the chain rule layer-by-layer, reusing intermediate computations.

Theorem 13.25 (Backpropagation). *Consider a neural network as a composition $f = f_L \circ f_{L-1} \circ \dots \circ f_1$ with loss $\mathcal{L} = \ell \circ f$. Let $\mathbf{z}^{(\ell)}$ denote the output of layer ℓ (with $\mathbf{z}^{(0)} = \mathbf{x}$). The gradient with respect to parameters in layer ℓ is:*

$$\frac{\partial \mathcal{L}}{\partial \theta_\ell} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell)}} \cdot \frac{\partial \mathbf{z}^{(\ell)}}{\partial \theta_\ell}$$

where the “upstream gradient” $\partial \mathcal{L} / \partial \mathbf{z}^{(\ell)}$ is computed recursively:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell-1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell)}} \cdot \frac{\partial \mathbf{z}^{(\ell)}}{\partial \mathbf{z}^{(\ell-1)}}$$

Remark 13.26 (Forward and Backward Passes). Backpropagation consists of two phases:

1. **Forward pass:** Compute all activations $\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(L)}$ and the loss $\mathcal{L}$.
2. **Backward pass:** Starting from $\partial \mathcal{L} / \partial \mathbf{z}^{(L)}$, propagate gradients backward through each layer.

Each intermediate activation must be stored during the forward pass for use in the backward pass. This is the primary memory cost of training.

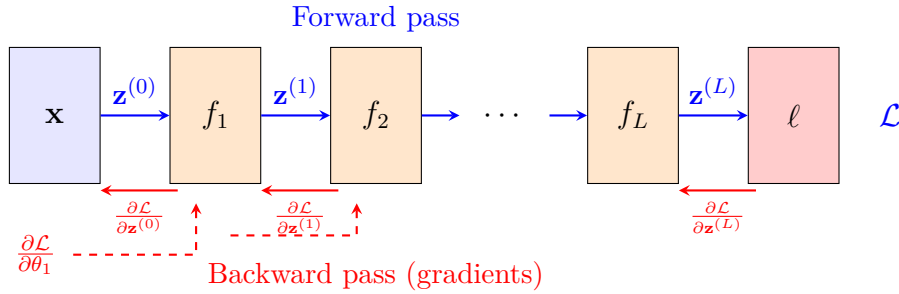


Figure 13.7: Backpropagation: forward pass computes activations; backward pass propagates gradients via the chain rule.

Example 13.27 (Backprop Through a Linear Layer). Consider layer $\mathbf{z} = W\mathbf{a} + \mathbf{b}$ where $\mathbf{a}$ is the input (previous layer’s activation). Given upstream gradient $\boldsymbol{\delta} = \partial \mathcal{L} / \partial \mathbf{z}$:

$$\frac{\partial \mathcal{L}}{\partial W} = \boldsymbol{\delta} \mathbf{a}^\top \quad (\text{outer product}) \quad (13.1)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \boldsymbol{\delta} \quad (\text{direct}) \quad (13.2)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}} = W^\top \boldsymbol{\delta} \quad (\text{propagate to previous layer}) \quad (13.3)$$

Example 13.28 (Backprop Through Activations). For element-wise activation $\mathbf{a} = \sigma(\mathbf{z})$, given upstream gradient $\boldsymbol{\delta} = \partial \mathcal{L} / \partial \mathbf{a}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \boldsymbol{\delta} \odot \sigma'(\mathbf{z})$$

where $\odot$ denotes element-wise multiplication. For ReLU: $\sigma'(z) = \mathbf{1}_{[z>0]}$, so gradients are “gated” by whether the neuron was active.

Remark 13.29 (Jacobian Perspective). For a function $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the Jacobian $J_{\mathbf{f}} \in \mathbb{R}^{m \times n}$ has entries $(J_{\mathbf{f}})_{ij} = \partial f_i / \partial x_j$. The chain rule becomes matrix multiplication:

$$J_{\mathbf{g} \circ \mathbf{f}} = J_{\mathbf{g}} \cdot J_{\mathbf{f}}$$

Backpropagation computes the *vector-Jacobian product* (VJP) $\mathbf{v}^\top J$ efficiently without forming the full Jacobian. This is why we propagate row vectors (gradients) backward, not column vectors.

Proposition 13.30 (Computational Complexity). *Let the forward pass require F floating-point operations. The backward pass requires approximately $2F$ operations—roughly twice the forward pass. This is because each operation in the forward pass corresponds to one multiplication and one addition in the backward pass.*

13.7 Optimization Algorithms

Plain SGD can be slow and sensitive to the learning rate. Modern optimizers adapt the learning rate and use momentum to accelerate convergence.

Definition 13.31 (Momentum). **SGD with momentum** maintains a velocity term that accumulates past gradients:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + \nabla \mathcal{L}(\theta_t) \quad (13.4)$$

$$\theta_{t+1} = \theta_t - \eta \mathbf{v}_{t+1} \quad (13.5)$$

where $\beta \in [0, 1]$ is the momentum coefficient (typically $\beta = 0.9$).

Remark 13.32 (Why Momentum Works). Momentum provides two benefits:

- **Acceleration:** In consistent gradient directions, velocity builds up, allowing faster progress.
- **Noise reduction:** Random fluctuations average out while consistent directions reinforce.

Viewing the sequence as an exponential moving average: if gradients are constant $\mathbf{g}$, then $\mathbf{v} \rightarrow \mathbf{g}/(1 - \beta)$, amplifying the effective step size.

Definition 13.33 (AdaGrad). **AdaGrad** adapts the learning rate for each parameter based on historical gradients:

$$\mathbf{s}_{t+1} = \mathbf{s}_t + (\nabla \mathcal{L}(\theta_t))^2 \quad (\text{accumulate squared gradients}) \quad (13.6)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\mathbf{s}_{t+1}} + \epsilon} \odot \nabla \mathcal{L}(\theta_t) \quad (13.7)$$

where operations are element-wise and $\epsilon \approx 10^{-8}$ prevents division by zero.

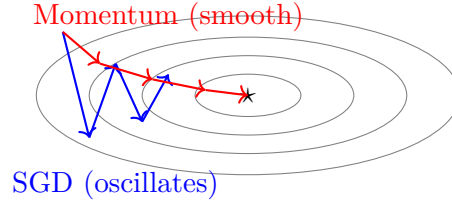


Figure 13.8: SGD oscillates in directions with high curvature; momentum smooths the trajectory by accumulating velocity.

Remark 13.34 (AdaGrad Intuition). Parameters with large gradients get smaller learning rates; those with small gradients get larger rates. This is useful for sparse features: rare features get larger updates when they do appear. The downside: learning rates only decrease, which can cause premature stopping.

Definition 13.35 (RMSProp). **RMSProp** fixes AdaGrad’s diminishing learning rate by using an exponential moving average:

$$\mathbf{s}_{t+1} = \rho \mathbf{s}_t + (1 - \rho)(\nabla \mathcal{L}(\theta_t))^2 \quad (13.8)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\mathbf{s}_{t+1} + \epsilon}} \odot \nabla \mathcal{L}(\theta_t) \quad (13.9)$$

where $\rho \approx 0.9$ controls the decay rate.

Definition 13.36 (Adam). **Adam** (Adaptive Moment Estimation) combines momentum with adaptive learning rates:

$$\mathbf{m}_{t+1} = \beta_1 \mathbf{m}_t + (1 - \beta_1) \nabla \mathcal{L}(\theta_t) \quad (\text{first moment}) \quad (13.10)$$

$$\mathbf{v}_{t+1} = \beta_2 \mathbf{v}_t + (1 - \beta_2) (\nabla \mathcal{L}(\theta_t))^2 \quad (\text{second moment}) \quad (13.11)$$

$$\hat{\mathbf{m}}_{t+1} = \frac{\mathbf{m}_{t+1}}{1 - \beta_1^{t+1}} \quad (\text{bias correction}) \quad (13.12)$$

$$\hat{\mathbf{v}}_{t+1} = \frac{\mathbf{v}_{t+1}}{1 - \beta_2^{t+1}} \quad (\text{bias correction}) \quad (13.13)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_{t+1} + \epsilon}} \odot \hat{\mathbf{m}}_{t+1} \quad (13.14)$$

Default hyperparameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

Remark 13.37 (Bias Correction). The bias correction terms $1/(1 - \beta^t)$ compensate for the initialization $\mathbf{m}_0 = \mathbf{v}_0 = \mathbf{0}$. Without correction, early estimates are biased toward zero. As $t \rightarrow \infty$, the correction approaches 1.

Definition 13.38 (AdamW). **AdamW** decouples weight decay from the adaptive learning rate:

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{\mathbf{m}}_{t+1}}{\sqrt{\hat{\mathbf{v}}_{t+1} + \epsilon}} + \lambda \theta_t \right)$$

where λ is the weight decay coefficient. In standard Adam with L2 regularization, the weight decay is scaled by the adaptive learning rate, which is undesirable.

Remark 13.39 (Practical Recommendations). • **AdamW** is the default choice for transformers and most deep learning.

- **SGD with momentum** can generalize better but requires more tuning; often used for CNNs.
- Learning rate schedules (warmup, cosine decay) are essential for good performance.
- The learning rate is typically the most important hyperparameter to tune.

13.8 Universal Approximation

Neural networks are powerful because they can approximate any continuous function to arbitrary precision.

Theorem 13.40 (Universal Approximation—Cybenko, 1989). *Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous sigmoidal function (i.e., $\sigma(x) \rightarrow 1$ as $x \rightarrow \infty$ and $\sigma(x) \rightarrow 0$ as $x \rightarrow -\infty$). Then for any continuous function $f : [0, 1]^n \rightarrow \mathbb{R}$ and any $\epsilon > 0$, there exists a single-hidden-layer network:*

$$g(\mathbf{x}) = \sum_{j=1}^m \alpha_j \sigma(\mathbf{w}_j^\top \mathbf{x} + b_j)$$

such that $\sup_{\mathbf{x} \in [0, 1]^n} |f(\mathbf{x}) - g(\mathbf{x})| < \epsilon$.

Remark 13.41 (What This Means). • A single hidden layer with enough neurons can approximate any continuous function.

- This is an **existence** result—it doesn’t say how many neurons are needed (could be exponentially many) or how to find the weights.
- Modern extensions show ReLU networks are also universal approximators.

Remark 13.42 (Width vs Depth). Universal approximation holds for width (wide shallow networks), but:

- **Wide networks:** Can approximate any function, but may need exponentially many neurons.
- **Deep networks:** Can represent certain functions exponentially more efficiently. Some functions require exponential width with bounded depth but polynomial size with sufficient depth.

In practice, deep networks generalize better and are more parameter-efficient. The residual stream interpretation (next section) explains why depth helps: each layer can incrementally refine the representation.

Theorem 13.43 (Depth Separation). *There exist functions computable by depth- k networks of polynomial size that require exponential size for depth- $(k - 1)$ networks. That is, depth provably helps for some function classes.*

13.9 Residual Connections and the Residual Stream

Definition 13.44 (Residual Connection). A **residual connection** (or skip connection) adds the input of a layer to its output:

$$y = x + F(x)$$

where F is the layer’s learned transformation. Instead of learning $y = F(x)$ directly, the network learns the *residual* $F(x) = y - x$.

Remark 13.45 (Why Residuals Work). The key insight is that learning $F(x) \approx 0$ (doing nothing) is easy—just set weights to zero. This provides:

- **Easy identity:** If a layer isn’t needed, it can learn to pass input through unchanged
- **Gradient flow:** Gradients flow directly through the addition, avoiding vanishing gradients
- **Depth without degradation:** Deeper networks can’t perform worse than shallower ones

Definition 13.46 (Residual Stream). The **residual stream** is a conceptual framework for understanding transformers. Think of a “river” of information flowing through the network:

$$x^{(0)} \xrightarrow{+\text{Attn}_1} x^{(1)} \xrightarrow{+\text{MLP}_1} x^{(2)} \xrightarrow{+\text{Attn}_2} \dots \xrightarrow{+\text{MLP}_L} x^{(2L)}$$

Each component *reads from* and *writes to* this shared stream via addition.

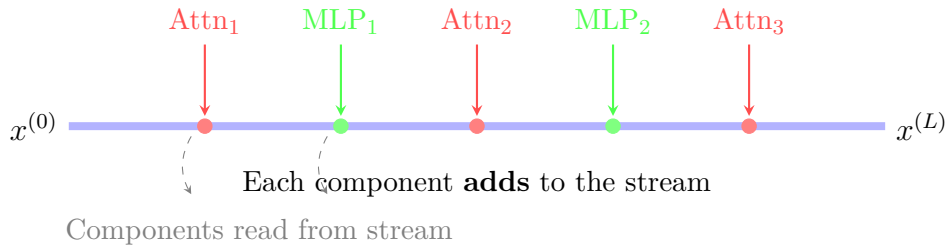


Figure 13.9: The residual stream: attention and MLP layers read from and write to a shared representation via addition.

Remark 13.47 (Residual Stream Interpretation). This view reveals important properties:

- **Superposition:** The stream holds a superposition of features from all previous layers
- **Direct paths:** Information from layer 1 can directly influence layer 10 via the residual
- **Additive contributions:** Each layer’s contribution can be analyzed independently
- **No information destruction:** Layers add information; they don’t overwrite it

Mathematically, the final output is:

$$x^{(L)} = x^{(0)} + \sum_{\ell=1}^L F_{\ell}(x^{(\ell-1)})$$

The output is the input plus the sum of all layer contributions.

13.10 The FFN Block

In transformers, the **FFN block** (feedforward block) is a specific two-layer network applied independently at each token position.

Definition 13.48 (FFN Block). The **FFN block** in a transformer is:

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

where:

- $x \in \mathbb{R}^{d_{\text{model}}}$ is the input (one token’s representation)
- $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ projects to a higher dimension
- σ is a nonlinear activation (GELU, SiLU, etc.)
- $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ projects back down
- Typically $d_{\text{ff}} = 4 \cdot d_{\text{model}}$

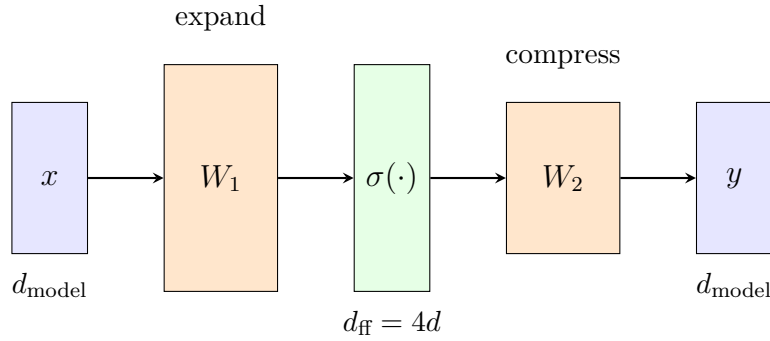


Figure 13.10: FFN block: expand to higher dimension, apply nonlinearity, then compress back to model dimension.

Remark 13.49 (FFN as Key-Value Memory). The FFN can be interpreted as an associative memory:

- Rows of W_1 are **keys** (patterns to detect)
- Columns of W_2 are **values** (information to retrieve)
- $\sigma(W_1 x)$ computes “match scores” for each key
- $W_2 \sigma(W_1 x)$ retrieves a weighted combination of values

This is where factual knowledge (“Paris is the capital of France”) appears to be stored. The keys detect contexts like “The capital of France is”, and values provide the completion.

Remark 13.50 (Position Independence). Critically, the FFN is applied **independently to each position**:

$$\text{FFN}([x_1, x_2, \dots, x_n]) = [\text{FFN}(x_1), \text{FFN}(x_2), \dots, \text{FFN}(x_n)]$$

There is no communication between positions in the FFN—that’s the job of attention. The FFN processes each token’s representation in isolation.

13.11 Modern Activation Functions

Beyond ReLU, modern architectures (especially transformers) use smoother activations:

Definition 13.51 (GELU). The **Gaussian Error Linear Unit**:

$$\text{GELU}(x) = x \cdot \Phi(x) \approx x \cdot \frac{1}{2} \left[1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right]$$

where Φ is the standard normal CDF. Smooth approximation to ReLU; used in GPT, BERT.

Definition 13.52 (SiLU/Swish). The **Sigmoid Linear Unit** (also called Swish):

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

Smooth, non-monotonic, self-gated. Used in many modern architectures.

Definition 13.53 (SwiGLU). The **Swish-Gated Linear Unit** combines SiLU with gating:

$$\text{SwiGLU}(x) = \text{SiLU}(W_1 x) \odot (W_3 x)$$

Uses an extra projection W_3 for gating. Used in LLaMA, PaLM.

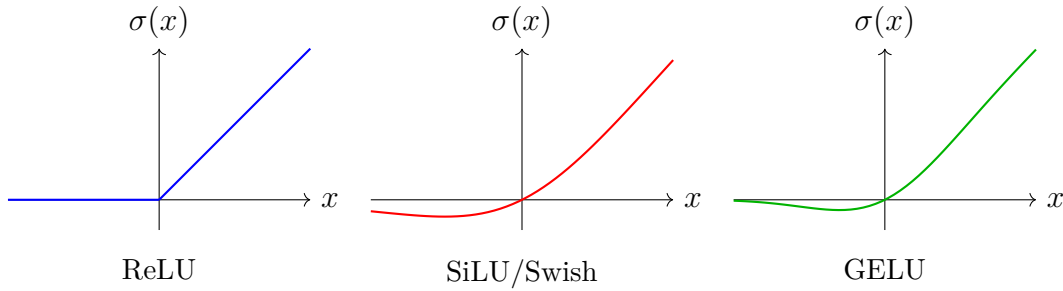


Figure 13.11: Modern activation functions: ReLU is piecewise linear; SiLU and GELU are smooth approximations used in transformers.

13.12 General Feedforward Neural Networks (MLP)

Having seen the specific FFN block used in transformers, we now define the general concept.

Definition 13.54 (Feedforward Neural Network / MLP). A **feedforward neural network** (or **multilayer perceptron**) is a composition of layers:

$$f(x) = (f_L \circ f_{L-1} \circ \cdots \circ f_1)(x)$$

where each layer $f_\ell(z) = \sigma(W_\ell z + b_\ell)$ applies a linear transformation followed by a nonlinearity.

Equivalently:

$$f(x) = W_L \sigma(W_{L-1} \sigma(\cdots \sigma(W_1 x + b_1) \cdots) + b_{L-1}) + b_L$$

Remark 13.55 (Terminology). “**Feedforward**” means information flows in one direction (input $\rightarrow$ output) with no cycles. This contrasts with recurrent networks where outputs feed back as inputs.

“**Multilayer perceptron**” is historical—the original perceptron used step functions; modern MLPs use smooth activations. The name persists.

The transformer’s FFN block is a specific 2-layer MLP with expansion ratio 4.

13.13 Embeddings

Definition 13.56 (Embedding). An **embedding** is a learned mapping from discrete objects to continuous vectors:

$$\text{Embed} : \mathcal{V} \rightarrow \mathbb{R}^d$$

where $\mathcal{V}$ is a finite vocabulary and d is the embedding dimension.

Remark 13.57 (Implementation). Embeddings are stored as a matrix $E \in \mathbb{R}^{|\mathcal{V}| \times d}$ where row i is the embedding of token i . Lookup is just indexing: $\text{Embed}(t) = E[i, :]$.

Equivalently, this is multiplication by a one-hot vector:

$$\text{Embed}(t) = \mathbf{1}_t^\top E$$

Remark 13.58 (Why Embeddings?). **Problem:** Discrete tokens have no inherent similarity. Token indices are arbitrary.

Solution: Learn a continuous space where:

- Similar items have similar vectors
- Relationships are encoded as directions (“king – man + woman $\approx$ queen”)
- The geometry captures meaningful structure

Chapter 14

Transformer Theory

14.1 Self-Attention Mechanism

Definition 14.1 (Scaled Dot-Product Attention). Given queries Q , keys K , and values V :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

The attention weights $A = \text{softmax}(QK^\top/\sqrt{d_k})$ form a stochastic matrix where A_{ij} is how much position i attends to position j .

Definition 14.2 (Self-Attention). In **self-attention**, queries, keys, and values all come from the same sequence:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

where $X \in \mathbb{R}^{n \times d}$ is the input sequence and W_Q, W_K, W_V are learned projections.

Remark 14.3 (Dimension Analysis). Let us trace the dimensions carefully:

- Input: $X \in \mathbb{R}^{n \times d_{\text{model}}}$ (sequence of n tokens, each d_{model} -dimensional)
- Projections: $W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$
- Queries/Keys: $Q, K \in \mathbb{R}^{n \times d_k}$
- Values: $V \in \mathbb{R}^{n \times d_v}$
- Attention scores: $QK^\top \in \mathbb{R}^{n \times n}$
- Attention weights: $A = \text{softmax}(QK^\top/\sqrt{d_k}) \in \mathbb{R}^{n \times n}$
- **Output:** $AV \in \mathbb{R}^{n \times d_v}$

Key observation: The output dimension equals d_v (the value dimension), not d_k (the key/query dimension). The attention matrix A is $n \times n$; multiplying by V (which is $n \times d_v$) gives $n \times d_v$.

In practice, typically $d_k = d_v = d_{\text{model}}/H$ where H is the number of heads.

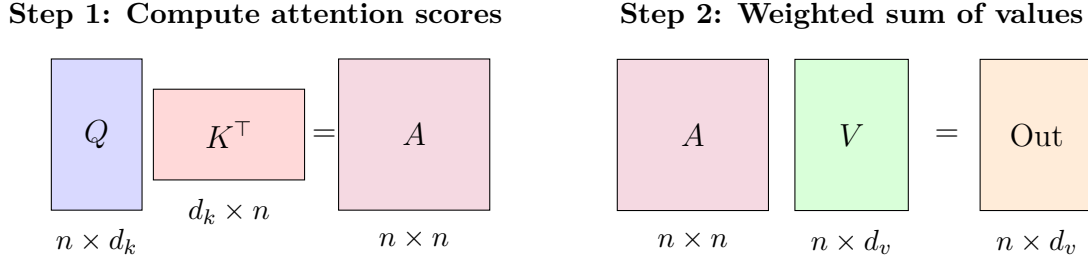


Figure 14.1: Attention as matrix multiplication: (1) query-key products yield attention weights; (2) weights aggregate values.

14.2 Multi-Head Attention

Single-head attention can only learn one type of relationship pattern. Multi-head attention runs multiple attention operations in parallel, allowing the model to simultaneously attend to information from different representation subspaces.

Definition 14.4 (Multi-Head Attention). Given H attention heads, **multi-head attention** computes:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O$$

where each head uses its own projections:

$$\text{head}_h = \text{Attention}(XW_Q^{(h)}, XW_K^{(h)}, XW_V^{(h)})$$

with $W_Q^{(h)}, W_K^{(h)} \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_V^{(h)} \in \mathbb{R}^{d_{\text{model}} \times d_v}$, and the output projection $W_O \in \mathbb{R}^{Hd_v \times d_{\text{model}}}$.

Remark 14.5 (Intuition: Why Multiple Heads?). Consider the sentence: “The cat sat on the mat because it was tired.”

Different relationships exist simultaneously:

- **Syntactic**: “it” $\rightarrow$ “cat” (pronoun resolution)
- **Positional**: “sat” $\rightarrow$ “on” (adjacent dependency)
- **Semantic**: “tired” $\rightarrow$ “cat” (who is tired?)
- **Causal**: “sat” $\rightarrow$ “tired” (why did it sit?)

A single attention pattern cannot capture all these simultaneously. Each head can specialize in one type of pattern.

Example 14.6 (Observed Head Specialization). Research analyzing trained transformers has found heads that specialize in:

- **Previous token heads**: Attend strongly to position $i - 1$
- **Induction heads**: Attend to tokens that followed similar contexts (covered later)

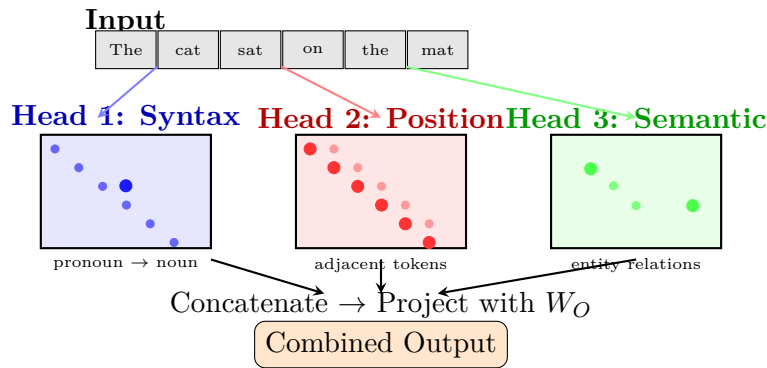


Figure 14.2: Multi-head attention: each head learns a different attention pattern, then outputs are concatenated and projected.

- **Rare word heads:** Focus attention on uncommon tokens
- **Delimiter heads:** Attend to punctuation and separators
- **Duplicate token heads:** Find repeated tokens in the sequence
- **Syntax heads:** Track grammatical structure (subject-verb, noun-adjective)

Not all heads are equally important—some can be pruned with minimal performance loss.

Remark 14.7 (Dimension Budget). To keep computation constant, the per-head dimension is reduced:

$$d_k = d_v = \frac{d_{\text{model}}}{H}$$

For GPT-3 with $d_{\text{model}} = 12288$ and $H = 96$ heads: $d_k = 128$ per head.

This means each head operates in a lower-dimensional subspace, forcing each head to focus on specific patterns rather than learning everything.

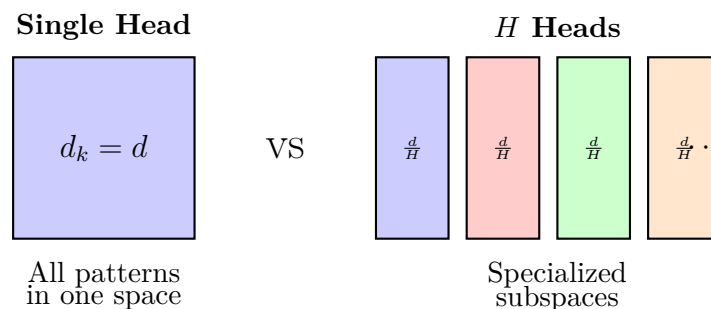


Figure 14.3: Single head uses full dimension; multiple heads partition into specialized subspaces.

Remark 14.8 (Why Concatenate Then Project?). After computing all heads, we concatenate and apply W_O :

$$\text{Output} = [\text{head}_1; \dots; \text{head}_H]W_O$$

The output projection W_O serves several purposes:

- **Mixing:** Combines information from different heads
- **Dimension matching:** Maps Hd_v back to d_{model}
- **Learning:** Allows the model to learn how to weight different heads' contributions

Without W_O , heads would be independent and couldn't share information.

14.3 Why Divide by $\sqrt{d_k}$?

The scaling factor $\sqrt{d_k}$ in attention is not arbitrary—it is precisely the factor needed to control the variance of dot products and prevent softmax saturation.

Theorem 14.9 (Variance of Dot Products). *Let $q, k \in \mathbb{R}^{d_k}$ be random vectors with independent components satisfying $\mathbb{E}[q_i] = \mathbb{E}[k_i] = 0$ and $\text{Var}(q_i) = \text{Var}(k_i) = 1$. Then:*

$$\mathbb{E}[q^\top k] = 0 \quad \text{and} \quad \text{Var}(q^\top k) = d_k$$

Proof. The dot product is $q^\top k = \sum_{i=1}^{d_k} q_i k_i$. We compute its mean and variance.

Mean:

$$\mathbb{E}[q^\top k] = \mathbb{E}\left[\sum_{i=1}^{d_k} q_i k_i\right] = \sum_{i=1}^{d_k} \mathbb{E}[q_i k_i] = \sum_{i=1}^{d_k} \mathbb{E}[q_i] \mathbb{E}[k_i] = \sum_{i=1}^{d_k} 0 \cdot 0 = 0$$

where we used independence of q_i and k_i .

Variance: Since $\mathbb{E}[q^\top k] = 0$, we have $\text{Var}(q^\top k) = \mathbb{E}[(q^\top k)^2]$.

$$\begin{aligned} \mathbb{E}[(q^\top k)^2] &= \mathbb{E}\left[\left(\sum_{i=1}^{d_k} q_i k_i\right)^2\right] \\ &= \mathbb{E}\left[\sum_{i=1}^{d_k} \sum_{j=1}^{d_k} q_i k_i q_j k_j\right] \\ &= \sum_{i=1}^{d_k} \sum_{j=1}^{d_k} \mathbb{E}[q_i k_i q_j k_j] \end{aligned}$$

For $i \neq j$, by independence:

$$\mathbb{E}[q_i k_i q_j k_j] = \mathbb{E}[q_i] \mathbb{E}[k_i] \mathbb{E}[q_j] \mathbb{E}[k_j] = 0$$

For $i = j$:

$$\mathbb{E}[q_i^2 k_i^2] = \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] = \text{Var}(q_i) \cdot \text{Var}(k_i) = 1 \cdot 1 = 1$$

Therefore:

$$\text{Var}(q^\top k) = \sum_{i=1}^{d_k} 1 = d_k$$

□

Corollary 14.10 (Standard Deviation Scaling). *The standard deviation of the dot product is $\sqrt{d_k}$. To obtain unit variance:*

$$\text{Var}\left(\frac{q^\top k}{\sqrt{d_k}}\right) = \frac{\text{Var}(q^\top k)}{d_k} = \frac{d_k}{d_k} = 1$$

This is why we divide by $\sqrt{d_k}$, not d_k .

Remark 14.11 (Why Variance Control Matters for Softmax). The softmax function is:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

When inputs z_i have large magnitude, softmax **saturates**:

Input	Output	Gradient
$[1, 0, 0, 0]$	$[0.46, 0.18, 0.18, 0.18]$	Healthy
$[5, 0, 0, 0]$	$[0.95, 0.02, 0.02, 0.02]$	Reduced
$[10, 0, 0, 0]$	$[0.9999, 10^{-4}, 10^{-4}, 10^{-4}]$	Nearly zero

Problem: With $d_k = 64$, unscaled dot products have standard deviation $\sqrt{64} = 8$. Values of ± 16 (two standard deviations) are common, causing extreme saturation.

Solution: Dividing by $\sqrt{d_k} = 8$ brings the standard deviation back to 1, keeping softmax in a well-behaved regime.

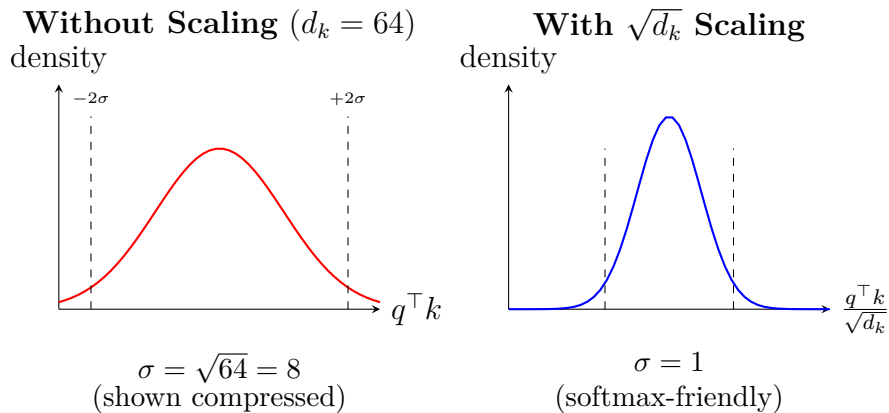


Figure 14.4: Scaling by $\sqrt{d_k}$ normalizes variance from d_k to 1, preventing softmax saturation.

Remark 14.12 (Numerical Example). Let $d_k = 64$ and suppose $q, k \sim \mathcal{N}(0, I_{64})$.

Unscaled: $q^\top k \sim \mathcal{N}(0, 64)$, so $\text{std}(q^\top k) = 8$.

A typical attention score matrix might look like:

$$QK^\top = \begin{pmatrix} 12.3 & -5.1 & 8.7 & \cdots \\ -3.2 & 15.8 & -11.4 & \cdots \\ \vdots & & \ddots & \end{pmatrix}$$

Applying softmax to $[12.3, -5.1, 8.7, \dots]$ gives nearly all weight to position 1.

Scaled: $\frac{q^\top k}{\sqrt{64}} = \frac{q^\top k}{8} \sim \mathcal{N}(0, 1)$.

$$\frac{QK^\top}{\sqrt{d_k}} = \begin{pmatrix} 1.54 & -0.64 & 1.09 & \cdots \\ -0.40 & 1.98 & -1.43 & \cdots \\ \vdots & & \ddots & \end{pmatrix}$$

Softmax of $[1.54, -0.64, 1.09, \dots]$ gives a smoother distribution with meaningful gradients.

Remark 14.13 (Connection to Initialization). The $\sqrt{d_k}$ scaling is part of a broader principle: **maintain unit variance throughout the network**.

- **Xavier/Glorot initialization:** $W_{ij} \sim \mathcal{N}(0, 1/d_{\text{in}})$ ensures output variance ≈ 1
- **He initialization:** $W_{ij} \sim \mathcal{N}(0, 2/d_{\text{in}})$ accounts for ReLU killing half the variance
- **Attention scaling:** $1/\sqrt{d_k}$ ensures attention logits have unit variance

All of these prevent activations from exploding or vanishing as signals propagate through the network.

Remark 14.14 (Alternative Scalings). Some architectures use different scalings:

- $1/d_k$ (linear scaling): Used in some linear attention variants. Scales more aggressively.
- $1/\sqrt{d_k \cdot n}$ (length-adjusted): Some works scale by sequence length n to account for the sum over n keys.
- **Learned scaling:** A learnable parameter τ replacing $\sqrt{d_k}$:

$$\text{softmax}\left(\frac{QK^\top}{\tau}\right)$$

This is called **temperature scaling**. Lower τ = sharper attention.

The $\sqrt{d_k}$ choice is principled (variance = 1) but not unique.

14.4 Attention as a Kernel Method

To understand the kernel interpretation of attention, we first need to understand what kernel methods are.

Definition 14.15 (Kernel Function). A **kernel** $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is a function that measures similarity between pairs of points. A kernel is **positive definite** if for any finite set $\{x_1, \dots, x_n\} \subset \mathcal{X}$, the Gram matrix is positive semi-definite.

Definition 14.16 (Gram Matrix). Given points $\{x_1, \dots, x_n\}$ and a kernel K , the **Gram matrix** $G \in \mathbb{R}^{n \times n}$ is:

$$G_{ij} = K(x_i, x_j)$$

Each entry is the kernel similarity between points i and j . The matrix captures all pairwise similarities.

A matrix is **positive semi-definite (PSD)** if $v^\top G v \geq 0$ for all $v \in \mathbb{R}^n$, equivalently, all eigenvalues are ≥ 0 .

Remark 14.17 (Gram Matrix in Attention). The attention score matrix $QK^\top$ is exactly a Gram matrix with the linear kernel:

$$(QK^\top)_{ij} = q_i^\top k_j = K_{\text{linear}}(q_i, k_j)$$

After applying softmax, we get the attention weights. The Gram matrix perspective helps us see attention as computing all pairwise similarities between queries and keys.

Remark 14.18 (The Kernel Trick). The power of kernels comes from the **kernel trick**: every positive definite kernel corresponds to an inner product in some (possibly infinite-dimensional) feature space:

$$K(x, y) = \langle \phi(x), \phi(y) \rangle$$

where $\phi : \mathcal{X} \rightarrow \mathcal{H}$ maps to a Hilbert space $\mathcal{H}$.

This means we can compute inner products in high-dimensional spaces without explicitly computing the features $\phi(x)$.

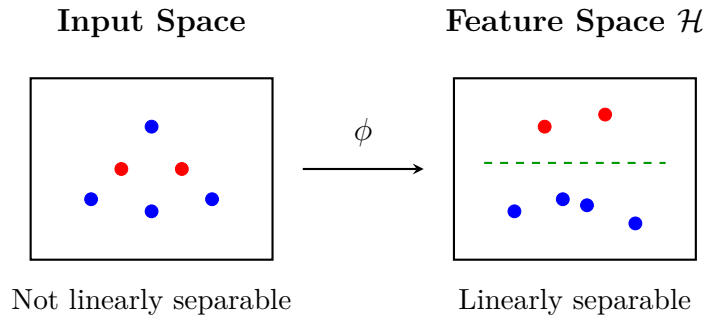


Figure 14.5: The kernel trick: feature map ϕ lifts data to a space where classes become linearly separable.

Example 14.19 (Common Kernels). Common kernel functions include:

- **Linear:** $K(x, y) = x^\top y$. Feature map is identity.
- **Polynomial:** $K(x, y) = (x^\top y + c)^d$. Implicitly computes all monomials up to degree d .
- **RBF/Gaussian:** $K(x, y) = \exp(-\|x - y\|^2 / 2\sigma^2)$. Infinite-dimensional feature space.
- **Softmax/Exponential:** $K(x, y) = \exp(x^\top y)$. This is the attention kernel!

Definition 14.20 (Kernel Regression). Given data $\{(x_i, y_i)\}_{i=1}^n$, **kernel regression** predicts at a new point x by:

$$\hat{f}(x) = \sum_{i=1}^n w_i(x) y_i \quad \text{where} \quad w_i(x) = \frac{K(x, x_i)}{\sum_{j=1}^n K(x, x_j)}$$

This is the **Nadaraya-Watson estimator**: a weighted average of training outputs, where weights are determined by kernel similarity to the query point.

Remark 14.21 (Attention IS Kernel Regression). Self-attention is exactly Nadaraya-Watson kernel regression. Let's see why.

Recall what softmax does: For a vector z , the softmax normalizes by the sum:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

So softmax *already includes* the division by the sum—that's its whole purpose!

Attention weights: Applying softmax to the attention scores:

$$A_{ij} = \text{softmax}_j \left(\frac{q_i^\top k_j}{\sqrt{d_k}} \right) = \frac{\exp(q_i^\top k_j / \sqrt{d_k})}{\sum_{j'} \exp(q_i^\top k_{j'} / \sqrt{d_k})}$$

This is exactly kernel regression: If we define $K(q, k) = \exp(q^\top k / \sqrt{d_k})$, then:

$$A_{ij} = \frac{K(q_i, k_j)}{\sum_{j'} K(q_i, k_{j'})}$$

which matches the Nadaraya-Watson weights. The attention output is:

$$\text{output}_i = \sum_j A_{ij} v_j = \sum_j \frac{K(q_i, k_j)}{\sum_{j'} K(q_i, k_{j'})} v_j$$

Interpretation:

- Query q_i = the point where we want to predict
- Keys k_j = the “locations” of data points
- Values v_j = the “outputs” at those locations

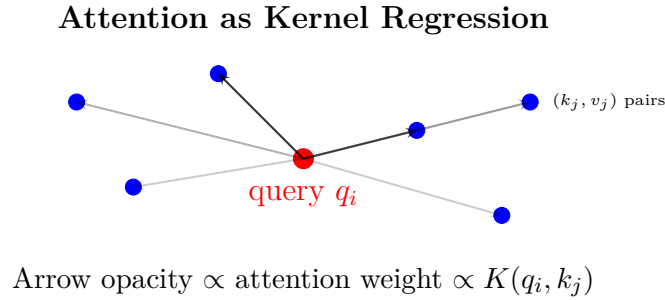


Figure 14.6: Attention as kernel regression: query retrieves a weighted average of values based on kernel similarity to keys.

- $K(q_i, k_j)$ = similarity between query and key (unnormalized)
- A_{ij} = normalized similarity (weights sum to 1)

Remark 14.22 (What Breaks vs Classical Kernels). Classical RKHS (Reproducing Kernel Hilbert Space) kernels satisfy:

1. **Fixed feature map:** $K(x, y) = \phi(x)^\top \phi(y)$ for some fixed ϕ
2. **Symmetry:** $K(x, y) = K(y, x)$
3. **Positive definiteness:** The Gram matrix is always PSD

Self-attention **violates** these:

- **Data-dependent features:** $q_i = W_Q x_i$ and $k_j = W_K x_j$ depend on the data
- **Asymmetric:** $K(q_i, k_j) \neq K(q_j, k_i)$ because $W_Q \neq W_K$
- **Learned projections:** The “kernel” changes during training

The kernel interpretation is thus an **analogy**, not an equivalence.

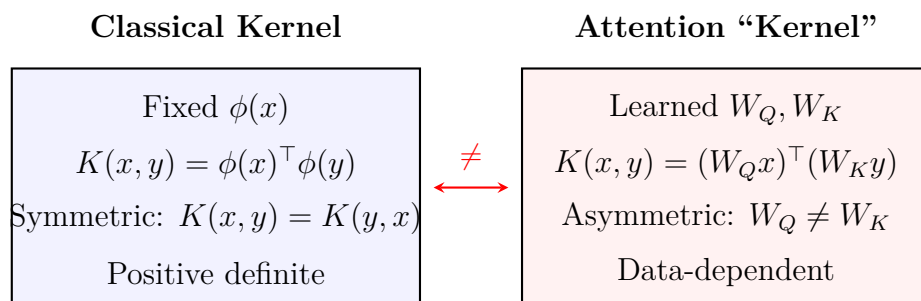


Figure 14.7: Classical kernels are symmetric and fixed; attention uses learned, asymmetric projections.

Remark 14.23 (Why the Asymmetry Matters). In attention, $W_Q \neq W_K$ creates an **asymmetric** relationship:

- “How much does token A attend to token B?” $\neq$ “How much does B attend to A?”
- This is semantically meaningful: “the cat sat on the mat” — “cat” should attend to “sat” (its verb), but “sat” should attend to “cat” (its subject) for different reasons.

If we forced $W_Q = W_K$ (symmetric attention), we would lose this expressivity.

Remark 14.24 (Why the Kernel Interpretation Matters). Given that attention is **not** a proper kernel method, why study this connection?

1. **Linear Attention and Efficiency:** The kernel view directly enables efficient $O(n)$ attention variants. If we decompose $K(q, k) = \phi(q)^\top \phi(k)$, then:

$$\text{Attention}(q_i) = \frac{\sum_j \phi(q_i)^\top \phi(k_j) v_j}{\sum_j \phi(q_i)^\top \phi(k_j)} = \frac{\phi(q_i)^\top \sum_j \phi(k_j) v_j}{\phi(q_i)^\top \sum_j \phi(k_j)}$$

The sums $\sum_j \phi(k_j) v_j^\top$ and $\sum_j \phi(k_j)$ can be computed **once** and reused for all queries—this is $O(n)$ instead of $O(n^2)$.

2. **Expressivity Analysis:** Kernel methods have well-studied approximation theory. The kernel view helps us understand what functions attention can learn (e.g., attention is a universal approximator of continuous permutation-equivariant functions on compact domains).
3. **Implicit Feature Spaces:** Thinking of $q^\top k$ as a kernel suggests attention operates in an implicit feature space—the model learns **what similarity means** through W_Q, W_K rather than hardcoding it.
4. **Connection to Other Architectures:** The kernel view unifies attention with:
 - Nadaraya-Watson regression (statistics)
 - Graph neural networks (message passing)
 - Memory networks (soft addressing)

The kernel analogy is imperfect but productive: it provides both practical algorithms (linear attention) and theoretical insights (expressivity, generalization).

14.5 Positional Encodings and Length Generalization

Definition 14.25 (Positional Encoding). Since attention is permutation-equivariant, we must inject position information. Common approaches:

- **Absolute (Learned):** Add learned vectors p_i to each position:

$$x_i \leftarrow x_i + p_i, \quad p_i \in \mathbb{R}^d \text{ learned}$$

- **Absolute (Sinusoidal):**

$$p_{i,2k} = \sin(i/10000^{2k/d}), \quad p_{i,2k+1} = \cos(i/10000^{2k/d})$$

- **Relative:** Attention score depends on $i - j$:

$$A_{ij} \propto \exp\left(\frac{q_i^\top k_j + q_i^\top r_{i-j}}{\sqrt{d_k}}\right)$$

- **RoPE** (Rotary Position Embedding): Rotate query/key vectors by position-dependent angles:

$$q_i \leftarrow R_{\theta_i} q_i, \quad k_j \leftarrow R_{\theta_j} k_j$$

where R_θ is a rotation matrix. Then $q_i^\top k_j$ depends on $i - j$.

Remark 14.26 (Sinusoidal Encoding: Why 10000?). The sinusoidal encoding uses wavelengths forming a geometric progression from 2π to $10000 \cdot 2\pi$:

$$\text{wavelength}_k = 2\pi \cdot 10000^{2k/d}$$

The base 10000 was chosen so that:

- **Low dimensions** ($k \approx 0$): wavelength $\approx 2\pi$, changes every position (captures fine-grained position)
- **High dimensions** ($k \approx d/2$): wavelength $\approx 2\pi \cdot 10000$, changes slowly (captures coarse position)
- **Coverage:** For sequences up to 10000 tokens, each dimension contributes meaningfully

The value 10000 is somewhat arbitrary—it just needs to be larger than the maximum expected sequence length. The key insight is the **geometric progression**: each dimension encodes position at a different frequency, like binary digits of a number.

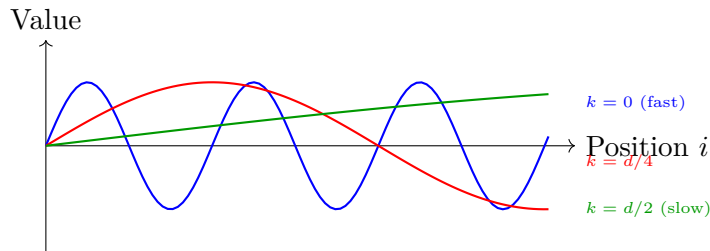


Figure 14.8: Sinusoidal encodings: different dimensions oscillate at different frequencies.

Remark 14.27 (Sinusoidal vs Learned: Historical Context). The original Transformer paper (Vaswani et al., 2017) proposed sinusoidal encodings for several reasons:

1. **Length extrapolation:** Sinusoidal functions are defined for any position, while learned embeddings require a fixed maximum length

2. **Relative position:** $\sin(a + b)$ and $\cos(a + b)$ can be expressed as linear combinations of $\sin(a)$, $\cos(a)$, $\sin(b)$, $\cos(b)$, so p_{i+k} is a linear function of p_i —this allows the model to learn relative positions
3. **No additional parameters:** Saves memory and training compute
4. **Inductive bias:** Smooth variation between positions encourages similar treatment of nearby tokens

However, the same paper showed learned embeddings performed comparably. Modern models typically use:

- **Learned absolute:** BERT, GPT-2 (simple, works well for fixed context)
- **RoPE:** LLaMA, GPT-NeoX (relative, better length generalization)
- **ALiBi:** Some models (linear bias, no learned parameters)

Sinusoidal encodings are now mostly historical—they demonstrated key principles but learned/RoPE methods dominate in practice.

Remark 14.28 (Why Absolute Encodings Fail at Length Generalization). With absolute positional encodings:

- The model learns position-specific patterns: “position 5 often has verbs”
- At position 1000 (unseen in training), the model encounters novel embeddings
- The attention patterns learned for positions 1–512 don’t transfer

The fundamental issue: **absolute positions encode “where am I”** rather than “**how far apart are we**”.

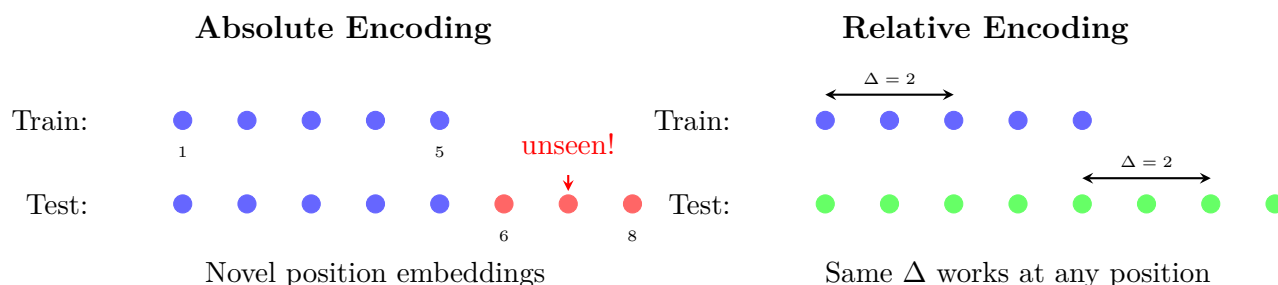


Figure 14.9: Absolute encodings fail on unseen positions; relative encodings generalize by encoding distances.

Remark 14.29 (What Positional Encodings Cannot Fix). Even with perfect relative encodings, length generalization can fail because:

1. **Attention capacity:** With n positions and fixed heads, each position can only meaningfully attend to $O(1)$ or $O(\log n)$ other positions effectively

2. **Distribution shift:** Longer sequences have different statistical properties (more repeated patterns, different frequency distributions)
3. **Algorithmic complexity:** Some tasks require $O(n)$ sequential steps, but transformers have $O(1)$ depth
4. **Training signal:** The model was optimized for short sequences; the loss landscape for long sequences may be different

14.6 Induction Heads

Definition 14.30 (Induction Head). An **induction head** is a two-attention-head circuit that implements pattern completion:

$$[A][B] \cdots [A] \rightarrow [B]$$

If the model has seen “[A][B]” earlier in context, and now sees “[A]”, it predicts “[B]”.

Remark 14.31 (Induction Head Mechanism). The circuit requires two heads working together:

Head 1 (Previous Token Head): Copies information from position $i - 1$ to position i . Each token “knows” what came before it.

Head 2 (Induction Head): At position n , attends to positions where the *previous token* matches the current token. Then copies what came *after* that match.

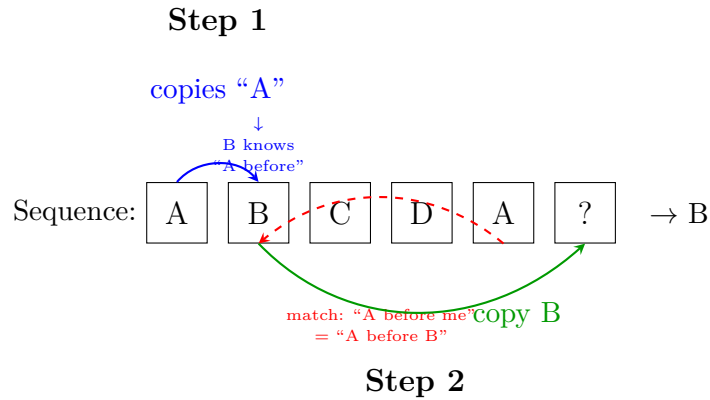


Figure 14.10: Induction head circuit: (1) previous token head copies A to B’s residual; (2) induction head matches and copies.

Remark 14.32 (Why Induction Heads Emerge Reliably). Induction heads appear consistently in transformers because:

1. **Gradient signal:** Next-token prediction heavily rewards copying patterns. When “AB...A” appears, predicting “B” gives strong gradient signal.

2. **Simplicity:** The induction circuit is one of the simplest useful computations attention can learn. It requires only:
 - Matching (via $QK^\top$)
 - Copying (via V projection)
3. **Weight sharing:** The same attention weights apply at all positions, so learning induction for one pattern generalizes.
4. **Phase transition:** Anthropic observed a sharp “phase change” where induction heads form suddenly during training, suggesting they represent a qualitative capability jump.

Remark 14.33 (Are Induction Heads Inevitable?). **Largely yes**, given:

- Next-token prediction objective
- Sufficient depth (≥ 2 layers for the two-head circuit)
- Repeated patterns in training data

What might prevent them:

- Single-layer transformers (cannot implement the two-head circuit)
- Training data with no repeated patterns
- Extreme regularization preventing memorization

In practice, induction heads are nearly universal in trained transformers.

14.7 Attention vs MLP: Routing vs Transformation

Remark 14.34 (Functional Separation). A useful mental model:

- **Attention:** “Where should information flow?” (routing, communication)
- **MLP:** “What should be computed?” (transformation, processing)

Attention moves information between positions; MLP processes information at each position.

Remark 14.35 (Can Each Simulate the Other?). **Can MLP simulate attention?** Not efficiently:

- MLP is position-wise; it cannot compare tokens at different positions
- To simulate attention, MLP would need $O(n^2)$ parameters for sequence length n
- Attention achieves cross-position comparison with $O(d^2)$ parameters (independent of n)

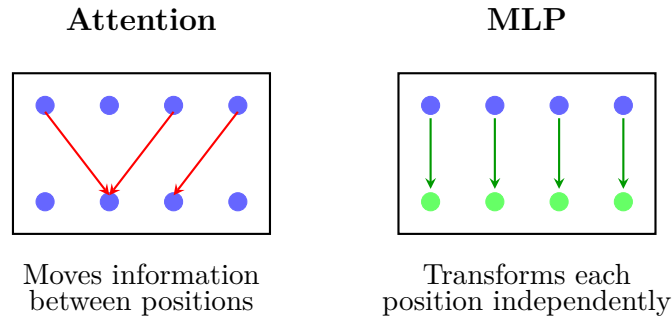


Figure 14.11: Attention routes information across positions; MLP processes each position independently.

Can attention simulate MLP? Partially:

- Attention can implement some nonlinear functions via softmax
- But MLP’s element-wise nonlinearity over d_{ff} dimensions is hard to match
- Attention is fundamentally about weighted averaging, not arbitrary transformations

Conclusion: The components are complementary, not interchangeable.

Remark 14.36 (Formal Perspective). Consider the computation graph:

- **Attention:** Implements a data-dependent sparse graph over positions. The “sparsity” is soft (via softmax).
- **MLP:** Implements a dense, fixed computation at each node.

This separation resembles message-passing neural networks:

- Attention = message passing (aggregate from neighbors)
- MLP = node update (transform aggregated information)

14.8 In-Context Learning

Definition 14.37 (In-Context Learning (ICL)). **In-context learning** is the ability of a model to learn new tasks from examples provided in the prompt, without updating parameters:

$$\text{prompt} = [\text{example}_1, \text{example}_2, \dots, \text{example}_k, \text{query}]$$

The model produces the correct answer to the query by “learning” from the examples.

Remark 14.38 (Competing Explanations). Three main theories attempt to explain ICL:

1. Implicit Bayesian Inference:

- The model implicitly maintains a posterior over “tasks” or “functions”

- Examples update this posterior; the query is answered by the posterior predictive
- Formally: $p(y|\text{query}, \text{examples}) = \int p(y|\text{query}, f)p(f|\text{examples})df$

2. Amortized Meta-Learning:

- During pretraining, the model learns a meta-algorithm for learning
- The forward pass implements gradient descent or a similar learning algorithm
- ICL is “learning to learn” compressed into weights

3. Pattern Matching in Embedding Space:

- No true “learning” occurs; the model retrieves similar patterns from training
- Examples activate relevant “task vectors” in embedding space
- ICL is sophisticated retrieval, not inference

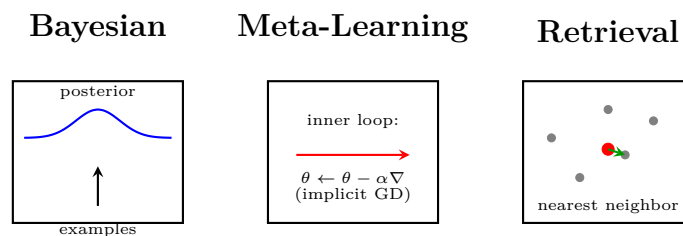


Figure 14.12: Three theories of in-context learning: Bayesian inference, implicit meta-learning, and pattern retrieval.

Remark 14.39 (Falsifying Each Theory). **To falsify Bayesian inference:**

- Show ICL performance doesn’t improve with more examples (Bayesian should converge)
- Show ICL fails with shuffled examples (Bayesian inference is permutation-invariant for i.i.d. examples)
- Find tasks where ICL succeeds but Bayesian inference would require exponential samples

To falsify meta-learning:

- Show ICL works for tasks never seen during pretraining (meta-learning requires task distribution)
- Demonstrate that attention patterns don’t resemble gradient descent updates
- Show ICL in 1-layer models (gradient descent needs depth)

To falsify pattern matching:

- Show ICL on truly novel tasks with no pretraining analogues
- Demonstrate compositional generalization beyond training distribution
- Show ICL improves with more compute at inference (retrieval shouldn't)

Remark 14.40 (Current Evidence). The evidence is mixed:

- Transformers *can* implement gradient descent in their forward pass (supports meta-learning)
- ICL is sensitive to example order (against pure Bayesian inference)
- ICL often fails on truly out-of-distribution tasks (supports retrieval)
- Scaling improves ICL substantially (mechanism unclear)

The truth is likely a combination: transformers learn a flexible algorithm that has Bayesian-like properties, meta-learning structure, and retrieval components.

14.9 Expressivity: Depth vs Width

Remark 14.41 (The Core Question). **Can a single-layer transformer with very wide attention simulate a deep transformer?**

Intuitively:

- More heads = more parallel computations
- Wider FFN = more features
- But can width substitute for depth?

Theorem 14.42 (Width Can Simulate Depth—At Exponential Cost). *A single-layer transformer with $O(L \cdot d^L)$ heads can simulate an L -layer transformer with d dimensions.*

Proof sketch: Enumerate all possible L -step computations and implement each with a separate head.

Remark 14.43 (Where Depth is Strictly Necessary). Depth provides exponential efficiency for:

1. Sequential Computation:

- Tasks requiring k sequential steps need $\Omega(k)$ layers
- Example: Composing k functions $f_k \circ f_{k-1} \circ \dots \circ f_1$
- Width cannot parallelize inherently sequential computation

2. Iterative Refinement:

- Many algorithms iterate: gradient descent, Newton’s method, belief propagation
- Each iteration refines the previous output
- Depth = number of refinement steps

3. Hierarchical Composition:

- Recognizing “noun phrase inside prepositional phrase inside sentence”
- Each layer builds higher-level abstractions
- Width gives more features at one level; depth gives more levels

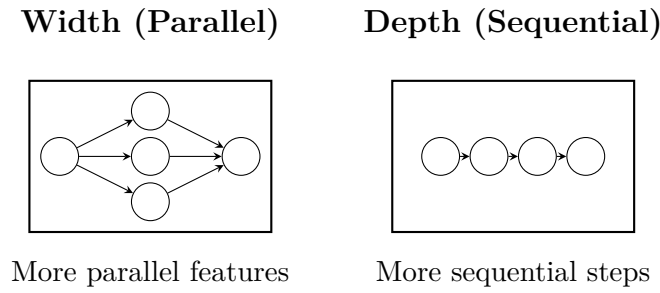


Figure 14.13: Width enables parallel feature computation; depth enables sequential refinement.

Remark 14.44 (Formal Results). **Circuit complexity:** Transformers with L layers can compute functions in TC^0 (constant-depth threshold circuits) with $O(L)$ “rounds” of adaptive computation.

Separation: There exist functions computable by depth- L transformers that require width $\exp(\Omega(L))$ for depth-1 transformers.

Practical implication: For most tasks, moderate depth (12–96 layers) with moderate width is more efficient than extreme width with minimal depth.

14.10 Information Bottlenecks in Attention

Remark 14.45 (The Bottleneck). With H attention heads and hidden dimension d , self-attention imposes a bottleneck:

Per-head capacity: Each head has dimension $d_k = d/H$. The attention matrix $A \in \mathbb{R}^{n \times n}$ can encode at most $n \cdot d_k$ bits of information about which positions to attend.

Rank constraint: The attention output for head h is:

$$\text{head}_h = A_h V_h \in \mathbb{R}^{n \times d_v}$$

where $\text{rank}(A_h V_h) \leq \min(n, d_v, \text{rank}(A_h))$.

Remark 14.46 (Scaling with Sequence Length). As sequence length n grows:

Attention is $O(n^2)$: Computing $QK^\top$ requires $O(n^2)$ operations and memory.

But information per position is $O(1)$: Each position receives a d -dimensional vector, regardless of n . Long-range dependencies must be compressed into this fixed capacity.

Effective context: Empirically, attention weights concentrate on $O(1)$ or $O(\log n)$ positions. Most positions receive negligible attention.

Typical Attention Pattern

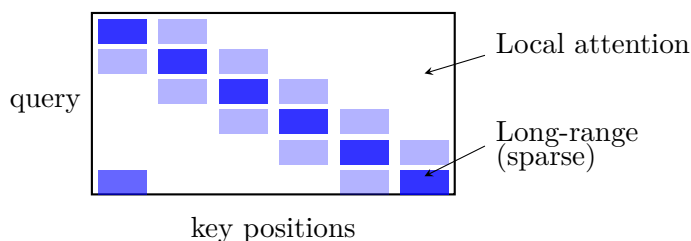


Figure 14.14: Attention concentrates on diagonal (local) with sparse long-range connections.

Remark 14.47 (Information-Theoretic View). Consider the mutual information between position i 's output and distant position j 's input:

$$I(\text{output}_i; \text{input}_j) \leq H(\text{attention weights on } j) \leq \log_2(\text{precision})$$

With float16 attention weights, each position can communicate at most ~ 16 bits per head to any other position. For a sequence of length 10,000, this is a severe bottleneck.

Implication: Transformers must learn to route important information through multiple hops or compress it efficiently. This is why intermediate “summary” tokens and hierarchical attention patterns emerge.

14.11 Optimization Landscape

Remark 14.48 (The Mystery). Transformers are highly non-convex with billions of parameters, yet:

- They train reliably with SGD/Adam
- Loss decreases smoothly (no catastrophic failures)
- Different random seeds converge to similar performance

Why are they “easy” to optimize?

Remark 14.49 (Role of Residual Connections). Beyond gradient flow, residuals shape the loss landscape:

1. **Linear path at initialization:**

- At init, layer outputs are small (weights near zero)
- The network is approximately $x + \epsilon \cdot F'(x) \approx x$
- The loss landscape is nearly convex initially

2. Gradual nonlinearity:

- As training progresses, $F'(x)$ grows
- Nonlinearity increases gradually
- The optimization “follows” a path from convex to non-convex

3. Ensemble effect:

- With residuals, a depth- L network has 2^L paths (include/exclude each layer)
- This implicit ensemble smooths the loss landscape

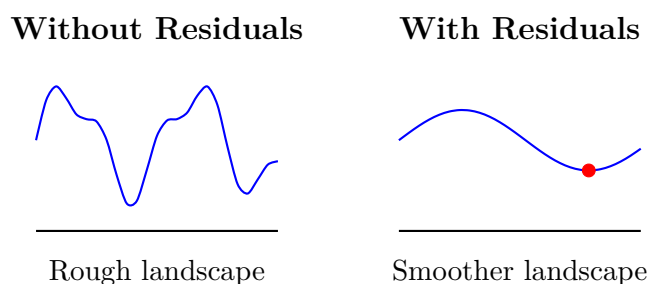


Figure 14.15: Residual connections smooth the loss landscape, making optimization easier.

Remark 14.50 (Role of Layer Normalization). LayerNorm provides:

1. Scale invariance:

- Activations are normalized to unit variance
- Gradient magnitudes are controlled regardless of layer

2. Implicit learning rate adaptation:

- LayerNorm’s Jacobian depends on activation statistics
- This creates an adaptive, per-neuron learning rate

3. Loss landscape conditioning:

- The Hessian eigenvalues are bounded more tightly
- This allows larger learning rates without divergence

Remark 14.51 (Additional Factors). • **Overparameterization:** More parameters than needed creates many equivalent minima

- **Data scale:** Large datasets provide consistent gradients (low variance)
- **Attention sparsity:** Attention weights are soft-sparse, reducing effective nonlinearity
- **Adam optimizer:** Adaptive learning rates handle different parameter scales

14.12 Length Generalization Failures

Remark 14.52 (The Problem). Transformers often fail to generalize to longer sequences, even when:

- The task is algorithmically length-invariant (e.g., addition, copying)
- Relative position encodings are used
- The model has “enough” capacity

Example 14.53 (Addition Failure). Train a transformer on 5-digit addition:

$$12345 + 67890 = 80235 \quad \checkmark$$

Test on 10-digit addition:

$$1234567890 + 9876543210 = ? \quad (\text{often wrong})$$

The algorithm is the same, but the model fails.

Remark 14.54 (Causes of Length Generalization Failure). **1. Attention Distribution Shift:**

- With more positions, attention is spread thinner
- Patterns that worked with 10 positions fail with 1000
- Even relative encodings don’t fix the attention *sharpness*

2. Positional Encoding Extrapolation:

- Learned positions: unseen indices have random embeddings
- Sinusoidal: extrapolates but at different frequencies
- RoPE: better but still degrades beyond training length

3. Depth Limitations:

- Some algorithms require $O(n)$ sequential steps
- Transformers have $O(L)$ depth, independent of n
- Example: Checking balanced parentheses needs a “stack” that grows with n

4. Training Distribution:

- The model is optimized for short sequences
- Features learned for length 100 may not transfer to length 10,000
- The loss landscape for long sequences is different

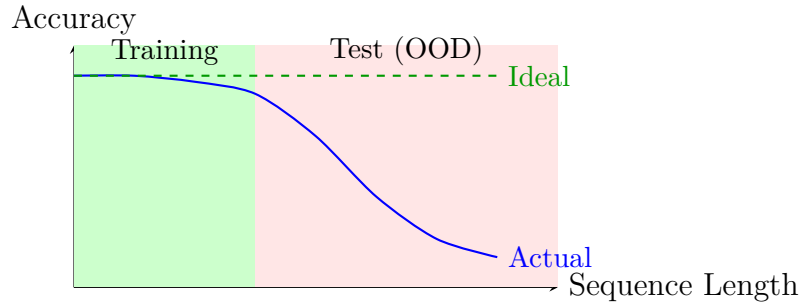


Figure 14.16: Length generalization failure: accuracy degrades beyond training sequence lengths.

14.13 Universality vs Efficiency

Theorem 14.55 (Transformer Universality). *Transformers are universal sequence-to-sequence approximators: for any continuous function $f : \mathbb{R}^{n \times d} \rightarrow \mathbb{R}^{n \times d}$ and $\epsilon > 0$, there exists a transformer T such that $\|T(X) - f(X)\| < \epsilon$ for all X in a compact domain.*

Remark 14.56 (But At What Cost?). Universality says nothing about efficiency. Key questions:

1. Width/depth requirements:

- Some functions require exponential width or depth
- Universality proofs often use $O(1/\epsilon^d)$ parameters

2. Sample complexity:

- Learning the approximation may require exponential data
- Existence $\neq$ learnability

Remark 14.57 (Comparison with Other Architectures). **Transformers vs RNNs:**

	Transformer	RNN
Parallel training	$O(1)$ depth	$O(n)$ sequential
Long-range deps	$O(1)$ path length	$O(n)$ path length
Memory	$O(n^2)$ attention	$O(1)$ state
Sequential tasks	$O(L)$ layers	$O(n)$ natural

Transformers vs State-Space Models (SSMs):

- SSMs (Mamba, S4) have $O(n)$ complexity vs $O(n^2)$
- SSMs maintain compressed state; may lose information
- SSMs are better for very long sequences; worse for tasks requiring full context access

Remark 14.58 (Function Classes). Functions efficiently representable by transformers:

- **Efficient:** Pattern matching, copying, soft retrieval, shallow reasoning
- **Possible but costly:** Deep recursion, arithmetic, precise counting
- **Likely exponential:** Arbitrary graph algorithms, general-purpose computation

The transformer architecture is biased toward:

- Parallel, shallow computation
- Soft, differentiable operations
- Pattern recognition over symbolic manipulation

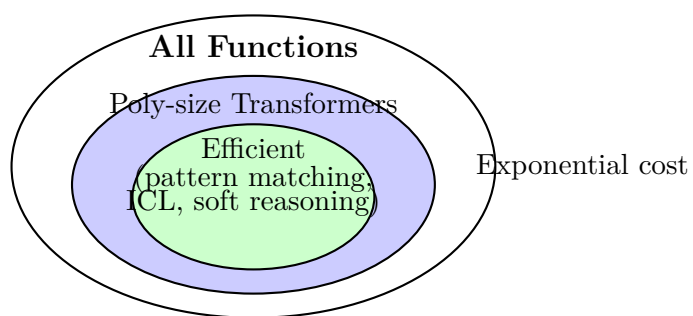


Figure 14.17: Hierarchy of function classes: transformers efficiently compute a subset of representable functions.

Chapter 15

Training Phases of Large Language Models

Modern LLMs are trained in distinct phases, each with different objectives, data, and which parameters are updated.

15.1 Overview of Training Phases

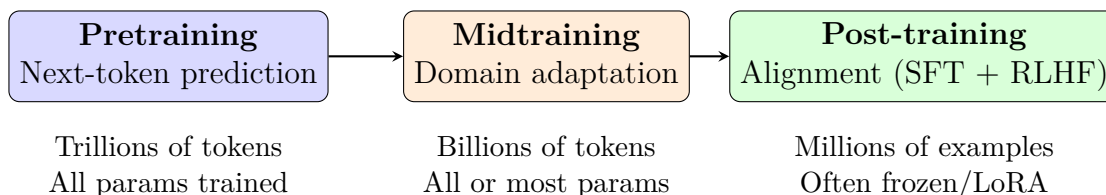


Figure 15.1: The three phases of LLM training: pretraining, midtraining, and post-training.

15.2 Pretraining

Definition 15.1 (Pretraining). **Pretraining** is the initial phase where the model learns general language understanding from a massive corpus using self-supervised learning.

Objective: Next-token prediction (autoregressive LM):

$$\mathcal{L}_{\text{pretrain}} = - \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t})$$

Data: Web crawls (Common Crawl), books, Wikipedia, code. Typically 1–15 trillion tokens.

Parameters: All parameters trained from random initialization.

Remark 15.2 (What the Model Learns). During pretraining, the model acquires:

- **Syntax:** Grammar, sentence structure, agreement
- **Semantics:** Word meanings, relationships, common sense
- **World knowledge:** Facts, dates, named entities
- **Reasoning patterns:** (to some extent) logical deduction, arithmetic

The model is “raw”—it completes text but doesn’t follow instructions.

15.3 Midtraining (Continued Pretraining)

Definition 15.3 (Midtraining). **Midtraining** extends pretraining on domain-specific data to specialize the model.

Objective: Same as pretraining (next-token prediction), but on curated domain data.

Data: Domain-specific corpora (medical, legal, code, scientific). Billions of tokens.

Remark 15.4 (Freezing During Midtraining). Common approaches:

1. **Full fine-tuning:** All parameters updated. Risk of catastrophic forgetting.
2. **Freeze embeddings:** Keep token embeddings fixed; train transformer layers.
3. **Freeze early layers:** Early layers capture syntax; freezing preserves basic patterns.
4. **Layerwise LR decay:** Earlier layers get smaller updates.

Typical practice: Train all parameters but with lower learning rate and mixed data.

15.4 Post-training (Alignment)

Post-training transforms a raw LM into an assistant that follows instructions.

15.4.1 Supervised Fine-Tuning (SFT)

Definition 15.5 (Supervised Fine-Tuning). **SFT** trains the model on (instruction, response) pairs:

$$\mathcal{L}_{\text{SFT}} = - \sum_{t=1}^T \log p_{\theta}(y_t | x, y_{<t})$$

where x is the instruction and y is the desired response.

15.4.2 RLHF / Preference Optimization

Definition 15.6 (RLHF). **RLHF** refines the model using human preferences:

1. Train a reward model on preference data
2. Use RL (PPO) to optimize:

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}} [R(x, y)] - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

Remark 15.7 (Freezing During Post-training). Common strategies:

1. **Full fine-tuning**: Update all parameters.
2. **LoRA**: Freeze pretrained weights; add low-rank adapters.
3. **Freeze embeddings**: Vocabulary unchanged; no need to update.
4. **Freeze early layers**: Alignment affects high-level behavior (later layers).

Remark 15.8 (Summary: What Gets Frozen).

	Pretraining	Midtraining	Post-training
Embeddings	Train	Train or Freeze	Usually Freeze
Early transformer layers	Train	Train (lower LR)	Often Freeze / LoRA
Late transformer layers	Train	Train	Train / LoRA
LM head (unembedding)	Train	Train	Train or Freeze
Layer norms	Train	Train	Train

15.5 Parameter-Efficient Fine-Tuning (PEFT)

Definition 15.9 (LoRA). **Low-Rank Adaptation** freezes pretrained weights W and adds trainable low-rank matrices:

$$h = Wx + BAx$$

where $B \in \mathbb{R}^{d_{\text{out}} \times r}$, $A \in \mathbb{R}^{r \times d_{\text{in}}}$, and $r \ll \min(d_{\text{out}}, d_{\text{in}})$.

At initialization, $B = 0$, so $BA = 0$ and output matches the frozen model.

Example 15.10 (LoRA Savings). For $W \in \mathbb{R}^{4096 \times 4096}$:

- Full: $4096^2 = 16.7\text{M}$ parameters
- LoRA ($r = 16$): $(4096 + 4096) \times 16 = 131\text{K}$ parameters (0.8%)

Remark 15.11 (Other PEFT Methods). • **QLoRA**: LoRA + 4-bit quantization. Fine-tune 65B on single GPU.

- **Prefix Tuning**: Learn virtual tokens prepended to input.
- **Adapters**: Insert small trainable layers between blocks.

15.6 Encoder-Decoder vs Decoder-Only Architecture

The original Transformer was designed for sequence-to-sequence tasks like translation. Modern language models use only the decoder.

Definition 15.12 (Original Transformer Architecture). The **Attention Is All You Need** architecture has two components:

- **Encoder:** Processes the input sequence (e.g., English sentence). Uses bidirectional self-attention—each token can attend to all others.
- **Decoder:** Generates the output sequence (e.g., French translation). Uses causal self-attention + cross-attention to the encoder.

Remark 15.13 (Cross-Attention: Q from Decoder, K/V from Encoder). In cross-attention:

$$Q = X_{\text{dec}}W_Q, \quad K = X_{\text{enc}}W_K, \quad V = X_{\text{enc}}W_V$$

Why this asymmetry?

- **Query from decoder:** “What information do I need to generate the next target token?”
- **Keys from encoder:** “Where in the source sentence is relevant information?”
- **Values from encoder:** “What is the actual content to retrieve?”

The decoder “asks questions” (Q) about the encoder’s representation (K, V). This allows each decoder position to selectively read from the encoded source.

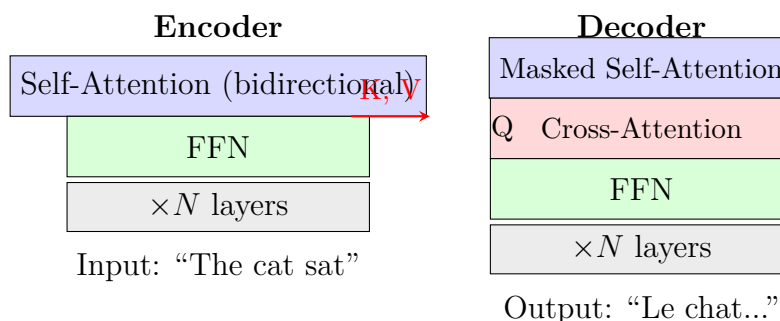


Figure 15.2: Encoder-decoder architecture: encoder provides K/V for cross-attention; decoder queries with Q.

Remark 15.14 (Why Decoder-Only for Language Models?). Modern LLMs (GPT, LLaMA, Claude) use **decoder-only** architecture because:

1. **Unified task:** Language modeling is next-token prediction, not translation. No separate source/target.

2. **Simplicity**: One model type, half the parameters for same depth.
3. **Flexibility**: Same architecture handles completion, chat, reasoning.
4. **Scaling**: Simpler architecture scales better empirically.

The encoder-decoder split only makes sense when input and output are fundamentally different (translation, summarization with fixed input).

Definition 15.15 (Decoder-Only Architecture). A **decoder-only** transformer for language modeling consists of:

1. Token embedding + positional encoding
2. $N \times$ blocks of:
 - Masked (causal) multi-head self-attention
 - Add & LayerNorm
 - Feed-forward network
 - Add & LayerNorm
3. Final LayerNorm
4. Linear projection to vocabulary
5. Softmax for next-token probabilities

No encoder, no cross-attention. The model only sees past tokens.

15.7 Attention Dimensions in Practice

Remark 15.16 (Typical Dimensions (Attention Is All You Need)). The original Transformer paper used:

- $d_{\text{model}} = 512$ (embedding dimension)
- $H = 8$ heads
- $d_k = d_v = d_{\text{model}}/H = 64$ per head
- $W_Q, W_K, W_V \in \mathbb{R}^{512 \times 64}$ for each head
- $W_O \in \mathbb{R}^{512 \times 512}$ (output projection)

The “base” model had $N = 6$ encoder and decoder layers. The “big” model used $d_{\text{model}} = 1024$, $H = 16$ heads.

Remark 15.17 (Why Separate W_Q , W_K , W_V Instead of One Matrix?). One might think: since we compute $QK^\top = (XW_Q)(XW_K)^\top = X(W_QW_K^\top)X^\top$, why not learn $W_{QK} = W_QW_K^\top$ directly?

The problem is dimensionality:

- $W_Q \in \mathbb{R}^{d \times d_k}$, $W_K \in \mathbb{R}^{d \times d_k} \Rightarrow 2d \cdot d_k$ parameters
- $W_{QK} = W_Q W_K^\top \in \mathbb{R}^{d \times d} \Rightarrow d^2$ parameters

For GPT-3 with $d = 12288$ and $d_k = 128$:

- Factored: $2 \times 12288 \times 128 = 3.1\text{M}$ parameters
- Combined: $12288^2 = 151\text{M}$ parameters ($48\times$ larger!)

The factorization is a **low-rank constraint**: we’re forcing W_{QK} to have rank at most d_k , which acts as regularization and saves massive compute/memory.

Remark 15.18 (Why Not $Q = K = V$?). If we set $W_Q = W_K = W_V$, attention becomes symmetric:

$$A_{ij} = A_{ji} \quad (\text{how much } i \text{ attends to } j = \text{how much } j \text{ attends to } i)$$

This loses expressivity:

- Can’t learn asymmetric relations (“subject attends to verb” vs “verb attends to subject”)
- Query-key separation lets the model learn **what to look for** (Q) separately from **what to match against** (K)
- Value separation lets the model learn **what information to extract** independently

Empirically, tied projections perform significantly worse.

15.8 KV Cache for Efficient Inference

During autoregressive generation, we can cache computed keys and values to avoid redundant computation.

Definition 15.19 (KV Cache). For a decoder generating token $t + 1$ given tokens $1, \dots, t$:

- **Without cache**: Recompute $K = [k_1, \dots, k_t, k_{t+1}]$ and $V = [v_1, \dots, v_t, v_{t+1}]$ from scratch.
- **With cache**: Store $K_{\text{cache}} = [k_1, \dots, k_t]$ and $V_{\text{cache}} = [v_1, \dots, v_t]$. Only compute k_{t+1}, v_{t+1} for the new token and append.

Remark 15.20 (KV Cache Memory Cost). The cache stores K, V for all layers and all past tokens:

$$\text{Memory} = 2 \times L \times n \times d_{\text{model}} \times \text{bytes per element}$$

For LLaMA-70B with $L = 80$ layers, $d = 8192$, context $n = 4096$, fp16:

$$2 \times 80 \times 4096 \times 8192 \times 2 \text{ bytes} = 10.7 \text{ GB per sequence}$$

This is why long-context models are memory-bound during inference.

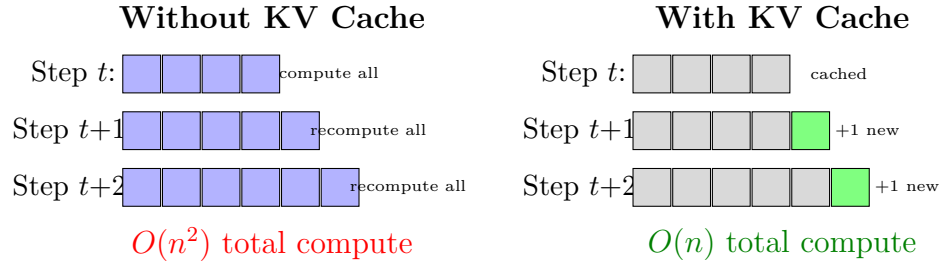


Figure 15.3: KV cache avoids recomputation: cached keys/values persist across generation steps.

Remark 15.21 (Why KV Cache Works with Causal Masking). The KV cache relies on causal (autoregressive) attention:

- Past tokens' keys/values don't depend on future tokens
- Once computed, k_i and v_i are fixed forever
- The query q_{t+1} only needs to attend to $k_1, \dots, k_{t+1}$

With bidirectional attention, each new token would change all previous representations, invalidating the cache.

15.9 Causal Masking

Definition 15.22 (Causal Mask). The **causal mask** (or autoregressive mask) prevents position i from attending to positions $j > i$:

$$M_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

Applied as: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$

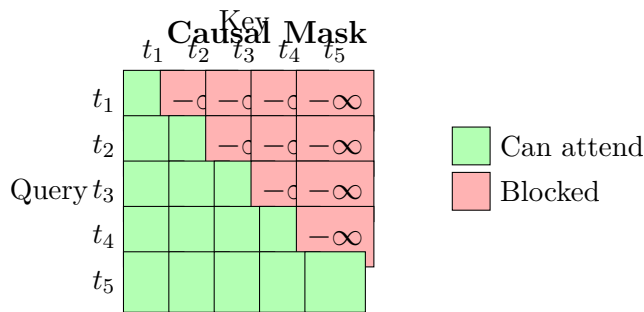


Figure 15.4: Causal mask: lower triangular allows each token to attend only to previous tokens.

Remark 15.23 (Why Causal Masking is Necessary). For autoregressive language modeling:

1. **Training:** We predict $P(x_t|x_1, \dots, x_{t-1})$. If token t could see token $t + 1$, it would “cheat”—the task becomes trivial.
2. **Inference:** At generation time, future tokens don’t exist yet. Training must match this.
3. **Parallelism:** Without masking, we’d need sequential training. With masking, we can compute all positions in parallel during training.

Remark 15.24 (Causal Mask and KV Cache Interaction). The causal mask is what makes KV caching possible:

- Token t only attends to tokens $1, \dots, t$ (causality)
- Therefore k_t and v_t only depend on $x_1, \dots, x_t$
- Once computed, these never change regardless of future tokens
- Cache is append-only: just add new k_{t+1}, v_{t+1}

Without causality, token t ’s representation would depend on all tokens, including future ones—caching would be invalid.

15.10 What Limits Context Window?

Remark 15.25 (Context Window Limitations). Several factors limit how long a context a transformer can handle:

1. **Positional Encodings:** Most are defined for fixed range.
 - Learned absolute: $p_1, \dots, p_N$ are trained embeddings. Position $N + 1$ has no embedding.
 - Sinusoidal: Defined for any position mathematically, but model wasn’t trained on long sequences.
 - RoPE: Rotation angles are defined modulo 2π . With base 10000 and $d = 128$, positions beyond ~ 10000 may have colliding angles.
2. **Attention Complexity:** $O(n^2)$ memory and compute. For $n = 1\text{M}$ tokens, attention matrix has 10^{12} entries.
3. **Training Distribution:** Model was optimized for sequences of length $\leq N$. Longer sequences are out-of-distribution.
4. **Attention Patterns:** Model may learn position-dependent heuristics that break for unseen positions.

Remark 15.26 (Extending Context Windows). Techniques for longer contexts:

- **Position interpolation:** Scale positions so trained range covers longer sequences. If trained on 4K, scale by 0.5 to fit 8K into $[0, 4K]$.
- **ALiBi:** Replace positional encodings with linear attention bias based on distance. No position embeddings to extrapolate.
- **Sparse attention:** Only attend to subset of tokens (local window + global tokens). Reduces $O(n^2)$ to $O(n)$.
- **Ring attention:** Distribute attention computation across devices for sequences that don't fit in memory.

15.11 The Role of Feed-Forward Networks

Remark 15.27 (Why Attention Alone is Insufficient). Self-attention has fundamental limitations:

1. **Linear in values:** Output is a weighted sum of values: $\sum_j \alpha_j v_j$. This is a linear combination—no nonlinearity applied to the content.
2. **No position-wise transformation:** Attention mixes information across positions but doesn't transform individual token representations.
3. **Rank bottleneck:** With H heads of dimension d_k , attention can only produce outputs in a Hd_k -dimensional subspace per layer.

Remark 15.28 (What FFN Provides). The feed-forward network $\text{FFN}(x) = W_2 \cdot \text{ReLU}(W_1 x + b_1) + b_2$ contributes:

1. **Nonlinearity:** ReLU (or GELU/SiLU) introduces nonlinear transformations, enabling the model to learn complex functions.
2. **Per-token processing:** Each token is processed independently, allowing position-specific transformations.
3. **Capacity expansion:** FFN typically has $d_{\text{ff}} = 4 \times d_{\text{model}}$. This “expands” the representation, applies nonlinearity, then “compresses” back.
4. **Memory/knowledge storage:** Evidence suggests FFN layers store factual associations (“Eiffel Tower $\rightarrow$ Paris”). Attention routes information; FFN stores it.

Remark 15.29 (Attention is Linear in V). Despite softmax nonlinearity in computing weights, attention output is:

$$\text{output}_i = \sum_j \underbrace{\text{softmax} \left(\frac{q_i^\top k_j}{\sqrt{d_k}} \right)}_{\alpha_{ij}} v_j = \sum_j \alpha_{ij} v_j$$

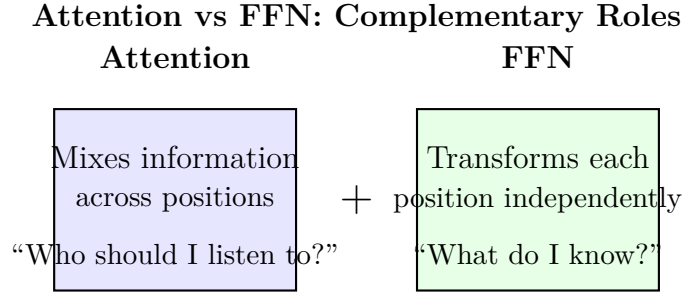


Figure 15.5: Attention aggregates across positions; FFN stores and retrieves knowledge at each position.

This is a **weighted average**—a linear combination of the v_j . The output lives in the convex hull of values.

FFN breaks this: $\text{ReLU}(Wx)$ can produce outputs outside any convex hull of inputs.

15.12 Computational Complexity of Self-Attention

Theorem 15.30 (Self-Attention Complexity). *For sequence length n and dimension d :*

- **Time complexity:** $O(n^2d)$
- **Memory complexity:** $O(n^2 + nd)$

Proof. Breaking down the computation:

Computing Q, K, V : Each is XW where $X \in \mathbb{R}^{n \times d}$, $W \in \mathbb{R}^{d \times d_k}$.

Cost: $3 \times O(n \cdot d \cdot d_k) = O(nd^2)$ (assuming $d_k \sim d$)

Computing $QK^\top$: Matrix multiply $(n \times d_k) \times (d_k \times n) = (n \times n)$.

Cost: $O(n^2d_k) = O(n^2d)$

Softmax: Applied to each of n rows of length n .

Cost: $O(n^2)$

Computing AV : Matrix multiply $(n \times n) \times (n \times d_v) = (n \times d_v)$.

Cost: $O(n^2d_v) = O(n^2d)$

Total time: Dominated by $O(n^2d)$ from $QK^\top$ and AV .

Memory: Must store attention matrix $A \in \mathbb{R}^{n \times n}$ giving $O(n^2)$, plus Q, K, V , output $\in \mathbb{R}^{n \times d}$ giving $O(nd)$. □

Remark 15.31 (Where the Quadratic Term Hurts). The n^2 term is problematic because:

- **Memory:** For $n = 100\text{K}$, attention matrix is 10^{10} floats = 40GB in fp32

- **Compute:** Scales poorly—doubling context $4\times$ the attention cost
- **Cannot parallelize over sequence:** Must compute all n^2 pairs

Note: The d factor is parallelizable across GPU cores. The n^2 factor requires sequential dependency (each query attends to all keys).

15.13 Permutation Equivariance and Positional Encodings

Theorem 15.32 (Self-Attention is Permutation Equivariant). *Let π be a permutation and $\pi(X)$ the row-permuted input. Then:*

$$\text{Attention}(\pi(X)) = \pi(\text{Attention}(X))$$

That is, permuting inputs just permutes outputs the same way.

Proof. Let P_π be the permutation matrix for π . Then $\pi(X) = P_\pi X$.

$$\begin{aligned} Q' &= P_\pi X W_Q = P_\pi Q \\ K' &= P_\pi X W_K = P_\pi K \\ V' &= P_\pi X W_V = P_\pi V \end{aligned}$$

The attention scores:

$$Q'(K')^\top = P_\pi Q (P_\pi K)^\top = P_\pi Q K^\top P_\pi^\top = P_\pi A P_\pi^\top$$

After softmax (applied row-wise, preserving structure):

$$A' = P_\pi A P_\pi^\top$$

The output:

$$A'V' = P_\pi A P_\pi^\top P_\pi V = P_\pi A V = P_\pi \cdot \text{Attention}(X)$$

□

Remark 15.33 (Consequence: Attention Sees a Set, Not a Sequence). Without positional encodings:

- “The cat sat on the mat” $\equiv$ “mat the on sat cat The” (same bag of tokens)
- Word order carries no information
- Language structure is lost

Positional encodings break this symmetry by making $x_i \neq x_j$ even if tokens are identical, because $x_i + p_i \neq x_j + p_j$.

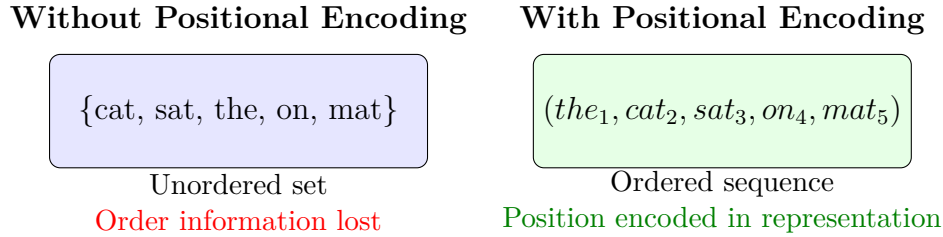


Figure 15.6: Positional encodings break permutation symmetry, converting sets into ordered sequences.

15.14 Why Transformers Excel at Lexical Matching

Remark 15.34 (Attention as Exact Match). Self-attention computes $q_i^\top k_j$ —a dot product measuring similarity. For identical tokens:

$$q_i = W_Q x_i, \quad k_j = W_K x_j$$

If $x_i = x_j$ (same token), then $q_i^\top k_j$ is maximized when W_Q, W_K are learned to detect this.

The model can easily learn: “attend strongly to tokens that match exactly.”

Remark 15.35 (Induction Heads Enable String Matching). Recall induction heads: $[A][B] \cdots [A] \rightarrow [B]$

This is precisely lexical/string matching:

1. Find previous occurrence of current token (lexical match)
2. Copy what came next (pattern completion)

This emerges naturally from the attention mechanism—no explicit string matching algorithm needed.

Remark 15.36 (Why Lexical Match is Easy for Attention). Consider the attention score for matching tokens $x_i = x_j$:

$$\text{score}_{ij} = (W_Q x_i)^\top (W_K x_j) = x_i^\top W_Q^\top W_K x_j$$

If $W_Q^\top W_K \approx I$ (identity), then $\text{score}_{ij} \approx x_i^\top x_j$, which is maximized when $x_i = x_j$.

Learning $W_Q^\top W_K \approx I$ is trivial—just initialize appropriately. This is why transformers are “born” with some string-matching ability.

15.15 Depth vs Width in Transformers

Remark 15.37 (Why Depth Helps Compositional Reasoning). Each transformer layer can be seen as one “step” of computation:

- Layer 1: Direct token relationships (adjacent words, simple patterns)
- Layer 2: Compositions of layer 1 patterns (phrases, local syntax)

- Layer k : k -step reasoning chains

For tasks requiring k sequential reasoning steps, you need at least k layers. **Depth enables sequential composition.**

Remark 15.38 (Why Width (More Heads) is Different). Adding more attention heads within a layer:

- Heads operate **in parallel**, not sequentially
- Each head captures a **different** relationship type
- More heads = more relationship types per layer, but not deeper reasoning

Analogy:

- **Depth**: More steps in a proof (sequential)
- **Width**: More facts available at each step (parallel)

You can't replace sequential reasoning with more parallel processing—some computations are inherently serial.

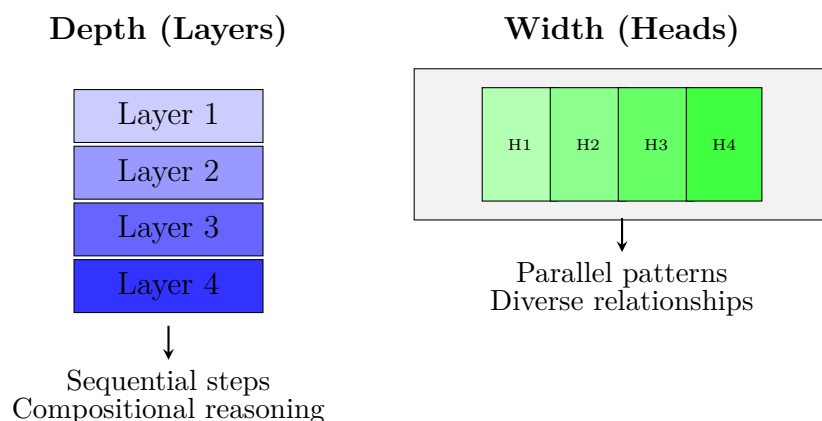


Figure 15.7: Depth vs width in transformer architecture: depth enables sequential compositional reasoning while width enables parallel pattern matching.

Chapter 16

Energy-Based Models

Energy-based models (EBMs) provide a unified framework for modeling data by associating a scalar *energy* with each configuration. Low energy configurations are “good” (likely), high energy configurations are “bad” (unlikely). This chapter develops the theory and connects EBMs to probabilistic modeling.

16.1 The Energy Function

Definition 16.1 (Energy-Based Model). An **energy-based model** defines a function $E_\theta : \mathcal{X} \rightarrow \mathbb{R}$ called the **energy function**, parameterized by θ . The model assigns:

- **Low energy** to configurations x that are “desirable” or consistent with observed data
- **High energy** to configurations that are “undesirable” or inconsistent

Learning involves adjusting θ so that training data receives lower energy than other configurations.

Remark 16.2 (Intuition). Think of energy as a “cost” or “incompatibility score”:

- A face detector assigns low energy to images containing faces
- A language model assigns low energy to grammatical sentences
- A physics simulator assigns low energy to stable configurations

The energy function captures what makes a configuration “good” without explicitly defining a probability distribution.

16.2 From Energy to Probability: The Boltzmann Distribution

To perform probabilistic inference, we convert energy to probability via the *Boltzmann distribution* (also called Gibbs distribution).

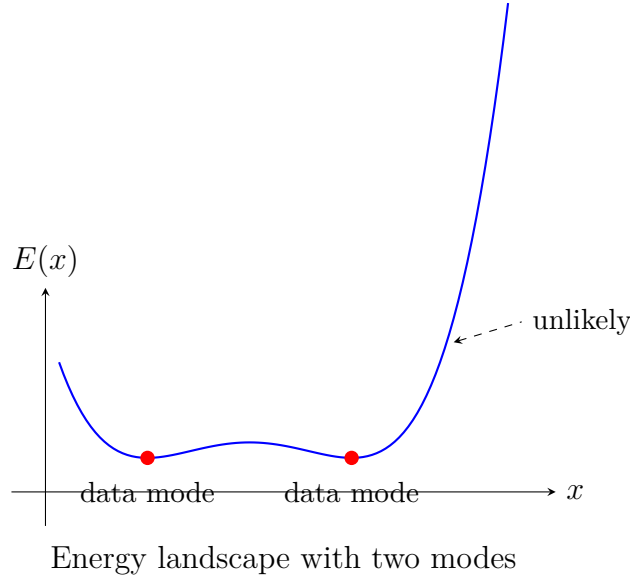


Figure 16.1: Energy function defining a landscape. Data concentrates at energy minima (modes).

Definition 16.3 (Boltzmann Distribution). Given an energy function $E_\theta(x)$, the associated **Boltzmann distribution** is:

$$p_\theta(x) = \frac{\exp(-E_\theta(x))}{Z_\theta}$$

where the **partition function** (normalizing constant) is:

$$Z_\theta = \int_{\mathcal{X}} \exp(-E_\theta(x)) dx$$

For discrete $\mathcal{X}$, the integral becomes a sum.

Remark 16.4 (Temperature Scaling). More generally, we can include a **temperature** parameter $T > 0$:

$$p_\theta(x; T) = \frac{\exp(-E_\theta(x)/T)}{Z_\theta(T)}$$

- $T \rightarrow 0$: Distribution concentrates on global minimum (deterministic)
- $T = 1$: Standard Boltzmann distribution
- $T \rightarrow \infty$: Distribution becomes uniform (maximum entropy)

Temperature controls the “sharpness” of the distribution around energy minima.

Theorem 16.5 (Log-Probability and Energy). *The log-probability under an EBM decomposes as:*

$$\log p_\theta(x) = -E_\theta(x) - \log Z_\theta$$

The partition function $\log Z_\theta$ is a constant with respect to x , so:

$$\nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x)$$

*The **score function** (gradient of log-density) equals the negative energy gradient.*

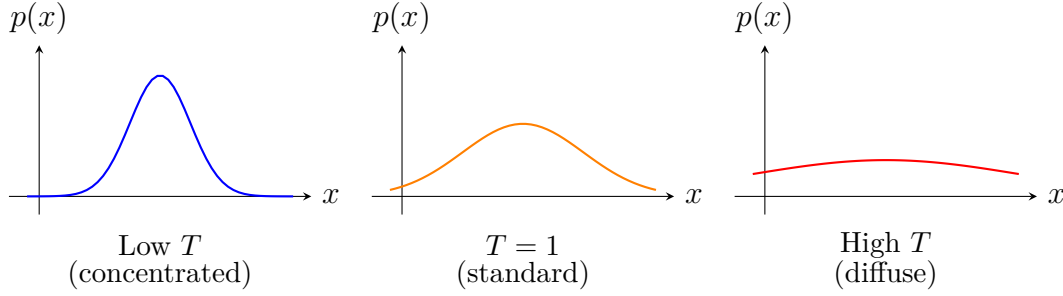


Figure 16.2: Effect of temperature on the Boltzmann distribution. Lower temperature concentrates probability mass at energy minima.

16.3 The Partition Function Problem

The central challenge in EBMs is computing the partition function Z_θ .

Remark 16.6 (Intractability of Z_θ). For most interesting energy functions, Z_θ is **intractable**:

- Continuous $\mathcal{X}$: Requires integrating over all possible inputs
- Discrete $\mathcal{X}$: Requires summing over exponentially many configurations
- No closed form: Neural network energy functions don't admit analytical integration

This intractability affects both:

1. **Evaluation**: Computing $p_\theta(x)$ for a given x
2. **Training**: Computing gradients $\nabla_\theta \log p_\theta(x)$

Theorem 16.7 (Gradient of Log-Likelihood). *For a single data point x , the gradient of the log-likelihood is:*

$$\nabla_\theta \log p_\theta(x) = -\nabla_\theta E_\theta(x) + \mathbb{E}_{x' \sim p_\theta}[\nabla_\theta E_\theta(x')]$$

Proof. Starting from $\log p_\theta(x) = -E_\theta(x) - \log Z_\theta$:

$$\nabla_\theta \log p_\theta(x) = -\nabla_\theta E_\theta(x) - \nabla_\theta \log Z_\theta$$

For the partition function term:

$$\begin{aligned} \nabla_\theta \log Z_\theta &= \frac{1}{Z_\theta} \nabla_\theta Z_\theta = \frac{1}{Z_\theta} \nabla_\theta \int \exp(-E_\theta(x')) dx' \\ &= \frac{1}{Z_\theta} \int \exp(-E_\theta(x')) \cdot (-\nabla_\theta E_\theta(x')) dx' \\ &= - \int p_\theta(x') \nabla_\theta E_\theta(x') dx' = -\mathbb{E}_{x' \sim p_\theta}[\nabla_\theta E_\theta(x')] \end{aligned}$$

Combining: $\nabla_\theta \log p_\theta(x) = -\nabla_\theta E_\theta(x) + \mathbb{E}_{x' \sim p_\theta}[\nabla_\theta E_\theta(x')]$. □

Remark 16.8 (Interpretation: Positive and Negative Phase). The gradient has two terms with intuitive meanings:

1. **Positive phase:** $-\nabla_{\theta} E_{\theta}(x)$ pushes down the energy of training data
2. **Negative phase:** $+\mathbb{E}_{x' \sim p_{\theta}}[\nabla_{\theta} E_{\theta}(x')]$ pushes up the energy of model samples

Training balances these forces: lower energy for real data, higher energy for “hallucinated” samples.

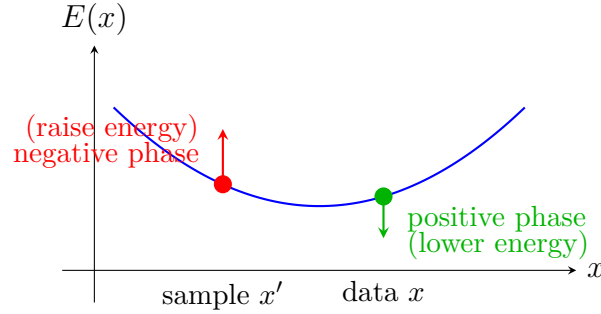


Figure 16.3: The two phases of EBM training: push down energy of data (positive phase), push up energy of model samples (negative phase).

16.4 Contrastive Divergence

Since sampling from p_{θ} is expensive, we approximate using MCMC.

Definition 16.9 (Contrastive Divergence (CD- k)). **Contrastive Divergence** approximates the negative phase expectation using short MCMC runs:

1. Initialize chain at a data point: $x^{(0)} = x_{\text{data}}$
2. Run k steps of MCMC (e.g., Gibbs sampling): $x^{(0)} \rightarrow x^{(1)} \rightarrow \dots \rightarrow x^{(k)}$
3. Approximate gradient:

$$\nabla_{\theta} \log p_{\theta}(x) \approx -\nabla_{\theta} E_{\theta}(x_{\text{data}}) + \nabla_{\theta} E_{\theta}(x^{(k)})$$

CD-1 (single MCMC step) is most common due to computational efficiency.

Remark 16.10 (Why CD Works). CD is biased but often effective because:

- Starting from data, the chain quickly moves to nearby regions the model considers likely
- These “negative samples” are precisely where the model is wrong
- The bias decreases as $k \rightarrow \infty$ (full equilibrium)

CD trades statistical accuracy for computational efficiency.

Remark 16.11 (Persistent Contrastive Divergence (PCD)). An improvement maintains **persistent chains** across gradient steps:

- Don't restart chains from data each iteration
- Continue chains from previous iteration's endpoints
- Chains gradually explore more of the distribution

PCD often produces better samples and more stable training than CD.

16.5 Score Matching

Score matching avoids computing Z_θ entirely by matching derivatives.

Definition 16.12 (Score Function). The **score function** of a distribution $p(x)$ is the gradient of its log-density:

$$s(x) = \nabla_x \log p(x)$$

For an EBM: $s_\theta(x) = -\nabla_x E_\theta(x)$ (independent of Z_θ !).

Theorem 16.13 (Score Matching Objective). *The **score matching** loss minimizes the expected squared difference between model and data scores:*

$$\mathcal{L}_{SM}(\theta) = \mathbb{E}_{x \sim p_{\text{data}}} \left[\frac{1}{2} \|s_\theta(x) - s_{\text{data}}(x)\|^2 \right]$$

*This is equivalent (up to a constant) to the **explicit score matching** objective:*

$$\mathcal{L}_{ESM}(\theta) = \mathbb{E}_{x \sim p_{\text{data}}} \left[\frac{1}{2} \|s_\theta(x)\|^2 + \nabla_x \cdot s_\theta(x) \right]$$

where $\nabla_x \cdot s_\theta(x) = \sum_i \frac{\partial s_{\theta,i}(x)}{\partial x_i}$ is the divergence (trace of Jacobian).

Remark 16.14 (Why Score Matching Works). The key insight is that $s_{\text{data}}(x)$ doesn't appear in the computable objective:

- The true data score $s_{\text{data}}(x) = \nabla_x \log p_{\text{data}}(x)$ is unknown
- But the cross-term $\mathbb{E}[s_\theta(x)^\top s_{\text{data}}(x)]$ can be converted via integration by parts
- The result only requires samples from p_{data} and gradients of s_θ

No partition function, no MCMC sampling from the model!

Remark 16.15 (Denoising Score Matching). Computing the divergence $\nabla_x \cdot s_\theta(x)$ is expensive for high dimensions. **Denoising score matching** offers an alternative:

1. Add noise to data: $\tilde{x} = x + \sigma\epsilon$, where $\epsilon \sim \mathcal{N}(0, I)$
2. Train to predict the score of the noised distribution:

$$\mathcal{L}_{DSM}(\theta) = \mathbb{E}_{x, \epsilon} \left[\left\| s_\theta(\tilde{x}) + \frac{\epsilon}{\sigma} \right\|^2 \right]$$

The optimal $s_\theta(\tilde{x})$ points back toward clean data—this is the foundation of **diffusion models**.

16.6 Noise Contrastive Estimation

Another approach: turn density estimation into classification.

Definition 16.16 (Noise Contrastive Estimation (NCE)). NCE trains the model to distinguish data from noise:

1. Choose a noise distribution $p_n(x)$ (e.g., Gaussian)
2. Create a classification problem: Is x from data or noise?
3. Train a classifier using the energy function

The NCE objective is:

$$\mathcal{L}_{\text{NCE}}(\theta) = \mathbb{E}_{x \sim p_{\text{data}}} [\log \sigma(E_n(x) - E_\theta(x))] + \mathbb{E}_{x \sim p_n} [\log \sigma(E_\theta(x) - E_n(x))]$$

where $E_n(x) = -\log p_n(x)$ is the noise energy and σ is the sigmoid.

Theorem 16.17 (NCE Consistency). *As the number of noise samples per data point $k \rightarrow \infty$, the NCE estimator converges to the maximum likelihood estimator. With k noise samples per data point, NCE estimates:*

$$\log p_\theta(x) \approx -E_\theta(x) - \log Z_\theta$$

where $\log Z_\theta$ is implicitly learned as a parameter.

Remark 16.18 (InfoNCE and Contrastive Learning). NCE inspired modern contrastive learning. The **InfoNCE** loss used in methods like SimCLR:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(s(x, x^+))}{\exp(s(x, x^+)) + \sum_{j=1}^k \exp(s(x, x_j^-))}$$

where x^+ is a positive pair (augmentation of x) and x_j^- are negative samples.

This is NCE applied to learn representations where similar items have low “energy” (high similarity).

16.7 Hopfield Networks

Hopfield networks are a classical EBM for associative memory.

Definition 16.19 (Hopfield Network). A **Hopfield network** is a recurrent network of n binary units $x_i \in \{-1, +1\}$ with symmetric connections. The energy function is:

$$E(x) = -\frac{1}{2} \sum_{i \neq j} w_{ij} x_i x_j - \sum_i b_i x_i = -\frac{1}{2} x^\top W x - b^\top x$$

where W is symmetric with zero diagonal, and b is the bias vector.

Theorem 16.20 (Hopfield Dynamics). *The asynchronous update rule:*

$$x_i \leftarrow \text{sign} \left(\sum_{j \neq i} w_{ij} x_j + b_i \right)$$

always decreases or maintains energy. The network converges to a local minimum.

Proof. When updating unit i , the energy change is:

$$\Delta E = -\Delta x_i \left(\sum_{j \neq i} w_{ij} x_j + b_i \right)$$

The update rule ensures Δx_i has the same sign as the sum, so $\Delta E \leq 0$. $\square$

Remark 16.21 (Associative Memory). Hopfield networks store patterns as energy minima:

- **Storage:** Given patterns $\{\xi^1, \dots, \xi^p\}$, set weights via Hebbian rule: $w_{ij} = \frac{1}{n} \sum_{\mu=1}^p \xi_i^\mu \xi_j^\mu$
- **Retrieval:** Initialize with corrupted pattern, run dynamics until convergence
- **Capacity:** Can reliably store approximately $0.14n$ patterns

The network performs pattern completion—given partial information, it recovers the full stored pattern.

16.8 Restricted Boltzmann Machines

RBMs are a tractable class of EBMs with hidden variables.

Definition 16.22 (Restricted Boltzmann Machine). An **RBM** consists of:

- Visible units $v \in \{0, 1\}^m$ (observed data)
- Hidden units $h \in \{0, 1\}^n$ (latent features)
- No connections within layers (“restricted”)

The energy function is:

$$E(v, h) = -v^\top W h - b^\top v - c^\top h$$

where $W \in \mathbb{R}^{m \times n}$ are weights, $b \in \mathbb{R}^m$ and $c \in \mathbb{R}^n$ are biases.

Theorem 16.23 (RBM Conditional Independence). *Due to the bipartite structure, conditionals factorize:*

$$\begin{aligned} p(h|v) &= \prod_{j=1}^n p(h_j|v) = \prod_{j=1}^n \sigma \left(\sum_i w_{ij} v_i + c_j \right)^{h_j} \\ p(v|h) &= \prod_{i=1}^m p(v_i|h) = \prod_{i=1}^m \sigma \left(\sum_j w_{ij} h_j + b_i \right)^{v_i} \end{aligned}$$

*This enables efficient **block Gibbs sampling**: sample all hidden units in parallel given visible, and vice versa.*

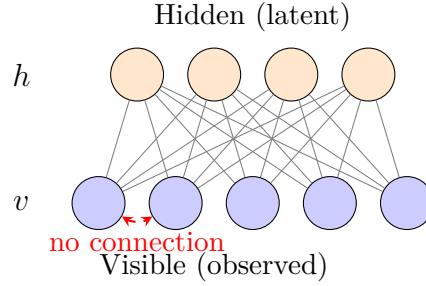


Figure 16.4: RBM architecture: bipartite graph with no within-layer connections, enabling efficient Gibbs sampling.

Remark 16.24 (RBM Training with CD). RBMs are trained using Contrastive Divergence:

1. **Positive phase:** Clamp visible units to data, sample hidden: $h^{(0)} \sim p(h|v^{(0)})$
2. **Reconstruction:** Sample visible from hidden: $v^{(1)} \sim p(v|h^{(0)})$
3. **Negative phase:** Sample hidden again: $h^{(1)} \sim p(h|v^{(1)})$
4. **Update:** $\Delta W \propto v^{(0)}h^{(0)\top} - v^{(1)}h^{(1)\top}$

This is CD-1: one step of Gibbs sampling approximates the negative phase.

16.9 Connection to Modern Generative Models

EBMs connect to many modern approaches in deep learning.

Remark 16.25 (Diffusion Models as Score-Based EBMs). Diffusion models can be viewed as a hierarchy of EBMs at different noise levels:

- Define noised distributions: $p_t(x) = \int p_{\text{data}}(x_0) \mathcal{N}(x; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I) dx_0$
- Train a **score network** $s_\theta(x, t) \approx \nabla_x \log p_t(x)$
- Generate by following the score (Langevin dynamics or probability flow ODE)

The score network implicitly defines an energy: $E_\theta(x, t) = -\int s_\theta(x', t) \cdot dx'$ (path integral).

Remark 16.26 (GANs vs EBMs). GANs and EBMs differ in how they handle the negative phase:

	EBM	GAN
Negative samples	MCMC from p_θ	Generator $G(z)$
Training signal	$\nabla_\theta E_\theta(x')$	Discriminator gradient
Mode coverage	Good (in principle)	Mode collapse risk
Sampling speed	Slow (MCMC)	Fast (feedforward)

GANs amortize MCMC sampling into a generator network.

Remark 16.27 (VAEs and the Free Energy). Variational autoencoders relate to EBMs through the **free energy**:

$$\mathcal{F}(x; q) = \mathbb{E}_{z \sim q(z|x)}[E_\theta(x, z)] - H(q(z|x))$$

The ELBO is: $\log p_\theta(x) \geq -\mathcal{F}(x; q)$. VAEs optimize this bound using:

- **Energy:** $E_\theta(x, z) = -\log p_\theta(x|z) - \log p(z)$
- **Variational distribution:** $q_\phi(z|x)$ (encoder)

Remark 16.28 (Modern Hopfield Networks and Attention). There's a deep connection between Hopfield networks and attention:

- **Classical Hopfield:** $E(x) = -\frac{1}{2}x^\top Wx$, capacity $\sim 0.14n$
- **Modern Hopfield:** $E(x) = -\text{lse}(Wx) + \frac{1}{2}x^\top x$, exponential capacity
- **Attention:** The softmax attention mechanism corresponds to one step of modern Hopfield dynamics

The query-key-value attention can be seen as energy-based pattern retrieval!

16.10 Sampling from EBMs: Langevin Dynamics

To generate samples, we need MCMC methods that use energy gradients.

Definition 16.29 (Langevin Dynamics). **Langevin dynamics** generates samples by simulating:

$$x_{t+1} = x_t - \frac{\epsilon}{2} \nabla_x E_\theta(x_t) + \sqrt{\epsilon} \cdot z_t, \quad z_t \sim \mathcal{N}(0, I)$$

where ϵ is the step size. As $\epsilon \rightarrow 0$ and $t \rightarrow \infty$, the distribution of x_t converges to $p_\theta(x) \propto \exp(-E_\theta(x))$.

Remark 16.30 (Intuition for Langevin Dynamics). The update has two components:

- **Gradient descent:** $-\frac{\epsilon}{2} \nabla_x E_\theta(x)$ moves toward lower energy (modes)
- **Noise injection:** $\sqrt{\epsilon} \cdot z$ prevents collapse to a single mode

The balance between these terms ensures samples from the full distribution, not just the mode.

Theorem 16.31 (Langevin as Gradient Flow + Brownian Motion). *Langevin dynamics is the discretization of the stochastic differential equation:*

$$dx = -\nabla_x E_\theta(x) dt + \sqrt{2} dW_t$$

where W_t is standard Brownian motion. The stationary distribution is exactly $p_\theta(x) \propto \exp(-E_\theta(x))$.

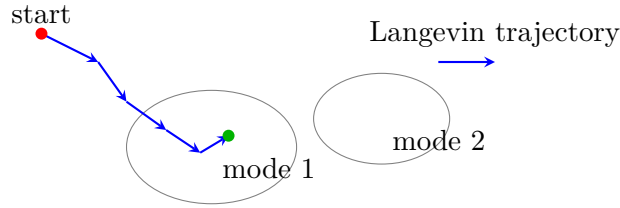


Figure 16.5: Langevin dynamics: noisy gradient descent that samples from the energy-based distribution.

Remark 16.32 (Metropolis-Adjusted Langevin (MALA)). Pure Langevin dynamics has discretization error. **MALA** adds a Metropolis-Hastings correction:

1. Propose: $x' = x - \frac{\epsilon}{2} \nabla_x E_\theta(x) + \sqrt{\epsilon} \cdot z$
2. Accept with probability: $\min\left(1, \frac{p_\theta(x')q(x|x')}{p_\theta(x)q(x'|x)}\right)$

This ensures exact sampling even with finite step size ϵ .

16.11 Summary: The EBM Perspective

Remark 16.33 (Why Energy-Based Models?).

- **Flexibility:** Any function $E_\theta(x)$ defines a valid model (no normalization required for energy)
- **Compositionality:** Energies add: $E_{\text{joint}}(x) = E_1(x) + E_2(x)$ combines constraints
- **Unification:** Many models (Hopfield, RBM, diffusion) are EBMs in disguise
- **Physical interpretation:** Connects to statistical mechanics and thermodynamics

Remark 16.34 (Key Training Methods).

Method	Avoids Z_θ ?	Key Idea
Maximum Likelihood	No	Requires MCMC for gradient
Contrastive Divergence	Partially	Short MCMC chains
Score Matching	Yes	Match score function
Noise Contrastive Est.	Yes	Classify data vs noise

Remark 16.35 (The Fundamental Trade-off). EBMs face a core tension:

- **Expressivity:** Flexible energy functions can model complex distributions
- **Tractability:** Sampling and evaluation require approximations

Modern approaches (diffusion models, flow matching) resolve this by:

1. Learning the **score** instead of the energy directly
2. Using **hierarchical** noise schedules for tractable sampling
3. Replacing MCMC with **amortized** sampling (learned generators)

16.12 EBMs vs Transformers

Energy-based models and transformers represent fundamentally different paradigms for modeling data.

Remark 16.36 (Architectural Philosophy).

- **Transformers:** Discriminative/autoregressive. Model $p(x_t|x_{<t})$ directly via next-token prediction. Fast feedforward sampling.
- **EBMs:** Generative/holistic. Model the full joint $p(x) \propto \exp(-E(x))$. Evaluate entire configurations simultaneously.

Remark 16.37 (Advantages of EBMs over Transformers).

1. **Bidirectional coherence:** EBMs score entire sequences holistically, avoiding the left-to-right bias of autoregressive models. Can naturally handle:
 - Infilling (filling in the middle)
 - Global consistency constraints
 - Non-sequential dependencies
2. **Compositionality:** Energy functions compose additively:

$$E_{\text{combined}}(x) = E_{\text{grammar}}(x) + E_{\text{style}}(x) + E_{\text{topic}}(x)$$

Multiple constraints can be combined at inference time without retraining.

3. **Flexible conditioning:** Any variable can be conditioned on any other—no fixed generation order. Useful for constraint satisfaction problems.
4. **Iterative refinement:** Sampling via MCMC naturally supports “thinking longer”—more compute at inference improves quality.
5. **Unnormalized modeling:** No need to model the full distribution—only relative preferences matter. This is natural for ranking, preference learning, and critic models.

Remark 16.38 (Disadvantages of EBMs vs Transformers).

1. **Slow sampling:** MCMC requires many iterative steps vs single feedforward pass. Transformers generate tokens in $O(n)$ forward passes; EBMs may need $O(1000+)$ MCMC steps.
2. **Training instability:** The negative phase requires sampling from the model, creating a chicken-and-egg problem. Mode collapse and divergence are common failure modes.
3. **No likelihood:** Cannot compute exact $p(x)$ due to intractable Z_θ . This makes:
 - Model comparison difficult (no perplexity)
 - Anomaly detection unreliable

- Density estimation approximate
4. **Scaling challenges:** Transformers benefit from clear scaling laws (more compute $\Rightarrow$ better). EBM scaling is less understood—MCMC doesn't parallelize as well.
 5. **Discrete data:** EBMs work best in continuous spaces where gradients flow. Discrete sequences (text) require:
 - Continuous relaxations (Gumbel-softmax)
 - Embedding space operations
 - Specialized discrete MCMC

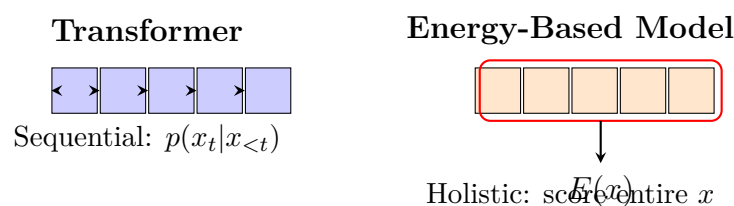


Figure 16.6: Transformers model sequences left-to-right; EBMs evaluate entire configurations holistically.

16.13 Reasoning in Energy-Based Models

A key insight: *inference-time compute* in EBMs can be viewed as a form of reasoning.

Definition 16.39 (Reasoning as Energy Minimization). In the EBM framework, **reasoning** is the iterative process of refining a configuration x to minimize energy:

$$x^{(t+1)} = x^{(t)} - \eta \nabla_x E_\theta(x^{(t)}) + \text{noise}$$

Each step can be interpreted as:

- **Gradient:** Direction toward more coherent/consistent states
- **Noise:** Exploration to escape local minima
- **Iterations:** “Thinking time”—more steps = deeper reasoning

Remark 16.40 (Test-Time Compute Scaling). Unlike transformers where inference is a single forward pass, EBMs naturally support **test-time compute scaling**:

- Easy problems: Few MCMC steps suffice
- Hard problems: Run more iterations to find lower-energy solutions
- The same model can trade off speed vs quality at inference time

This mirrors human reasoning: harder problems require more “thinking.”

Remark 16.41 (Constraint Satisfaction as Reasoning). Many reasoning tasks are constraint satisfaction problems. EBMs handle these naturally:

1. **Sudoku:** $E(x) = \sum_{\text{constraints}} \mathbb{I}[\text{violated}]$. MCMC finds valid solutions.
2. **Planning:** $E(\tau) = -R(\tau) + \lambda \cdot \text{constraint_violations}(\tau)$
3. **Logical inference:** Energy penalties for logical inconsistencies

The energy function encodes “what makes a solution good”; MCMC searches for it.

Remark 16.42 (Chain-of-Thought as Energy Landscape Navigation). An intriguing connection to chain-of-thought reasoning:

- **CoT in transformers:** Generate intermediate tokens z before answer y
- **EBM interpretation:** The chain of thought z helps navigate to low-energy regions

$$E(x, z, y) = E_{\text{problem}}(x) + E_{\text{reasoning}}(z|x) + E_{\text{answer}}(y|z)$$

- Each reasoning step reduces energy toward the final answer

This suggests CoT works by providing “stepping stones” through the energy landscape.

Remark 16.43 (Amortized vs Non-Amortized Reasoning).

	Amortized (Transformers)	Non-Amortized (EBMs)
Training	Learn to predict directly	Learn energy landscape
Inference	Single forward pass	Iterative optimization
Generalization	Interpolation	Can extrapolate via search
Novel problems	May fail silently	Can reason (slowly)

Amortized models are fast but brittle; EBMs are slow but can handle out-of-distribution inputs via search.

16.14 Notable Energy-Based Models

A survey of influential EBMs throughout deep learning history.

Remark 16.44 (Classical EBMs (1980s–2000s)).

- **Hopfield Networks** (1982): Associative memory with $\sim 0.14n$ pattern capacity. Showed neural networks can store and retrieve patterns.
- **Boltzmann Machines** (1985): Hinton & Sejnowski. General EBM with hidden units, but intractable training.
- **Restricted Boltzmann Machines** (2006): Hinton. Tractable training via CD, enabled deep belief networks. Key breakthrough for deep learning revival.

Remark 16.45 (Deep Energy-Based Models (2010s)).

- **Deep Boltzmann Machines** (Salakhutdinov & Hinton, 2009): Multiple hidden layers, better feature learning, but difficult to train.
- **NADE/MADE** (2011–2015): Autoregressive density estimators that can be viewed as tractable EBMs with specific structure.
- **Neural Autoregressive Flows**: Transform simple distributions through invertible networks—implicit EBMs with tractable sampling.

Remark 16.46 (Modern EBMs (2019–Present)).

- **IGEBM** (Du & Mordatch, 2019): “Implicit Generation and Modeling with Energy-Based Models.” Showed deep ConvNets as energy functions work for image generation via Langevin dynamics.
- **JEM** (Grathwohl et al., 2020): “Joint Energy-based Model.” Unified classifier and generative model: $p(x, y) \propto \exp(-E_\theta(x, y))$. A classifier’s logits define an energy!
- **Score-Based Generative Models** (Song & Ermon, 2019): Learn $\nabla_x \log p(x)$ directly via denoising score matching. Foundation for diffusion models.
- **Energy-Based Models for Text** (Deng et al., 2020): Residual EBMs that augment autoregressive LMs with global coherence scoring.
- **Modern Hopfield Networks** (Ramsauer et al., 2020): Exponential storage capacity, connection to transformer attention. Energy: $E(\xi) = -\text{lse}(X^\top \xi) + \frac{1}{2} \|\xi\|^2$

Remark 16.47 (State-of-the-Art Applications).

- **Diffusion Models** (DDPM, 2020; Stable Diffusion, DALL-E): Score-based EBMs dominate image generation. Technically EBMs trained via denoising score matching.
- **Classifier-Free Guidance**: Use energy interpretation to strengthen conditioning: $\tilde{E}(x|c) = E(x|c) - w \cdot E(x|\emptyset)$
- **Energy-Based Reward Models**: In RLHF, the reward model $r(x, y)$ is essentially an energy function for preference learning.
- **Contrastive Learning** (SimCLR, CLIP): InfoNCE objective is NCE applied to representation learning—an implicit EBM over embeddings.

16.15 EBMs vs Diffusion Models: A Detailed Comparison

Diffusion models are often called “score-based generative models”—they are a specific, highly successful instantiation of the EBM framework.

Definition 16.48 (Diffusion Models as Hierarchical EBMs). A diffusion model defines a sequence of distributions at noise levels $t \in [0, T]$:

$$p_t(x) = \int p_{\text{data}}(x_0) \mathcal{N}(x; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I) dx_0$$

The model learns a **score network** $s_\theta(x, t) \approx \nabla_x \log p_t(x)$.

This implicitly defines an energy at each noise level:

$$E_\theta(x, t) = - \int^x s_\theta(x', t) \cdot dx' \quad (\text{up to constant})$$

Remark 16.49 (Key Differences).

Aspect	Classical EBM	Diffusion Model
What's learned	Energy $E_\theta(x)$	Score $s_\theta(x, t) = -\nabla_x E$
Training	Contrastive divergence	Denoising score matching
Noise levels	Single distribution	Hierarchy of noise levels
Sampling	MCMC (slow, general)	Ancestral/ODE (fast, structured)
Partition function	Intractable	Avoided via score

Remark 16.50 (Why Diffusion Models Succeeded Where Classical EBMs Struggled).

1. **Stable training:** Denoising score matching has a simple regression loss—no adversarial dynamics or MCMC in the training loop.

$$\mathcal{L}_{\text{DSM}} = \mathbb{E}_{x_0, \epsilon, t} [\|s_\theta(x_t, t) - \nabla_{x_t} \log p(x_t|x_0)\|^2]$$

The target $\nabla_{x_t} \log p(x_t|x_0) = -\epsilon/\sigma_t$ is known in closed form!

2. **Coarse-to-fine generation:** High noise levels capture global structure; low noise levels add details. This curriculum makes learning easier.
3. **Structured sampling:** Instead of random-walk MCMC, diffusion uses a principled denoising process:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} (x_t + (1 - \alpha_t)s_\theta(x_t, t)) + \sigma_t z$$

This is deterministic up to the noise injection—much more directed than Langevin dynamics.

4. **Theoretical guarantees:** The forward diffusion has a known stationary distribution (Gaussian). Reversal is well-characterized by Anderson's theorem.

Remark 16.51 (When to Use Each).

- **Use Diffusion Models when:**
 - High-quality sample generation is the goal
 - Data is continuous (images, audio, video)

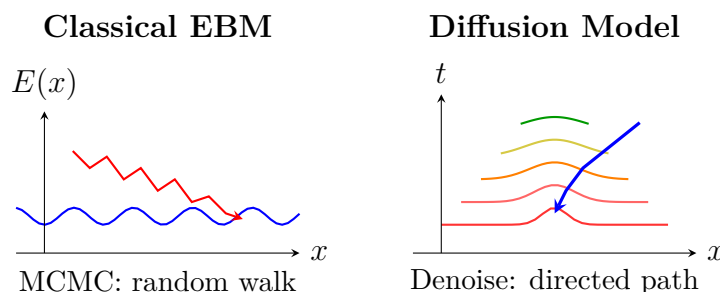


Figure 16.7: Classical EBMs use random-walk MCMC; diffusion models follow a structured denoising trajectory through noise levels.

- Training stability is important
- You want well-understood scaling behavior
- **Use Classical EBMs when:**
 - You need to compose multiple energy functions at test time
 - The task involves constraint satisfaction or optimization
 - You want a unified model for classification and generation (JEM)
 - Working with discrete/structured data where diffusion is awkward
 - You need explicit energy values for ranking or decision-making

Remark 16.52 (Hybrid Approaches). Modern systems often combine both paradigms:

- **Classifier guidance:** Use a classifier’s energy to guide diffusion sampling
- **Composable diffusion:** Add energy terms from multiple diffusion models
- **Energy-based refinement:** Use EBM to refine outputs from fast generators
- **Diffusion for EBM training:** Use diffusion to generate negative samples for EBM training

Part VII

Reinforcement Learning

Chapter 17

Classical Reinforcement Learning

17.1 Markov Decision Processes

The foundation of reinforcement learning is the *Markov Decision Process* (MDP), which formalizes sequential decision-making.

Definition 17.1 (Markov Decision Process). An MDP is a tuple $(\mathcal{S}, \mathcal{A}, p_0, P, R, \gamma)$ where:

- $\mathcal{S}$ is the state space,
- $\mathcal{A}$ is the action space,
- $p_0(s)$ is the initial state distribution,
- $P(s'|s, a)$ is the transition probability (environment dynamics),
- $R(s, a)$ is the reward function,
- $\gamma \in [0, 1)$ is the discount factor.

Remark 17.2 (The Markov Property). The “Markov” in MDP refers to the *Markov property*: the future depends only on the current state, not on how we got there:

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = P(s_{t+1}|s_t, a_t).$$

This means the state s_t captures all relevant history. Formally, the state is a *sufficient statistic* for predicting future rewards and states.

Why this matters:

- Algorithms can make decisions based solely on current state—no need to track full history.
- Value functions $V(s)$ and $Q(s, a)$ are well-defined as functions of state alone.
- The Bellman equations hold, enabling dynamic programming.

When does it fail? In practice, the true state may be *partially observable*. For example:

- A robot with limited sensors cannot observe its full position.
- In games, the opponent’s strategy depends on history you didn’t observe.
- In LLMs, the “state” (input prompt) often doesn’t capture conversation context.

When Markov property fails, we need POMDPs (Partially Observable MDPs) or recurrent architectures that maintain memory.

Definition 17.3 (Policy). A policy $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ maps states to distributions over actions. The agent’s goal is to find a policy maximizing expected cumulative reward.

Remark 17.4 (Reward Notation). $R(s, a)$ is the reward *function*—it takes a state-action pair and returns a number. When we write R_t , we mean the reward received at time t , i.e., $R_t = R(s_t, a_t)$.

In classical RL, R is given by the environment (e.g., +1 for winning a game). In RLHF (chapter 18), we don’t have a true reward—instead we *learn* a reward model $r_\phi(x, y)$ from human preferences. The lowercase r with subscript ϕ emphasizes that it’s a learned approximation, not ground truth.

Remark 17.5 (Notation). We write $\pi(a|s)$ for the probability of action a given state s , and $\pi(\cdot|s)$ to denote the entire distribution over actions (the dot is a placeholder for the random variable). Thus $a \sim \pi(\cdot|s)$ means “sample action a from the distribution $\pi(\cdot|s)$.” Similarly, $\tau \sim \pi_\theta$ means “sample a trajectory $\tau = (s_0, a_0, s_1, a_1, \dots)$ by rolling out the policy π_θ .”

17.2 Value Functions

Value functions quantify the expected return from states or state-action pairs.

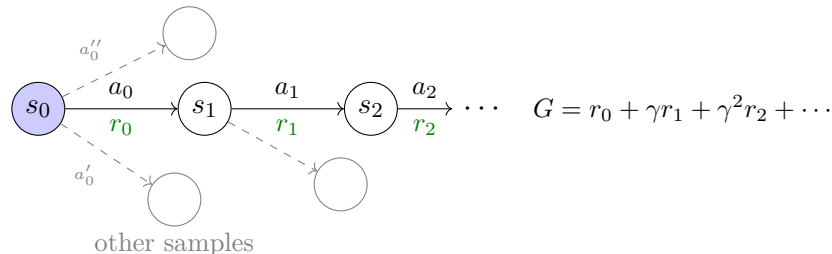
Definition 17.6 (State Value Function). The value function under policy π is

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right].$$

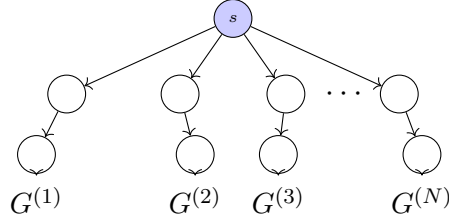
The conditioning $s_0 = s$ means we average over all trajectories that start from state s . A trajectory $\tau = (s_0, a_0, s_1, a_1, s_2, a_2, \dots)$ is generated by:

$$a_t \sim \pi(\cdot|s_t), \quad s_{t+1} \sim P(\cdot|s_t, a_t), \quad r_t = R(s_t, a_t).$$

Branching occurs due to stochasticity in both the policy and environment:



Sampling N trajectories from s and averaging their returns gives:



$$V^\pi(s) \approx \frac{1}{N} (G^{(1)} + G^{(2)} + \dots + G^{(N)})$$

In practice, this expectation is estimated via Monte Carlo sampling:

$$V^\pi(s) \approx \frac{1}{N} \sum_{i=1}^N G^{(i)}, \quad \text{where } G^{(i)} = \sum_{t=0}^T \gamma^t R(s_t^{(i)}, a_t^{(i)})$$

and each trajectory $\tau^{(i)} = (s, a_0^{(i)}, s_1^{(i)}, \dots)$ is sampled by starting from s and following π .

Definition 17.7 (Action-Value Function). The Q-function under policy π is

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, a_0 = a \right].$$

Here the conditioning $s_0 = s, a_0 = a$ means we fix the initial state-action pair and average over all continuations under π .

The relationship between them:

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot|s)} [Q^\pi(s, a)] = \sum_a \pi(a|s) Q^\pi(s, a).$$

Definition 17.8 (Advantage Function). The advantage function measures how much better an action is compared to the average:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

17.3 Bellman Equations

The Bellman equations transform an intractable problem (optimize over infinite trajectories) into a tractable one (solve a system of equations). They are the foundation of nearly all RL algorithms.

17.3.1 The Equations

Proposition 17.9 (Bellman Expectation Equation). *The value function satisfies the recursive relation:*

$$V^\pi(s) = \sum_a \pi(a|s) \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \right].$$

Proof. Start from the definition and split the first step from the rest:

$$\begin{aligned}
V^\pi(s) &= \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid s_0 = s \right] \\
&= \mathbb{E}_\pi \left[R_0 + \sum_{t=1}^{\infty} \gamma^t R_t \mid s_0 = s \right] \\
&= \mathbb{E}_\pi \left[R_0 + \gamma \sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid s_0 = s \right] \\
&= \mathbb{E}_{a \sim \pi(\cdot|s)} \left[R(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} \left[\mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid s_0 = s' \right] \right] \right]
\end{aligned}$$

The inner expectation is exactly $V^\pi(s')$ by the Markov property. Writing out the expectations as sums:

$$V^\pi(s) = \sum_a \pi(a|s) \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \right]. \quad \square$$

Remark 17.10 (Intuition). The Bellman equation says: the value of state s equals the immediate reward plus the discounted value of where we end up. This recursive structure is what enables dynamic programming algorithms (value iteration, policy iteration).

Proposition 17.11 (Bellman Optimality Equation). *The optimal value function $V^* = \max_\pi V^\pi$ satisfies:*

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right].$$

Similarly for the optimal Q -function:

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a').$$

Proof. The optimal policy at state s must choose the action that maximizes expected return. Suppose we act optimally from s onwards:

$$\begin{aligned}
V^*(s) &= \max_\pi V^\pi(s) \\
&= \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right].
\end{aligned}$$

The max over policies reduces to max over the first action because, by the principle of optimality, whatever action we take now, we should act optimally thereafter—and $V^*(s')$ already captures optimal future behavior.

For Q^* : if we take action a in state s and then act optimally:

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a'). \quad \square$$

17.3.2 Why Bellman Equations Matter

Remark 17.12 (From Definition to Algorithm). Without Bellman equations, computing $V^\pi(s)$ requires:

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid s_0 = s \right] \quad \text{— average over infinitely many infinite trajectories.}$$

The Bellman equation converts this into a *system of $|\mathcal{S}|$ linear equations*:

$$V^\pi(s) = \sum_a \pi(a|s) \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \right] \quad \text{for each } s \in \mathcal{S}.$$

This can be solved exactly (matrix inversion) or iteratively.

Definition 17.13 (Value Iteration). To find the optimal value function V^* , repeatedly apply the Bellman optimality operator:

$$V_{k+1}(s) \leftarrow \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V_k(s') \right] \quad \text{for all } s.$$

Start with arbitrary V_0 . As $k \rightarrow \infty$, $V_k \rightarrow V^*$.

Remark 17.14 (Why It Converges). The Bellman operator is a *contraction* with factor $\gamma < 1$. By the Banach fixed-point theorem, repeated application converges to the unique fixed point V^* .

Definition 17.15 (Q-Learning Update). When the environment dynamics $P(s'|s, a)$ are unknown, we learn Q^* from experience. After observing transition (s, a, r, s') :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{Bellman target}} - Q(s, a) \right].$$

This is a stochastic approximation of value iteration, using samples instead of the full expectation.

Remark 17.16 (The Bellman Equation in Disguise). Q-learning works because it's trying to satisfy:

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \right].$$

Each update nudges $Q(s, a)$ toward the right-hand side. At convergence, the Bellman equation holds.

Remark 17.17 (Connection to Policy Gradient). Even policy gradient methods use Bellman ideas:

- The **advantage** $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ requires estimating V^π or Q^π .

- **Actor-critic** methods learn V^π (the critic) using Bellman updates, then use it to compute advantages for policy gradient (the actor).
- **GAE** (Generalized Advantage Estimation) uses the Bellman structure: $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ is the *TD error*, measuring how wrong the current V is.

The Bellman equation is not an alternative to policy gradient—it’s a tool used *within* policy gradient methods.

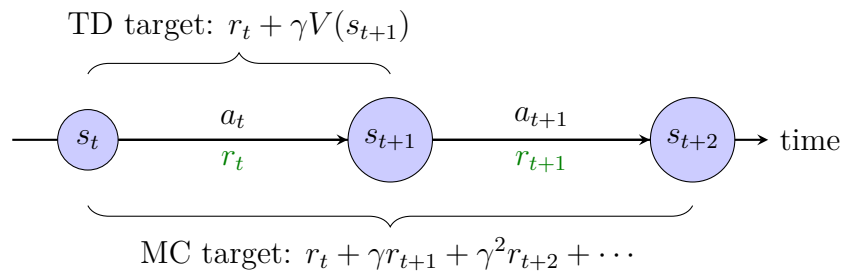
17.4 Temporal Difference Learning

TD learning bridges Monte Carlo methods (which wait for complete episodes) and dynamic programming (which requires a model).

Definition 17.18 (TD(0) Update). After observing transition (s_t, a_t, r_t, s_{t+1}) , update the value function:

$$V(s_t) \leftarrow V(s_t) + \alpha \underbrace{\left[r_t + \gamma V(s_{t+1}) - V(s_t) \right]}_{\delta_t \text{ (TD error)}}.$$

The TD error δ_t measures how “surprised” we are: if $r_t + \gamma V(s_{t+1}) > V(s_t)$, the outcome was better than expected.



Remark 17.19 (TD vs Monte Carlo).

	Monte Carlo	TD(0)
Updates	End of episode	Every step
Target	$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$	$r_t + \gamma V(s_{t+1})$
Bias	Unbiased	Biased (uses estimate V)
Variance	High	Lower
Requires	Complete episodes	Only (s, a, r, s') tuples

TD methods *bootstrap*—they update estimates based on other estimates. This introduces bias but dramatically reduces variance.

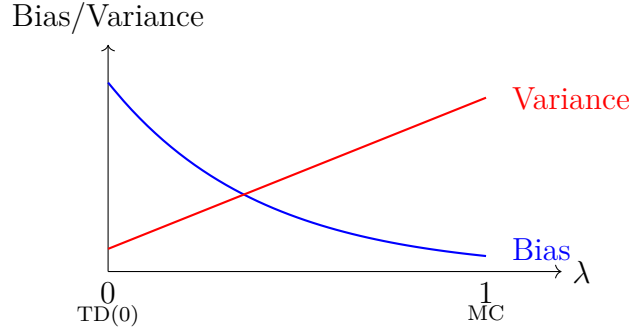
Definition 17.20 (TD(λ) and Eligibility Traces). TD(λ) interpolates between TD(0) ($\lambda = 0$) and Monte Carlo ($\lambda = 1$):

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

where $G_t^{(n)} = r_t + \gamma r_{t+1} + \dots + \gamma^{n-1} r_{t+n-1} + \gamma^n V(s_{t+n})$ is the n -step return.

Equivalently, using *eligibility traces*:

$$e_t(s) = \gamma \lambda e_{t-1}(s) + \mathbf{1}[s_t = s], \quad V(s) \leftarrow V(s) + \alpha \delta_t e_t(s).$$



17.5 Policy Gradient Methods

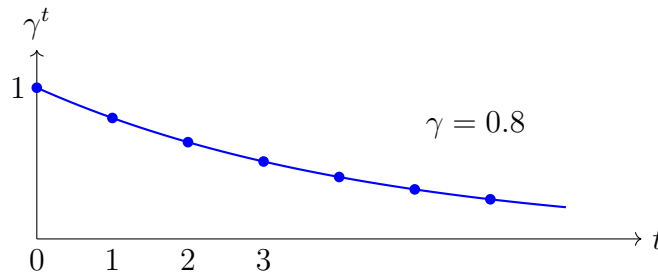
When the policy is parameterized by θ , we directly optimize the expected return.

17.5.1 Why This Objective?

The goal is to find a policy that maximizes total reward over time. But which total?

- **Finite horizon:** $\sum_{t=0}^T R_t$ — sum rewards for T steps. Simple, but what is T ?
- **Infinite horizon (undiscounted):** $\sum_{t=0}^{\infty} R_t$ — can diverge to ∞ .
- **Infinite horizon (discounted):** $\sum_{t=0}^{\infty} \gamma^t R_t$ — always finite if rewards are bounded.

The discounted sum $\sum_{t=0}^{\infty} \gamma^t R_t$ with $\gamma \in [0, 1)$ is the standard choice:



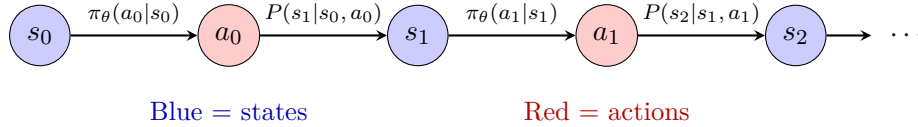
Remark 17.21 (Why Discount?). The discount factor $\gamma < 1$ serves multiple purposes:

1. **Convergence:** Ensures $\sum_{t=0}^{\infty} \gamma^t R_t < \infty$ even for infinite horizons.
2. **Preference for sooner rewards:** Reward now is worth more than reward later (like money).
3. **Uncertainty:** The further into the future, the less certain our predictions.

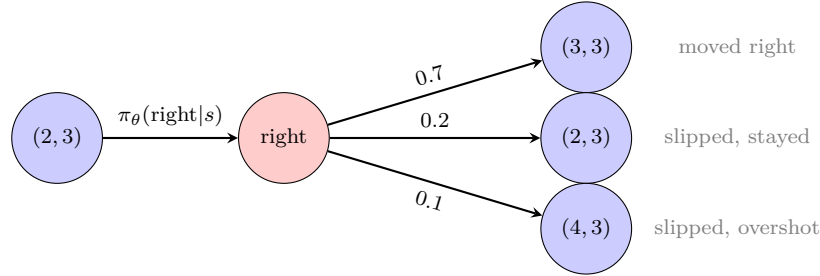
4. **Effective horizon:** Most of the weight is on the first $\approx \frac{1}{1-\gamma}$ steps. For $\gamma = 0.99$, this is ≈ 100 steps; for $\gamma = 0.9$, only ≈ 10 steps.

Remark 17.22 (Trajectory Structure). A trajectory $\tau = (s_0, a_0, s_1, a_1, \dots)$ is generated by alternating between policy and environment:

1. **Agent chooses action:** Given s_t , policy outputs a_t with probability $\pi_\theta(a_t|s_t)$.
2. **Environment transitions:** Given (s_t, a_t) , next state is s_{t+1} with probability $P(s_{t+1}|s_t, a_t)$.



Example 17.23 (Classic RL — Robot on Slippery Floor). State s = robot position. Action a = movement command. The robot is at $(2, 3)$ and commands “right”:

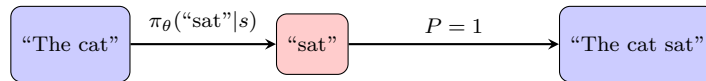


The transition probabilities $P(s'|s, a)$ form a distribution over next states:

$$P((3, 3) | (2, 3), \text{right}) = 0.7, \quad P((2, 3) | (2, 3), \text{right}) = 0.2, \quad P((4, 3) | (2, 3), \text{right}) = 0.1.$$

The agent chose “right”, but *physics decides where it actually lands*. This environment randomness is outside the agent’s control.

Example 17.24 (LLM Text Generation). State s = tokens so far. Action a = next token. The LLM has generated “The cat” and samples “sat”:



There is only *one possible next state*: the concatenation. The transition is deterministic:

$$P(\text{“The cat sat”} | \text{“The cat”, “sat”}) = 1.$$

All other next states have probability 0. The “environment” is just string concatenation—it contributes *zero randomness*.

Remark 17.25 (Where Does Randomness Live?). This distinction is fundamental:

	Policy $\pi_\theta(a s)$	Environment $P(s' s, a)$
Robot	stochastic	stochastic
LLM	stochastic	deterministic ($P = 1$)

In LLMs, *all trajectory randomness comes from the policy*. This is why LLM fine-tuning (RLHF, DPO, etc.) focuses entirely on modifying π_θ —there is no environment to model or fight against.

Definition 17.26 (RL Objective). The objective is the expected discounted return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]$$

where $\tau = (s_0, a_0, s_1, a_1, \dots)$ is a trajectory sampled by following policy π_θ .

Theorem 17.27 (Policy Gradient Theorem). *The gradient of the objective is*

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t \right]$$

where $G_t = \sum_{k=t}^{\infty} \gamma^{k-t} R(s_k, a_k)$ is the return from time t .

Proof. Starting from $J(\theta) = \mathbb{E}_\tau[R(\tau)]$:

$$\begin{aligned} \nabla_\theta J(\theta) &= \nabla_\theta \int p_\theta(\tau) R(\tau) d\tau \\ &= \int R(\tau) \nabla_\theta p_\theta(\tau) d\tau \\ &= \int R(\tau) p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) d\tau \\ &= \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau) \nabla_\theta \log p_\theta(\tau)]. \end{aligned}$$

For the trajectory probability, by chain rule:

$$p_\theta(\tau) = p_0(s_0) \prod_{t=0}^{\infty} \pi_\theta(a_t|s_t) \cdot P(s_{t+1}|s_t, a_t).$$

Taking logs:

$$\log p_\theta(\tau) = \underbrace{\log p_0(s_0)}_{\text{no } \theta} + \sum_t \underbrace{\log \pi_\theta(a_t|s_t)}_{\text{depends on } \theta} + \sum_t \underbrace{\log P(s_{t+1}|s_t, a_t)}_{\text{no } \theta}.$$

Only π_θ terms depend on θ , so $\nabla_\theta \log p_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)$.

Substituting with $R(\tau) = \sum_{t=0}^{\infty} \gamma^t R_t$:

$$\nabla_\theta J = \mathbb{E} \left[\left(\sum_{t'} \gamma^{t'} R_{t'} \right) \left(\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \right) \right].$$

The Causality Argument: Consider the cross terms $R_{t'} \cdot \nabla_\theta \log \pi_\theta(a_t|s_t)$ for $t' < t$:

- $R_{t'}$ depends only on $(s_{t'}, a_{t'})$, which are determined *before* action a_t is chosen.
- $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ depends on action a_t .
- Since $a_t \sim \pi_{\theta}(\cdot | s_t)$ is sampled *independently* of the past reward $R_{t'}$ (given s_t):

$$\mathbb{E}[R_{t'} \cdot \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] = \mathbb{E}[R_{t'}] \cdot \underbrace{\mathbb{E}_{a_t}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]}_{=0}.$$

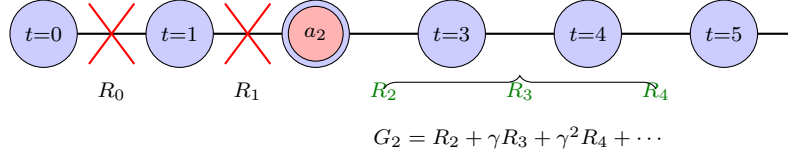
The second factor is zero because $\mathbb{E}_{a \sim \pi}[\nabla \log \pi(a)] = \sum_a \pi(a) \frac{\nabla \pi(a)}{\pi(a)} = \nabla \sum_a \pi(a) = \nabla 1 = 0$.

Thus, only $t' \geq t$ terms contribute. Rearranging:

$$\nabla_{\theta} J = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot \underbrace{\sum_{t' \geq t} \gamma^{t'} R_{t'}}_{=\gamma^t G_t} \right] = \mathbb{E} \left[\sum_t \gamma^t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t \right].$$

(The γ^t can be absorbed into how we weight time steps, giving the stated form.) $\square$

Remark 17.28 (Intuition for Causality). The action at time t cannot influence rewards that already happened ($t' < t$). Including past rewards in the gradient estimate only adds noise—they're independent of the action we're evaluating. By using G_t (future returns only), we reduce variance without introducing bias.



Definition 17.29 (Policy Gradient Methods). A *policy gradient* algorithm directly maximizes $J(\theta)$ by stochastic gradient ascent on the parameters of a differentiable policy π_{θ} . The vanilla update is

$$\theta_{k+1} = \theta_k + \alpha \mathbb{E}_t[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t],$$

where $\hat{A}_t$ is an estimator of the advantage $A^{\pi}(s_t, a_t)$ (often the Monte Carlo return minus a baseline). All later variants (e.g., TRPO, PPO) modify this basic gradient ascent step to control variance and stabilize updates.

Remark 17.30 (Variance Reduction via Baselines). The policy gradient theorem gives us:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t \right]$$

where $G_t = \sum_{k=t}^{\infty} \gamma^{k-t} R_k$ is the return from time t . We can subtract any *baseline* $b(s_t)$ that depends only on the state:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot (G_t - b(s_t)) \right].$$

This doesn't change the expectation because:

$$\mathbb{E}_{a_t \sim \pi_\theta(\cdot|s_t)} \left[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot b(s_t) \right] = b(s_t) \cdot \underbrace{\sum_a \nabla_\theta \pi_\theta(a|s_t)}_{=\nabla_\theta 1=0} = 0.$$

The optimal baseline (minimizing variance) is $b(s_t) = V^\pi(s_t)$. With this choice, $G_t - V^\pi(s_t)$ is an estimate of the advantage $A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$.

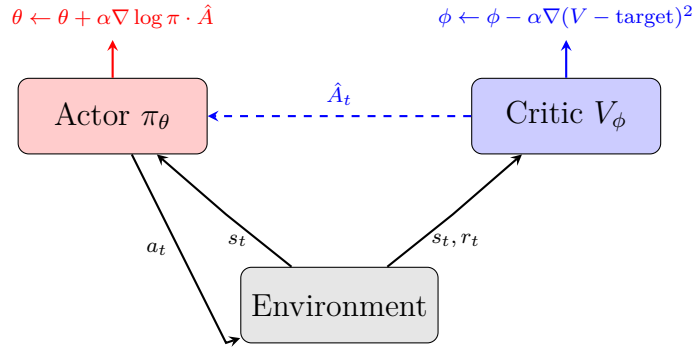
17.5.2 Actor-Critic Methods

The Actor-Critic architecture combines policy gradient (actor) with value function learning (critic).

Definition 17.31 (Actor-Critic). An *actor-critic* algorithm maintains two components:

- **Actor** $\pi_\theta(a|s)$: the policy, updated via policy gradient.
- **Critic** $V_\phi(s)$ or $Q_\phi(s, a)$: estimates value, updated via TD learning.

The critic provides low-variance advantage estimates for the actor's gradient.



Remark 17.32 (Why Actor-Critic?).

	REINFORCE	Actor-Critic	Pure Critic (DQN)
Variance	High	Low	N/A
Bias	None	Some (from V)	None
Continuous actions	Yes	Yes	No
Sample efficiency	Low	Medium	High

17.5.3 Generalized Advantage Estimation (GAE)

GAE provides a practical way to estimate advantages with controllable bias-variance tradeoff.

Definition 17.33 (Generalized Advantage Estimation). Given TD errors $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$, the GAE estimator is:

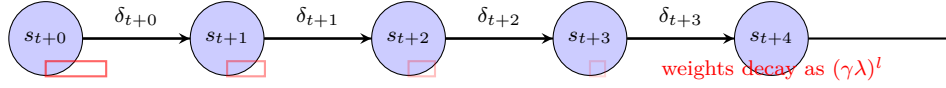
$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l} = \delta_t + \gamma \lambda \delta_{t+1} + (\gamma \lambda)^2 \delta_{t+2} + \dots$$

where $\lambda \in [0, 1]$ controls bias-variance tradeoff.

Remark 17.34 (GAE Special Cases). • $\lambda = 0$: $\hat{A}_t = \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ (1-step TD, high bias, low variance)

- $\lambda = 1$: $\hat{A}_t = \sum_{l=0}^{\infty} \gamma^l \delta_{t+l} = G_t - V(s_t)$ (Monte Carlo, no bias, high variance)

In practice, $\lambda \approx 0.95$ works well—enough bootstrapping to reduce variance, but not so much that bias dominates.



$$\hat{A}_t^{\text{GAE}} = \delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots$$

Proposition 17.35 (GAE as Exponentially-Weighted Average). *GAE can be written as:*

$$\hat{A}_t^{\text{GAE}} = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_t^{(n)}$$

where $\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l r_{t+l} + \gamma^n V(s_{t+n}) - V(s_t)$ is the n -step advantage estimate. This shows GAE averages over all n -step estimates, with exponentially decaying weights.

17.6 Proximal Policy Optimization (PPO)

Definition 17.36 (Proximal Policy Optimization). Proximal Policy Optimization (PPO) is a first-order policy gradient algorithm that still aims to maximize the true return $J(\theta)$ but *actually* optimizes a clipped surrogate objective $\mathcal{L}^{\text{PPO}}(\theta)$ (defined below) using samples from $\pi_{\theta_{\text{old}}}$. It constrains the change between the new policy π_{θ} and the data-collecting policy $\pi_{\theta_{\text{old}}}$ (via probability-ratio clipping or an explicit KL penalty), allowing relatively large learning rates while keeping updates stable. In practice, PPO alternates between (i) collecting trajectories with $\pi_{\theta_{\text{old}}}$ and (ii) several epochs of gradient ascent on $\mathcal{L}^{\text{PPO}}$ using minibatches.

17.6.1 The Problem with Vanilla Policy Gradient

Recall the policy gradient objective $J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)]$. The gradient tells us how to improve:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_t [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot A_t]$$

where A_t is the advantage (how much better action a_t was than average).

Problem: Gradient descent takes a step $\theta \rightarrow \theta + \alpha \nabla_{\theta} J$. But how big should α be? Too small = slow. Too large = policy changes drastically, samples become invalid, training collapses.

Answer: control the *size of the policy change*, not just the learning rate. Trust-region style methods bound the KL divergence between π_{θ} and $\pi_{\theta_{\text{old}}}$ so that updates stay within

a region where old samples are still informative. PPO implements this idea with a clipped surrogate (and optionally a KL penalty), which effectively caps the probability ratio $r_t(\theta)$ so a large α cannot cause destructive jumps. In practice one still uses an optimizer like Adam, but clipping/penalty provides the main safeguard.

17.6.2 The Importance Sampling Trick

We want to reuse data collected under an old policy $\pi_{\theta_{\text{old}}}$ to improve a new policy π_θ . Using importance sampling:

$$\mathbb{E}_{a \sim \pi_\theta} [f(a)] = \mathbb{E}_{a \sim \pi_{\theta_{\text{old}}}} \left[\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} f(a) \right].$$

Definition 17.37 (Probability Ratio). The probability ratio measures how much more (or less) likely action a_t is under the new policy compared to the old:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}.$$

- $r_t = 1$: new and old policies agree on this action
- $r_t > 1$: new policy makes this action *more* likely
- $r_t < 1$: new policy makes this action *less* likely

Using this ratio, we can write a *policy-gradient surrogate* that we optimize using data from $\pi_{\theta_{\text{old}}}$:

$$\mathcal{L}^{\text{PG}}(\theta) = \mathbb{E}_t [r_t(\theta) \cdot A_t].$$

This equals $J(\theta)$ to first order around θ_{old} , but can be computed using old samples.

17.6.3 Why We Need Clipping

The problem: if $r_t(\theta)$ becomes very large, the surrogate $\mathcal{L}^{\text{PG}}(\theta) = \mathbb{E}[r_t A_t]$ gives huge gradients, causing destructive updates. We want to constrain how far the new policy can drift from the old one.

Definition 17.38 (PPO-Clip Objective).

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}_t [\min(r_t(\theta) \cdot A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot A_t)]$$

where $\epsilon \approx 0.2$ and $\text{clip}(r, 1 - \epsilon, 1 + \epsilon)$ clamps r to $[1 - \epsilon, 1 + \epsilon]$.

Remark 17.39 (How Clipping Works). The clipping behaves differently depending on whether the action was good ($A_t > 0$) or bad ($A_t < 0$):

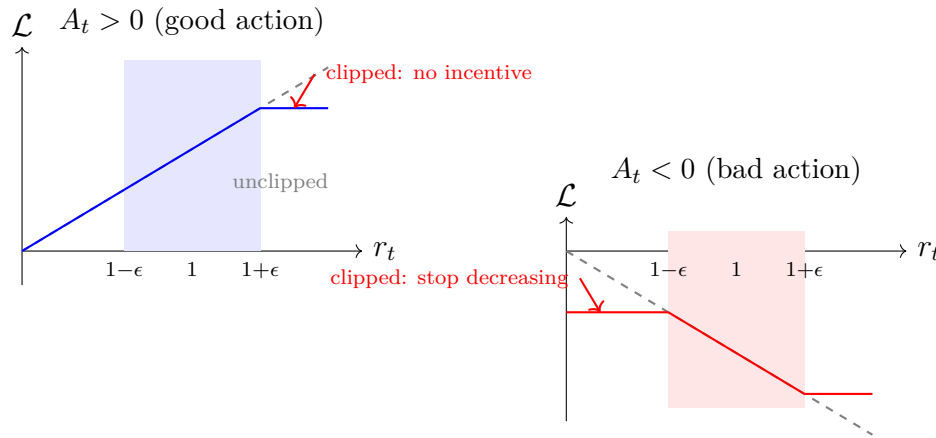
Case 1: Good action ($A_t > 0$)

- If $r_t < 1 - \epsilon$: action became less likely. Gradient pushes to increase r_t .

- If $r_t \in [1 - \epsilon, 1 + \epsilon]$: normal gradient, objective is $r_t \cdot A_t$.
- If $r_t > 1 + \epsilon$: action became much more likely. The min kicks in, objective becomes $(1 + \epsilon)A_t$ —flat, no gradient to increase r_t further.

Case 2: Bad action ($A_t < 0$)

- If $r_t > 1 + \epsilon$: action became more likely (bad!). Gradient pushes to decrease r_t .
- If $r_t \in [1 - \epsilon, 1 + \epsilon]$: normal gradient, objective is $r_t \cdot A_t$ (negative).
- If $r_t < 1 - \epsilon$: action became much less likely. The min kicks in (clipped value is less negative), objective becomes $(1 - \epsilon)A_t$ —flat, no further decrease.



Remark 17.40 (The Pessimistic Bound). The min in PPO is *pessimistic*—it takes the lower of the clipped and unclipped objectives. This ensures:

- We never get “credit” for making a good action too likely ($r_t > 1 + \epsilon$).
- We never get “credit” for making a bad action too unlikely ($r_t < 1 - \epsilon$).

The effect: the policy can improve within the trust region, but cannot exploit the surrogate objective by moving too far.

Remark 17.41 (Notation: J vs $\mathcal{L}$). $J(\theta)$ is the true objective we want to maximize. $\mathcal{L}^{\text{PG}}(\theta)$ and $\mathcal{L}^{\text{PPO}}(\theta)$ are *surrogate* objectives that:

1. Can be computed from samples collected under $\pi_{\theta_{\text{old}}}$
2. Approximates $J(\theta)$ near θ_{old}
3. Has safety mechanisms (clipping) to prevent bad updates

Workflow: collect samples with current policy, optimize $\mathcal{L}^{\text{PPO}}$ (the clipped surrogate), update policy, repeat. Keep J for the true return, and use $\mathcal{L}$ symbols only for surrogate training objectives to avoid notational whiplash.

Remark 17.42 (PPO in RLHF). PPO is the algorithm typically used for the “RL Fine-Tuning” stage of RLHF (chapter 18). The connection:

- **Policy:** π_θ is the language model being fine-tuned.
- **Reward:** $r_\phi(x, y)$ is the learned reward model (plus KL penalty).
- **Advantage:** Estimated from generated responses using GAE or simpler baselines.
- **Clipping:** Prevents the LLM from changing too drastically in one update.

The PPO objective becomes:

$$\mathcal{L}(\theta) = \mathbb{E}_{x, y \sim \pi_{\theta_{\text{old}}}} \left[\min \left(\frac{\pi_\theta(y|x)}{\pi_{\theta_{\text{old}}}(y|x)} \hat{A}, \text{clip}(\cdot) \cdot \hat{A} \right) \right]$$

where $\hat{A}$ incorporates the reward $r_\phi(x, y)$ and KL penalty $-\beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$.

Note: PPO’s clipping and RLHF’s KL penalty both serve to keep the policy from drifting too far—they’re complementary regularization mechanisms.

Chapter 18

Reinforcement Learning for Language Models

This chapter applies RL to language models, where the goal is to align model outputs with human preferences. The MDP structure from chapter 17 still applies, but with important differences:

Remark 18.1 (LLMs as MDPs). For a language model generating response $y = (y_1, y_2, \dots, y_T)$ to prompt x :

- **State:** $s_t = (x, y_1, \dots, y_{t-1})$ — the prompt plus tokens generated so far.
- **Action:** $a_t = y_t$ — the next token to generate.
- **Policy:** $\pi_\theta(a_t|s_t) = \pi_\theta(y_t|x, y_{<t})$ — the LLM’s next-token distribution.
- **Transition:** $P(s_{t+1}|s_t, a_t) = 1$ — *deterministic!* The next state is just $(x, y_1, \dots, y_t)$.
- **Reward:** Often given only at the end: $R(s_T, a_T) = r_\phi(x, y)$, with $R(s_t, a_t) = 0$ for $t < T$.

Remark 18.2 (Key Differences from Classical RL). **1. Deterministic environment.** In robotics, the same action leads to different outcomes due to physics. In LLMs, appending token y_t to state s_t always gives the same next state. All randomness comes from the policy.

2. Sparse, delayed reward. Classical RL often has dense rewards (e.g., distance to goal at each step). LLM rewards are typically given only after the full response—the reward model $r_\phi(x, y)$ evaluates complete outputs.

3. No simulator. We cannot reset to arbitrary states or query the reward function cheaply. Each sample requires a forward pass through the LLM.

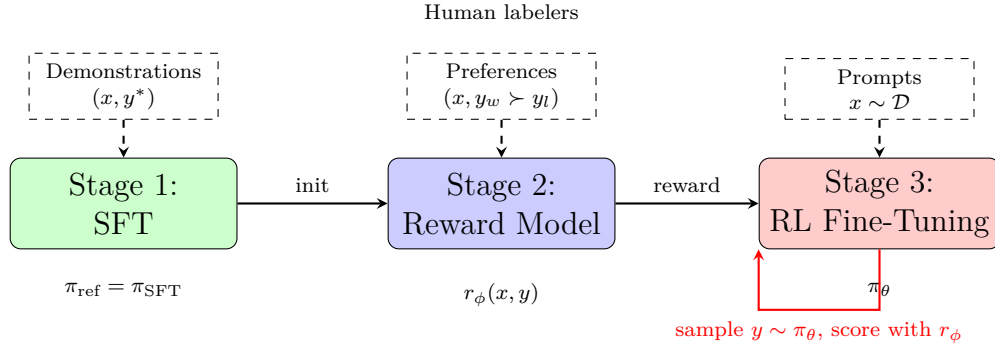
4. The reference policy matters. Unlike classical RL where we start from random initialization, LLMs begin with a pre-trained model that already has useful behavior. We want to *improve* upon it, not destroy it. This motivates the KL constraint in RLHF.

5. Preference data, not reward labels. Humans find it easier to compare “Is response A better than B?” than to assign absolute scores. This leads to preference-based methods (Bradley-Terry, DPO).

18.1 RLHF: Reinforcement Learning from Human Feedback

Fine-tuning language models with RL proceeds in three stages:

1. **Supervised Fine-Tuning (SFT)**: Train on high-quality demonstrations.
2. **Reward Modeling**: Train a reward model $r_\phi(x, y)$ on human preference data.
3. **RL Fine-Tuning**: Optimize the policy against the reward model.



Remark 18.3 (Data Requirements). Each stage requires different data:

Stage	Data Type	Typical Size
SFT	High-quality (x, y) pairs	10K–100K
Reward Model	Preference comparisons $(y_w \succ y_l)$	10K–500K
RL	Prompts only (responses sampled)	10K–100K

18.1.1 The RLHF Objective

Definition 18.4 (RLHF Objective). Given a prompt x , the objective is to maximize reward while staying close to the reference policy π_{ref} (typically the SFT model):

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x)} \left[r_\phi(x, y) - \beta D_{\text{KL}}(\pi_\theta(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x)) \right].$$

Example 18.5 (RLHF in Action). Prompt: $x = \text{“Explain why the sky is blue.”}$

The policy $\pi_\theta(\cdot|x)$ is a distribution over all possible responses. For illustration, suppose there are only 3 possible outputs:

Response y	$\pi_\theta(y x)$	$\pi_{\text{ref}}(y x)$	$r_\phi(x, y)$
y_1 : “Rayleigh scattering...”	0.50	0.40	0.9
y_2 : “The sky appears blue...”	0.45	0.35	0.85
y_3 : “Blue blue blue...”	0.05	0.25	0.1

Notation: $\pi_\theta(y_1|x) = 0.50$ is a number. $\pi_\theta(\cdot|x) = (0.50, 0.45, 0.05)$ is the entire distribution.

Computing the KL divergence:

$$D_{\text{KL}}(\pi_\theta(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x)) = \sum_y \pi_\theta(y|x) \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}.$$

With our numbers:

$$\begin{aligned} D_{\text{KL}} &= 0.50 \log \frac{0.50}{0.40} + 0.45 \log \frac{0.45}{0.35} + 0.05 \log \frac{0.05}{0.25} \\ &= 0.50 \cdot 0.223 + 0.45 \cdot 0.251 + 0.05 \cdot (-1.609) \\ &= 0.112 + 0.113 - 0.080 = 0.145. \end{aligned}$$

The KL is positive because π_θ shifted probability *toward* good responses (y_1, y_2) and *away* from the bad one (y_3).

The full objective for this prompt:

$$\underbrace{0.50 \cdot 0.9 + 0.45 \cdot 0.85 + 0.05 \cdot 0.1}_{\mathbb{E}_{y \sim \pi_\theta}[r_\phi(x, y)] = 0.838} - \beta \cdot \underbrace{0.145}_{D_{\text{KL}}}.$$

If $\beta = 0.1$, the objective is $0.838 - 0.0145 = 0.824$. The reward term wants to push even more probability to y_1 , but the KL term resists moving too far from π_{ref} .

Remark 18.6 (How RLHF Works in Practice). The example above is a simplification. In reality:

1. Where do responses come from? We *sample* from the policy: $y \sim \pi_\theta(\cdot|x)$. This means running the LLM autoregressively, sampling one token at a time. We do *not* use beam search—we need stochastic samples because RL requires exploring different outputs. Typically we sample k responses per prompt (e.g., $k = 4$).

2. Who assigns the rewards? The reward model $r_\phi(x, y)$ —a neural network trained on human preference data. During reward model training, humans compared pairs of responses and labeled which was better. The reward model learned to predict these preferences.

3. How do we compute D_{KL} without enumerating all responses? We cannot sum over all possible y —the space is astronomically large. Instead, rewrite the KL as an expectation:

$$D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}}) = \mathbb{E}_{y \sim \pi_\theta(\cdot|x)} \left[\log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \right] = \mathbb{E}_{y \sim \pi_\theta} [\log \pi_\theta(y|x) - \log \pi_{\text{ref}}(y|x)].$$

For a *single sample* $y \sim \pi_\theta(\cdot|x)$, the quantity $\log \pi_\theta(y|x) - \log \pi_{\text{ref}}(y|x)$ is an unbiased estimate of the KL.

Computing $\log \pi_\theta(y|x)$: For an LLM, this is just the sum of log-probabilities of each token:

$$\log \pi_\theta(y|x) = \sum_{t=1}^T \log \pi_\theta(y_t|x, y_{<t}).$$

Run the LLM on the response, read off the probability it assigns to each token, sum the logs. Do this for both π_θ and π_{ref} , subtract.

Remark 18.7 (Why the KL Penalty?). Without the KL term, we would simply maximize $\mathbb{E}[r_\phi(x, y)]$. This fails for several reasons:

1. **Reward hacking:** The reward model r_ϕ is imperfect—trained on limited data. The policy will find adversarial outputs that score high on r_ϕ but are actually garbage. Example: repetitive text, nonsense that exploits quirks in the reward model.
2. **Distribution shift:** The reward model was trained on outputs from π_{ref} . As π_θ drifts away, r_ϕ 's predictions become unreliable (it's being evaluated out-of-distribution).
3. **Mode collapse:** Without regularization, the policy might converge to always outputting the same high-reward response, losing diversity.
4. **Preserve capabilities:** π_{ref} (the SFT model) already produces coherent, grammatical text. We want to improve *alignment*, not destroy language ability.

The KL term acts as a “leash”—the policy can improve, but not wander too far from known-good behavior.

Remark 18.8 (KL Divergence Direction: Why $D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$?). The KL divergence is *asymmetric*: $D_{\text{KL}}(P \parallel Q) \neq D_{\text{KL}}(Q \parallel P)$. The two directions have different names and behaviors:

Note on naming: some communities swap the labels “forward” and “reverse” depending on which distribution they call the “target.” The unambiguous way to read the direction is: *which distribution the expectation is taken under*.

Definition 18.9 (KL Divergence Directions). For distributions P and Q :

$$\textbf{Forward KL: } D_{\text{KL}}(P \parallel Q) = \mathbb{E}_{x \sim P} \left[\log \frac{P(x)}{Q(x)} \right] \quad (\text{expectation under } P)$$

$$\textbf{Reverse KL: } D_{\text{KL}}(Q \parallel P) = \mathbb{E}_{x \sim Q} \left[\log \frac{Q(x)}{P(x)} \right] \quad (\text{expectation under } Q)$$

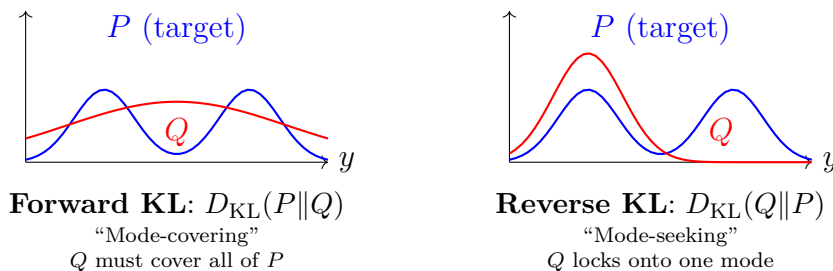
The naming convention: P is the “target” or “true” distribution, Q is the “approximation.”

In RLHF, π_{ref} is the reference (playing the role of P) and π_θ is what we’re learning (playing the role of Q). We use:

$$D_{\text{KL}}(\underbrace{\pi_\theta}_Q \parallel \underbrace{\pi_{\text{ref}}}_P) = \mathbb{E}_{y \sim \pi_\theta} \left[\log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \right] \quad (\textbf{Reverse KL: } D_{\text{KL}}(Q \parallel P))$$

This is reverse KL because the **expectation is taken under** π_θ (the distribution we’re learning), not under π_{ref} (the reference).

Why does RLHF use reverse KL? The two directions have very different behaviors:



- **Forward KL** $D_{\text{KL}}(P\|Q)$: Expectation under P . Penalizes Q whenever $P(x) > 0$ but $Q(x) \approx 0$. Effect: Q must *cover* all modes of P , even if it means spreading mass where P is low. This is **mode-covering** (also called “mean-seeking” or “inclusive”).
- **Reverse KL** $D_{\text{KL}}(Q\|P)$: Expectation under Q . Penalizes Q whenever $Q(x) > 0$ but $P(x) \approx 0$. Effect: Q avoids placing mass where P is low, typically *locking onto* a single mode of P . This is **mode-seeking** (also called “zero-forcing” or “exclusive”).

In **RLHF**, we use reverse KL $D_{\text{KL}}(\pi_\theta\|\pi_{\text{ref}})$ because:

1. The expectation is under π_θ , which we can sample from (we generate from the policy).
2. Mode-seeking behavior is desirable: we want π_θ to concentrate on high-quality outputs rather than spreading probability to cover everything π_{ref} might generate.
3. If π_θ tries to put mass where π_{ref} has near-zero probability, the penalty $\log(\pi_\theta/\pi_{\text{ref}}) \rightarrow +\infty$ explodes. This prevents the policy from generating outputs that the reference model would never produce.

Remark 18.10 (When to Question the KL Penalty). The KL penalty is not always the right choice:

1. **Over-regularization**: If β is too large, the model barely moves from π_{ref} . You pay the cost of RL but get minimal improvement. Some tasks genuinely require the model to behave very differently from the SFT baseline.
2. **Bad reference policy**: If π_{ref} is itself flawed (e.g., trained on low-quality data), anchoring to it perpetuates those flaws.
3. **Tuning β is hard**: Too small = reward hacking. Too large = no learning. The optimal β depends on the task, reward model quality, and is often found by expensive hyperparameter search.
4. **Alternative regularization**: Methods like GRPO use group-based normalization instead of KL, arguing that relative ranking within a batch is more robust than absolute distance from a reference.

18.1.2 Reward Modeling

The key insight: humans are better at *comparing* options than assigning absolute scores. Instead of asking “Rate this response 1-10,” we ask “Which response is better?”

Remark 18.11 (Why Bradley-Terry?). The Bradley-Terry model (1952) was originally developed for ranking chess players. The idea: each player has an underlying “strength” s_i , and the probability that player i beats player j is:

$$P(i \text{ beats } j) = \frac{s_i}{s_i + s_j} = \frac{1}{1 + s_j/s_i} = \sigma\left(\log \frac{s_i}{s_j}\right) = \sigma(\log s_i - \log s_j).$$

If we define $r_i = \log s_i$ as the “rating,” we get $P(i \text{ beats } j) = \sigma(r_i - r_j)$.

Key properties:

- *Transitivity*: If $r_A > r_B > r_C$, then $P(A \succ B) > 0.5$ and $P(B \succ C) > 0.5$.
- *Invariance*: Only *differences* in reward matter, not absolute values. Adding a constant to all rewards doesn't change preferences.
- *Calibration*: When $r_w - r_l = 0$, we get $P = 0.5$ (tie). As the difference grows, $P \rightarrow 1$.

For LLMs, we replace player strength with response quality: $r_\phi(x, y)$ is the “reward” or quality score for response y to prompt x .

Definition 18.12 (Bradley-Terry Model). Given preference pairs (y_w, y_l) where y_w is preferred over y_l , the probability of the preference under the Bradley-Terry model is

$$P(y_w \succ y_l | x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

where $\sigma(z) = 1/(1 + e^{-z})$ is the sigmoid function.

Definition 18.13 (Reward Model Loss). The reward model is trained to minimize the negative log-likelihood:

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)) \right].$$

18.2 DPO: Direct Preference Optimization

DPO eliminates the need for a separate reward model by directly optimizing the policy on preference data.

18.2.1 Derivation

Proposition 18.14 (Optimal Policy under KL Constraint). *The optimal policy for the RLHF objective has closed form:*

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{r(x, y)}{\beta}\right)$$

where $Z(x) = \sum_y \pi_{\text{ref}}(y|x) \exp(r(x, y)/\beta)$ is the partition function.

Proof. The RLHF objective with KL constraint is:

$$\max_{\pi} \mathbb{E}_{y \sim \pi} [r(x, y)] - \beta D_{\text{KL}}(\pi \| \pi_{\text{ref}}).$$

Expanding the KL divergence:

$$\begin{aligned} &= \mathbb{E}_{y \sim \pi} [r(x, y)] - \beta \mathbb{E}_{y \sim \pi} \left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \right] \\ &= \mathbb{E}_{y \sim \pi} \left[r(x, y) - \beta \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \right]. \end{aligned}$$

Taking the functional derivative with respect to $\pi(y|x)$ and setting to zero (with Lagrange multiplier for normalization) gives the Boltzmann distribution. $\square$

Corollary 18.15 (Implicit Reward). *Rearranging the optimal policy, the reward can be expressed as:*

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x).$$

Theorem 18.16 (DPO Loss). *Substituting the implicit reward into the Bradley-Terry preference model, the partition functions cancel:*

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right].$$

Proof. Under the Bradley-Terry model with the implicit reward:

$$\begin{aligned} P(y_w \succ y_l) &= \sigma(r(x, y_w) - r(x, y_l)) \\ &= \sigma \left(\beta \log \frac{\pi^*(y_w|x)}{\pi_{\text{ref}}(y_w|x)} + \beta \log Z(x) - \beta \log \frac{\pi^*(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \beta \log Z(x) \right) \\ &= \sigma \left(\beta \log \frac{\pi^*(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi^*(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right). \end{aligned}$$

Replacing π^* with the learned policy π_θ and taking negative log-likelihood gives the DPO loss. $\square$

Remark 18.17. The key insight is that the partition function $Z(x)$ cancels in the preference comparison, making the loss tractable without needing to compute the intractable normalization constant.

18.2.2 Gradient Analysis

Proposition 18.18 (DPO Gradient). *The gradient of the DPO loss is:*

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\beta \mathbb{E} [\sigma(\hat{r}_l - \hat{r}_w) (\nabla_\theta \log \pi_\theta(y_w|x) - \nabla_\theta \log \pi_\theta(y_l|x))]$$

where $\hat{r}_y = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ is the implicit reward.

Remark 18.19. The weighting $\sigma(\hat{r}_l - \hat{r}_w)$ is high when the model incorrectly ranks the pair (assigns higher implicit reward to the losing response). This focuses learning on mistakes.

18.3 IPO: Identity Preference Optimization

IPO addresses potential overfitting in DPO by using a different loss formulation.

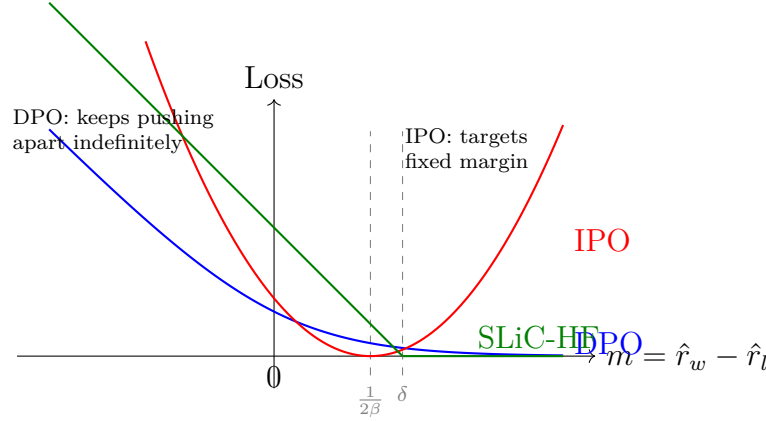
Definition 18.20 (IPO Loss).

$$\mathcal{L}_{\text{IPO}}(\theta) = \mathbb{E}_{(x, y_w, y_l)} \left[\left(\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \frac{1}{2\beta} \right)^2 \right].$$

Remark 18.21. IPO uses a squared loss that targets a fixed margin $1/(2\beta)$ between log-ratios, rather than pushing them apart indefinitely as in DPO.

18.3.1 Visualizing DPO vs IPO vs SLiC-HF

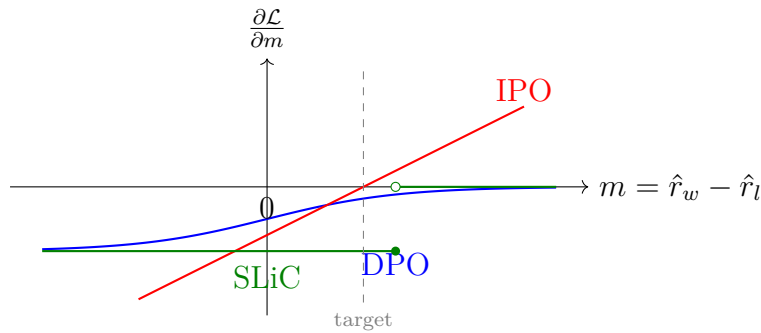
The key difference between these methods is how their loss behaves as the margin $m = \hat{r}_w - \hat{r}_l$ (implicit reward difference) changes:



Remark 18.22 (Comparing Loss Behaviors). • **DPO** ($-\log \sigma(m)$): Loss $\rightarrow 0$ as $m \rightarrow \infty$. The model is always incentivized to increase the margin further. This can lead to *over-optimization*—the model keeps pushing probabilities to extremes even after the preference is satisfied.

- **IPO** ($(m - \frac{1}{2\beta})^2$): Loss is minimized at $m = \frac{1}{2\beta}$. Once the target margin is achieved, pushing further *increases* the loss. This prevents over-optimization.
- **SLiC-HF** ($\max(0, \delta - m)$): Loss is exactly zero once $m \geq \delta$. No gradient once the margin is satisfied—the model stops optimizing this pair entirely.

Remark 18.23 (Gradient Magnitude Comparison). The gradient magnitude reveals how aggressively each method learns:



- **DPO**: Gradient $\rightarrow 0$ as $m \rightarrow \infty$, but never exactly zero. Always some signal.
- **IPO**: Gradient flips sign at the target. Pushes back if you overshoot.
- **SLiC**: Constant gradient until margin reached, then exactly zero.

18.4 GRPO: Group Relative Policy Optimization

GRPO, introduced by DeepSeek, removes the need for a critic network by using group-based advantage estimation.

18.4.1 Motivation

Standard policy gradient methods require a value function $V(s)$ to compute advantages. GRPO instead samples multiple outputs for each prompt and computes advantages relative to the group.

Definition 18.24 (GRPO Objective). For each prompt x , sample a group of G outputs $\{y_1, \dots, y_G\}$ from the old policy $\pi_{\theta_{\text{old}}}$. The GRPO objective is:

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathbb{E}_x \left[\frac{1}{G} \sum_{i=1}^G \min(\rho_i A_i, \text{clip}(\rho_i, 1 - \epsilon, 1 + \epsilon) A_i) - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) \right]$$

where $\rho_i = \frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)}$ is the importance ratio.

Definition 18.25 (Group Relative Advantage). The advantage for output y_i is computed by normalizing rewards within the group:

$$A_i = \frac{r(x, y_i) - \text{mean}(\{r(x, y_j)\}_{j=1}^G)}{\text{std}(\{r(x, y_j)\}_{j=1}^G)}.$$

Remark 18.26. This normalization ensures zero mean and unit variance advantages within each group, providing a natural baseline without a learned critic.

18.4.2 Token-Level KL Penalty

Definition 18.27 (Per-Token KL). The KL divergence is computed at the token level:

$$D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) = \sum_{t=1}^{|y|} D_{\text{KL}}(\pi_\theta(\cdot | x, y_{<t}) \| \pi_{\text{ref}}(\cdot | x, y_{<t})).$$

18.4.3 Comparison with PPO

Aspect	PPO	GRPO
Critic network	Required	Not required
Advantage estimation	GAE with $V(s)$	Group normalization
Memory overhead	High (critic params)	Lower
Samples per prompt	Typically 1	Multiple (G)

18.5 REINFORCE Leave-One-Out (RLOO)

RLOO is another critic-free method that uses leave-one-out baselines.

Definition 18.28 (RLOO Baseline). For output y_i in a group, the baseline is the mean reward of all other outputs:

$$b_i = \frac{1}{G-1} \sum_{j \neq i} r(x, y_j).$$

The advantage is simply $A_i = r(x, y_i) - b_i$.

Remark 18.29. Unlike GRPO’s normalized advantages, RLOO preserves the reward scale, which may be important when reward magnitudes carry meaning.

18.6 KTO: Kahneman-Tversky Optimization

KTO is designed for non-paired preference data (just labels of good/bad, not comparisons).

Definition 18.30 (KTO Loss).

$$\mathcal{L}_{\text{KTO}}(\theta) = \mathbb{E}_{x,y} \left[w(y) \left(1 - \sigma \left(\beta \text{sign}(y) \cdot (\log r_\theta(x, y) - z_{\text{ref}}) \right) \right) \right]$$

where:

- $\log r_\theta(x, y) = \log \pi_\theta(y|x) - \log \pi_{\text{ref}}(y|x)$,
- $\text{sign}(y) = +1$ if y is desirable, -1 if undesirable,
- z_{ref} is a reference point (KL from reference on desirable examples),
- $w(y)$ weights desirable vs undesirable examples (often with loss aversion $\lambda > 1$ for undesirable).

Remark 18.31. KTO is inspired by Kahneman and Tversky’s prospect theory: humans are loss-averse and evaluate outcomes relative to a reference point, not absolutely.

18.7 Reinforcement Learning with Verifiable Rewards (RLVR)

A paradigm shift in LLM training emerged with *Reinforcement Learning with Verifiable Rewards* (RLVR)—using programmatically checkable correctness signals instead of learned reward models or human preferences. This approach, notably used by DeepSeek-R1, enables scalable RL training for reasoning tasks.

18.7.1 Motivation: Beyond Human Preferences

Traditional RLHF has fundamental limitations:

- **Reward model bottleneck:** The learned reward model $r_\phi(x, y)$ is an imperfect proxy for human preferences, and optimizing against it leads to reward hacking.
- **Annotation cost:** Collecting high-quality human preferences is expensive and slow, limiting scale.
- **Subjectivity:** For tasks with objectively correct answers (math, code, logic), human preferences add noise rather than signal.

RLVR sidesteps these issues by using *verifiable rewards*—signals that can be computed automatically and are provably correct.

Definition 18.32 (Verifiable Reward). A reward function $r(x, y)$ is *verifiable* if it can be computed programmatically without human judgment, typically by checking the output against a ground-truth answer or executing it in an environment.

18.7.2 Sources of Verifiable Rewards

Domain	Verification Method	Reward Signal
Mathematics	Final answer matching	$r = \mathbf{1}[\text{extracted_answer} = \text{ground_truth}]$
Code	Test case execution	$r = \frac{\# \text{ passed tests}}{\# \text{ total tests}}$
Logic puzzles	Constraint satisfaction	$r = \mathbf{1}[\text{all constraints satisfied}]$
Formal proofs	Proof assistant verification (Lean, Coq)	$r = \mathbf{1}[\text{proof checks}]$
Games	Win/loss outcome	$r \in \{-1, 0, +1\}$

Remark 18.33 (Format Rewards). In addition to correctness rewards, RLVR often includes *format rewards* to encourage structured reasoning:

$$r_{\text{format}}(y) = \begin{cases} +\alpha & \text{if } y \text{ contains proper } \texttt{<think>...</think>} \text{ tags} \\ -\alpha & \text{otherwise} \end{cases}$$

This encourages the model to “show its work” in a parseable format without requiring human annotation.

18.7.3 The RLVR Training Pipeline

The typical RLVR pipeline has three stages:

1. **Cold Start (Optional SFT):** Initialize with a base model or perform light supervised fine-tuning on reasoning examples. Some approaches (like DeepSeek-R1-Zero) skip this entirely.

2. **RL with Verifiable Rewards:** Apply policy gradient methods (typically GRPO) with verifiable rewards:

- Sample multiple completions $\{y_i\}_{i=1}^G$ for each prompt x
- Compute verifiable rewards $r(x, y_i)$ for each completion
- Update policy using group-relative advantages

3. **Rejection Sampling + SFT (Optional):** Distill the RL-trained model by:

- Sampling many completions and keeping only correct ones
- Fine-tuning a new model on this curated dataset

18.7.4 Mathematical Formulation

RLVR typically uses GRPO (??) as the optimization algorithm. The key difference from RLHF is the reward source.

Definition 18.34 (RLVR Objective). Given a dataset of problems $\mathcal{D} = \{(x_i, a_i^*)\}$ where a_i^* is the ground-truth answer, the RLVR objective is:

$$\max_{\theta} \mathbb{E}_{x \sim \mathcal{D}} \left[\mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)} \left[r_{\text{verify}}(y, a^*) \right] \right] - \beta D_{\text{KL}}(\pi_{\theta} \| \pi_{\text{ref}})$$

where the verifiable reward is:

$$r_{\text{verify}}(y, a^*) = \mathbf{1}[\text{extract_answer}(y) = a^*].$$

In practice, GRPO is applied with:

$$\mathcal{L}_{\text{RLVR}}(\theta) = -\mathbb{E}_x \left[\frac{1}{G} \sum_{i=1}^G \min \left(\rho_i \tilde{A}_i, \text{clip}(\rho_i, 1 - \epsilon, 1 + \epsilon) \tilde{A}_i \right) \right]$$

where the advantage is computed from verifiable rewards:

$$\tilde{A}_i = \frac{r_{\text{verify}}(y_i, a^*) - \bar{r}}{\sigma_r + \varepsilon}, \quad \bar{r} = \frac{1}{G} \sum_{j=1}^G r_{\text{verify}}(y_j, a^*).$$

Remark 18.35 (Why GRPO for RLVR?). GRPO is particularly well-suited for RLVR because:

1. **No critic needed:** Verifiable rewards provide direct signal; no need to train a value network.
2. **Group normalization handles sparse rewards:** When most completions are wrong ($r = 0$) and few are correct ($r = 1$), normalization ensures meaningful gradients.
3. **Scales to long generations:** Reasoning traces can be thousands of tokens; critic-based methods struggle with credit assignment over such horizons.

18.7.5 DeepSeek-R1: A Case Study

DeepSeek-R1 demonstrated that pure RL with verifiable rewards can induce sophisticated reasoning behaviors without supervised fine-tuning on chain-of-thought data.

Example 18.36 (DeepSeek-R1-Zero). Starting from DeepSeek-V3-Base (no instruction tuning), applying GRPO with only:

- Accuracy rewards: $r = 1$ if final answer correct, 0 otherwise
- Format rewards: Small bonus for using `<think>` tags

The model spontaneously developed:

- Extended chain-of-thought reasoning
- Self-verification (“let me check this...”)
- Backtracking when detecting errors
- Reflection on approach (“maybe I should try a different method”)

These behaviors emerged purely from RL optimization—not from imitating human demonstrations.

Remark 18.37 (The “Aha Moment”). A striking observation from DeepSeek-R1-Zero training: the model learned to allocate more “thinking tokens” to harder problems. This emergent test-time compute scaling was not explicitly programmed but arose from the RL objective—longer reasoning chains were rewarded when they led to correct answers.

18.7.6 RLVR vs RLHF: Key Differences

Aspect	RLHF	RLVR
Reward source	Learned model $r_\phi(x, y)$	Programmatic verification
Ground truth	Human preferences (subjective)	Correct answer (objective)
Reward hacking	Major concern	Minimal (rewards are exact)
Scalability	Limited by annotation	Limited by verifiable problems
Applicable domains	Open-ended generation	Math, code, logic, games
Training signal	Dense (per-response)	Sparse (binary correctness)
Emergent behaviors	Style matching	Reasoning strategies

18.7.7 Challenges and Limitations

1. **Domain restriction:** RLVR only works for tasks with verifiable answers. Open-ended tasks (creative writing, general chat) still require RLHF or similar.
2. **Answer extraction:** Reliably extracting the final answer from free-form reasoning is non-trivial. Regex-based extraction can fail on edge cases.
3. **Reward sparsity:** Binary correctness rewards provide limited signal. A response that’s “almost right” gets the same $r = 0$ as a completely wrong one.
4. **Reasoning shortcuts:** Models may learn to pattern-match to answers without genuine reasoning, especially on problems similar to training data.
5. **Distribution mismatch:** Training on math/code problems may not transfer well to general reasoning capabilities.

Remark 18.38 (Combining RLVR and RLHF). Modern training pipelines often combine both:

1. RLVR on math/code to develop core reasoning capabilities
2. RLHF on general tasks to align style and safety

This hybrid approach leverages the scalability of verifiable rewards where possible while maintaining broad capabilities.

18.7.8 Process Reward Models: A Middle Ground

An intermediate approach uses *Process Reward Models* (PRMs) that provide step-level feedback rather than outcome-only rewards.

Definition 18.39 (Process Reward Model). A PRM $r_{\text{PRM}}(x, y_{\leq t})$ scores the reasoning process at each step, not just the final answer:

$$r_{\text{PRM}}(x, y_{\leq t}) = P(\text{step } t \text{ is correct} \mid x, y_{\leq t}).$$

PRMs offer:

- **Denser signal:** Feedback at each reasoning step, not just the end
- **Better credit assignment:** Identify where reasoning went wrong
- **Verifiable training data:** PRM training data can be generated by checking intermediate steps

However, PRMs still require significant annotation effort to label step-level correctness, though this can sometimes be automated for mathematical derivations.

18.8 Comparison of Alignment Methods

This section provides a comprehensive comparison of all preference optimization and RL alignment methods. The goal is to make each method identifiable by its loss function structure.

18.8.1 Quick Reference: Identifying Methods by Loss Function

Method	Loss Function	Identifying Feature
PPO	$\min(r_t A_t, \text{clip}(r_t, 1 \pm \epsilon) A_t)$	Clipped ratio $\times$ advantage
DPO	$-\log \sigma(\beta(\hat{r}_w - \hat{r}_l))$	σ of log-ratio <i>difference</i>
IPO	$(\hat{r}_w - \hat{r}_l - \frac{1}{2\beta})^2$	<i>Squared</i> loss with margin
GRPO	PPO-clip + group-normalized A_i	Advantages: $\frac{r_i - \mu}{\sigma}$
RLOO	REINFORCE with $b_i = \frac{1}{G-1} \sum_{j \neq i} r_j$	Leave-one-out baseline
KTO	$w(y)(1 - \sigma(\beta \cdot \text{sign}(y) \cdot \Delta))$	Uses $\text{sign}(y)$ for good/bad
SLiC-HF	$\max(0, \delta - \hat{r}_w + \hat{r}_l)$	Hinge loss (margin δ)
ORPO	$\mathcal{L}_{\text{SFT}} - \lambda \log \sigma(\text{log-odds diff})$	Combines SFT + odds ratio
SimPO	$-\log \sigma\left(\frac{\beta}{ y }(\log \pi_w - \log \pi_l) - \gamma\right)$	Length-normalized, no π_{ref}

Notation: $\hat{r}_y = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ is the implicit reward (log-ratio).

18.8.2 Detailed Loss Functions

For precise implementation, here are the complete loss functions:

PPO (Proximal Policy Optimization)

$$\mathcal{L}^{\text{PPO}}(\theta) = -\mathbb{E}_t [\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ and A_t is the advantage estimate (typically from GAE).

Remark 18.40 (Sequence vs Token-Level Ratios (How It’s Implemented for LLMs)). In classical RL, $r_t(\theta)$ is defined at a *single time step* (one action given one state). For an LLM, the “action” is the *next token*, so this definition still applies with $a_t = y_t$ and $s_t = (x, y_{<t})$.

Sometimes you will also see a *sequence-level* ratio for a full sampled response $y = (y_1, \dots, y_T)$:

$$\frac{\pi_\theta(y|x)}{\pi_{\theta_{\text{old}}}(y|x)} = \prod_{t=1}^T \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t | x, y_{<t})}.$$

In practice, implementations compute *token-level log-probabilities* and sum them:

$$\log \pi_\theta(y|x) = \sum_{t=1}^T \log \pi_\theta(y_t | x, y_{<t}),$$

and they often apply clipping and KL penalties *per token* (with masking) for stability and easier bookkeeping.

Key identifiers:

- Ratio r_t between new and *old* policy (not reference!)
- Clipping to $[1 - \epsilon, 1 + \epsilon]$
- Requires advantage estimation (critic network or GAE)

DPO (Direct Preference Optimization)

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

Equivalent forms:

$$\begin{aligned} &= -\mathbb{E} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x) \cdot \pi_{\text{ref}}(y_l|x)}{\pi_\theta(y_l|x) \cdot \pi_{\text{ref}}(y_w|x)} \right) \right] \\ &= -\mathbb{E} \left[\log \sigma(\hat{r}_w - \hat{r}_l) \right] \end{aligned}$$

Key identifiers:

- Sigmoid of *difference* of log-ratios
- Requires π_{ref} (reference policy)
- No clipping, no advantage—pure preference loss

IPO (Identity Preference Optimization)

$$\mathcal{L}_{\text{IPO}}(\theta) = \mathbb{E}_{(x, y_w, y_l)} \left[\left(\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \frac{1}{2\beta} \right)^2 \right]$$

Key identifiers:

- **Squared loss** (MSE) instead of log-sigmoid
- Targets a fixed margin $\frac{1}{2\beta}$ rather than pushing apart indefinitely
- Prevents overfitting to preference data

GRPO (Group Relative Policy Optimization)

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\mathbb{E}_x \left[\frac{1}{G} \sum_{i=1}^G \min(\rho_i A_i, \text{clip}(\rho_i, 1 - \epsilon, 1 + \epsilon) A_i) \right] + \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$$

where $\rho_i = \frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)}$ and the advantage is group-normalized:

$$A_i = \frac{r(x, y_i) - \bar{r}}{\sigma_r}, \quad \bar{r} = \frac{1}{G} \sum_{j=1}^G r(x, y_j), \quad \sigma_r = \text{std}(\{r_j\}).$$

Key identifiers:

- PPO-style clipping
- **Group-normalized advantages** (z-score within batch)
- **No critic network**—advantage from reward normalization
- Explicit KL penalty (not just implicit through reference)

RLOO (REINFORCE Leave-One-Out)

$$\mathcal{L}_{\text{RLOO}}(\theta) = -\mathbb{E}_x \left[\frac{1}{G} \sum_{i=1}^G \log \pi_\theta(y_i|x) \cdot \left(r(x, y_i) - \frac{1}{G-1} \sum_{j \neq i} r(x, y_j) \right) \right]$$

Key identifiers:

- REINFORCE gradient (no clipping)
- **Leave-one-out baseline:** $b_i = \frac{1}{G-1} \sum_{j \neq i} r_j$
- Preserves reward scale (unlike GRPO's normalization)

KTO (Kahneman-Tversky Optimization)

$$\mathcal{L}_{\text{KTO}}(\theta) = \mathbb{E}_{x,y} \left[\lambda_y \cdot \max \left(0, 1 - \sigma \left(\beta \cdot \text{sign}(y) (\hat{r}_y - z_0) \right) \right) \right]$$

where:

- $\text{sign}(y) = +1$ for desirable, -1 for undesirable responses
- $z_0 = \mathbb{E}_{x', y' \sim \mathcal{D}_{\text{desirable}}} [\hat{r}_{y'}]$ is a reference point
- λ_y implements loss aversion ($\lambda_{\text{undesirable}} > \lambda_{\text{desirable}}$)

Key identifiers:

- Uses $\text{sign}(y)$ to handle **unpaired** good/bad data
- Reference point z_0 (prospect theory)
- Asymmetric weighting (loss aversion)

SLiC-HF (Sequence Likelihood Calibration)

$$\mathcal{L}_{\text{SLiC}}(\theta) = \mathbb{E}_{(x, y_w, y_l)} \left[\max \left(0, \delta - \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} + \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

Key identifiers:

- **Hinge loss** (max with zero)
- Fixed margin δ (hyperparameter)
- Loss is zero once margin is achieved—no further optimization

ORPO (Odds Ratio Preference Optimization)

$$\mathcal{L}_{\text{ORPO}}(\theta) = \mathcal{L}_{\text{SFT}}(y_w) - \lambda \log \sigma \left(\log \frac{\text{odds}_\theta(y_w|x)}{\text{odds}_\theta(y_l|x)} \right)$$

where $\text{odds}_\theta(y|x) = \frac{\pi_\theta(y|x)}{1-\pi_\theta(y|x)}$.

Key identifiers:

- **Combines SFT loss** with preference loss
- Uses **odds ratio** instead of probability ratio
- **No reference policy** π_{ref} needed

SimPO (Simple Preference Optimization)

$$\mathcal{L}_{\text{SimPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\frac{\beta}{|y_w|} \log \pi_\theta(y_w|x) - \frac{\beta}{|y_l|} \log \pi_\theta(y_l|x) - \gamma \right) \right]$$

Key identifiers:

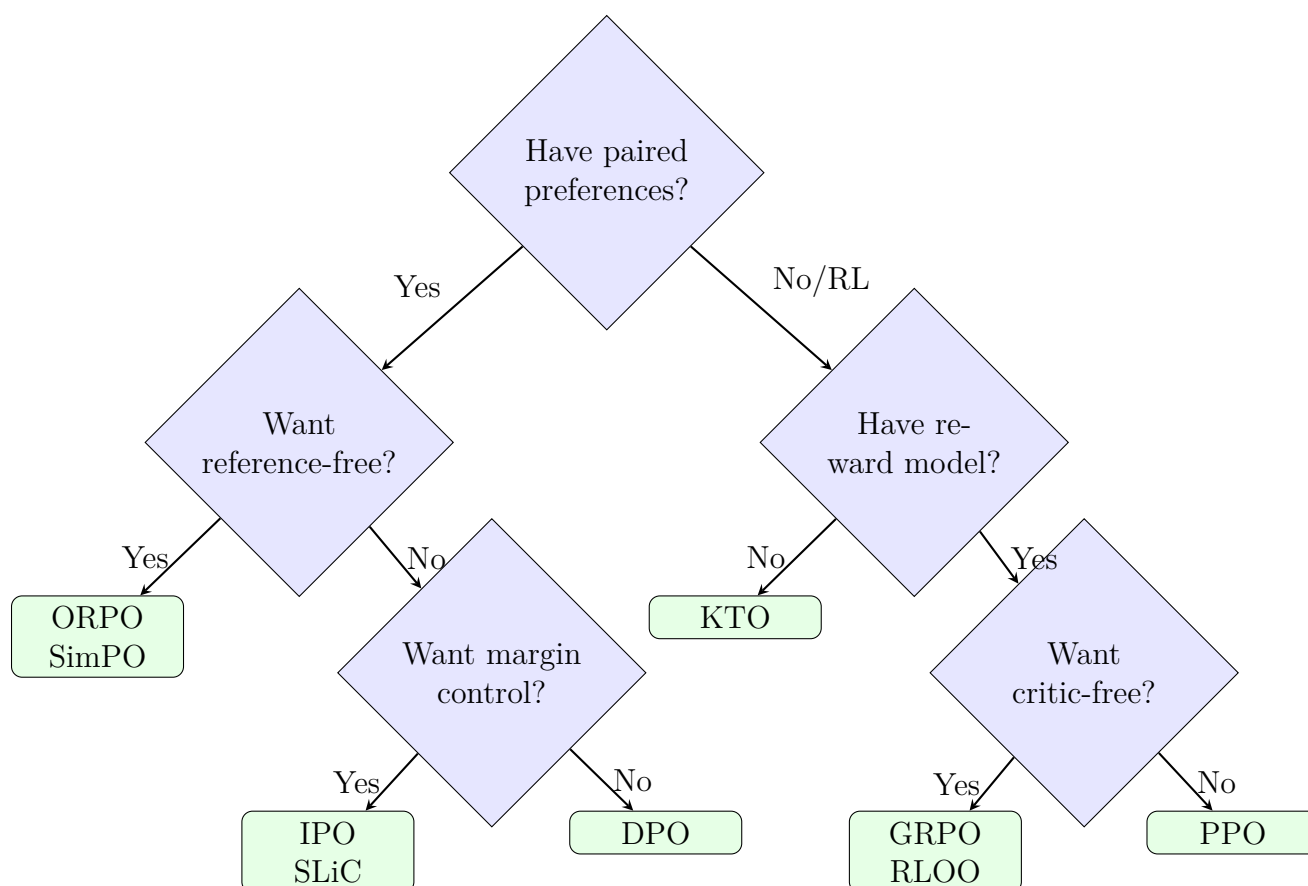
- **No reference policy** π_{ref}
- **Length normalization** by $|y|$ (average log-prob per token)
- Target margin γ between winner and loser

18.8.3 Taxonomy: What Makes Each Method Different?

Method	Needs π_{ref}	Needs Critic	Data Type	Online?	Loss Type
PPO	No*	Yes	RL rollouts	Yes	Clipped surrogate
DPO	Yes	No	Paired prefs	No	Log-sigmoid
IPO	Yes	No	Paired prefs	No	MSE
GRPO	Yes	No	RL rollouts	Yes	Clipped surrogate
RLOO	No	No	RL rollouts	Yes	REINFORCE
KTO	Yes	No	Unpaired	No	Prospect theory
SLiC-HF	Yes	No	Paired prefs	No	Hinge
ORPO	No	No	Paired prefs	No	SFT + odds
SimPO	No	No	Paired prefs	No	Log-sigmoid

*PPO in RLHF typically uses KL penalty to π_{ref} , but the core PPO algorithm doesn't require it.

18.8.4 Visual Decision Tree



18.8.5 Gradient Comparison

Understanding the gradients helps identify *how* each method learns:

Method	Gradient Structure
DPO	$\nabla_{\theta} \mathcal{L} \propto \underbrace{\sigma(\hat{r}_l - \hat{r}_w)}_{\text{error weight}} (\nabla \log \pi_{\theta}(y_w) - \nabla \log \pi_{\theta}(y_l))$
IPO	$\nabla_{\theta} \mathcal{L} \propto \underbrace{(\hat{r}_w - \hat{r}_l - \frac{1}{2\beta})}_{\text{linear error}} (\nabla \log \pi_{\theta}(y_w) - \nabla \log \pi_{\theta}(y_l))$
PPO/GRPO	$\nabla_{\theta} \mathcal{L} \propto A_t \nabla \log \pi_{\theta}(a_t s_t)$ (clipped when r_t out of range)
SLiC-HF	$\nabla_{\theta} \mathcal{L} = 0$ when margin satisfied; else like DPO

Key insight: DPO’s weighting $\sigma(\hat{r}_l - \hat{r}_w)$ is large when the model is “wrong” (assigns higher implicit reward to the loser). IPO’s linear weighting is gentler and prevents over-optimization.

18.8.6 When to Use What

- **DPO:** Default choice for offline preference optimization. Simple, effective, well-understood.
- **IPO:** When you observe DPO overfitting or want more controlled optimization.
- **SimPO/ORPO:** When you don’t want to maintain a reference policy (simpler infrastructure).
- **KTO:** When you only have thumbs-up/thumbs-down data, not pairwise comparisons.
- **GRPO:** Online RL without the complexity of training a critic; DeepSeek’s choice.
- **RLOO:** Similar to GRPO but preserves reward scale; simpler baseline computation.
- **PPO:** When you need fine-grained control, have compute for a critic, or are doing classic RL.
- **SLiC-HF:** When you want to stop optimizing once a margin is achieved.